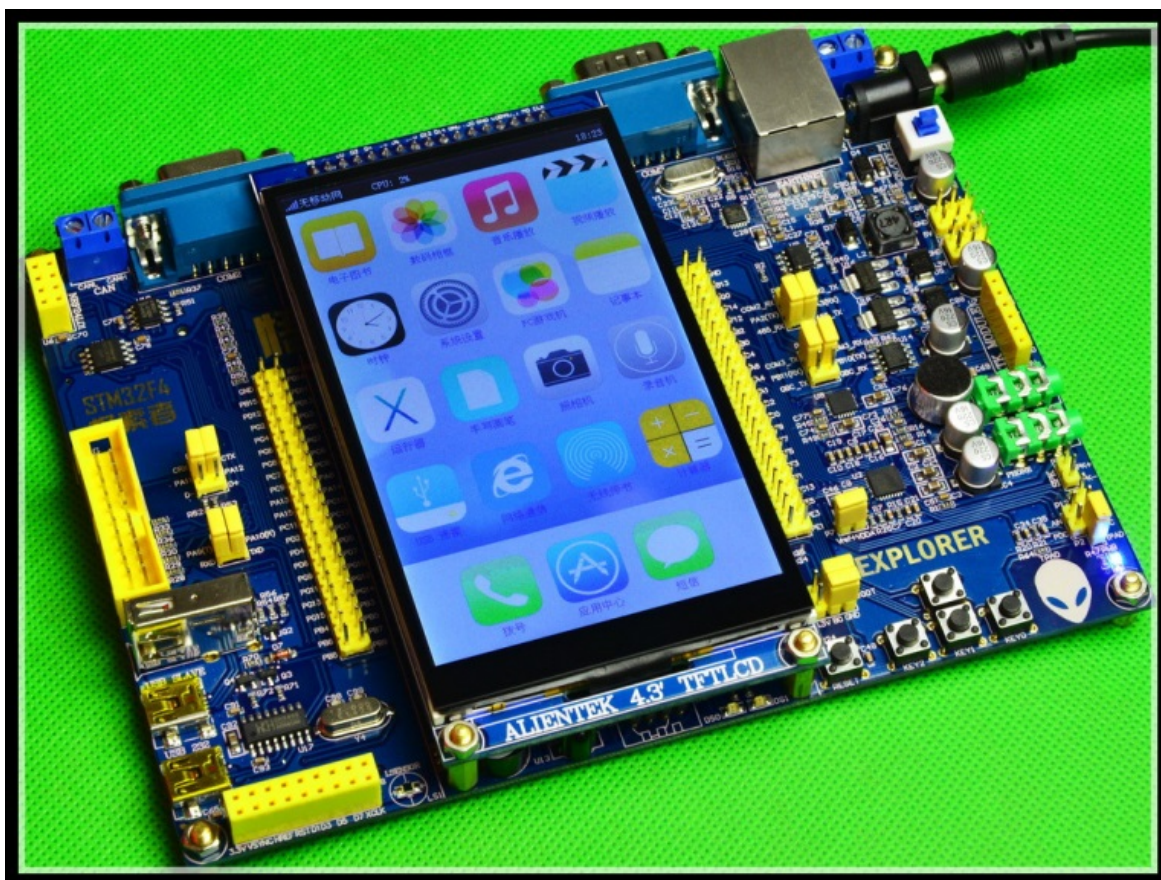


# EMWIN 开发手册

## V3.0

—ALIENTEK STM32F4 EMWIN 开发教程



官方店铺: <http://shop62057469.taobao.com>

技术论坛: [www.openedv.com](http://www.openedv.com)

技术交流群: 333121886

淘宝店铺: <http://eboard.taobao.com>

技术支持论坛 (开源电子网) : [www.openedv.com](http://www.openedv.com)

官方网站: [www.alientek.com](http://www.alientek.com)

最新资料下载链接: <http://www.openedv.com/posts/list/13912.htm>

E-mail: [389063473@qq.com](mailto:389063473@qq.com) QQ: [389063473](https://www.qq.com/)

咨询电话: [020-38271790](tel:020-38271790)

团队: [正点原子团队](#)

正点原子, 做最全面优秀的嵌入式开发平台软硬件供应商。

## 友情提示

如果您想及时免费获取“正点原子”最新资料, 敬请关注正点原子微信公众平台。我们将实时给您发布重大资料发布消息。



### 关注方法:

- (1) 微信“扫一扫”, 扫描右侧二维码, 添加关注
- (2) 微信->添加朋友->公众号->输入“正点原子”->关注
- (3) 微信->添加朋友->输入“alientek\_stm32”->关注



## 文档更新说明

| 版本   | 版本更新说明                                                                                                                                                                                                                                                             | 负责人 | 校审 | 发布日期       |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|----|------------|
| V1.0 | 初稿：<br><br>第一章 STemWin 无操作系统移植<br><br>第二章 STemWin 带操作系统移植                                                                                                                                                                                                          | 左忠凯 | 刘军 | 2014.11.4  |
| V2.0 | 修改：<br><br>1、修改 STemWin 中触摸屏驱动文件：<br>GUI_X_Touch_Analog.c 文件，使其支持 GT9147 电容触摸芯片(用于 ALIENTEK 4.3 寸电容屏)。<br><br>2、修改 LCDConf_FlexColor_Template.c 文件中的<br>函数 LCD_X_Config(), 极大的提高电容触摸屏的触摸精度！<br><br>新增：<br><br>第三章 在 PC 上仿真 STemWin 移植                              | 左忠凯 | 刘军 | 2014.12.10 |
| V3.0 | 第四章 文本显示<br><br>第五章 数值显示<br><br>第六章 2D 绘图<br><br>第七章 显示位图<br><br>第八章 颜色<br><br>第九章 存储设备<br><br>第十章 窗口管理<br><br>第十一章 GUIBuilder 的使用<br><br>第十二章 对话框<br><br>第十三章 窗口小工具(控件)<br><br>第十四章 BUTTON 控件<br><br>第十五章 CHECKBOX 控件<br><br>第十六章 DROPDOWN 控件<br><br>第十七章 EDIT 控件 | 左忠凯 | 刘军 | 2015.4.13  |

| 版本   | 版本更新说明                                                                                                                                                           | 负责人 | 校审 | 发布日期      |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|----|-----------|
| V3.0 | 第十八章 FRAMEWIN 控件<br>第十九章 字体<br>第二十章 XBF 格式字体显示<br>第二十一章 TTF 格式字体显示<br>第二十二章 汉字显示(外部存储器)<br>第二十三章 BMP 图片显示<br>第二十四章 JPEG 图片显示<br>第二十五章 GIF 图片显示<br>第二十六章 PNG 图片显示 | 左忠凯 | 刘军 | 2015.4.13 |



# 目录

|                                         |    |
|-----------------------------------------|----|
| EMWIN开发手册 .....                         | 1  |
| 第一章STemWin无操作系统移植 .....                 | 13 |
| 1.1 移植准备工作 .....                        | 14 |
| 1.2 向工程添加文件 .....                       | 18 |
| 1.3 修改工程文件 .....                        | 19 |
| 1.4 触摸屏移植 .....                         | 23 |
| 1.5 综合测试程序编写 .....                      | 28 |
| 1.6 下载验证 .....                          | 30 |
| 第二章 STemWin带操作系统移植 .....                | 31 |
| 2.1 移植准备工作 .....                        | 32 |
| 2.2 向工程添加以及修改相应文件 .....                 | 32 |
| 2.3 编写综合测试程序 .....                      | 35 |
| 2.4 下载验证 .....                          | 35 |
| 第三章 在PC上仿真STemWin .....                 | 37 |
| 3.1 准备emWin仿真文件 .....                   | 38 |
| 3.2 打开emWin仿真文件 .....                   | 38 |
| 3.3 进行仿真 .....                          | 40 |
| 3.4 emWin仿真文件介绍 .....                   | 42 |
| 3.4.1 Application文件 .....               | 42 |
| 3.4.2 Config文件 .....                    | 43 |
| 3.4.3 GUI文件 .....                       | 46 |
| 3.4.4 Sample文件 .....                    | 46 |
| 3.5 Hello Word! .....                   | 47 |
| 第四章 文本显示 .....                          | 51 |
| 4.1 基本文本显示 .....                        | 52 |
| 4.2 文本显示API函数 .....                     | 52 |
| 4.3 重点API函数简介 .....                     | 53 |
| 4.3.1 GUI_SetTextMode() .....           | 53 |
| 4.3.2 GUI_SetTextAlign() .....          | 54 |
| 4.3.3 GUI_Clear()和GUI_DispcEOL() .....  | 54 |
| 4.3.4 GUI_DispcStringInRectWrap() ..... | 55 |

|                                |           |
|--------------------------------|-----------|
| 4.4 演示实例 .....                 | 55        |
| <b>第五章 数值显示 .....</b>          | <b>58</b> |
| 5.1 数值显示API函数 .....            | 59        |
| 5.2 显示十进制数值 .....              | 59        |
| 5.3 显示浮点数值 .....               | 61        |
| 5.4 二进制和十六进制数值显示 .....         | 62        |
| 5.5 综合演示例程 .....               | 63        |
| <b>第六章 2D绘图 .....</b>          | <b>65</b> |
| 6.1 图形库基本API简介 .....           | 66        |
| 6.1.1 画笔大小 .....               | 66        |
| 6.1.2 基本绘图例程 .....             | 66        |
| 6.1.3 Alpha混色 .....            | 67        |
| 6.2 其他图形绘制API函数 .....          | 68        |
| 6.2.1 绘制线条 .....               | 68        |
| 6.2.2 其他图形绘制函数 .....           | 70        |
| 6.3 综合演示例程 .....               | 71        |
| <b>第七章 显示位图 .....</b>          | <b>77</b> |
| 7.1 位图转换器 .....                | 78        |
| 7.1.1 位图转换器简介 .....            | 78        |
| 7.1.2 位图转换器的使用 .....           | 78        |
| 7.2 位图绘制API简介 .....            | 81        |
| 7.3 综合演示例程 .....               | 82        |
| <b>第八章 颜色 .....</b>            | <b>85</b> |
| 8.1 预定义的基本颜色 .....             | 86        |
| 8.2 调色板 .....                  | 86        |
| 8.3 颜色API函数 .....              | 88        |
| 8.3 综合演示例程 .....               | 88        |
| <b>第九章 存储设备 .....</b>          | <b>92</b> |
| 9.1 存储设备简介 .....               | 93        |
| 9.1.1 存储设备初窥 .....             | 93        |
| 9.1.2 内存要求 .....               | 94        |
| 9.2 存储设备基本API函数 .....          | 95        |
| 9.2.1 GUI_MEMDEV_Clear() ..... | 96        |

|                                |            |
|--------------------------------|------------|
| 9.2.2 GUI_MEMDEV_CopyToLCD()   | 96         |
| 9.2.3 GUI_MEMDEV_CopyToLCDAt() | 96         |
| 9.2.4 GUI_MEMDEV_Create()      | 97         |
| 9.2.5 GUI_MEMDEV_Delete()      | 97         |
| 9.2.6 GUI_MEMDEV_Select()      | 97         |
| 9.2.7 存储设备演示例程                 | 97         |
| 9.3 分段存储设备                     | 100        |
| 9.3.1 分段存储简介                   | 100        |
| 9.3.2 分段存储设备演示例程               | 100        |
| 9.4 自动设备对象                     | 103        |
| 9.4.1 自动设备对象简介                 | 103        |
| 9.4.2 自动设备对象API函数              | 103        |
| 9.4.3 自动设备对象演示例程               | 106        |
| <b>第十章 窗口管理器</b>               | <b>110</b> |
| 10.1 窗口术语说明                    | 111        |
| 10.2 窗口回调机制                    | 112        |
| 10.2.1 不使用回调渲染                 | 112        |
| 10.2.2 使用回调渲染                  | 112        |
| 10.2.3 背景窗口重绘和回调               | 113        |
| 10.2.4 无效化                     | 113        |
| 10.2.5 渲染透明窗口                  | 114        |
| 10.2.6 自动使用存储设备                | 114        |
| 10.3 消息                        | 114        |
| 10.3.1 消息结构                    | 114        |
| 10.3.2 消息类型                    | 115        |
| 10.4 窗口管理器API函数                | 116        |
| 10.5 演示例程                      | 119        |
| 10.5.1 窗口重绘                    | 119        |
| 10.5.1 窗口管理API函数使用             | 122        |
| 10.5.3 窗口裁剪                    | 127        |
| <b>第十一章 GUIBulider的使用</b>      | <b>132</b> |
| 11.1 GUIBulider简介              | 133        |
| 11.2 GUIBulider使用步骤            | 135        |

|                                      |            |
|--------------------------------------|------------|
| 11.3 GUIBulider输出 .....              | 146        |
| 11.4 修改C文件.....                      | 149        |
| 11.5 LCD显示结果 .....                   | 151        |
| <b>第十二章 对话框 .....</b>                | <b>152</b> |
| 12.1 对话框基本原理 .....                   | 153        |
| 12.2 创建对话框 .....                     | 153        |
| 12.2.1 对话框资源表 .....                  | 153        |
| 12.2.2 对话框回调函数 .....                 | 154        |
| 12.2.3 初始化对话框 .....                  | 155        |
| 12.3 定义小工具功能 .....                   | 157        |
| 12.4 对话框API函数.....                   | 159        |
| 12.4.1 GUI_CreateDialogBox() .....   | 159        |
| 12.4.2 GUI_ExecCreatedDialog() ..... | 159        |
| 12.4.3 GUI_ExecDialogBox().....      | 159        |
| 12.4.4 GUI_EndDialog() .....         | 160        |
| <b>第十三章 窗口小工具(控件) .....</b>          | <b>161</b> |
| 13.1 小工具基础知识 .....                   | 162        |
| 13.1.1 可用小工具 .....                   | 162        |
| 13.1.2 了解重绘机制 .....                  | 163        |
| 13.1.3 如何使用小工具 .....                 | 164        |
| 13.2 通用控件API函数.....                  | 165        |
| 13.2.1 用于控件的WM函数.....                | 165        |
| 13.2.2 常用API函数.....                  | 165        |
| 13.2.3 用户绘制控件 .....                  | 166        |
| <b>第十四章 BUTTON控件 .....</b>           | <b>168</b> |
| 14.1 BUTTON控件简介 .....                | 169        |
| 14.2 通知代码.....                       | 169        |
| 14.3 键盘反应 .....                      | 169        |
| 14.4 常用API函数.....                    | 169        |
| 14.5 BUTTON控件基础例程 .....              | 170        |
| 14.5.1 程序设计 .....                    | 170        |
| 14.5.2 下载验证 .....                    | 173        |
| 14.6 BUTTON控件显示位图 .....              | 174        |

|                              |            |
|------------------------------|------------|
| 14.6.1 制作位图C文件 .....         | 174        |
| 14.6.2 GUIBulider设计界面 .....  | 176        |
| 14.6.3 程序设计 .....            | 180        |
| 14.6.4 下载验证 .....            | 184        |
| <b>第十五章 CHECKBOX控件 .....</b> | <b>185</b> |
| 15.1 通知代码 .....              | 186        |
| 15.2 键盘反应 .....              | 186        |
| 15.3 CHECKBOX控件API函数 .....   | 186        |
| 15.4 CHECKBOX控件演示例程 .....    | 187        |
| 15.4.1 程序设计 .....            | 187        |
| 15.4.2 下载验证 .....            | 191        |
| <b>第十六章 DROPDOWN控件 .....</b> | <b>192</b> |
| 16.1 DROPDOWN控件简介 .....      | 193        |
| 16.2 通知代码 .....              | 193        |
| 16.3 键盘反应 .....              | 194        |
| 16.4 DROPDOWN控件API函数 .....   | 194        |
| 16.5 DROPDOWN控件演示例程 .....    | 195        |
| 16.5.1 程序设计 .....            | 195        |
| 16.5.2 下载验证 .....            | 197        |
| <b>第十七章 EDIT控件 .....</b>     | <b>198</b> |
| 17.1 EDIT控件简介 .....          | 199        |
| 17.2 通知代码 .....              | 199        |
| 17.3 键盘反应 .....              | 199        |
| 17.4 EDIT控件API函数 .....       | 199        |
| 17.5 EDIT控件演示例程 .....        | 201        |
| 17.5.1 程序设计 .....            | 201        |
| 17.5.2 下载验证 .....            | 205        |
| <b>第十八章 FRAMEWIN控件 .....</b> | <b>206</b> |
| 18.1 FRAMEWIN控件简介 .....      | 207        |
| 18.2 FRAMEWIN控件API函数 .....   | 208        |
| 18.3 FRAMEWIN控件演示例程 .....    | 210        |
| 18.3.1 程序设计 .....            | 210        |
| 18.3.2 下载验证 .....            | 214        |

|                                       |     |
|---------------------------------------|-----|
| 第十九章 字体 .....                         | 216 |
| 19.1 字体简介 .....                       | 217 |
| 19.2 字体类型 .....                       | 217 |
| 19.3 字体格式 .....                       | 218 |
| 19.3.1 C文件格式 .....                    | 218 |
| 19.3.2 系统独立字体(SIF)格式 .....            | 218 |
| 19.3.3 外部位图字体(XBF)格式 .....            | 219 |
| 19.3.4 TrueType字体(TTF)格式 .....        | 219 |
| 19.4 声明自定义字体 .....                    | 220 |
| 19.5 选择字体 .....                       | 220 |
| 19.6 字体常用API函数 .....                  | 220 |
| 第二十章 XBF格式字体显示 .....                  | 222 |
| 20.1 XBF格式字库制作 .....                  | 223 |
| 20.2 XBF字体相关API函数 .....               | 228 |
| 20.2.1 GUI_XBF_CreateFont()函数 .....   | 228 |
| 20.2.2 GUI_XBF_DeleteFont() .....     | 229 |
| 20.3 程序设计 .....                       | 229 |
| 20.3.1 制作字体 .....                     | 229 |
| 20.3.2 演示例程 .....                     | 233 |
| 20.3.3 启用UTF8 编码 .....                | 239 |
| 20.4 下载验证 .....                       | 241 |
| 第二十一章 TTF格式字体显示 .....                 | 243 |
| 21.1 TTF字体简介 .....                    | 244 |
| 21.2 TTF字体相关API函数 .....               | 244 |
| 21.2.1 GUI_TTF_CreateFont()函数 .....   | 244 |
| 21.2.2 GUI_TTF_SetCacheSize()函数 ..... | 245 |
| 21.3 程序设计 .....                       | 246 |
| 21.3.1 移植 5.26 版本EMWIN .....          | 246 |
| 21.3.2 移植Freetype库 .....              | 246 |
| 21.3.3 Freetype内存管理 .....             | 248 |
| 21.3.4 准备TTF字库 .....                  | 249 |
| 21.3.4 程序源码 .....                     | 249 |
| 21.4 下载验证 .....                       | 252 |



|                   |     |
|-------------------|-----|
| 第二十二章 汉字显示(外部存储器) | 254 |
| 22.1 汉字显示简介       | 255 |
| 22.2 实验工程准备       | 255 |
| 22.3 字库制作         | 255 |
| 22.3.1 汉字字库制作     | 255 |
| 22.3.2 英文字符字库制作   | 257 |
| 22.4 程序设计         | 257 |
| 22.4.1 汉字字体制作     | 257 |
| 22.4.2 字库更新及测试    | 264 |
| 22.5 下载验证         | 266 |
| 第二十三章 BMP图片显示     | 268 |
| 23.1 支持格式         | 269 |
| 23.2 BMP文件API函数   | 269 |
| 23.3 程序设计         | 269 |
| 23.4 下载验证         | 276 |
| 第二十四章 JPEG图片显示    | 278 |
| 24.1 支持的JPEG压缩方法  | 279 |
| 24.2 RAM使用要求      | 279 |
| 24.3 JPEG文件API函数  | 279 |
| 24.4 程序设计         | 279 |
| 24.5 下载验证         | 285 |
| 第二十五章 GIF图片显示     | 288 |
| 25.1 GIF文件支持      | 289 |
| 25.2 GIF文件API函数   | 289 |
| 25.3 程序设计         | 289 |
| 25.4 下载验证         | 296 |
| 第二十六章 PNG图片显示     | 298 |
| 26.1 基础工程准备       | 299 |
| 26.2 RAM使用        | 299 |
| 26.3 PNG文件API函数   | 300 |
| 26.4 程序设计         | 300 |
| 26.4 下载验证         | 305 |



## 第一章 STemWin 无操作系统移植

我们在使用 M3 内核 MCU 的时候会移植 UCGUI 来制作精美的 UI, UCGUI 的高级版本就是 emWin, 而 STemWin 是 SEGGER 授权给 ST 的 emWin 版本, ST 的芯片可以免费使用 STemWin, 而且 STemWin 针对 ST 的芯片做了优化。本章我们将向大家介绍如何在 STM32F407 开发板上移植 STemWin, 在本章的移植过程中适配了 ALIENTEK 的 2.8 寸, 3.5 寸, 4.3 寸和 7 寸(SSD1963) 这四种不同尺寸和分辨率的屏幕。本章分为如下几个部分:

- 1.1 移植准备工作
- 1.2 向工程添加文件
- 1.3 修改工程文件
- 1.4 触摸屏移植
- 1.5 综合测试程序编写
- 1.6 下载验证

## 1.1 移植准备工作

### 1) 建立移植基础工程

我们在移植 STemWin 之前首先要建立一个基础工程，然后在这个基础工程上添加文件，因为在以后的 STemWin 实验中我们使用了内存管理，因此这个基础工程就使用内存管理实验的工程，我们将内存管理实验的整个工程拷贝过来。这里我们要对内存管理实验的工程进行一点小的修改，因为在内存管理实验的工程中存在与 STemWin 重名的文件和变量，这些重名的文件和变量需要修改一下，修改步骤如下：

1、将 HARDWARE 文件夹下的 LCD 驱动程序文件 lcd.h 和 lcd.c 改为其他的名字，这里改为 ILI93xx.h 和 ILI93xx.c。

2、将 ILI93xx.c 中的 LCD\_Init 改为其他名字，这里改为 TFTLCD\_Init。

3、在 ILI93xx.h 中我们有如下代码，我们定义了一个 LCD，在这里将 LCD 改为 TFTLCD，记得将整个工程中的 LCD 都要改为 TFTLCD。

4、STM32F4xx 内置 FPU，STemWin 针对 STM32F4 做了相应的修改，支持内部 FPU 和 ST 的 DSP 库，所有如果我们是给 STM32F4xx 等 M4 内核的 MCU 移植 STemWin 的话就要使能 STM32F4xx 的 FPU，并将其 DSP 库添加到工程中，如果是给 STM32F1xx 等 M3 内核的 MCU 移植的话就不需要这一步。

首先我们先在 ST 官网上下载 STM32F4xx 的标准外设和 DSP 库，下载地址为：[http://www.st.com/web/en/catalog/tools/FM147/CL1794/SC961/SS1743/PF257901?s\\_searchtype=keyword#](http://www.st.com/web/en/catalog/tools/FM147/CL1794/SC961/SS1743/PF257901?s_searchtype=keyword#)。下载界面如图 1.1.1 所示，可以看出当前最新的库为 1.4.0。

Related Tools and Software

| Part Number | Description                                                                                                                                        |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| STM32CubeF4 | Embedded software for STM32 F4 series (HAL low level drivers, USB, TCP/IP, File system, RTOS, Graphic - coming with examples running on ST boards) |

Get Software Top

| Part Number   | Version | Marketing Status | Order From ST            |
|---------------|---------|------------------|--------------------------|
| STSW-STM32065 | 1.4.0   | Active           | <a href="#">Download</a> |

(\*) Suggested Resale Price per unit (USD) for BUDGETARY USE ONLY. For quotes, prices in local currency, please contact your local ST Sales Office or our Distributors  
 (\*\*) The Material Declaration forms available on st.com may be generic documents based on the most commonly used package within a package family. For this reason, they may not be 100% accurate for a specific device. Please contact our sales support for information on specific devices.

图 1.1.1 STM32F4xx 库下载

下载解压后打开 Libraries 文件，打开后如图 1.1.2 所示，里面包含有 STM32F4xx 的标准外设库和 DSP 库，DSP 库在 CMSIS 文件中，我们将 CMSIS 复制到我们基础工程的 FWLIB 文件中。CMSIS 文件大概有 133M，里面包含有一些我们用不到的文件，而我们只用到 DSP 库，所以在这里将这些不用的文件删除，最终只留下 Include 和 Lib 这两个文件。Include 为头文件，Lib 为针对不同的编译环境而做的 DSP 库，有 ARM，G++，GCC。因为我们使用的是 KEIL 所以只留 ARM 这个文件，其他两个也删除掉，如果使用的是 IAR 就保留 GCC 这个文件。

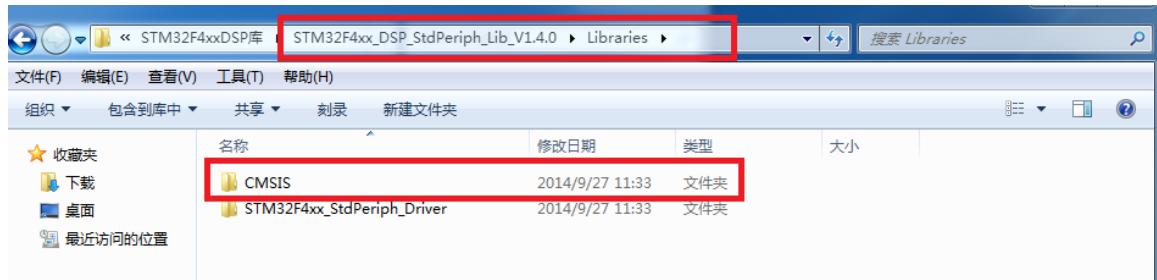


图 1.1.2 STM32F4xx 库文件

要使用 STM32F4 的 FPU，我们首先需要开启 FPU，打开 system\_stm32f4xx.c 文件，里面有一个 SystemInit() 函数，这个函数完成系统初始化，在一开始就有 FPU 设置选项，如图 1.1.3 所示，

```

417 void SystemInit(void)
418 {
419     /* FPU settings -----*/
420     #if (__FPU_PRESENT == 1) && (__FPU_USED == 1)
421         SCB->CPACR |= ((3UL << 10*2) | (3UL << 11*2)); /* set CP10 and CP11 Full Access */
422     #endif
423     /* Reset the RCC clock configuration to the default reset state -----*/
424     /* Set HSION bit */
425     RCC->CR |= (uint32_t)0x00000001;
426
427     /* Reset CFGR register */
428     RCC->CFGR = 0x00000000;
429

```

图 1.1.3 设置 FPU

从图 1.1.3 中我们可以看出如果要使用 FPU 的话 \_\_FPU\_PRESENT 和 \_\_FPU\_USED 要为 1，根据 arm\_math.h 中的要求，如果要使用 DSP 库的话还要定义 ARM\_MATH\_CM4 和 \_\_CC\_ARM，因此我们按图 1.1.4 所示添加 \_\_FPU\_PRESENT，\_\_FPU\_USED，ARM\_MATH\_CM4 和 \_\_CC\_ARM 的定义。

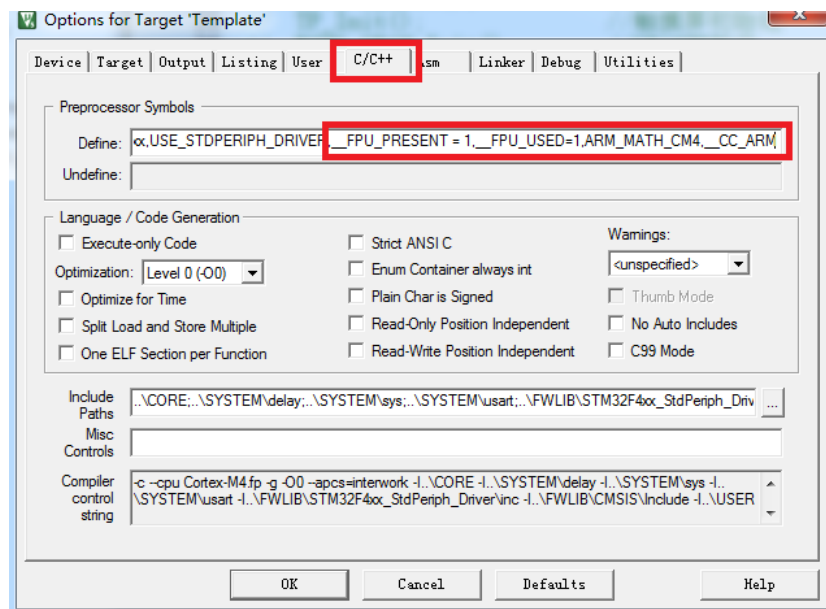


图 1.1.4 定义 \_\_FPU\_PRESENT 和 \_\_FPU\_USED

最后我们设置 keil 软件使用 FPU，如图 1.1.5 所示。

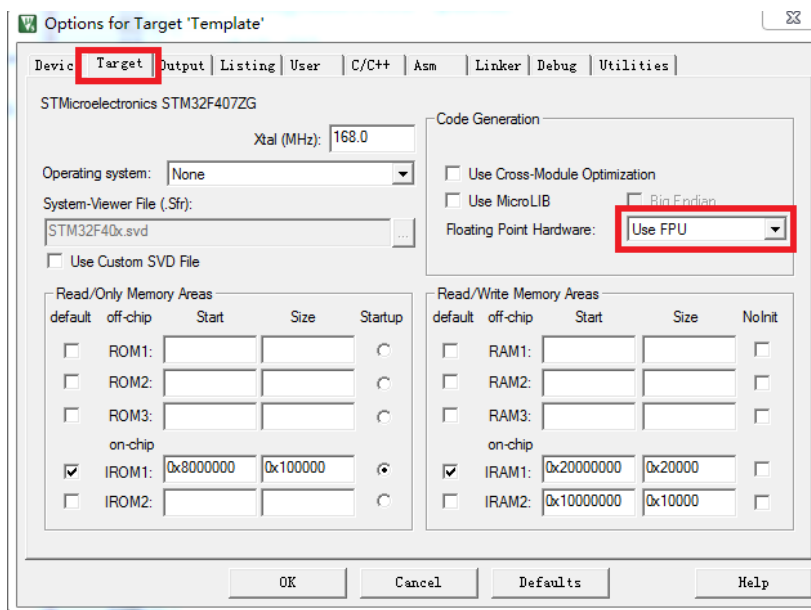


图 1.1.5 选择使用 FPU

上面步骤完成后就可以将 DSP 库添加到工程中，在 CMSIS 下的 Include 文件中有个 arm\_math.h 头文件，Lib 下的 ARM 文件中有个 arm\_cortexM4lf\_math.lib 的库文件。这两个就是我们最终要使用的 DSP 库文件，将这两个文件添加到工程中，可以看到在 ARM 文件中有许多的 lib 文件，这些 lib 文件很大，我们只使用了 arm\_cortexM4lf\_math.lib 这一个文件，其他就可以删掉了，这时候我们在看一下 CMSIS 文件的大小只有 11M 左右。添加完后的工程如图 1.1.6 所示。至此，移植 STemWin 所需的基础工程就建立完成。

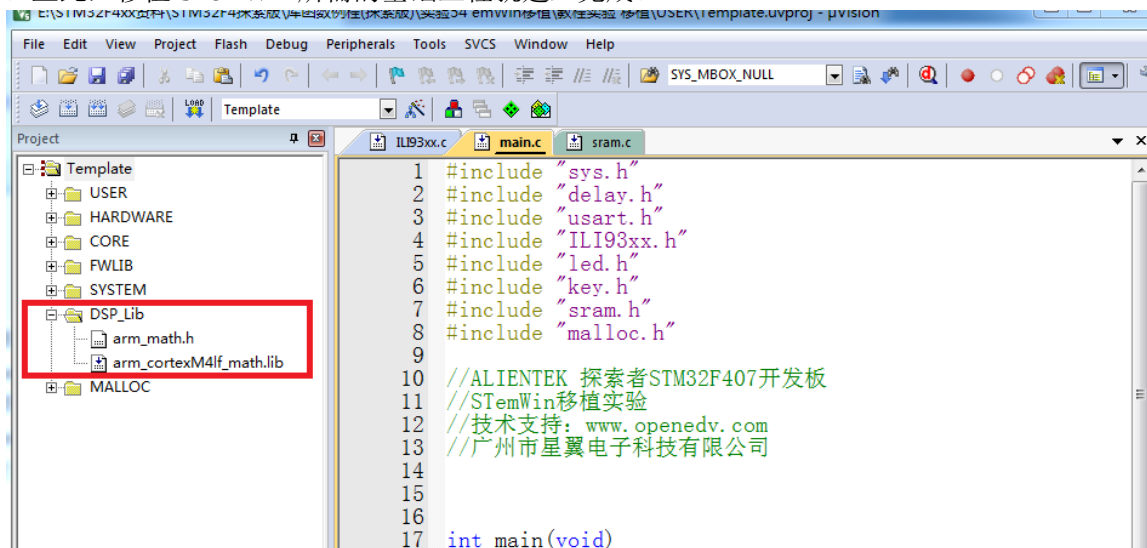


图 1.1.6 添加 DSP 后的工程

## 2) 准备 STemWin 源文件

要移植 STemWin，我们首先要有源文件，STemWin 源码我们可以在 ST 官网下载到，下载地址：[http://www.st.com/web/catalog/tools/FM147/CL1794/SC961/SS1743/LN1734/PF259225?s\\_searchtype=partnumber](http://www.st.com/web/catalog/tools/FM147/CL1794/SC961/SS1743/LN1734/PF259225?s_searchtype=partnumber) 下载界面如图 1.1.7 所示。当前 STemWin 版本为 1.1.2，对应着 5.22 版本的 emWin，我们已经下载好放在光盘中，路径:6.软件资料->EMWIN 学习资料->STemWin 源码

## Related Tools and Software

| Related Tools and Software |                                                                                                                                                           |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Part Number                | Description                                                                                                                                               |
| STM32CubeF2                | Embedded software for STM32 F2 series (HAL low level drivers, USB, TCP/IP, File system, RTOS, Graphic - coming with examples running on ST boards)        |
| STM32CubeF3                | Embedded software for STM32 F3 series (HAL low level drivers, USB, File system, RTOS, Touch Sensing, Graphic - coming with examples running on ST boards) |
| STM32CubeF4                | Embedded software for STM32 F4 series (HAL low level drivers, USB, TCP/IP, File system, RTOS, Graphic - coming with examples running on ST boards)        |

## Get Software

[Top](#)

| Part Number | Version | Marketing Status | Order From ST            |
|-------------|---------|------------------|--------------------------|
| STemWin     | 1.1.2   | Active           | <a href="#">Download</a> |

(\*) Suggested Resale Price per unit (USD) for BUDGETARY USE ONLY. For quotes, prices in local currency, please contact your local [ST Sales Office](#) or our [Distributors](#)

(\*\*) The Material Declaration forms available on st.com may be generic documents based on the most commonly used package within a package family. For this reason, they may not be 100% accurate for a specific device. Please contact our [sales support](#) for information on specific devices.

图 1.1.7 STemWin 下载页面

下载解压后的文件名为 STemWin\_Library\_V1.1.2, STemWin 源文件就在这个文件夹了, 但是这个文件夹里还有其他一些文件, 这些文件我们不用管, 我们只需要提出我们想要的 STemWin 源码, 在 STemWin\_Library\_V1.1.2 下的 Libraries 文件夹里面找到 STemWinLibrary522 这个文件, 这个就是 STemWin 源码, 如图 1.1.8 所示, 注意图中的路径, 我们从名字可以看出这是在 5.22 版本的 emWin 上修改而来的。

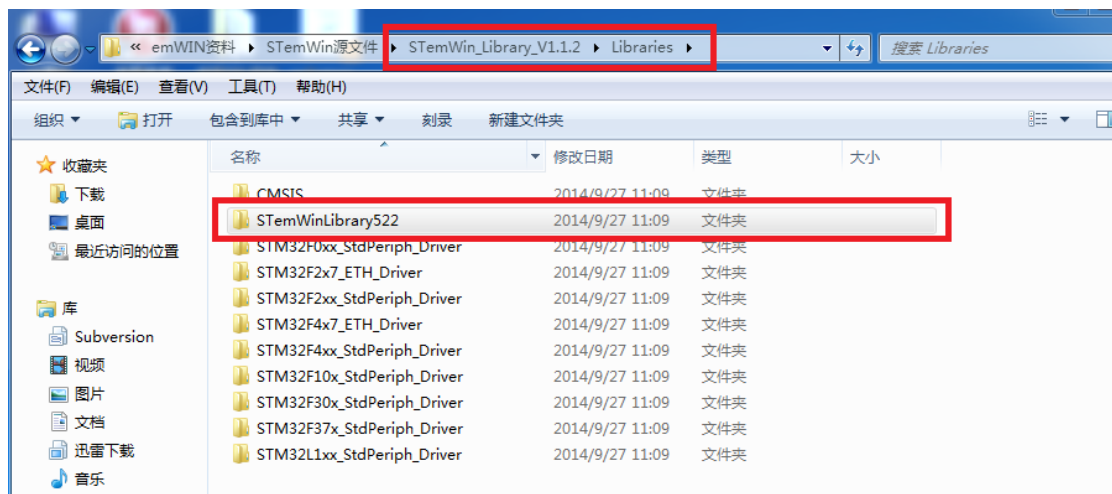


图 1.1.8 STemWin 源码

打开 STemWinLibrary522, 里面有 6 个文件, 每个文件的说明如表 1.1.1 所示。

| 文件            | 说明                   |
|---------------|----------------------|
| Config        | LCD 接口以及 emWin 配置文件, |
| Documentation | STemWin 的函数说明文档      |
| inc           | STemWin 函数的头文件       |
| Lib           | STemWin 的函数库         |
| OS            | 裸机或使用 OS 情况下的驱动      |
| Software      | 使用到的一些软件             |

表 1.1.1 STemWin 各文件说明

## 1.2 向工程添加文件

移植所需要的基础工程和 STemWin 源码准备好之后就可以将 STemWin 添加到工程中，在基础工程目录下新建 EMWIN 文件夹，然后将 STemWin 源码复制到 EMWIN 文件夹中，最后按图 1.2.1 所示向工程中添加 STemWin 和相应的头文件路径。

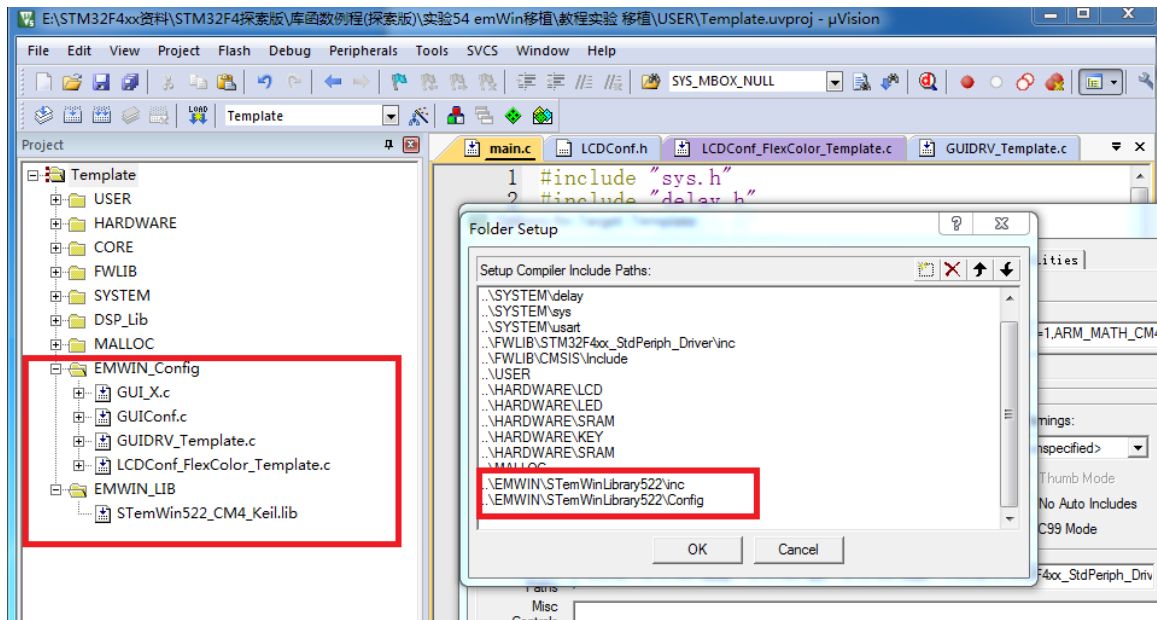


图 1.2.1 添加 STemWin 源码后的工程

图 1.2.1 中各个文件的功能以及在 STemWin 源码中的位置如表 1.2.1 所示。

| 文件                           | 功能                | 位置         |
|------------------------------|-------------------|------------|
| GUI_X.c                      | GUI 所依赖的外部配置/系统   | OS 文件下     |
| GUIConf.c                    | GUI 初始化配置         | Config 文件下 |
| GUIConf.h                    | GUI 配置设置          |            |
| GUIDRV_Template.c            | GUI 读点、画点、画线等函数配置 |            |
| LCDConf_FlexColor_Template.c | LCD 初始化配置         | Lib 文件下    |
| STemWin522_CM4_Keil.lib      | STemWin 函数库       |            |

表 1.2.1 工程中各个文件功能与位置

我们打开 STemWin 源码 STemWinLibrary522 下的 Lib 文件，如图 1.2.2 所示。STemWin 为不同的编译环境做了不同的库，而且分为带 OS 和不带 OS 两个版本。由于 Lib 文件太大，所以只保留适合我们所使用开发环境的带 OS 和不带 OS 的两种库文件。因为我使用的是 KEIL，所以我只保留 STemWin522\_CM4\_OS\_Keil.lib 和 STemWin522\_CM4\_Keil.lib 这两个库，其他的删除掉，如果使用的是其他内核的 MCU 和开发环境的话请选择相应的库文件，本章讲解的是无操作系统 STemWin 的移植，因此在工程中添加 STemWin522\_CM4\_Keil.lib 这个库文件，如果带操作系统的话就选择另一个库文件。上一节说了 STemWinLibrary522 文件下有 6 个文件，这里为了减小工程所占内存的大小，将 Documentation 和 Software 这两个文件删除掉，这样整个工程就小了很多。

工程添加完成后编译，提示没有 LCDConf.h 文件，我们新建一个 LCDConf.h 文件放到 STemWin 源码下的 Config 文件下，LCDConf.h 中可以不写任何程序，添加完成后再编译一次就没有任何错误了，如果还有错误提示的话根据错误类型来修改，直至编译没有错误提示。



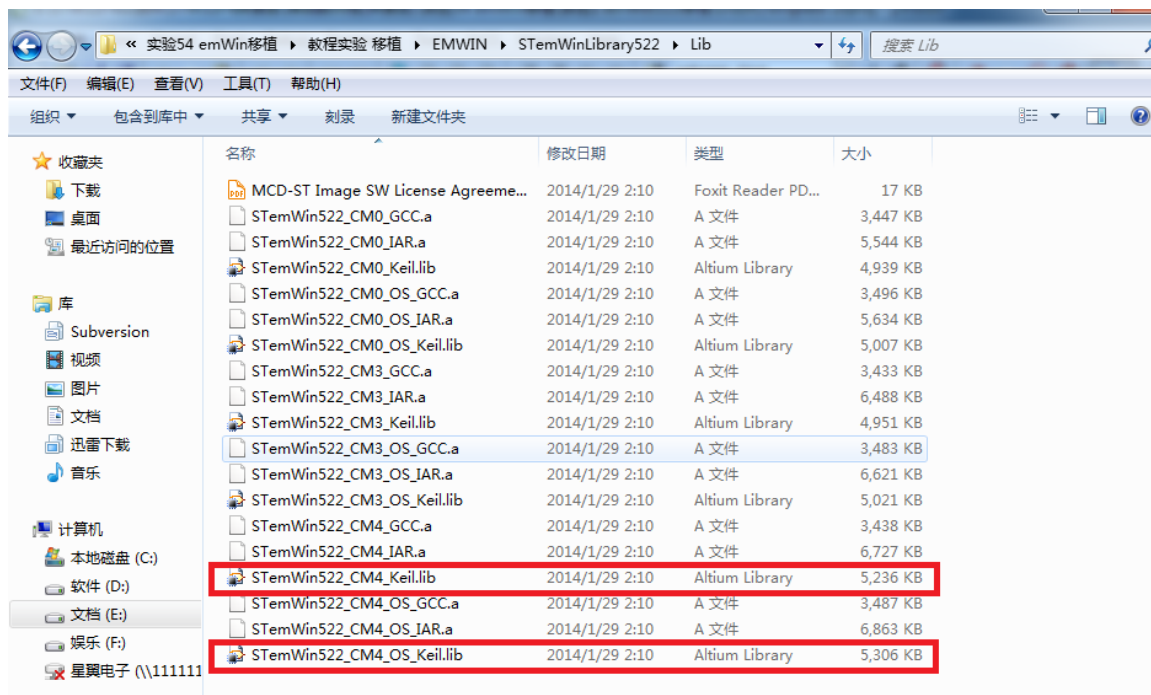


图 1.2.2 STemWin 库文件

### 1.3 修改工程文件

#### 1) 修改 GUIConf.h 文件

在 GUIConf.h 文件中定义了我们是否使用操作系统、鼠标、窗口管理等信息，还定义了默认字体，GUIConf.h 文件代码如下。

```
#ifndef GUICONF_H
#define GUICONF_H

#define GUI_NUM_LAYERS          2          // 显示的最大层数
#define GUI_OS                  (0)        // 不使用操作系统
#define GUI_SUPPORT_TOUCH       (0)        // 不支持触摸
#define GUI_DEFAULT_FONT        &GUI_Font6x8 //默认字体
#define GUI_SUPPORT_MOUSE       (1)        //支持鼠标
#define GUI_WINSUPPORT          (1)        //支持窗口管理
#define GUI_SUPPORT_MEMDEV      (1)        //支持存储设备
#define GUI_SUPPORT_DEVICES     (1)        //使用设备指针

#endif
```

#### 2) 修改 GUIConf.c 文件

在 GUIConf.c 文件中定义了 STemWin 所使用的内存，在这里使用内存管理实验中的内存管理函数来为 STemWin 分配内存，这样做的好处就是可以使用外部 1M 的 SRAM，GUIConf.c 文件代码如下，我们通过定义 USE\_EXRAM 来决定是否使用外部 SRAM。

```
#include "GUI.h"
#include "sram.h"
#include "malloc.h"
```

```

#define USE_EXRAM 0//不使用外部 SRAM
#define GUI_NUMBYTES (100*1024) //设置 EMWIN 内存大小
#define GUI_BLOCKSIZE 0X80 //块大小

//GUI_X_Config
//初始化的时候调用,用来设置 emwin 所使用的内存
void GUI_X_Config(void)
{
    if(USE_EXRAM) //使用外部 RAM
    {
        //从外部 SRAM 中分配 GUI_NUMBYTES 字节的内存
        U32 *aMemory = mymalloc(SRAMEX,GUI_NUMBYTES);
        //为存储管理系统分配一个存储块
        GUI_ALLOC_AssignMemory((void*)aMemory, GUI_NUMBYTES);
        //设置存储快的平均尺寸,该区越大,可用的存储快数量越少
        GUI_ALLOC_SetAvBlockSize(GUI_BLOCKSIZE);
        GUI_SetDefaultFont(GUI_FONT_6X8); //设置默认字体
    }else //使用内部 RAM
    {
        //从内部 RAM 中分配 GUI_NUMBYTES 字节的内存
        U32 *aMemory = mymalloc(SRAMIN,GUI_NUMBYTES);
        //为存储管理系统分配一个存储块
        GUI_ALLOC_AssignMemory((U32 *)aMemory, GUI_NUMBYTES);
        //设置存储快的平均尺寸,该区越大,可用的存储快数量越少
        GUI_ALLOC_SetAvBlockSize(GUI_BLOCKSIZE);
        GUI_SetDefaultFont(GUI_FONT_6X8); //设置默认字体
    }
}

```

### 3) 修改 GUIDRV\_Template.c 文件

在这个文件中我们要完成 STemWin 的打点、读点函、填充等函数的实现,最后还优化了 16BPP, 这个文件是移植的重点, 这个文件中的代码优化好的话会极大的提升 STemWin 的性能, 在这个文件中我们需要修改四个函数: \_SetPixelIndex(), \_GetPixelIndex(), \_FillRect() 和 \_DrawBitLine16BPP(), 修改后的这四个函数如下:

```

//STemWin 中的打点函数
static void _SetPixelIndex(GUI_DEVICE * pDevice, int x, int y, int PixelIndex) {
    LCD_Fast_DrawPoint(x,y,PixelIndex); //调用 ILI93xx.c 文件中的快速打点函数
}

//STemWin 中的读点函数
static unsigned int _GetPixelIndex(GUI_DEVICE * pDevice, int x, int y) {
    unsigned int PixelIndex;
    #if (LCD_MIRROR_X == 1) || (LCD_MIRROR_Y == 1) || (LCD_SWAP_XY == 1)
        int xPhys, yPhys;

```

```

    xPhys = LOG2PHYS_X(x, y);
    yPhys = LOG2PHYS_Y(x, y);
#else
    #define xPhys x
    #define yPhys y
#endif
GUI_USE_PARA(pDevice);
GUI_USE_PARA(x);
GUI_USE_PARA(y);
{
    PixelIndex = LCD_ReadPoint(x,y);    //调用 ILI93xx.c 文件中的读点函数
}
#if (LCD_MIRROR_X == 0) && (LCD_MIRROR_Y == 0) && (LCD_SWAP_XY == 0)
    #undef xPhys
    #undef yPhys
#endif
return PixelIndex;
}

//STemWin 中向指定区域填充指定颜色函数
static void _FillRect(GUI_DEVICE * pDevice, int x0, int y0, int x1, int y1) {
    LCD_Fill(x0,y0,x1,y1,LCD_COLORINDEX);    //调用 ILI93xx.c 文件中的填充函数
}

//STemWin 中绘制 16BPP 位图函数
static void _DrawBitLine16BPP(GUI_DEVICE * pDevice, int x, int y, U16 const GUI_UNI_PTR * p,
int xsize) {
    LCD_PIXELINDEX pixel;
    LCD_SetCursor(x,y);
    *(__IO uint16_t*)(UCGUI_LCD_CMD) = lcddev.wramcmd;    //写入颜色值
    for (;xsize > 0; xsize--, x++, p++)
    {
        pixel = *p;
        *(__IO uint16_t*)(UCGUI_LCD_DATA) = pixel;
    }
}

```

#### 4) 修改 LCDConf\_FlexColor\_Template.c 文件

这个文件有 LcdWriteReg()、LcdWriteData()、LcdWriteDataMultiple()、LcdReadDataMultiple()、LCD\_X\_Config()和 LCD\_X\_DisplayDriver()这 6 个函数。因为 STemWin 自带有一些 LCD 驱动 IC 的驱动程序，使用前 4 个函数可以直接使用 STemWin 的 LCD 驱动程序，但是我们在实际使用中会有很多 LCD 驱动 IC 是 STemWin 自带的驱动程序不支持的，所以这里我们没用这 4 个函数而是使用我们自己的 LCD 初始化函数 TFTLCD\_Init()函数来初始化 LCD，并且通过 GUIDRV\_Template.c 文件将打点和读点等函数封装起来传递给 STemWin，在这里我们把这 4 个

函数删除掉，只留下 LCD\_X\_Config()和 LCD\_X\_DisplayDriver()这两个函数，LCD\_X\_Config()函数代码如下所示。

```
void LCD_X_Config(void) {
    //创建显示驱动器件
    GUI_DEVICE_CreateAndLink(&GUIDRV_Template_API, GUICC_M565, 0, 0);
    LCD_SetSizeEx (0, lcddev.width, lcddev.height);
    LCD_SetVSizeEx (0, lcddev.width, lcddev.height);
}
```

上面代码中的 GUI\_DEVICE\_CreateAndLink()函数用来创建显示驱动器件，设置用于存取显示器的颜色转换程序，并将驱动器件连接到给定层的器件列表。函数的第一个参数是一个指向结构体的指针，这个结构体就是显示器的驱动，因为 STemWin 自带一些液晶 IC 的驱动，因此这里可以选择相应的驱动程序，具体使用请查阅 emWin 手册的“显示驱动”章节。第二个参数是指定所使用的调色板，STemWin 中有很多调色板可以使用，具体使用请查阅 emWin 手册的“颜色”章节，在这里我们使用的是 GUICC\_M565 的调色板，也就是 RGB565，最后我们根据 LCD 尺寸来设置屏幕的大小。

### 5) 编写测试代码

到这一步 STemWin 的移植基本已经完成，我们可以编写测试代码来测试一下移植是成功，测试代码如下。

```
int main(void)
{
    delay_init(168);          //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断分组配置
    uart_init(115200);        //串口波特率设置
    LED_Init();               //LED 初始化
    TFTLCD_Init();            //LCD 初始化
    FSMC_SRAM_Init();         //SRAM 初始化
    mem_init(SRAMIN);         //初始化内部内存池
    mem_init(SRAMEX);         //初始化外部内存池
    mem_init(SRAMCCM);        //初始化 CCM 内存池

    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_CRC,ENABLE); //开启 CRC 时钟
    GUI_Init();               //STemWin 初始化
    GUI_SetBkColor(GUI_BLUE); //设置背景颜色
    GUI_SetColor(GUI_YELLOW); //设置颜色
    GUI_Clear();              //清屏
    GUI_SetFont(&GUI_Font24_ASCII); //设置字体
    GUI_DispStringAt("Hello Word!",0,0); //在指定位置显示字符串
    while(1);
}
```

我们要使用 STemWin 的话一定要先使能 CRC 的时钟，否则 STemWin 不能使用。测试代码很简单，是在 LCD 上的(0,0)坐标处显示”Hello Word!”，可以在 LCD 初始化函数 TFTLCD\_Init()中使用 LCD\_Scan\_Dir()函数来改变横竖屏，在这里我们使用的是横屏，显示效果如图 1.3.1 所示。



图 1.3.1 测试效果图

## 1.4 触摸屏移植

我们知道 STemWin 是支持触摸屏的，所以这一节我们移植触摸屏部分。

### 1) 修改和添加触摸屏底层驱动程序

首先肯定是要先将底层的触摸屏驱动添加到工程中，这个大家都知道怎么添加的，这里需要将 touch.c, gt9147.c, ott2001a 和 ft5206.c 这四个文件添加到工程中，注意还有它们相对应的头文件路径。

### 2) 添加 GUI\_X\_Touch\_Analog.c 文件

在工程中添加 GUI\_X\_Touch\_Analog.c 这个文件，文件代码如下：

```
#include "GUI.h"
#include "touch.h"
#include "ILI93xx.h"
#include "usart.h"

void GUI_TOUCH_X_ActivateX(void)
{
    // XPT2046_WriteCMD(0x90);
}

void GUI_TOUCH_X_ActivateY(void)
{
    //XPT2046_WriteCMD(0xd0);
}

int GUI_TOUCH_X_MeasureX(void)
```

```

{
    int32_t xvalue;
    //电容屏的触摸值获取(使用 NT5510 和 SSD1963 芯片的 4.3 寸和 7 寸屏都是电容屏)
    if((lcddev.id == 0X5510) || (lcddev.id == 0X1963))
    {
        tp_dev.scan(0);
        xvalue=tp_dev.x[0];
        return xvalue;
    }else                //电阻屏
    {
        return TP_Read_XOY(0XD0); //CMD_RDX=0XD0
    }
}

int GUI_TOUCH_X_MeasureY(void)
{
    int32_t yvalue;
    //电容屏的触摸值获取(使用 NT5510 和 SSD1963 芯片的 4.3 寸和 7 寸屏都是电容屏)
    if((lcddev.id == 0X5510) || (lcddev.id == 0X1963))
    {
        tp_dev.scan(0);
        yvalue = tp_dev.y[0];
        return yvalue;
    }else                //电阻屏
    {
        return TP_Read_XOY(0X90); //CMD_RDX=0XD0
    }
}

```

GUI\_X\_Touch\_Analog.c 文件有四个函数：GUI\_TOUCH\_X\_ActivateX()，GUI\_TOUCH\_X\_ActivateY()，GUI\_TOUCH\_X\_MeasureX()和 GUI\_TOUCH\_X\_MeasureY()。其中前两个我们没有使用到，STemWin 真正调用 GUI\_TOUCH\_X\_MeasureX() 和 GUI\_TOUCH\_X\_MeasureY 这两个函数来获取触摸屏按下时的 X 轴和 Y 轴 AD 值，这里我们根据 LCD 的 ID 的不同来判断触屏使用的是电容屏还是电阻屏，并分别作相应的处理，也就是调用不同的函数。

### 3) 修改 GUIConf.h 和 LCDConf\_FlexColor\_Template.c 文件

将 GUIConf.h 文件中的 GUI\_SUPPORT\_TOUCH 定义为 1，使 STemWin 支持触摸屏。在 LCDConf\_FlexColor\_Template.c 文件的开始添加如下代码。

```

#define TOUCH_AD_TOP        160 //根据实际情况填写!!!!
#define TOUCH_AD_BOTTOM    3990
#define TOUCH_AD_LEFT      160
#define TOUCH_AD_RIGHT     3990

```

以上定义了触摸屏的上下左右边界的 AD 值，此值为实际测量自己的触摸屏得出来的，此值可以通过串口输出，也可以使用 STemWin 提供的函数来获取，函数如下所示。



```
void Mytouch_MainTask(void)
{
    GUI_PID_STATE TouchState;
    int          xPhys;
    int          yPhys;
    GUI_Init();
    GUI_SetFont(&GUI_Font20_ASCII);
    GUI_CURSOR_Show();
    GUI_CURSOR_Select(&GUI_CursorCrossL);
    GUI_SetBkColor(GUI_WHITE);
    GUI_SetColor(GUI_BLACK);
    GUI_Clear();
    GUI_DispString("Measurement of\nA/D converter values");
    while (1)
    {
        GUI_TOUCH_GetState(&TouchState); // Get the touch position in pixel
        xPhys = GUI_TOUCH_GetxPhys();      // Get the A/D measurement result in x
        yPhys = GUI_TOUCH_GetyPhys();      // Get the A/D measurement result in y
        GUI_SetColor(GUI_BLUE);
        GUI_DispStringAt("Analog input:\n", 0, 40);
        GUI_GotoY(GUI_GetDispPosY() + 2);
        GUI_DispString("x:");
        GUI_DispDec(xPhys, 4);
        GUI_DispString(", y:");
        GUI_DispDec(yPhys, 4);
        GUI_SetColor(GUI_RED);
        GUI_GotoY(GUI_GetDispPosY() + 4);
        GUI_DispString("\nPosition:\n");
        GUI_GotoY(GUI_GetDispPosY() + 2);
        GUI_DispString("x:");
        GUI_DispDec(TouchState.x, 4);
        GUI_DispString(", y:");
        GUI_DispDec(TouchState.y, 4);
        delay_ms(50);
    };
}
```

当点击触摸屏的时候，此函数将点击处的触摸屏 AD 值显示在 LCD 上，程序执行结果如图 1.4.1 所示，屏幕为竖屏，其中蓝色为原始的 AD 值，红色的为屏幕坐标值。

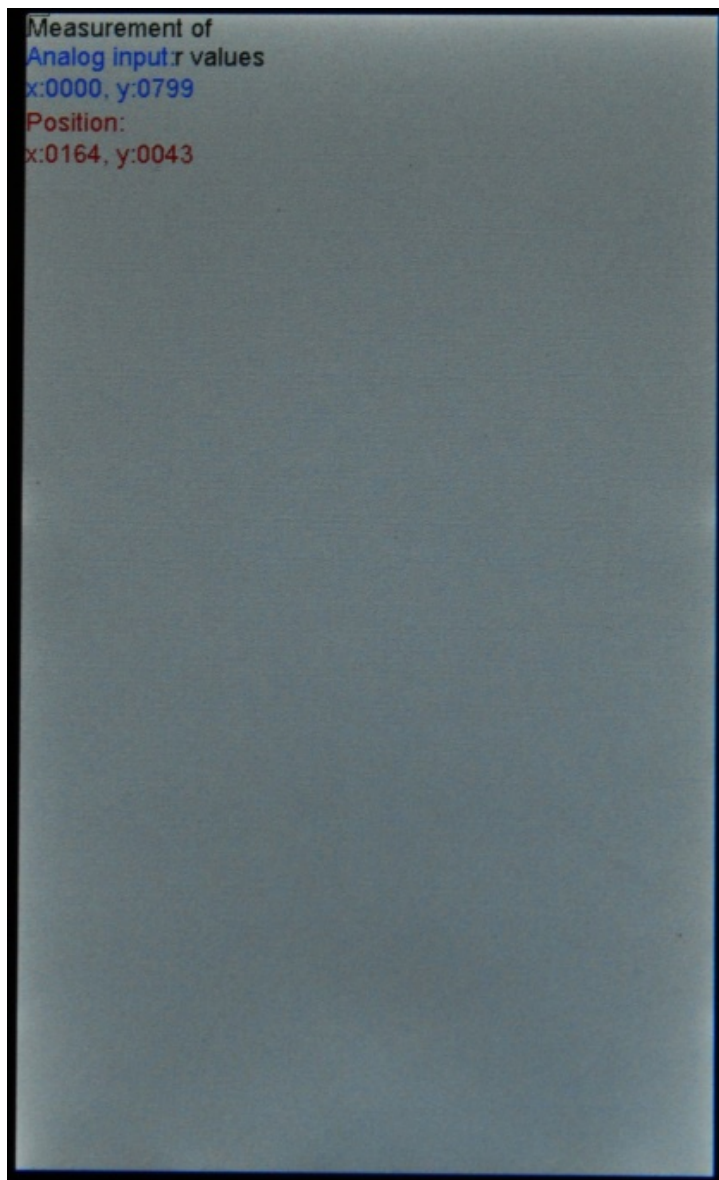


图 1.4.1 触摸屏 AD 值获取

最后修改 LCDConf\_FlexColor\_Template.c 文件中的 LCD\_X\_Config()函数，修改后的函数如下所示。

```
void LCD_X_Config(void) {  
    //创建显示驱动器件  
    GUI_DEVICE_CreateAndLink(&GUIDRV_Template_API,    GUICC_M565,    0,    0);  
    LCD_SetSizeEx    (0, lcddev.width, lcddev.height);  
    LCD_SetVSizeEx    (0, lcddev.width, lcddev.height);  
    if(lcddev.id == 0X5510) //0X5510 为 4.3 寸 800x480 的屏  
    {  
        if(lcddev.dir == 0) //竖屏    // 2001:3520 6352  
        {  
            // 9147:3832 6288  
            GUI_TOUCH_Calibrate(GUI_COORD_X,0,480,0,479);  
            GUI_TOUCH_Calibrate(GUI_COORD_Y,0,800,0,799);  
        }  
    }  
}
```



```

    }else //横屏
    {
        //GUI_TOUCH_SetOrientation(GUI_SWAP_XY|GUI_MIRROR_Y);
        GUI_TOUCH_Calibrate(GUI_COORD_X,0,480,0,479);
        GUI_TOUCH_Calibrate(GUI_COORD_Y,0,800,0,799);
    }
}
}else if(lcddev.id == 0X1963)//1963 为 7 寸屏 800*480
{
    if(lcddev.dir == 0) //竖屏
    {
        // 6368 3816
        //GUI_TOUCH_SetOrientation(GUI_SWAP_XY|GUI_MIRROR_Y);
        GUI_TOUCH_Calibrate(GUI_COORD_X,0,800,0,799);
        GUI_TOUCH_Calibrate(GUI_COORD_Y,0,480,0,479);
    }else //横屏
    {
        GUI_TOUCH_Calibrate(GUI_COORD_X,0,800,0,799);
        GUI_TOUCH_Calibrate(GUI_COORD_Y,0,480,0,479);
    }
}
}else if(lcddev.id == 0X5310 || lcddev.id == 0X6804) //0X5310 0X6804 为 3.5 寸 320x480
{
    if(lcddev.dir == 0) //竖屏
    {
        GUI_TOUCH_Calibrate(GUI_COORD_X,0,320,3931,226);
        GUI_TOUCH_Calibrate(GUI_COORD_Y,0,480,3812,196);
    }else //横屏
    {
        GUI_TOUCH_SetOrientation(GUI_SWAP_XY|GUI_MIRROR_Y);
        GUI_TOUCH_Calibrate(GUI_COORD_X,0,320,3931,226);
        GUI_TOUCH_Calibrate(GUI_COORD_Y,0,480,3812,196);
    }
}
}
else //其他屏幕全部默认为 2.8 寸 320X240
{
    if(lcddev.dir == 0) //竖屏
    {
        GUI_TOUCH_Calibrate(GUI_COORD_X,0,lcddev.width,155,3903);
        GUI_TOUCH_Calibrate(GUI_COORD_Y,0,lcddev.height,188,3935);
    }else //横屏
    {
        GUI_TOUCH_SetOrientation(GUI_SWAP_XY|GUI_MIRROR_Y);
        GUI_TOUCH_Calibrate(GUI_COORD_X,0,240,155,3903);
        GUI_TOUCH_Calibrate(GUI_COORD_Y,0,320,188,3935);
    }
}

```

```
}
}
```

注意！这里在处理电容屏的时候我们并没有测量上面那四个 AD 值，并且写入函数中。因为电容屏直接读出来的点坐标就是 LCD 屏幕上相对应的点坐标，不需要校准，但是 GUI\_TOUCH\_Calibrate() 函数的第 4,5 两个参数还是要写的，这两个参数本来是需要测量的，这里我们就直接根据屏幕的大小写进去，比如如果屏幕 X 轴大小为 480，Y 轴大小为 800，函数 GUI\_TOUCH\_Calibrate() 的参数就可以按照如下程序配置，注意第 4,5 个参数的设置。

```
GUI_TOUCH_Calibrate(GUI_COORD_X,0,480,0,479);
GUI_TOUCH_Calibrate(GUI_COORD_Y,0,800,0,799);
```

至此，触摸屏部分移植完成。

## 1.5 综合测试程序编写

移植完成以后我们可以移植 STemWin 的官方 Demo 来测试移植是否正确，这个官方的 Demo 需要操作系统提供系统时钟，本节移植教程中我们并没有使用操作系统，这里使用一个定时器来提供这个系统时钟，官方 Demo 中还需要定时调用 GUI\_TOUCH\_Exec() 函数来定时处理触摸屏事件，因此这里我们使用定时器 3 来提供系统时钟，使用定时器 4 来周期调用 GUI\_TOUCH\_Exec() 函数处理触摸屏事件。定时器 3 和 4 的中断服务函数代码如下。

//定时器 3 中断服务程序

```
void TIM3_IRQHandler(void)
{
    if(TIM_GetITStatus(TIM3,TIM_IT_Update)==SET) //溢出中断
    {
        OS_TimeMS++;
    }
    TIM_ClearITPendingBit(TIM3,TIM_IT_Update); //清除中断标志位
}
```

//定时器 4 中断服务程序

```
void TIM4_IRQHandler(void)
{
    if(TIM_GetITStatus(TIM4,TIM_IT_Update)==SET) //溢出中断
    {
        TIM_ClearITPendingBit(TIM4,TIM_IT_Update); //清除中断标志位
        GUI_TOUCH_Exec();
    }
}
```

从定时器 3 的中断服务程序中可以看出我们通过对 OS\_TimeMS 这个全局变量加一来提供系统时钟。接下来我们将这个 Demo 移植到我们的工程中，这个移植非常简单，只要将这个 Demo 拷贝到我们的工程文件中，并将其加入到我们的工程中就可以了，编译后会提示在 GUIDEMO.c 文件中不能打开 bsp.h 这个头文件，我们将添加 bsp.h 文件的这句话注销掉，添加完成后的工程如图 1.5.1 所示。

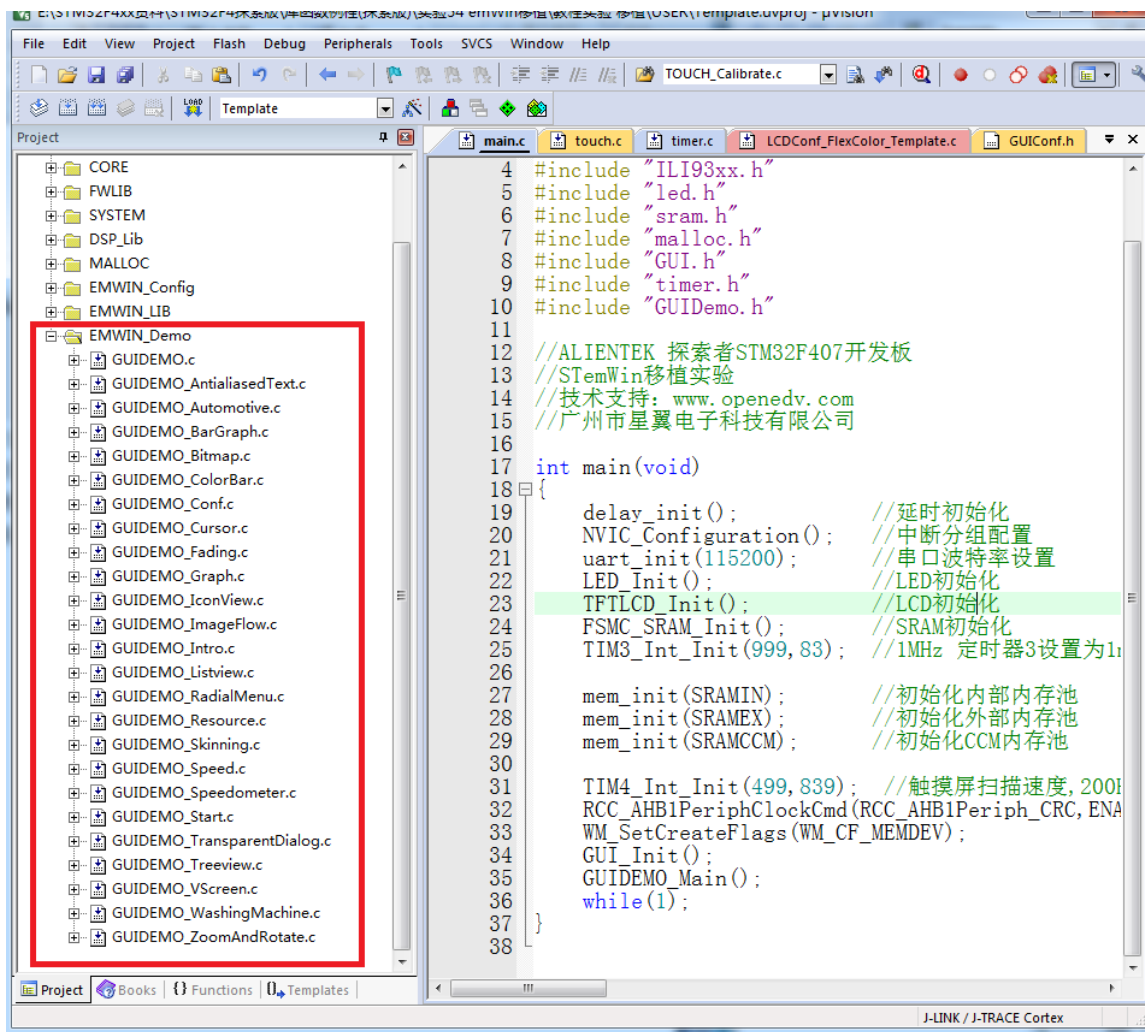


图 1.5.1 添加 Demo

接下来我们编写主函数，主函数代码如下，在主函数中我们首先完成相应外设的初始化，然后初始化 GUI，初始化完成后调用 GUIDEMO\_Main()函数，记得一定要开启 CRC 时钟

```
int main(void)
{
    delay_init(168);           //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断分组配置
    uart_init(115200);         //串口波特率设置
    LED_Init();                //LED 初始化
    TFTLCD_Init();             //LCD 初始化
    TP_Init();                 //触摸屏初始化
    FSMC_SRAM_Init();          //SRAM 初始化
    TIM3_Int_Init(999,83);     //1KHz 定时器 3 设置为 1ms

    mem_init(SRAMIN);          //初始化内部内存池
    mem_init(SRAMEX);          //初始化外部内存池
    mem_init(SRAMCCM);         //初始化 CCM 内存池
```

```

TIM4_Int_Init(499,839);    //触摸屏扫描速度,200HZ.
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_CRC,ENABLE);//开启 CRC 时钟
WM_SetCreateFlags(WM_CF_MEMDEV);
GUI_Init();
GUIDEMO_Main();
while(1);
}

```

为了流畅的使用 STemWin 我们需要修改启动文件 startup\_stm32f40\_41xxx.s 的堆栈大小, 如图 1.5.2 所示。

```

31 ; limitations under the License.
32 ;
33 ;*****
34 ;
35 ; Amount of memory (in bytes) allocated for Stack
36 ; Tailor this value to your application needs
37 ; <h> Stack Configuration
38 ; <o> Stack Size (in Bytes) <0x0-0xFFFFFFFF:8>
39 ; </h>
40 ;
41 Stack_Size EQU 0x00000800 ;如果使用STemWin的话此值应该设置大一点
42 ;
43 Stack_Mem AREA STACK, NOINIT, READWRITE, ALIGN=3
44 __initial_sp SPACE Stack_Size
45
46
47

```

图 1.5.2 修改后的启动文件堆栈大小

## 1.6 下载验证

至此, 无操作系统的 STemWin 移植完成, 编译下载程序, 程序运行如图 1.5.3 所示。

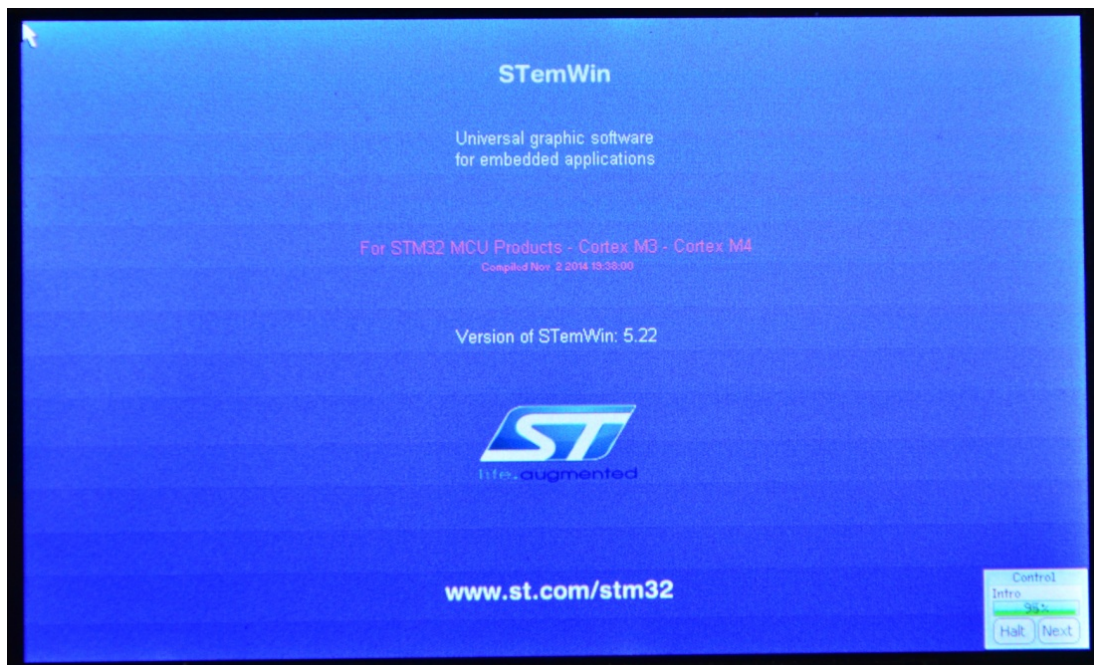


图 1.5.3 STemWin 测试

## 第二章 STemWin 带操作系统移植

在上一节中我们讲解了不带操作系统的 STemWin 移植,在这一章我们讲解带有操作系统的 STemWin 移植,系统使用的是 UCOS II,关于 UCOS II 的移植请参考《ALIENTEK STM32F4 UCOS 开发手册》的移植章节,本章的移植过程是在上一章的基础上进行的,有了上一章的基础本章相对来说简单很多。本章分为如下几个部分:

- 2.1 移植准备工作
- 2.2 向工程添加以及修改相应文件
- 2.3 综合测试程序编写
- 2.4 下载验证

## 2.1 移植准备工作

本章是在上一章不带操作系统实验移植基础上进行的，我们要先建一个带有 UCOS II 的基础工程，然后在这个基础工程上完成我们今天的移植工作。我们在上一章的工程中移植 UCOS II 操作系统，关于 UCOS 操作系统的移植请参考《ALIENTEK STM32F4 UCOS 开发手册》中的 UCOSII 移植部分，我们这里需要将 UCOSII 的系统节拍数 OS\_TICKS\_PER\_SEC 改为 1000，也就是 1ms，我们在 UCOS II 移植实验中定义的是 200，移植好 UCOS II 后的工程如图 2.1.1 所示，到这里基础工程建立完成。

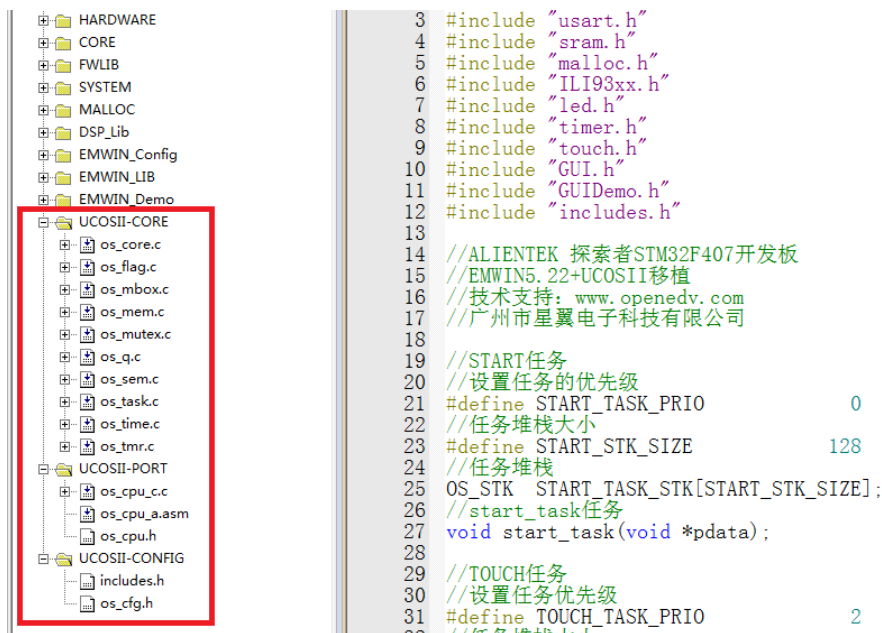


图 2.1.1 带 UCOS II 的基础工程

## 2.2 向工程添加以及修改相应文件

在无操作系统的工程中我们有一个 GUI\_X.c 的文件，这里用我们提供的 GUI\_X\_UCOSII.c 文件替代 GUI\_X.c 文件，并将 GUI\_X\_UCOSII.c 文件添加到工程中，添加完以后的工程如图 2.2.1 所示。

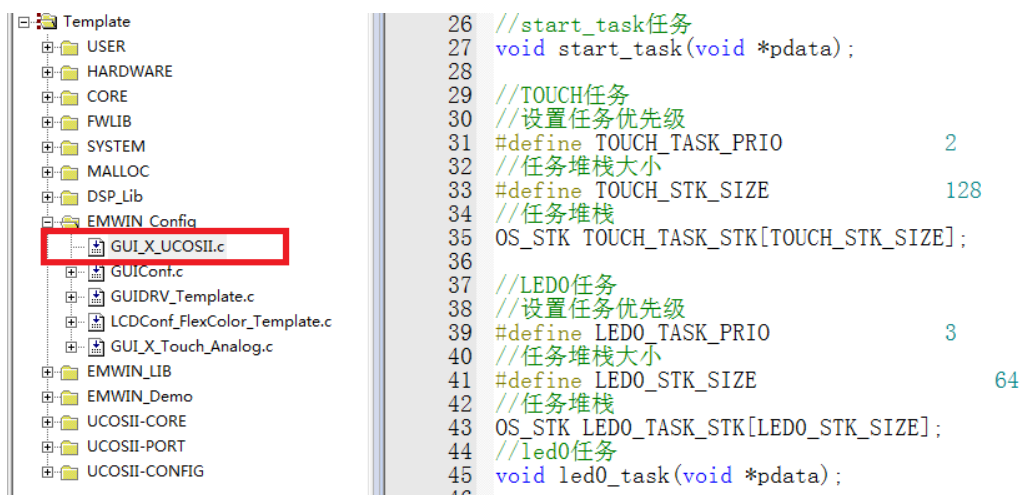


图 2.2.1 添加 GUI\_X\_UCOSII.c 后的工程



在 GUI\_X\_UCOSII.c 文件里面主要有以下一些内容。

### 1) 时间相关函数

```
int GUI_X_GetTime(void) {
    return ((int)OSTimeGet());    //获取系统时间,此处时间单位为 1ms
}

//GUI 延时函数
void GUI_X_Delay(int period) {
    INT32U ticks;
    ticks = (period * 1000) / OS_TICKS_PER_SEC;
    OSTimeDly(ticks);
}
```

上面两个函数比较好理解，分别为 STemWin 获取系统时间和 STemWin 延时函数，这两个函数都是通过调用 UCOS II 里面与时间相关的函数来实现的。

### 2) 信号量、消息邮箱相关函数

```
void GUI_X_InitOS(void)
{
    DispSem = OSSemCreate(1);        //创建初始值为 1 的信号量
    EventMbox = OSMboxCreate((void*)0); //创建消息邮箱
}

void GUI_X_Lock(void)
{
    INT8U err;
    OSSemPend(DispSem,0,&err); //请求信号量
}

void GUI_X_Unlock(void)
{
    OSSemPost(DispSem);    //发送信号量
}

void GUI_X_WaitEvent(void)
{
    INT8U err;
    (void)OSMboxPend(EventMbox,0,&err); //请求消息邮箱
}

void GUI_X_SignalEvent(void)
{
    (void)OSMboxPost(EventMbox,(void*)1); //发送消息邮箱
}
```

上面一共有 5 个函数:GUI\_X\_InitOS(),GUI\_X\_Lock(),GUI\_X\_Unlock(),GUI\_X\_WaitEvent() 和 GUI\_X\_SignalEvent()。这五个函数分别为创建信号量和消息邮箱，上锁与解锁，等待事件发生。其中上锁和解锁是通过信号量来实现的，等待事件发生是通过消息邮箱来实现的。

### 3) 与键盘相关函数

下面这几个函数与键盘有关，这几个函数很好理解。



```
static void CheckInit(void)
{
    if(KeyIsInitd == OS_FALSE){
        KeyIsInitd = OS_TRUE;
        GUI_X_Init();
    }
}

void GUI_X_Init(void)
{
    KeySem = OSSemCreate(0); //创建初始值为 0 的信号量
}

int GUI_X_GetKey(void)
{
    int r;
    r = KeyPressed;
    CheckInit();
    KeyPressed = 0;
    return (r);
}

int GUI_X_WaitKey(void)
{
    int r;
    INT8U err;
    CheckInit();
    if(KeyPressed == 0)
    {
        OSSemPend(KeySem,0,&err); //请求信号量
    }
    r = KeyPressed;
    KeyPressed = 0;
    return (r);
}

void GUI_X_StoreKey(int k)
{
    KeyPressed = k;
    OSSemPost(KeySem);
}
```

#### 4) 修改相应文件

在上一章不带操作系统的移植中我们使用的 STemWin 库文件是 STemWin522\_CM4\_Keil.lib, 这一章我们使用了操作系统, 因此应该使用 STemWin522\_CM4\_OS\_Keil.lib 这个库。添加完成后我们还有更改 GUIConf.h 中 GUI\_OS 的定义, 这里我们定义 GUI\_OS 为 1, 支持操作系统, 我们还需要定义可以调用 STemWin 的任务的最大数目, 如图 2.2.2 所示。

```

6 #define GUI_OS (1) // 使用操作系统
7 #define GUI_MAXTASK (5) // 最大可调用EMWIN的任务数
8 #define GUI_SUPPORT_TOUCH (1) // 支持触摸
9

```

图 2.2.2 GUIConf.h 配置文件

## 2.3 编写综合测试程序

测试程序同样采用官方 Demo，在 main.c 文件中我们建了 4 个任务，如表 2.3.1 所示。

表 2.3.1 测试程序任务函数

| 任务名            | 任务说明                              |
|----------------|-----------------------------------|
| start_task     | 开始任务，初始化 STemWin 并且创建其他 3 个任务     |
| emwindemo_task | 官方 Demo 的演示任务，我们看到的实际效果就是由这个任务完成的 |
| touch_task     | 通过调用 GUI_TOUCH_Exec()函数处理触摸屏事件    |
| led0_task      | LED 亮灭                            |

以上四个任务都非常简单，大家可以看一下我们的 main.c 文件，接下来就是主函数的编写，主函数也非常简单，主函数代码如下。

```

int main(void)
{
    delay_init(168);           //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断分组配置
    uart_init(115200);        //串口波特率设置
    TFTLCD_Init();            //初始化 LCD
    TP_Init();                 //初始化触摸屏
    LED_Init();                //LED 初始化
    FSMC_SRAM_Init();          //SRAM 初始化
    mem_init(SRAMIN);          //内部 RAM 初始化
    mem_init(SRAMEX);          //外部 RAM 初始化
    mem_init(SRAMCCM);         //CCM 初始化

    OSInit();                  //初始化 UCOS
    OSTaskCreate(start_task,    //start_task 任务
                  (void*)0,     //参数
                  (OS_STK*)&START_TASK_STK[START_TASK_SIZE-1], //任务堆栈栈顶
                  START_TASK_PRIO); //任务优先级
    OSStart(); //开启 UCOS
}

```

在主函数中我们主要完成了外设的初始化和创建 start\_task 任务，我们注意到这里并没有初始化定时器 3 和定时器 4。在上一章中我们使用定时器 3 来为 STemWin 提供系统时钟，使用定时器 4 来定时处理触摸事件。在本章中我们移植有 UCOS II 操作系统，我们使用 UCOS II 系统来为 STemWin 提供系统时钟，对于触摸事件的处理我们可以建立一个任务来完成，因此这里就不需要使用定时器了。

## 2.4 下载验证

代码编译成功之后，下载代码到我们的开发板上，可以看到 LCD 上显示如图 2.4.1 所示界

面，可以看到 Demo 开始运行，LED0 开始闪烁。



图 2.4.1 程序运行效果图

## 第三章 在 PC 上仿真 STemWin

Segger 官方给我们提供了很多例程，这些例程涵盖了 emWin 的所有功能，我们学习 emWin 其实就是学习这些例程，但是这些例程都是基于 PC 的，也就是说在 PC 上仿真运行的。因此学会如何在 PC 上仿真运行 emWin 非常重要！再者，学会了在 PC 上仿真 emWin，即使我们没有开发板也同样可以学习如何使用 emWin，本章主要分为如下几部分：

- 3.1 准备 emWin 仿真文件
- 3.2 打开 emWin 仿真文件
- 3.3 进行仿真
- 3.4 emWin 仿真文件介绍
- 3.5 Hello Word!

### 3.1 准备 emWin 仿真文件

ST官方没有提供STemWin的PC仿真文件，但是STemWin是基于emWin的，只是针对ST的MCU做了一点简单的修改，因此我们完全可以使用emWin的PC仿真文件。emWin的仿真文件我们可以在segger官网下载到，下载地址：<https://www.segger.com/downloads.html?pid=26>，打开以后页面如图 3.1.1 所示。

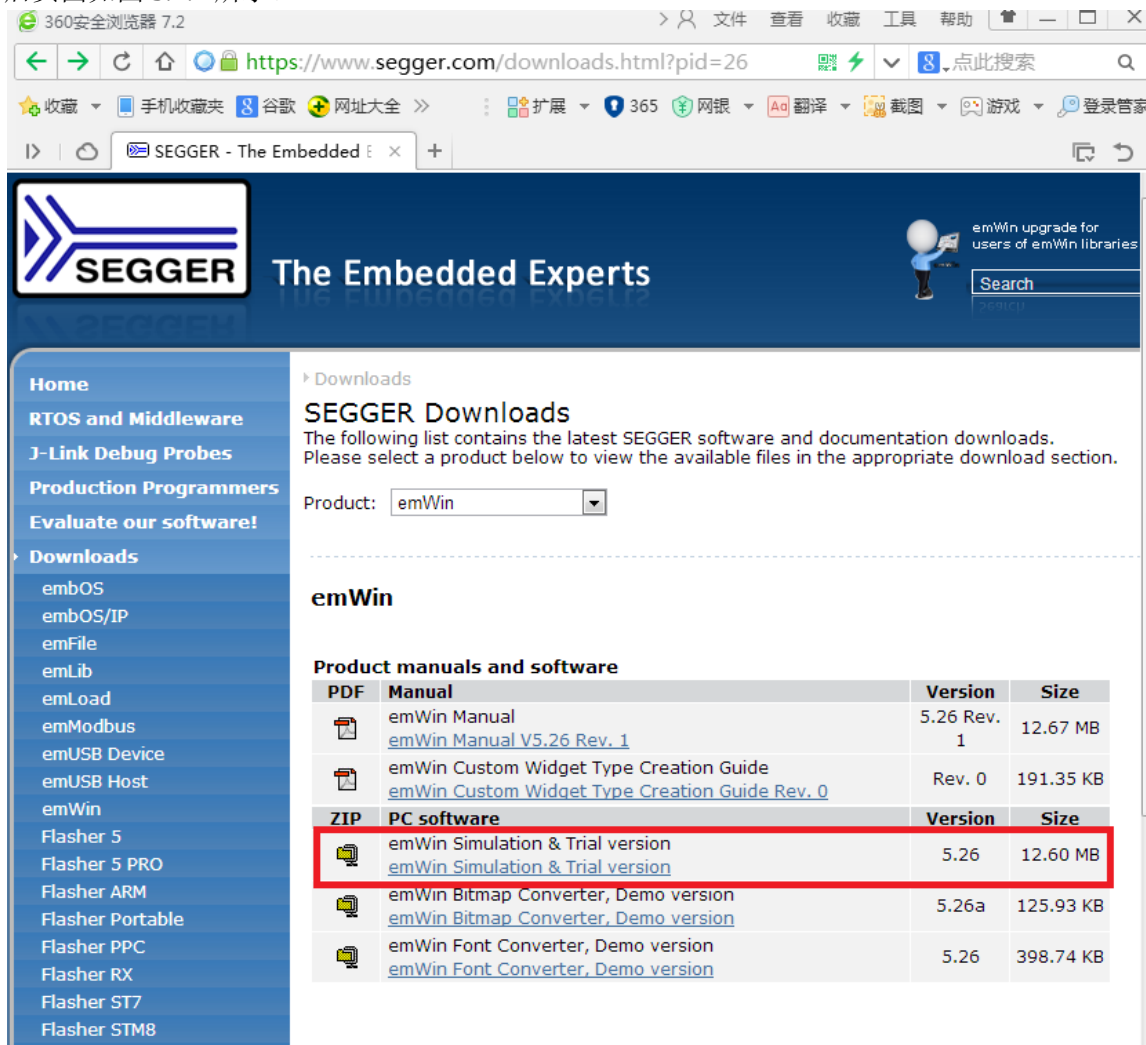


图 3.1.1 emWin 下载界面

当前最新的 emWin 版本为 5.26，但是我们使用的 STemWin 为 5.22 版本的，我们完全可以使用 5.26 的来仿真，最多是 5.26 里面有些小工具可能 5.22 里面没有，大家不要纠结于这个。下载下来后的文件为：SeggerEval\_WIN32\_MSVC\_MinGW\_GUI\_V526，我们已经放到了光盘中了，路径为：6，软件资料->3，EMWIN 学习资料->emWin5.26 PC 仿真文件->SeggerEval\_WIN32\_MSVC\_MinGW\_GUI\_V526。

这里我们也将 emWin5.26 的源码下载下来放到光盘，路径：6，软件资料->3，EMWIN 学习资料->emWin5.26 源码，感兴趣的朋友可以移植 emWin5.26 到开发板中。

### 3.2 打开 emWin 仿真文件

emWin 模拟需要 Microsoft Visual C++ (6.00 或更高版本)的 IDE，我使用的是 VS2013 也就是 Visula Studio 2013，关于 VS2013 软件的安装这里不做讲解，默认大家都已经安装好了 VS2013。

1、在 E 盘根目录下新建一个文件夹 emWin5.26 simulate，当然我们可以在 C 盘、D 盘等建立文件夹。将 emWin5.26 的 PC 仿真文件拷贝到 emWin5.26 simulate 文件夹中并解压，**注意，不能出现中文路径！**解压后如图 3.2.1 所示。

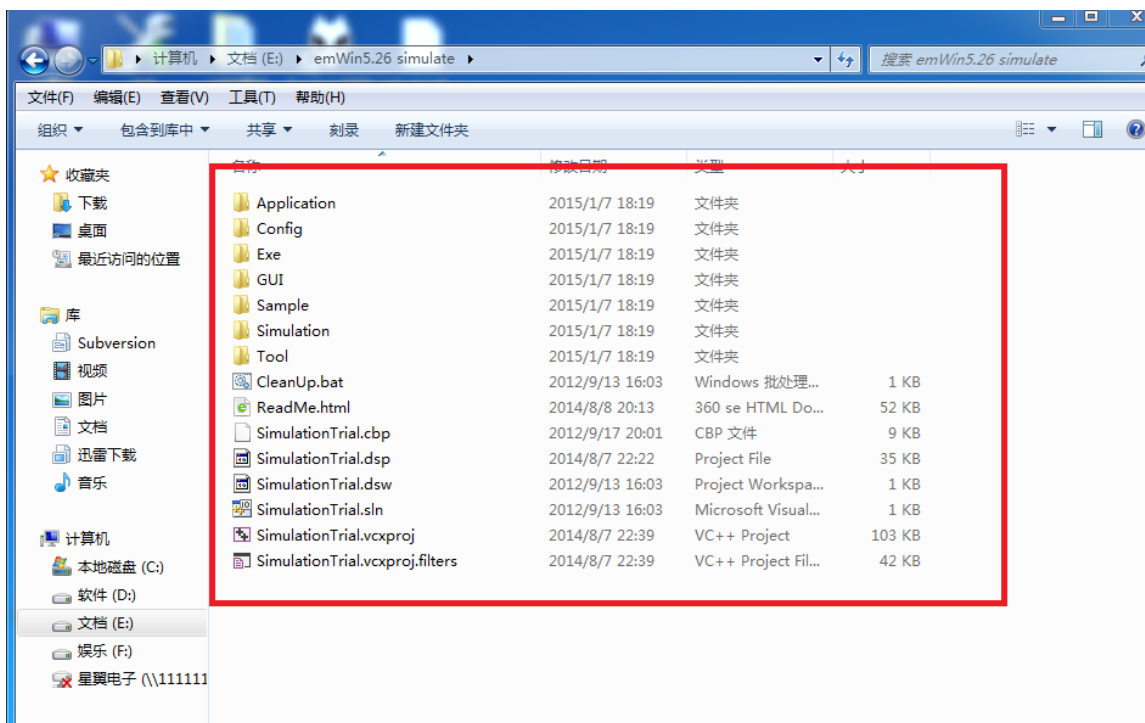


图 3.2.1 解压后的 emWin5.26 仿真文件

2、使用 VS2013 打开仿真工程，点击 SimulationTrial.sln 打开工程，如图 3.2.2 所示。

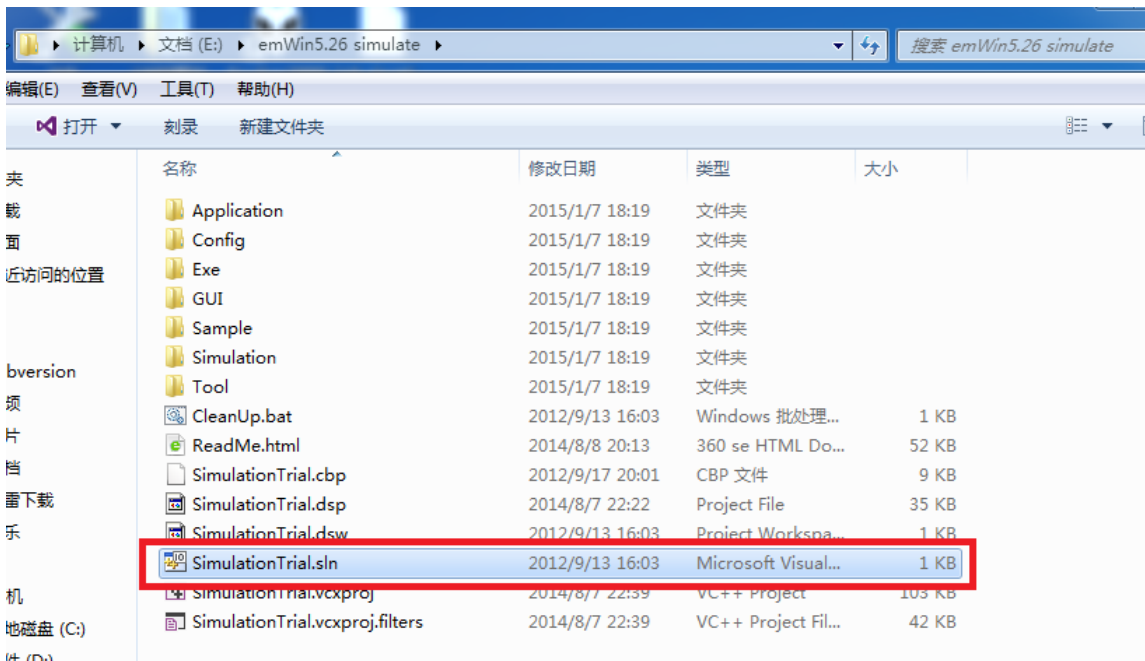


图 3.2.2 打开仿真文件

如图出现如图 3.2.3 所示的提示，点击“确定”就可以了。

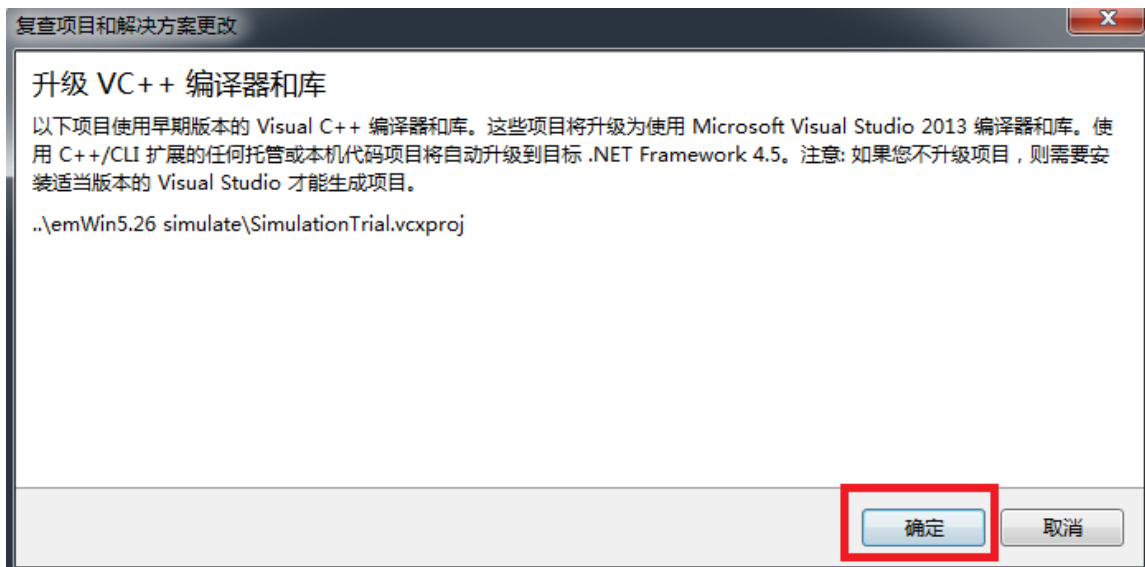


图 3.2.3 VS2013 提示

点击“确定”以后就会打开 emWin5.26 的 PC 仿真工程，如图，3.2.4 所示。

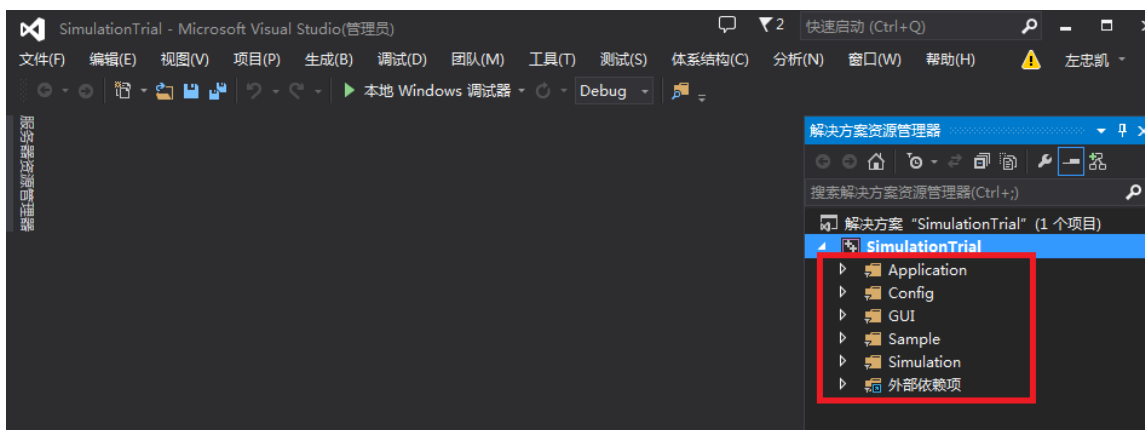


图 3.2.4 打开后的仿真文件

从图 3.2.4 中可以看出有 6 个文件：Application，Config，GUI，Sample，Simulation，外部依赖项，各个文件的简介如下：

**Application:** 官方的演示程序源代码

**Config:** emWin 以及 LCD 的配置文件，包含 GUIConf.c, GUIConf.h, LCDConf.c, LCDConf.h 和 SIMConf.c 这 5 个文件。

**GUI:** emWin 的库文件和库的头文件。

**Sample:** emWin 官方提供的一些应用小例程，我们学习 emWin 其实主要就是学习这个文件。

**Simulation 和外部依赖项:** 这个是配置 emWin 的运行环境等等，我们不用管这个东西。

### 3.3 进行仿真

打开 emWin 仿真工程后，按下键盘的 F5 键或者如图 3.3.1 红框所示的地方即可编译并运行整个工程。



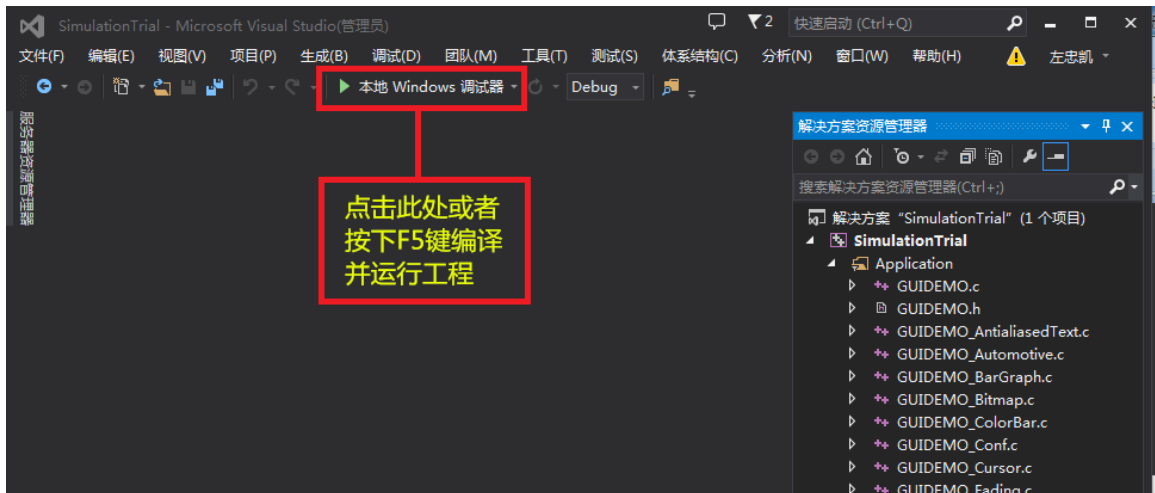


图 3.3.1 编译并运行工程

编译完成后运行的是 emWin 自带的综合演示 Demo，也就是 Application 中的那些.c 和.h 文件，运行如图 3.3.2 所示。

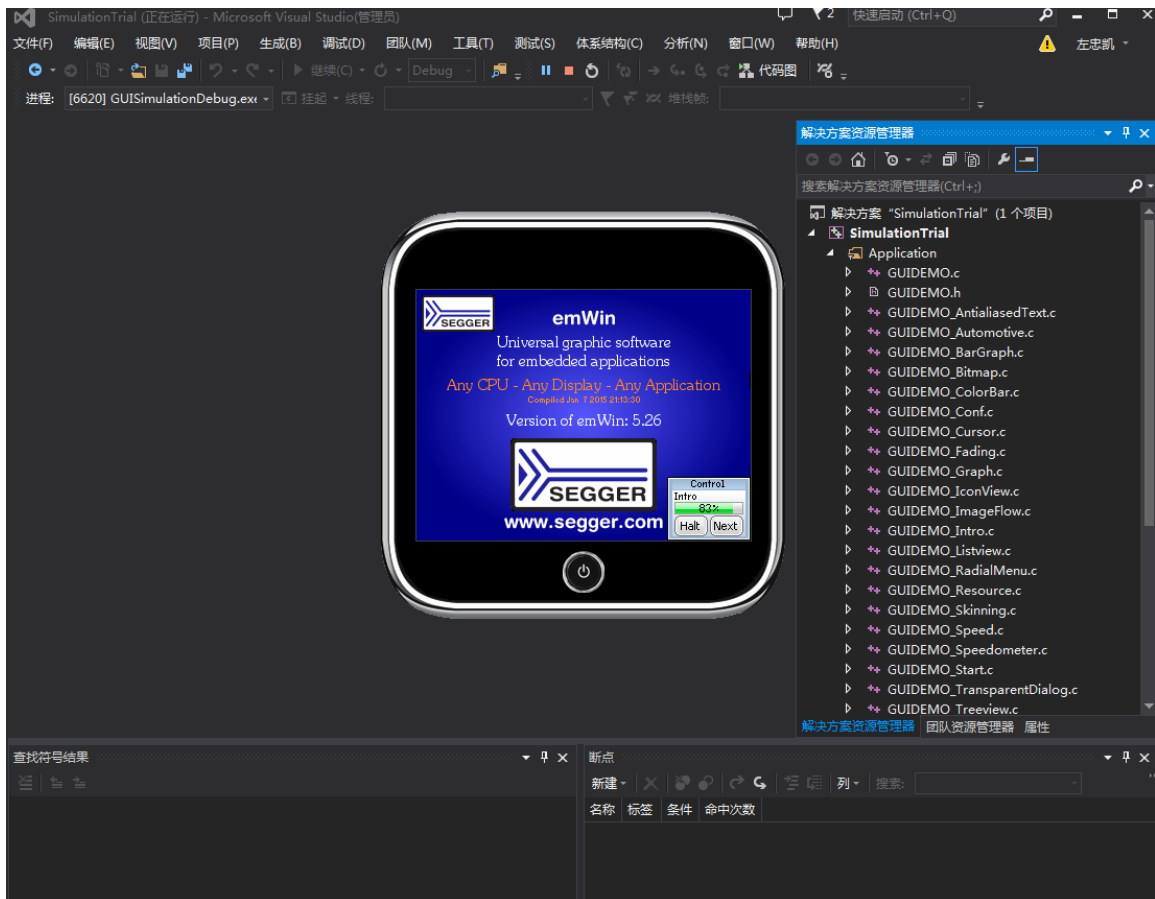


图 3.3.2 emWin 仿真工程运行

emWin 的这个自带的综合 Demo 功能非常强大，而且很绚丽，图 3.3.3 展示了其中部分例程的仿真截图。



图 3.3.3 emWin 部分仿真截图

## 3.4 emWin 仿真文件介绍

### 3.4.1 Application 文件

上一节我们简单的讲解了一下 emWin 仿真工程的文件,我们先来看一下 Application 文件,如图 3.4.1 所示。

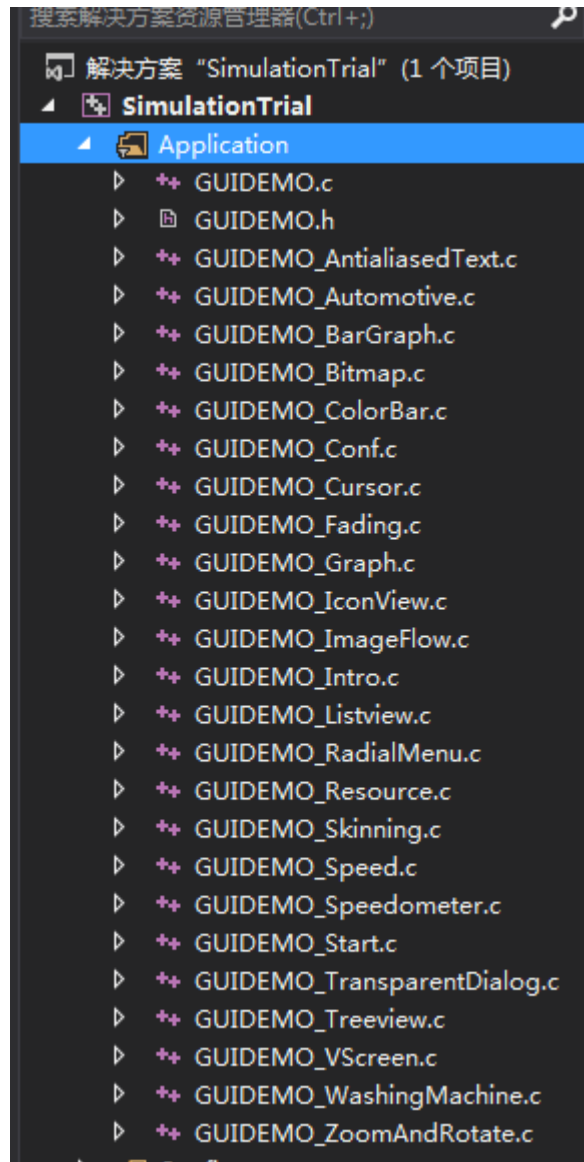


图 3.4.1 Application 文件

我们在上一节中看到的绚丽的界面就是这些.C 文件构成的，但是这些例程比较高级，不利于我们初学 EMWIN，当我们对 EMWIN 有了一定的了解后再来看这些文件就会轻松很多。

### 3.4.2 Config 文件

GUIConf.c 文件主要是给 EMWIN 分配内存的，代码如下：

```
Purpose      : Display controller initialization
-----END-OF-HEADER-----
*/
#include "GUI.h"
/*****
*
*      Defines
*
*****/
```

```

*/
//
// Define the available number of bytes available for the GUI
//
#define GUI_NUMBYTES 0x400000
//
// Define the average block size
//
#define GUI_BLOCKSIZE 0x80

/*****
*
*      Public code
*
*****/
*/
/*****
*
*      GUI_X_Config
*
* Purpose:
*   Called during the initialization process in order to set up the
*   available memory for the GUI.
*/
void GUI_X_Config(void) {
    //
    // 32 bit aligned memory area
    //
    static U32 aMemory[GUI_NUMBYTES / 4];
    //
    // Assign memory to emWin
    //
    GUI_ALLOC_AssignMemory(aMemory, GUI_NUMBYTES);
    GUI_ALLOC_SetAvBlockSize(GUI_BLOCKSIZE);
}

```

从上面的代码可以看出，GUIConf.c 文件里面只有一个 GUI\_X\_Config() 函数，这个函数就是给 emWin 分配内存，宏 GUI\_NUMBYTES 定义了应该分配的内存大小，默认为 0X400000，也就是 4M 的内存。最后调用 GUI\_ALLOC\_AssignMemory() 函数将这 4M 的内存分配给 emWin，调用 GUI\_ALLOC\_SetAvBlockSize() 设置存储块的平均大小。

GUIConf.h 用来配置以及裁剪 emWin，比如是否支持操作系统、触摸、鼠标、窗口和存储设备等，设置显示层数，默认字体等，代码如下。

```

#ifndef GUICONF_H
#define GUICONF_H

```

```

/*****
*
*      Multi layer/display support
*/
#define GUI_NUM_LAYERS      16    // Maximum number of available layers

/*****
*
*      Multi tasking support
*/
#define GUI_OS              (1)   // Compile with multitasking support

/*****
*
*      Configuration of available packages
*/
#define GUI_SUPPORT_TOUCH   (1)   // Support a touch screen (req. win-manager)
#define GUI_SUPPORT_MOUSE   (1)   // Support a mouse
#define GUI_WINSUPPORT      (1)   // Window manager package available
#define GUI_SUPPORT_MEMDEV  (1)   // Memory devices available

/*****
*
*      Configuration of window manager
*/
// Enable sending of WM_NOTIFY_VIS_CHANGED messages
#define WM_SUPPORT_NOTIFY_VIS_CHANGED (1)

/*****
*
*      Default font
*/
#define GUI_DEFAULT_FONT    &GUI_Font6x8

#endif  /* Avoid multiple inclusion */

```

当我们需要裁剪或配置 emWin 某些功能的时候就可以修改 GUIConf.h 中的这些宏定义。

LCDConf.c 文件用来设置模拟 LCD 的大小，配置显示驱动和颜色转换格式等，这个文件我们只需要最常修改的是两个宏：

```

#define XSIZE_PHYS  320
#define YSIZE_PHYS  240

```

修改上面这两个宏可以更改我们仿真时的 LCD 大小，这个很常用，LCDConf.c 代码就不贴出来了，这个文件中的其他地方我们不要去修改。

LCDConf.h 文件里面没有东西，SIMConf.c 是在仿真 emWin 的时候对 PC 环境的一些特定设置，我们不需要了解。

### 3.4.3 GUI 文件

GUI 文件就是 emWin 的真身了，这个文件里面是 emWin 源码的 lib 文件和相应的头文件，emWin 已经不提供源码了，都是以 lib 文件形式给出的。在 PC 版的仿真文件中 GUI.lib 只适用于 PC 平台，我们在移植 emWin 到开发板的时候不能使用这个 lib 文件！segger 官方有专门针对 Cortex-M 系列 MCU 的 lib 文件。但是这两个 lib 文件里面的 API 函数都是一样的，因此在 PC 上仿真通过的代码是完全可以在开发板上运行的。

### 3.4.4 Sample 文件

这个文件是 segger 针对 emWin 各个小控件、功能做的演示例程，我们学习 emWin 主要就是学习这个文件里面代码！相对与 Application 文件，这个文件中的代码比较简单易懂，我们在看 emWin 的官方手册的时候里面的演示例程都在这个文件中，这个文件中的部分截图如图 3.4.2 所示。

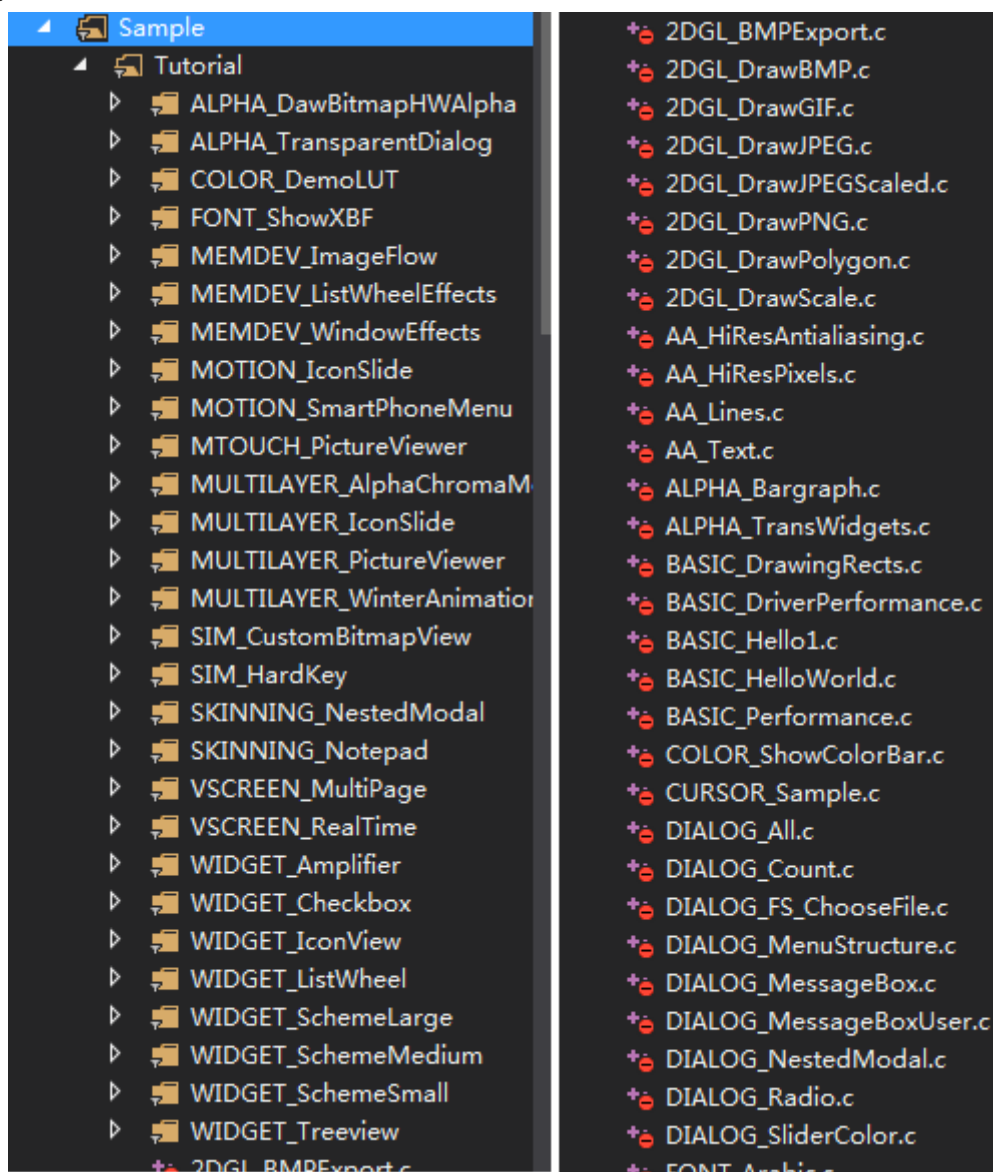


图 3.4.2 emWin 部分演示例程

可以看出 Sample 中的例程是非常多的，涵盖了 emWin 的所有内容，本手册后面的 emWin



例程就是移植和讲解的这些文件，因此使用我们的 emWin 开发手册即使没有板子也是可以学习的，使用 PC 仿真文件即可。

### 3.5 Hello Word!

本节我们就讲解一下如何使用 Sample 文件中这些例程，默认情况下 emWin 运行的是 Application 文件中的那的高大上的例程，但是不利于我们学习，Sample 中的文件才是我们的菜，接下来就讲解一下如何运行 Sample 中的例程。

1、由于默认运行的是 Application 中的例程，因此我们首先需要将 Application 中的代码“屏蔽”掉，把这些.c 和.h 文件从生成中排除掉，选中 Application 下的所有文件，然后点击鼠标右键，选择“属性”，如图 3.5.1 所示。

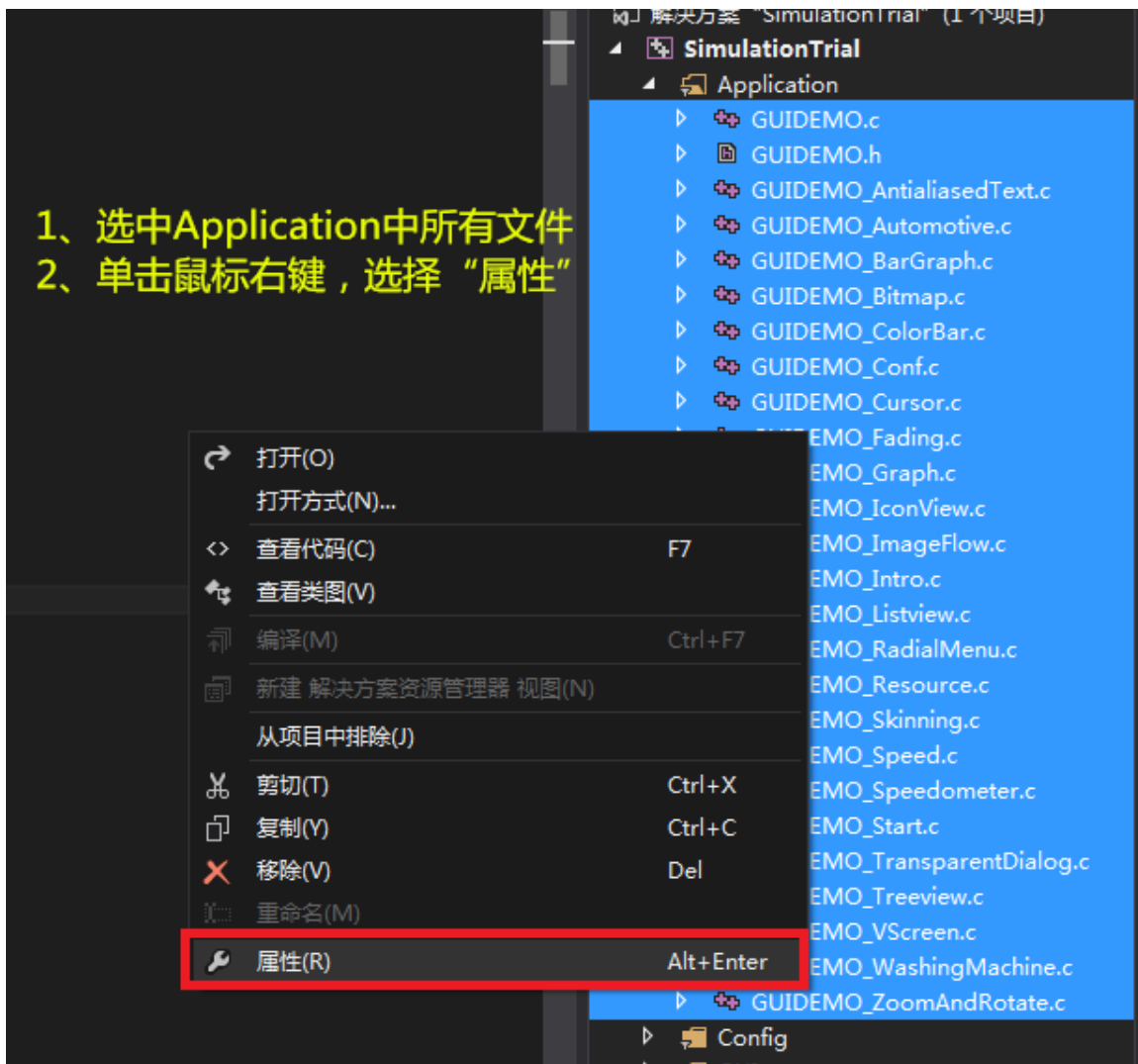


图 3.5.1 将 Application 中文件从生成中排除(1)

点击属性以后会出现如图 3.5.2 所示的提示。



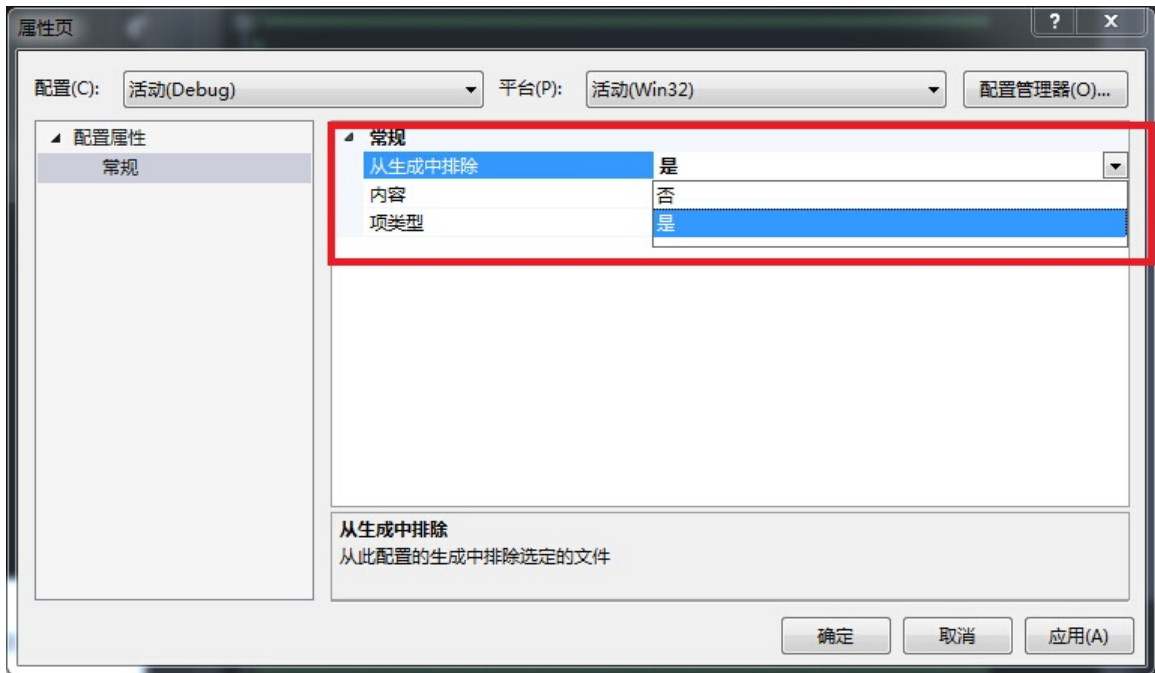


图 3.5.2 将 Application 中文件从生成中排除(2)

在图 3.5.2 中的 “从生成中排除” 选择 “是”，然后点击 “确定”，完成以后 Application 中的文件就被 “屏蔽” 掉了，如图 3.5.3 所示。

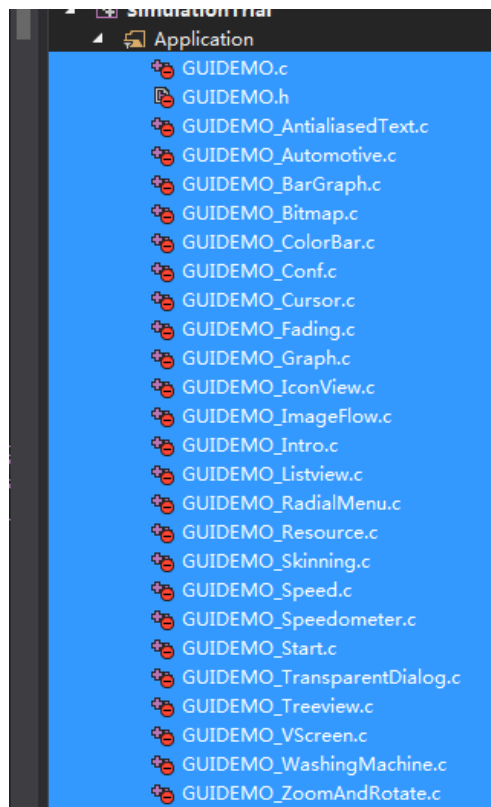


图 3.5.3 将 Application 文件从生成中排除

2、将 Application 中的文件全部从生成中排除以后就需要将 Sample 中我们想要仿真的文件添加到生成中，这里我们选择 Sample 中的 BASIC\_HelloWord.c 文件。同样选中文件，单击鼠

标右键，选择“属性”，如图 3.5.4 所示。

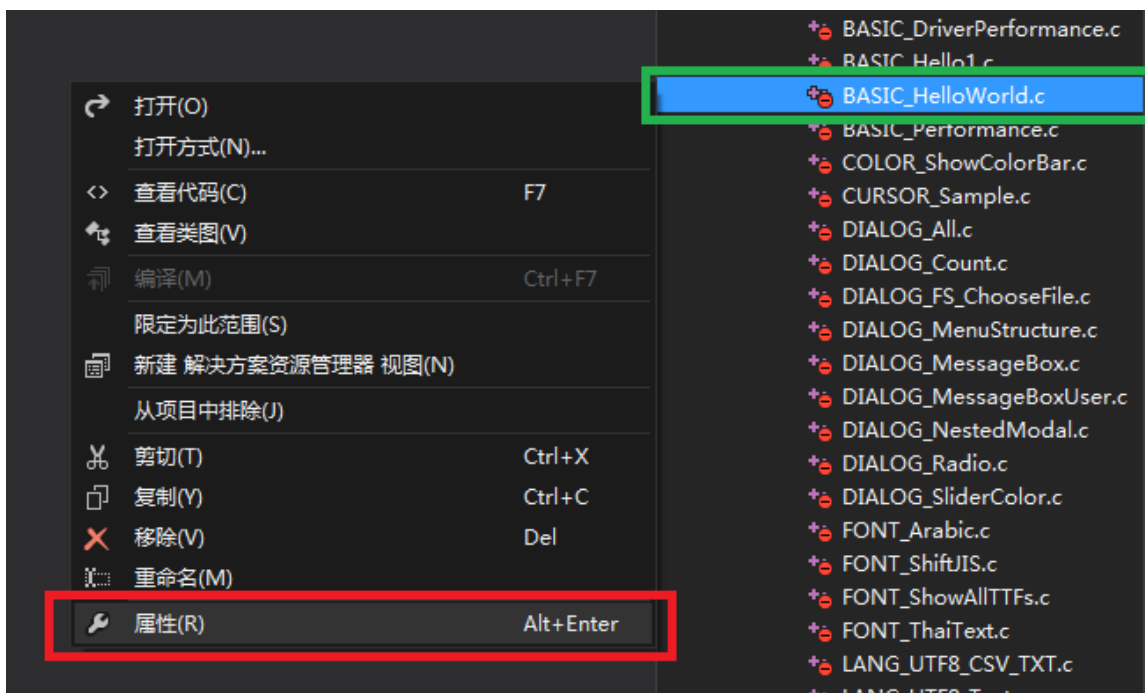


图 3.5.4 将文件添加到生成中(1)

点击“属性”以后会出现如图 3.5.5 所示提示，注意从图 3.5.5 中可以看出此时 BASIC\_HelloWord.c 文件是从生成中排除掉的，这里我们选择“否”，点击“确定”，即可将文件添加到生成中。

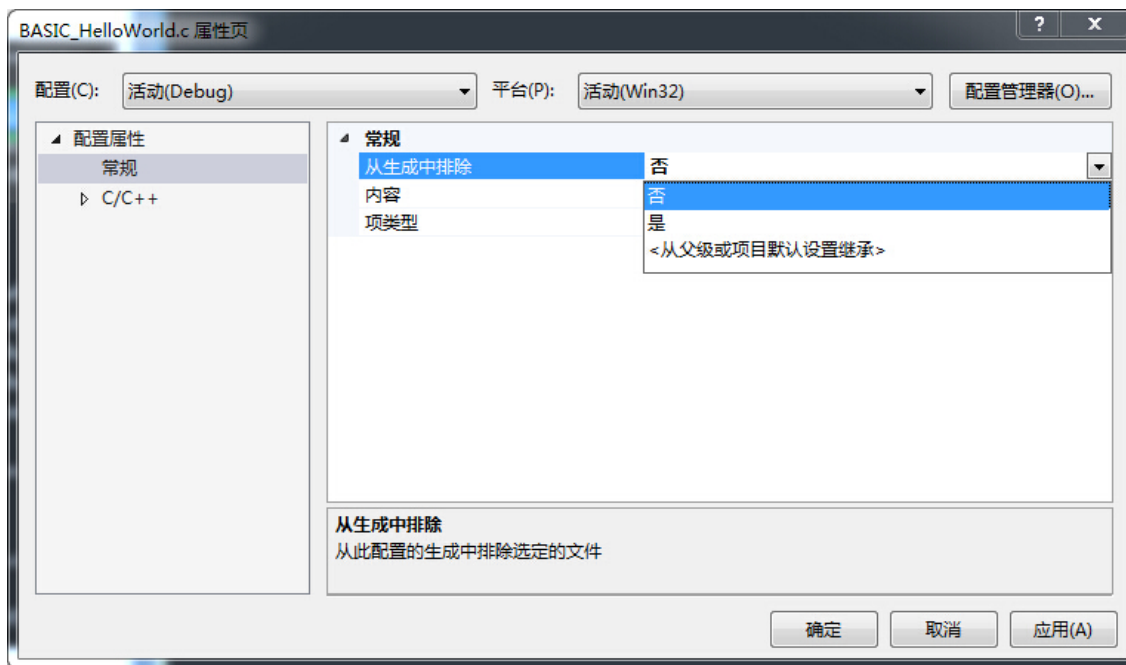


图 3.5.5 将文件添加到生成中(2)

添加完成以后，如图 3.5.6 所示，注意文件 BASIC\_HelloWord.c 和其他没有添加到生成中的文件前面标号的区别。

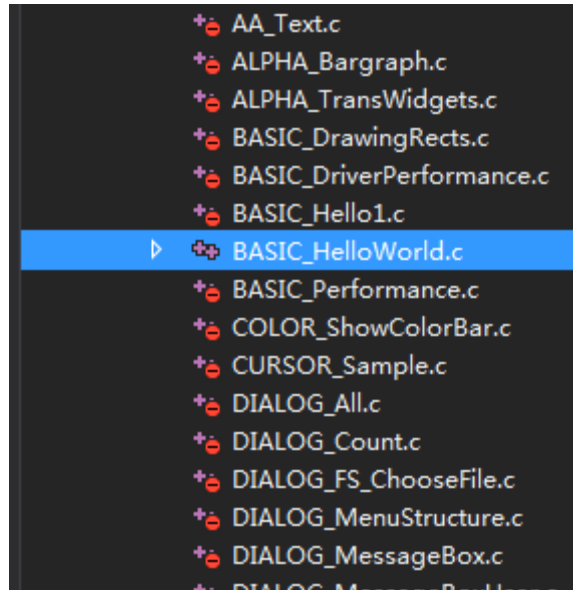


图 3.5.6 文件添加完成

添加完成以后，我们按下 F5 或者点击 VS2013 的“本地 Windows 调试器”编译并运行代码，结果如图 3.5.7 所示。



图 3.5.7 代码运行效果截图

当我们需要仿真其他的文件的时候，我们按照上面的方法，首先将 BASIC>HelloWord.c 文件从生成中删除，然后将需要仿真的文件添加到生成中，至此，emWin 在 PC 上的仿真就讲解完成了，大家可以自行多练习练习，仿真一下其他的文件。

## 第四章 文本显示

我们最常用的功能就是在 LCD 上显示各种字符，提示信息等，STemWin 提供了大量的文本显示 API 函数，也可以设置各种的显示效果，字体，颜色等等，本章就讲解一下 STemWin 的文本显示功能，本章分为以下几个部分：

- 4.1 基本文本显示
- 4.2 文本显示 API 函数
- 4.3 重点 API 函数简介
- 4.4 演示实例

## 4.1 基本文本显示

要在 LCD 上显示文本，只需调用例程 GUI\_DisString() 并以要显示的文本作为参数即可。例如：

```
GUI_DisString("Hello world!");
```

上面这行代码就是在 LCD 上显示“Hello world!”这行字符串，在 STemWin 的文本显示中我们可以使用表 4.1.1 中所示的控制字符。

| 字符代码 | ASCII 代码 | C  | 描述                                                                                     |
|------|----------|----|----------------------------------------------------------------------------------------|
| 10   | LF       | \n | 换行。<br>当前文本位置改变至下一行的开始。默认为：X = 0。<br>Y += 字体 - 距离（单位：像素）（如例程 GUI_GetFontDistY() 计算得出）。 |
| 13   | CR       | \r | 回车。<br>当前文本位置改变至当前行的开始。默认为：X = 0。                                                      |

表 4.1.1 STemWin 中可使用的控制字符

控制符\n和\r非常实用，换行符可以作为字符串的一部分，这样字符串就可以拆分为多行来显示。

## 4.2 文本显示 API 函数

STemWin 针对文本显示提供了很多的 API 函数，如表 4.2.1 所示，我们学习 STemWin 的文本显示主要就是学习这些 API 函数的使用。

| 例程                        | 描述                    |
|---------------------------|-----------------------|
| 显示文本的例程                   |                       |
| GUI_DisChar()             | 在当前位置显示单个字符。          |
| GUI_DisCharAt()           | 在指定位置显示单个字符。          |
| GUI_DisChars()            | 按指定次数显示字符。            |
| GUI_DispNextLine()        | 将光标移至下一行的开始。          |
| GUI_DisString()           | 在当前位置显示字符串。           |
| GUI_DisStringAt()         | 在指定位置显示字符串。           |
| GUI_DisStringAtCEOL()     | 在指定位置显示字符串，并清除至行末。    |
| GUI_DisStringHCenterAt()  | 在指定位置水平居中显示字符串。       |
| GUI_DisStringInRect()     | 在指定的矩形区域中显示字符串。       |
| GUI_DisStringInRectEx()   | 在指定的矩形区域中显示字符串，并可旋转   |
| GUI_DisStringInRectWrap() | 在指定的矩形区域中显示字符串，并可自动换行 |
| GUI_DisStringLen()        | 在当前位置显示指定字符数的字符串。     |
| GUI_WrapGetNumLines()     | 为指定自动换行模式获取文本行数。      |
| 选择文本绘制模式                  |                       |
| GUI_GetTextMode()         | 返回当前文本模式。             |
| GUI_SetTextMode()         | 设置文本绘制模式。             |
| GUI_SetTextStyle()        | 设置要使用的文本样式。           |
| 选择文本对齐模式                  |                       |

续表 4.2.1

|                    |                    |
|--------------------|--------------------|
| GUI_GetTextAlign() | 返回当前文本对齐模式。        |
| GUI_SetLBorder()   | 设置换行后的左边界。         |
| GUI_SetTextAlign() | 设置文本对齐模式。          |
| 设置当前文本位置           |                    |
| GUI_GotoX()        | 设置当前               |
| GUI_GotoXY()       | 设置当前               |
| GUI_GotoY()        | 设置当前               |
| 返回当前文本位置           |                    |
| GUI_GetDispPosX()  | 返回当前               |
| GUI_GetDispPosY()  | 返回当前               |
| 清除窗口或部分窗口的例程       |                    |
| GUI_Clear()        | 清除活动窗口             |
| GUI_DispCEOL()     | 清除从当前文本位置到行末的显示内容。 |

表 4.2.1 文本显示 API 函数

### 4.3 重点 API 函数简介

#### 4.3.1 GUI\_SetTextMode()

##### GUI\_SetTextMode()

###### 描述

按照指定的参数设置文本模式。

###### 原型

```
int GUI_SetTextMode(int TextMode);
```

| 参数       | 描述                            |
|----------|-------------------------------|
| TextMode | 设置的文本模式，可以是 TEXTMODE 标记的任意组合。 |

###### 参数 TextMode 的允许值（可以通过“OR”操作进行组合）

|                     |                            |
|---------------------|----------------------------|
| GUI_TEXTMODE_NORMAL | 设置为显示正常文本。这是默认设置，该数值等同于 0。 |
| GUI_TEXTMODE_REV    | 设置为显示反转文本。                 |
| GUI_TEXTMODE_TRANS  | 设置为显示透明文本。                 |
| GUI_TEXTMODE_XOR    | 设置为反相显示的文本。                |

###### 返回值

之前选定的文本模式。

STemWin 有多种文本绘制模式，使用函数 GUI\_SetTextMode()来设置文本的绘制模式，STemWin 提供了多种可以组合使用的标记：

###### 正常文本

通过指定 GUI\_TEXTMODE\_NORMAL 或 0,可以正常显示文本。

###### 反转文本

通过指定 GUI\_TEXTMODE\_RE,可以反转显示文本。通常的黑底白字显示方式将变为白底黑字显示。

###### 透明文本

通过指定 GUI\_TEXTMODE\_TRANS,可以显示为透明文本。透明文本表示文本写在屏幕上已经可见的内容之上。不同之处在于，屏幕上原有的内容仍然可见，而在正常文本中，背景

会替换为当前选择的背景色。

#### 异或文本

通过指定 GUI\_TEXTMODE\_XOR，可以使用异或模式显示文本。通常情况下，用白色绘制的（实际字符）显示是反相的。如果背景色是黑色，效果与默认模式（正常文本）是一样的。如果背景是白色，输出与反转文本一样。如果使用彩色，反相的像素由下式计算：

新像素颜色 = 颜色的值 - 实际像素颜色 - 1。

#### 透明反转文本

通过指定 GUI\_TEXTMODE\_TRANS | GUI\_TEXTMODE\_REV，可以显示为透明反转文本。与透明文本一样，它不会覆盖背景，而且和反转文本一样，该文本会反转显示。

### 4.3.2 GUI\_SetTextAlign()

STemWin 中可以设置文本的对齐模式，使用函数 GUI\_SetTextAlign()来设置文本的对齐模式，函数如下：

#### GUI\_SetTextAlign()

##### 描述

为当前窗口的字符串输出设置文本对齐模式。

##### 原型

```
int GUI_SetTextAlign(int TextAlign);
```

| 参数                        | 描述                         |
|---------------------------|----------------------------|
| <a href="#">TextAlign</a> | 设置的文本对齐模式，可以是水平和垂直对齐标记的组合。 |

| 参数 <a href="#">TextAlign</a> 的允许值<br>(水平和垂直标记可以通过“OR”操作进行组合) |                    |
|--------------------------------------------------------------|--------------------|
| 水平对齐                                                         |                    |
| GUI_TA_LEFT                                                  | X轴方向左对齐（默认）。       |
| GUI_TA_HCENTER                                               | X轴方向居中。            |
| GUI_TA_RIGHT                                                 | X轴方向右对齐。           |
| 垂直对齐                                                         |                    |
| GUI_TA_TOP                                                   | 在字符 Y 轴方向顶部对齐（默认）。 |
| GUI_TA_VCENTER                                               | Y轴方向居中。            |
| GUI_TA_BOTTOM                                                | 在字体 Y 轴底部像素线对齐。    |

##### 返回值

所选的文本对齐模式。

### 4.3.3 GUI\_Clear()和 GUI\_DispCEOL()

使用 GUI\_Clear()来清除当前窗口，GUI\_DispCEOL()函数清除当前窗口(或显示)从当前文本位置到行末。

#### GUI\_Clear()

##### 描述

清除当前窗口。

##### 原型

```
void GUI_Clear(void);
```

##### 其他信息

如果没有定义窗口，则当前窗口为整个显示区。在这种情况下，整个显示区都会被清除。



## GUI\_DispCEOL()

### 描述

清除当前窗口（或显示）从当前文本位置到行末的内容，高度为当前字体高度。

### 原型

```
void GUI_DispCEOL(void);
```

## 4.3.4 GUI\_DispStringInRectWrap()

我们可以在 LCD 上指定的矩形区域内显示指定的字符串，emWin 提供了函数 GUI\_DispStringInRectWrap()来实现这种功能，emWin 中对此函数描述如下：

## GUI\_DispStringInRectWrap()

### 描述

在当前窗口指定的矩形区域内的指定位置处，使用当前字体显示字符串，并可自动换行（可选）。

### 原型

```
void GUI_DispStringInRectWrap(const char GUI_UNI_PTR * s,
                              GUI_RECT * pRect,
                              int TextAlign,
                              GUI_WRAPMODE WrapMode);
```

| 参数               | 描述                                                                                                                                              |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>s</b>         | 显示的字符串。                                                                                                                                         |
| <b>pRect</b>     | 要写入的客户端窗口矩形区域（单位：像素）。                                                                                                                           |
| <b>TextAlign</b> | 对齐标记，可以通过“OR”操作进行组合。垂直对齐和水平对齐标记应该组合使用。可用的标记有：<br>垂直对齐：GUI_TA_TOP、GUI_TA_BOTTOM、GUI_TA_VCENTER。<br>水平对齐：GUI_TA_LEFT、GUI_TA_RIGHT、GUI_TA_HCENTER。 |
| <b>WrapMode</b>  | （参见下表）                                                                                                                                          |

| 参数 WrapMode 的允许值  |                |
|-------------------|----------------|
| GUI_WRAPMODE_NONE | 不执行自动换行。       |
| GUI_WRAPMODE_WORD | 根据字对文本进行自动换行。  |
| GUI_WRAPMODE_CHAR | 根据字符对文本进行自动换行。 |

### 其他信息

如需执行字换行，而指定的矩形区域又太小，则会对该字进行字符换行。

一般我们在使用 GUI\_DispStringInRectWrap()函数的时候，需要实现定义一个矩形区域，然后在调用函数来在 LCD 上显示字符串，如下所示：

```
GUI_RECT Rect= {10,10,100,100}; //显示矩形的左上角坐标为(10,10)，右下角坐标为(100,100)
```

```
GUI_DispStringInRectWrap("Hello Word!",&Rect,GUI_TA_LEFT,GUI_WRAPMODE_NONE);
```

## 4.4 演示实例

关于文本显示我们提供了一个演示实例来展示如何使用这些 API 函数，本实验完整工程为：**EMWIN 实验 3 文本显示**，程序如下：

```
//WEWIN 文本显示例程
```

```
void emwin_texttest(void)
```

```
{
```

```
    int i;
```

```
    char acText[] = "This example demonstrates text wrapping";
```

```
    GUI_RECT Rect = {200,240,259,299}; //定义矩形显示区域
```

```

GUI_WRAPMODE aWm[] = {GUI_WRAPMODE_NONE,
                        GUI_WRAPMODE_CHAR,
                        GUI_WRAPMODE_WORD};

GUI_SetBkColor(GUI_BLUE);      //设置背景颜色
GUI_Clear();                   //清屏
GUI_SetFont(&GUI_Font24_ASCII); //设置字体
GUI_SetColor(GUI_YELLOW);      //设置字体颜色
GUI_DispString("HELLO WORD!");

GUI_SetFont(&GUI_Font8x16);     //设置字体
GUI_SetPenSize(10);             //设置笔大小
GUI_SetColor(GUI_RED);         //红色字体
GUI_DrawLine(300,50,500,130);  //绘线
GUI_DrawLine(300,130,500,50);  //绘线
GUI_SetBkColor(GUI_BLACK);     //设置黑色背景
GUI_SetColor(GUI_WHITE);       //设置字体颜色为白色
GUI_SetTextMode(GUI_TM_NORMAL); //正常模式
GUI_DispStringHCenterAt("GUI_TM_NORMAL",400,50);
GUI_SetTextMode(GUI_TM_REV);   //反转文本
GUI_DispStringHCenterAt("GUI_TM_REV" ,400,66);
GUI_SetTextMode(GUI_TM_TRANS); //透明文本
GUI_DispStringHCenterAt("GUI_TM_TRANS" ,400,82);
GUI_SetTextMode(GUI_TM_XOR);   //异或文本
GUI_DispStringHCenterAt("GUI_TM_XOR" ,400,98);
GUI_SetTextMode(GUI_TM_TRANS|GUI_TM_REV); //透明反转文本
GUI_DispStringHCenterAt("GUI_EM_TRANS|GUI_TM_REV",400,114);

GUI_SetTextMode(GUI_TM_TRANS); //透明文本
for(i=0;i<3;i++)
{
    GUI_SetColor(GUI_WHITE);
    GUI_FillRectEx(&Rect);
    GUI_SetColor(GUI_BLACK);
    //在当前窗口指定的矩形区域内显示字符串(并可自动换行)
    GUI_DispStringInRectWrap(acText,&Rect,GUI_TA_LEFT,aWm[i]);
    Rect.x0 += 70;
    Rect.x1 += 70;
}
}

```

在上面这个演示实例中我们需要重点掌握的函数有:

|                  |        |
|------------------|--------|
| GUI_SetBkColor() | 设置背景颜色 |
| GUI_SetFont()    | 设置字体   |

- GUI\_SetColor()            设置前景色(如文本，画线等颜色)  
GUI\_SetTextMode()        设置文本显示模式  
GUI\_DispString()         在指定的位置显示字符串  
GUI\_DispStringHCenterAt()    在指定位置水平居中显示字符串  
GUI\_DispStringInRectWrap()    在指定的矩形区域中显示字符串，并可自动换行

上面提到的几个 API 函数很常用，大家必须掌握，有关文本显示的 API 函数还有很多，大家自行尝试学习使用这些 API 函数的使用，上面的演示函数 emwin\_texttest()在开发板上的实际运行效果如图 4.4.1 所示。

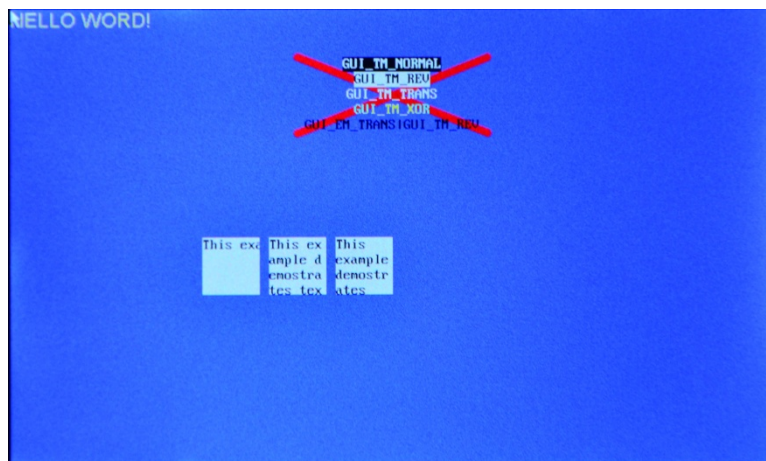


图 4.4.1 实例运行效果图

注意，图 4.4.1 为 4.3 寸屏(800\*480)上的演示效果，2.8 或者 3.5 寸屏会显示其中一部分！

本章内容为最基础的，大家务必掌握常用 API 函数的使用，本章只列举了几个函数做简单的讲解，其他没有列举出来的 API 函数大家可以参考 emWin 的中文手册来学习使用。

## 第五章 数值显示

上一章中我们讲解了如何在 LCD 上显示字符串，所以我们可以使用字符串的方式在 LCD 上显示数值。但是，这样的话我们要先将数值转换为字符串，使用起来很不方便。很庆幸，emWin 给我们提供了直接显示数值的函数，emWin 支持各种十进制、十六进制和二进制输出，本章我们就讲解一下如何使用这些 API 函数，本章分为以下几个部分：

- 5.1 数值显示 API 函数
- 5.2 显示十进制数值
- 5.3 显示浮点数值
- 5.4 二进制和十六进制数值显示
- 5.5 综合演示例程

## 5.1 数值显示 API 函数

有关数值显示的 API 函数如表 5.1.1 所示。

| API 函数                 | 描述                             |
|------------------------|--------------------------------|
| <b>显示十进制数值</b>         |                                |
| GUI_DisDec()           | 在当前位置显示指定字符数的十进制数值。            |
| GUI_DisDecAt()         | 在指定位置显示指定字符数的十进制数值。            |
| GUI_DisDecMin()        | 在当前位置显示最小字符数的十进制数值。            |
| GUI_DisDecShift()      | 在当前位置显示指定字符数、带小数点的十进制长数值。      |
| GUI_DisDecSpace()      | 在当前位置显示指定字符数的十进制数值，用空格代替首位的    |
| GUI_DisDecSDec()       | 在当前位置显示指定字符数的十进制数值并显示符号。       |
| GUI_DisDecSDecShift()  | 在当前位置显示指定字符数、带小数点的十进制长数值并显示符号。 |
| <b>显示浮点数值</b>          |                                |
| GUI_DisFloat()         | 显示指定字符数的浮点数值。                  |
| GUI_DisFloatFix()      | 显示指定小数点右边位数的浮点数值。              |
| GUI_DisFloatMin()      | 显示最小字符数的浮点数值。                  |
| GUI_DisSFloatFix()     | 显示指定小数点右边位数的浮点数值并显示符号。         |
| GUI_DisSFloatMin()     | 显示最小字符数的浮点数值并显示符号。             |
| <b>显示二进制数值</b>         |                                |
| GUI_DisBin()           | 在当前位置显示二进制数值。                  |
| GUI_DisBinAt()         | 在指定位置显示二进制数值。                  |
| <b>显示十六进制数值</b>        |                                |
| GUI_DisHex()           | 在当前位置显示十六进制数值。                 |
| GUI_DisHexAt()         | 在指定位置显示十六进制数值。                 |
| <b>emWin 版本</b>        |                                |
| GUI_GetVersionString() | 返回 emWin 的当前版本。                |

表 5.1.1 数值显示 API 函数

## 5.2 显示十进制数值

有关十进制数值显示的 API 函数有多种，这里我们可以 PC 上使用 VS2013 上仿真使用一下这些 API 函数，看看它们都有什么区别，程序很简单，如下所示：

```
void MainTask(void)
{
    GUI_Init();
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_DispStringHCenterAt("ALIENK NUM DISPLAY",160,0);
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_Font13_ASCII);

    //十进制数值 API 函数测试
    GUI_DisStringAt("GUI_DisStringAt(): ",0,20);
}
```

```

GUI_DispDec(123,4);
GUI_DispString(" ");
GUI_DispDec(-123, 4);

GUI_DispStringAt("GUI_DispDecAt():   ", 0, 35);
GUI_DispDecAt(100,100,35,3);
GUI_DispDecAt(-564, 130, 35, 4);

GUI_DispStringAt("GUI_DispDecShift(): ", 0, 50);
GUI_DispDecShift(1236,5,2);
GUI_DispString(" ");
GUI_DispDecShift(-1236, 6, 2);

GUI_DispStringAt("GUI_DispDecSpace(): ", 0, 65);
GUI_DispDecSpace(3265,5);

GUI_DispStringAt("GUI_DispSDec():     ", 0, 80);
GUI_DispSDec(200,4);

GUI_DispStringAt("GUI_DispSDecShift():   ", 0, 95);
GUI_DispSDecShift(12345,7,3);
while (1);
}

```

上面程序简单的展示了一下如何使用 STemWin 提供的十进制数值显示 API 函数，程序运行结果如图 4.2.1 所示。

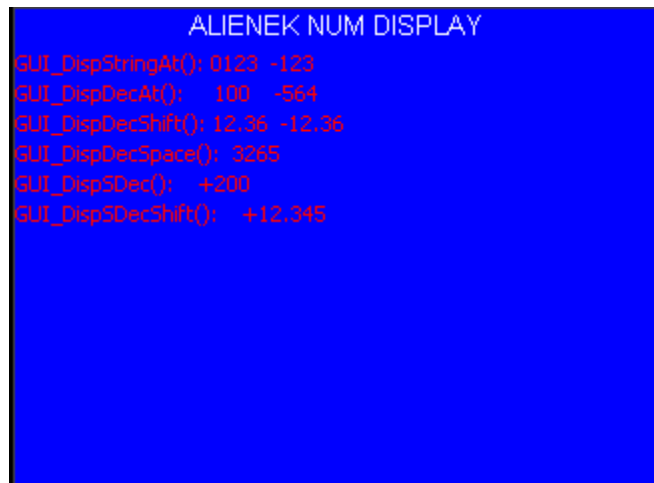


图 4.2.1 十进制数值显示

GUI\_DispDec()和 GUI\_DispDecAt()支持首位为 0 的格式，比如，如果我们要显示十进制数：123，当我们设置函数显示位数参数为 4 的时候，那么实际显示出来的就是：0123，首位用 0 来填充了，并且当数值为负的时候就会在前面显示一个“-”号，还有其他的一些函数支持首位为 0 的格式。当我们要显示的数据包含正负号或者小数点的时候，函数参数里面的显示位数也要将正负号和小数点算进去。

### 5.3 显示浮点数值

浮点显示 API 函数演示程序如下：

```
void MainTask(void)
{
    float f = 123.45678;

    GUI_Init();
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_DispStringHCenterAt("ALIENNEK NUM DISPLAY",160,0);
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_Font13_ASCII);

    //浮点数值显示 API 函数演示
    GUI_DispStringAt("GUI_DispFloat():\n", 0, 20);
    GUI_DispFloat(f, 9);
    GUI_GotoX(100);
    GUI_DispFloat(-f, 9);
    GUI_DispStringAt("GUI_DispFloatFix():\n", 0, 45);
    GUI_DispFloatFix(f, 9, 2);
    GUI_GotoX(100);
    GUI_DispFloatFix(f, 9, 2);
    GUI_DispStringAt("GUI_DispSFloatFix():\n", 0, 70);
    GUI_DispSFloatFix(f, 9, 2);
    GUI_GotoX(100);
    GUI_DispSFloatFix(-f, 9, 2);
    GUI_DispStringAt("GUI_DispFloatMin():\n", 0, 95);
    GUI_DispFloatMin(f, 3);
    GUI_GotoX(100);
    GUI_DispFloatMin(-f, 3);
    GUI_DispStringAt("GUI_DispSFloatMin():\n", 0, 120);
    GUI_DispSFloatMin(f, 3);
    GUI_GotoX(100);
    GUI_DispSFloatMin(-f, 3);
    while (1);
}
```

上面程序演示了一下在 emWin 中显示浮点数值，大家可以用 VS2013 在 PC 上仿真一下，程序很简单，大家对照着 emWin 的中文手册看一下各个函数和相对应的参数的含义，程序运行结果如图 5.3.1 所示。



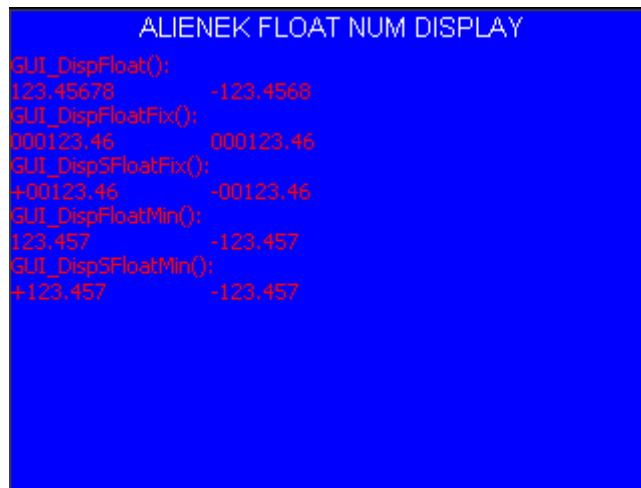


图 5.3.1 浮点数值 API 函数演示

## 5.4 二进制和十六进制数值显示

有时候我们需要显示 2 进制和十六进制的数值，在 emWin 中为我们提供了这样的 API 函数，极大的方便了我们的使用，下面我们使用一小段程序来展示一下如何使用 emWin 提供的 API 函数直接显示二进制和十六进制数值，程序如下：

```
void MainTask(void)
{
    int dec = 20;
    GUI_Init();
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_DisStringHCenterAt("ALIENK BIN&HEX NUM DISPLAY",160,0);
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_Font13_ASCII);

    //二进制数值显示 API 函数演示
    GUI_DisStringAt("GUI_DisBin():",0,20);
    GUI_DisBin(dec,6);
    GUI_DisStringAt("GUI_DisBinAt():",0,35);
    GUI_DisBinAt(dec,87,35,6);

    //十六进制数值显示 API 函数演示
    GUI_DisStringAt("GUI_DisHex():",0,50);
    GUI_DisHex(dec,2);
    GUI_DisStringAt("GUI_DisHexAt():",0,65);
    GUI_DisHexAt(dec,87,65,2);
    while (1);
}
```

关于二进制和十六进制数值的显示 emWin 分别只提供了 2 个 API 函数，上面程序运行结

果如图 5.4.1 所示。

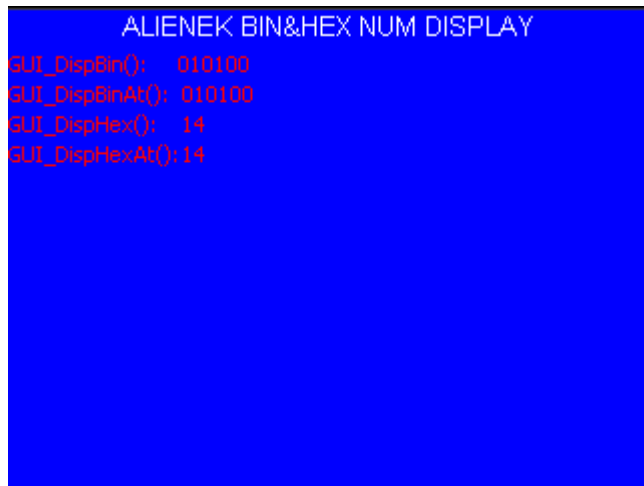


图 5.4.1 二进制和十六进制数值显示

## 5.5 综合演示例程

这里我们编写了一个综合演示例程在开发板上运行，实验完整工程为：**EMWIN 实验 4 数值显示**，此工程可以直接下载到开发板里面观察运行结果，本程序是基于 800\*480 分辨率屏幕的，如果使用 2.8 或者 3.5 寸屏的话可能显示不完全，这一点大家要注意一下，程序源码如下：

//emwin 数值显示例程

```
void numdisplay_demo(u16 x,u16 y)
{
    char* emwinversion;           //emwin 版本号
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetTextMode(GUI_TM_TRANS); //透明显示模式

    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_SetColor(GUI_YELLOW);
    GUI_DispStringHCenterAt("ALIENK NUM DISPLAY DEMO",400,10);

    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_Font20_ASCII);

    GUI_DispStringAt("GUI_DispDecAt():",x,y);
    GUI_DispDecAt(200,x+150,y,3); //显示数值 200

    GUI_DispStringAt("GUI_DispFloat():",x,y+20);
    GUI_GotoXY(x+150,y+20); //移动光标
    GUI_DispFloat(49.35,5); //显示浮点数 49.35
    GUI_DispString(" ");
    GUI_DispFloat(-0.345,6); //显示浮点数-0.345
```

```
GUI_DispStringAt("GUI_DispBinAt():",x,y+40);
GUI_DispBinAt(8,x+150,y+40,4);           //显示十进制数 8 的二进制值

GUI_DispStringAt("GUI_DispHex():",x,y+60);
GUI_GotoXY(x+150,y+60);                  //移动光标
GUI_DispHex(100,3);                      //显示十进制数 100 的 16 进制值

GUI_DispStringAt("GUI_DispHexAt():",x,y+80);
GUI_DispHexAt(100,x+150,y+80,3);        //在指定位置显示十进制数 100 的 16 进制值

GUI_DispStringAt("GUI_GetVersionString():",x,y+100);
emwinversion = (char*)GUI_GetVersionString(); //获取当前 emwin 版本号
GUI_DispStringAt(emwinversion,x+200,y+100); //显示 emwin 版本号
}
```

程序在开发板上的运行结果如图 5.5.1 所示（此处使用的是 4.3 寸 800\*480 屏幕）

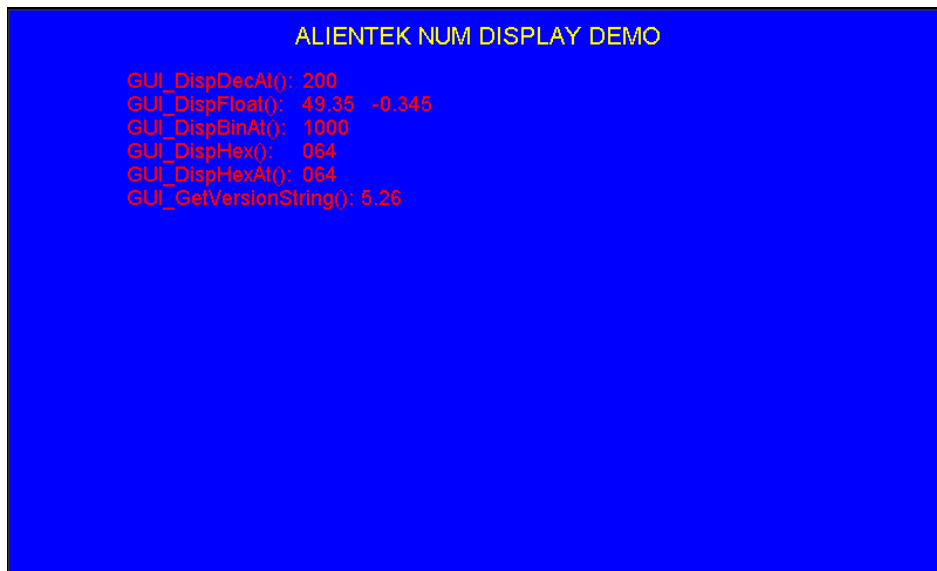


图 5.5.1 显示效果图

## 第六章 2D 绘图

emWin 提供有完整的 2-D 图形库，我们可以使用这些 2-D 图形库来绘制一些 2D 图形，我们可以绘制直线，曲线，多边形等图形，本章我们就讲解一下 emWin 的 2-D 图形库的使用，本章分为以下几个部分：

- 6.1 图形库基本 API 简介
- 6.2 其他图形绘制 API 函数
- 6.3 综合演示例程

## 6.1 图形库基本 API 简介

### 6.1.1 画笔大小

在绘制图形的时候我们常常需要不同粗细的线条，其实就是改变画笔的大小，emWin 中提供了 2 个关于画笔大小的 API 函数：GUI\_GetPenSize()和 GUI\_SetPenSize()，这两个函数分别用来获取当前画笔大小和改变当前画笔大小。

### 6.1.2 基本绘图例程

emWin 给我们提供了绘制点，水平线，垂直线，面和图形等基本绘制函数，这些函数在所有的绘制模式下都可使用，我们最常使用的也是这些函数，表 6.1.1 给出了 emWin 的基本绘制函数。

| 函数                         | 描述                |
|----------------------------|-------------------|
| GUI_ClearRect()            | 为矩形区域填充背景颜色。      |
| GUI_CopyRect()             | 复制显示器中的一个矩形区域。    |
| GUI_DrawGradientH()        | 绘制用水平颜色梯度填充的矩形。   |
| GUI_DrawGradientV()        | 绘制用垂直颜色梯度填充的矩形。   |
| GUI_DrawGradientRoundedH() | 绘制用水平颜色梯度填充的圆角矩形。 |
| GUI_DrawGradientRoundedV() | 绘制用垂直颜色梯度填充的圆角矩形。 |
| GUI_DrawPixel()            | 绘制单个像素。           |
| GUI_DrawPoint()            | 绘制点。              |
| GUI_DrawRect()             | 绘制矩形。             |
| GUI_DrawRectEx()           | 绘制矩形。             |
| GUI_DrawRoundedFrame()     | 绘制圆角框。            |
| GUI_DrawRoundedRect()      | 绘制圆角矩形。           |
| GUI_FillRect()             | 绘制填充的矩形。          |
| GUI_FillRectEx()           | 绘制填充的矩形。          |
| GUI_FillRoundedRect()      | 绘制填充的圆角矩形。        |
| GUI_InvertRect()           | 倒转矩形区域。           |

表 6.1.1 基本绘制函数

下面我们编写一小段程序，在 PC 上仿真几个基本绘制函数，看看效果，程序代码如下：

```
void MainTask(void)
{
    GUI_Init();
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_DispStringHCenterAt("ALIENK BIN&HEX NUM DISPLAY", 160, 0);
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_Font13_ASCII);

    //基本绘制例程函数演示
    GUI_SetBkColor(GUI_YELLOW);
```

```

GUI_ClearRect(5,20,95,110); //此函数使用背景颜色填充
GUI_SetBkColor(GUI_BLUE);
//绘制水平颜色梯度填充的矩形
GUI_DrawGradientH(100, 20, 190, 110, 0X4117BB, 0XC6B6F5);
//绘制垂直颜色梯度填充的圆角矩形
GUI_DrawGradientRoundedV(195, 20, 285, 110,25, 0X4117BB, 0XC6B6F5);
GUI_DrawRect(5,115,95,205); //在指定位置绘制矩形
GUI_FillRect(100,115,190,205); //在指定位置绘制填充的矩形区域
GUI_FillRoundedRect(195,115,285,205,25); //在指定位置绘制填充的圆角矩形
while(1);
}

```

上面程序展示了一下几个基本绘制函数的使用，在 PC 模拟器上的运行效果如图 6.1.1 所示。

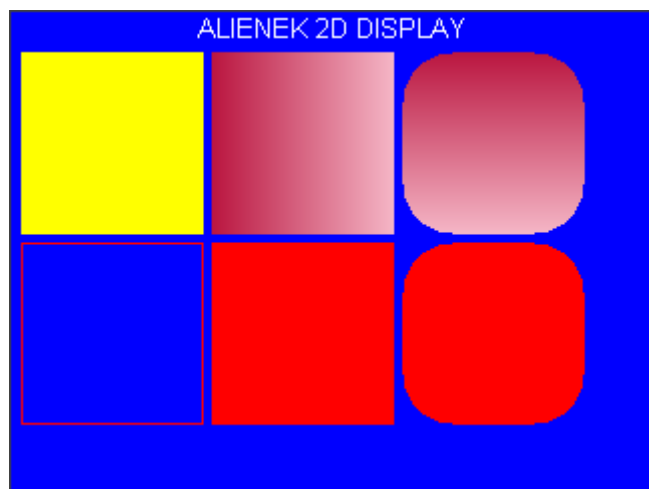


图 6.1.1 基本绘制函数演示

### 6.1.3 Alpha 混色

Alpha 混合是一种合并前景图像与背景来创建半透明效果的方法。Alpha 值决定可见的像素数以及背景透露的像素数。Alpha 混合完全自动执行。唯一需要做的就是使用 GUI\_EnableAlpha() 启用 Alpha 混合，关于 Alpha 混色的 API 函数如表 6.1.2 所示。

| 函数                     | 描述                |
|------------------------|-------------------|
| GUI_EnableAlpha()      | 启用                |
| GUI_RestoreUserAlpha() | 恢复之前的用户           |
| GUI_SetAlpha()         | 设置当前的             |
| GUI_SetUserAlpha()     | 设置其他值，以用于计算要使用的实际 |

表 6.1.2 Alpha 混色 API 函数

我们使用函数 GUI\_EnableAlpha() 来启用 Alpha，使用函数 GUI\_SetAlpha() 来设置 Alpha 的值，当值为 0 时表示不透明，为 255 时表示完全透明。使用 Alpha 会增加 CPU 的负荷，所以在操作完成后一定要将 Alpha 值设定为 0，即禁用 Alpha，或者也使用 GUI\_EnableAlpha() 来关闭 Alpha 功能，只需要将函数 GUI\_EnableAlpha() 的参数设置为 0 即可。

下面我们在 PC 上仿真展示一下 Alpha 的工作原理。

```

void MainTask(void)
{

```

```

GUI_Init();
GUI_SetBkColor(GUI_WHITE);
GUI_Clear();
GUI_SetFont(&GUI_Font16_ASCII);
GUI_SetColor(GUI_BLACK);
GUI_DispStringHCenterAt("ALIENNEK ALPHA DISPLAY", 160, 0);

GUI_EnableAlpha(1);      //开启 Alpha
GUI_SetColor(GUI_BLACK);
GUI_DispStringHCenterAt("Alphablending", 45, 61);
GUI_SetAlpha(40);        //设置 Alpha 值
GUI_SetColor( GUI_RED);
GUI_FillRect(0, 20, 49, 69);
GUI_SetAlpha(80);        //设置 Alpha 值
GUI_SetColor(GUI_GREEN);
GUI_FillRect(20, 40, 69, 89);
GUI_SetAlpha(40);        //设置 Alpha 值
GUI_SetColor(GUI_BLUE);
GUI_FillRect(40, 60, 89, 109);
GUI_EnableAlpha(0);      //关闭 Alpha
while (1);
}

```

效果如下如 6.1.2 所示。

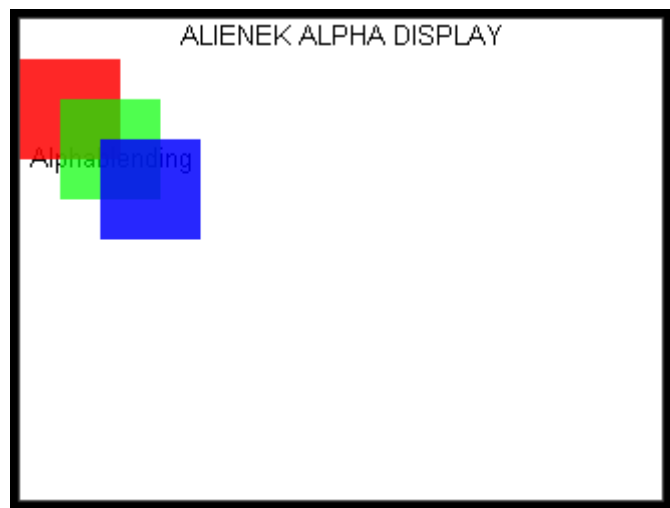


图 6.1.2 Alpha 混色演示

## 6.2 其他图形绘制 API 函数

### 6.2.1 绘制线条

在平常的使用中，我们在很多地方需要绘制线条，emWin 中提供了很多关于绘制线条的 API 函数，我们可以用这些函数方便的绘制出各种线条，这些 API 函数如表 6.2.1 所示。



| 函数                 | 描述                                |
|--------------------|-----------------------------------|
| GUI_DrawHLine()    | 绘制水平线。                            |
| GUI_DrawLine()     | 绘制从某个指定起点到指定终点的线条(绝对坐标)           |
| GUI_DrawLineRel()  | 绘制从当前位置到按 X 和 Y 距离指定的终点的线条 （相对坐标） |
| GUI_DrawLineTo()   | 绘制从当前位置到指定终点的线条。                  |
| GUI_DrawPolyLine() | 绘制折线。                             |
| GUI_DrawVLine()    | 绘制垂直线。                            |
| GUI_GetLineStyle() | 返回当前的线条样式。                        |
| GUI_MoveRel()      | 相对于其当前位置移动线条指针。                   |
| GUI_MoveTo()       | 将线条指针移动到给定位置。                     |
| GUI_SetLineStyle() | 设置当前的线条样式。                        |

表 6.1.2 绘制线条 API 函数

下面我们在 PC 上仿真展示一下这些函数的实际效果，程序如下。

```
const GUI_POINT Points[] =
{
    {10,220},
    {56,100},
    {102,220},
    {148,100},
    {194,220}
};

void MainTask(void)
{
    GUI_Init();
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_SetColor(GUI_RED);
    GUI_DispStringHCenterAt("ALIENK DRAW LINE DISPLAY", 160, 0);

    GUI_DrawHLine(30,10,250);//绘制一条水平线

    GUI_SetPenSize(3);
    GUI_DrawLine(10,50,250,50);//绘制一条指定起点和终点的直线
    GUI_SetPenSize(1);

    GUI_SetLineStyle(GUI_LS_DASH); //设置线条样式为虚线
    GUI_DrawLine(10, 70, 250, 70); //绘制一条指定起点和终点的直线
    GUI_SetLineStyle(GUI_LS_SOLID); //设置线条样式为实线

    GUI_DrawVLine(300,30,239); //绘制垂直线

    GUI_SetPenSize(2);
```

```
GUI_DrawPolyLine(&Points,GUI_COUNTOF(Points),0,0); //连接预定的点列表
while (1);
}
```

效果如图 6.2.1 所示。

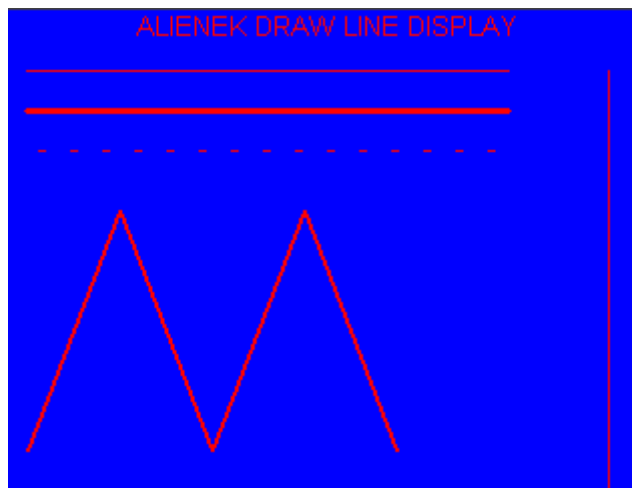


图 6.2.1 线条绘制演示

### 6.2.2 其他图形绘制函数

emWin 也提供了其他图形绘制函数，比如绘制多边形、圆、椭圆、弧线、线图和饼图等 API 函数，如表 6.2.2 所示。

| 函数                   | 描述          |
|----------------------|-------------|
| 绘制多边形                |             |
| GUI_DrawPolygon()    | 绘制多边形的轮廓。   |
| GUI_EnlargePolygon() | 扩展多边形。      |
| GUI_FillPolygon()    | 绘制填充的多边形。   |
| GUI_MagnifyPolygon() | 放大多边形。      |
| GUI_RotatePolygon()  | 按指定角度旋转多边形。 |
| 绘制圆                  |             |
| GUI_DrawCircle()     | 绘制圆的轮廓。     |
| GUI_FillCircle()     | 绘制填充的圆。     |
| 绘制椭圆                 |             |
| GUI_DrawEllipse()    | 绘制椭圆的轮廓。    |
| GUI_FillEllipse()    | 绘制填充的椭圆。    |
| 绘制弧线                 |             |
| GUI_DrawArc()        | 绘制弧线。       |
| 绘制线图                 |             |
| UI_DrawGraph()       | 绘制线图。       |
| 绘制饼图                 |             |
| GUI_DrawPie()        | 绘制圆形扇区。     |

表 6.2.2 其他图形绘制 API 函数

emWin 官方的仿真文件给我们提供了很多这些 API 函数的例程，大家可以运行一下这些程

序，观察一下这些函数的运行结果，这里就不一一给大家展示了。

### 6.3 综合演示例程

这里我们编写了一个综合演示例程在开发板上运行演示 emWin 的 2D 绘图 API 函数，实验完整工程为：[EMWIN 实验 5 2D 绘图](#)，此工程可以直接下载到开发板里面观察运行结果，本程序是基于 800\*480 分辨率屏幕的，如果使用 2.8 或者 3.5 寸屏的话可能显示不完全，这一点大家要注意一下，程序源码如下：

```
const GUI_POINT aPoints[]={
    {40,20},
    {0,20},
    {20,0}
};

const GUI_POINT aPoints1[]={
    {0,20},
    {40,20},
    {20,0}
};

const GUI_POINT aPointArrow[]={
    { 0, -5},
    {-40, -35},
    {-10, -25},
    {-10, -85},
    { 10, -85},
    { 10, -25},
    { 40, -35},
};

GUI_POINT aEnlargedPoints[GUI_COUNTOF(aPoints)];    //多边形放大后坐标
GUI_POINT aMagnifiedPoints[GUI_COUNTOF(aPoints1)];

//2D 图形绘制例程
void display_2d(void)
{
    int i;
    GUI_SetBkColor(GUI_BLUE);        //设置背景颜色
    GUI_Clear();

    GUI_SetColor(GUI_YELLOW);
    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_DispStringHCenterAt("ALIENTEK 2D DISPALY",400,10);
```

```

GUI_SetColor(GUI_RED);
GUI_SetFont(&GUI_Font16_ASCII);

GUI_SetBkColor(GUI_GREEN);
GUI_ClearRect(10,50,100,150); //在一个矩形区域填充绿色背景
GUI_SetBkColor(GUI_BLUE);      //背景改回蓝色

//绘制用水平颜色梯度填充的矩形
GUI_DrawGradientH(110,50,210,150,0X4117BB,0XC6B6F5);
//绘制用垂直颜色梯度填充的矩形
GUI_DrawGradientV(220,50,320,150,0X4117BB,0XC6B6F5);
//绘制水平颜色梯度填充的圆角矩形
GUI_DrawGradientRoundedH(330,50,430,150,25,0X4117BB,0XC6B6F5);
//绘制垂直颜色梯度填充的圆角矩形
GUI_DrawGradientRoundedV(440,50,540,150,25,0X4117BB,0XC6B6F5);
GUI_DrawRect(550,50,650,150);      //在当前窗口中指定的位置绘制矩形
GUI_FillRect(660,50,760,150);      //在当前窗口中指定位置绘制填充的矩形区域

GUI_SetPenSize(5);                  //设置画笔颜色,单位像素点
GUI_DrawLine(10,160,110,260);      //绘制线条

for(i=0;i<50;i+=3) GUI_DrawCircle(200,220,i); //绘制圆, 无法绘制半径大于 180 的圆
//在指定位置绘制指定尺寸的填充的圆,无法绘制半径大于 180 的圆
GUI_FillCircle(320,220,50);
GUI_SetPenSize(2);
GUI_SetColor(GUI_CYAN);
//在指定位置绘制指定尺寸的椭圆的轮廓 无法处理大于 180 的 rx/ry 参数
GUI_DrawEllipse(450,220,70,50);
GUI_SetColor(GUI_MAGENTA);
GUI_FillEllipse(450,220,65,45); //在指定位置绘制指定尺寸的填充的椭圆

drawarcscale();
draw_graph();
}

//Alpha 混合显示
void alpha_display(void)
{
    GUI_EnableAlpha(1);              //使能 Alpha 功能
    GUI_SetBkColor(GUI_WHITE);      //设置背景颜色
    GUI_Clear();

    GUI_SetTextMode(GUI_TM_TRANS);  //文本透明显示

```

```

GUI_SetColor(GUI_BLACK);
GUI_SetFont(&GUI_Font32_1);
GUI_DispStringHCenterAt("ALIENTEK ALPHA TEST",400,220);

GUI_SetAlpha(0x40);           //设置 alpha 值 0x00-0XFF
GUI_SetColor(GUI_RED);
GUI_FillRect(100,50,300,250); //绘制红色矩形

GUI_SetAlpha(0x80);
GUI_SetColor(GUI_GREEN);
GUI_FillRect(200,150,400,350); //绘制绿色矩形

GUI_SetAlpha(0xC0);
GUI_SetColor(GUI_BLUE);
GUI_FillRect(300,250,500,450); //绘制蓝色矩形

GUI_SetAlpha(0x80);
GUI_SetColor(GUI_YELLOW);
GUI_FillRect(400,150,600,350); //绘制黄色矩形

GUI_SetAlpha(0x40);
GUI_SetColor(GUI_BROWN);
GUI_FillRect(500,50,700,250); //绘制棕色矩形
GUI_SetAlpha(0);             //恢复 alpha 通道
}

//绘制弧线 绘制简易仪表盘
void drawarcscale(void)
{
    int x0 = 630;
    int y0 = 380;
    int i;
    char ac[4];
    GUI_SetPenSize( 5 );
    GUI_SetTextMode(GUI_TM_TRANS);
    GUI_SetFont(&GUI_FontComic18B_ASCII);
    GUI_SetColor(GUI_BLACK);
    GUI_DrawArc( x0,y0,150, 150,-30, 210 );
    for (i=0; i<= 23; i++) {
        float a = (-30+i*10)*3.1415926/180;
        int x = -141*cos(a)+x0;
        int y = -141*sin(a)+y0;
        if (i%2 == 0)

```

```

        GUI_SetPenSize( 5 );
    else
        GUI_SetPenSize( 4 );
    GUI_DrawPoint(x,y);
    if (i%2 == 0) {
        x = -123*cos(a)+x0;
        y = -130*sin(a)+y0;
        sprintf(ac, "%d", 10*i);
        GUI_SetTextAlign(GUI_TA_VCENTER);
        GUI_DispStringHCenterAt(ac,x,y);
    }
}

//绘制线图
void draw_graph(void)
{
    I16 aY[400];
    int i;
    for(i=0;i<GUI_COUNTOF(aY);i++)  aY[i] = rand()%100;
    GUI_ClearRect(0,300,400,400);  //每次显示前先清除显示区域
    GUI_DrawGraph(aY,GUI_COUNTOF(aY),0,300);
}

//绘制多边形
void draw_polygon(void)
{
    int i, Mag, magy = 50, Count = 4;
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetColor(GUI_YELLOW);
    GUI_Clear();

    GUI_SetTextMode(GUI_TM_TRANS);  //透明显示
    GUI_SetFont(&GUI_Font24_ASCII); //设置字体
    GUI_DispStringHCenterAt("ALIENTEK PLAYGON DISPLAY",400,10);

    GUI_SetColor(GUI_WHITE);
    GUI_SetDrawMode(GUI_DM_XOR); //设置绘制模式
    GUI_FillPolygon(aPoints,GUI_COUNTOF(aPoints),140,110);
    for(i=1;i<10;i++)
    {
        //放大坐标
        GUI_EnlargePolygon(aEnlargedPoints,aPoints,GUI_COUNTOF(aPoints),i * 5);
    }
}

```

```

//绘制放大坐标后的多边形
GUI_FillPolygon(aEnlargedPoints,GUI_COUNTOF(aPoints),140,110);
}
GUI_SetDrawMode(GUI_DM_NORMAL); //恢复默认的绘制模式

GUI_SetColor(GUI_GREEN);
for (Mag = 1; Mag <= 4; Mag *= 2, Count /= 2)
{
    int magx = 600;
    //按指定系数放大多边形
    GUI_MagnifyPolygon(aMagnifiedPoints, aPoints, GUI_COUNTOF(aPoints), Mag);
    for (i = Count; i > 0; i--, magx += 40 * Mag)
    {
        GUI_FillPolygon(aMagnifiedPoints, GUI_COUNTOF(aPoints), magx, magy);
        //绘制多边形
    }
    magy += 20 * Mag;
}

GUI_SetFont(&GUI_Font8x16);
GUI_DispStringAt("Polygons of arbitrary shape ", 100, 300);
GUI_DispStringAt("in any color", 220, 320);
GUI_SetColor(GUI_DARKRED);
GUI_FillPolygon (&aPointArrow[0],7,200,400);    //绘制多边形
}

```

程序运行结果如图 6.3.1、6.3.2、6.3.3 所示。

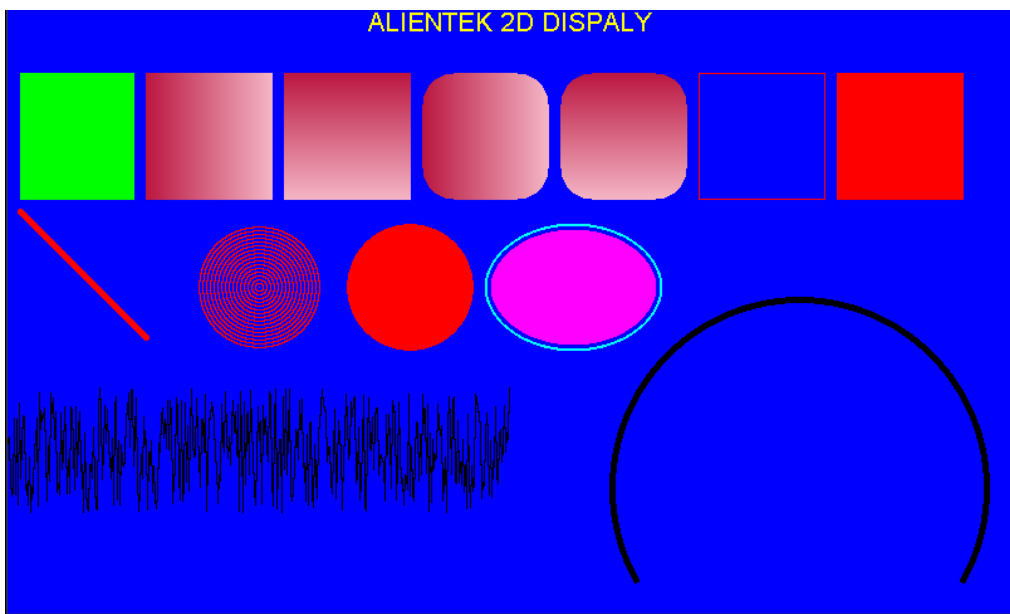


图 6.3.1 2D 绘图



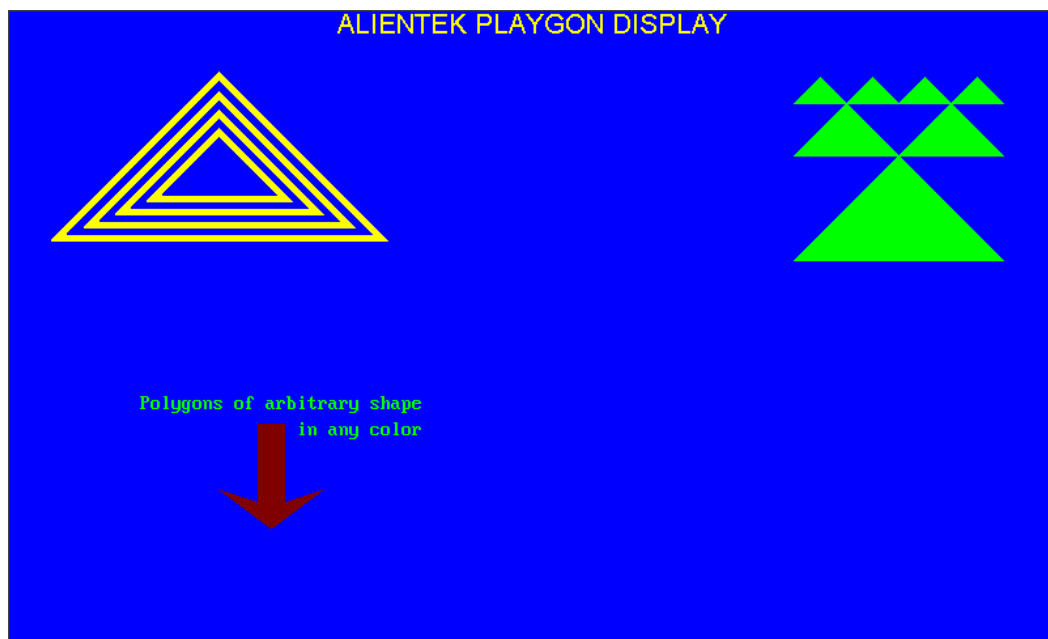


图 6.3.2 多边形绘制

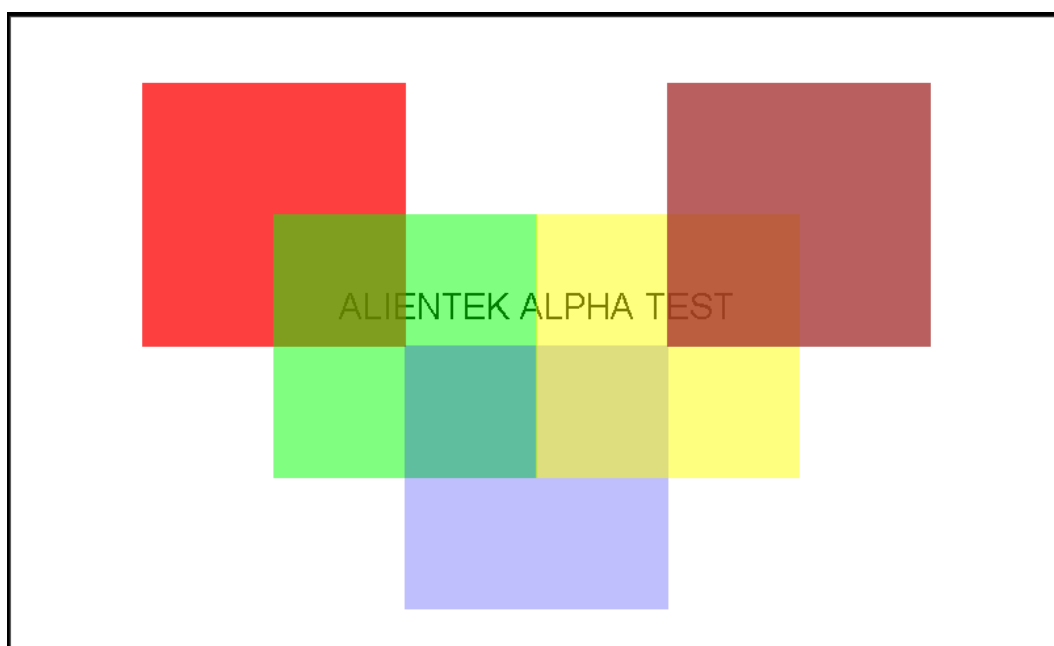


图 6.3.3 Alpha 测试

## 第七章 显示位图

很多情况下我们需要在 LCD 上显示图片,本章我们讲解一下如何使用位图转换器将图片转换为 C 文件,然后调用 emWin 中的相应函数来显示这个 C 文件位图。本章暂时不讲解如何在屏幕上直接显示 GIF、JPEG、BMP 和 PNG 等格式的图片,这些格式图片的直接显示我们会在以后的章节中讲解,本章分为如下几个部分:

- 7.1 位图转换器
- 7.2 位图绘制 API 简介
- 7.3 综合演示例程

## 7.1 位图转换器

### 7.1.1 位图转换器简介

位图转换器的主要作用是将位图从 PC 格式转换为 C 文件。emWin 可使用的位图在 C 文件中通常定义为 GUI\_BITMAP 结构。这些结构会非常大，手动生成这些位图很费时并且效率很低。因此建议使用位图转换器，自动从位图生成 C 文件。它的另一项有用功能是将图像另存为 C 流文件。与常规 C 文件相比，其优势在于这些数据流可位于需要将 C 文件置于的可寻址 CPU 区域的任何媒体中的任何位置。还具有色彩转换功能，因此可以减少所生成的 C 代码。通常可以减少每个像素的位数以减少内存消耗。位图转换器显示已转换的图像。位图转换器可执行很多简单的功能，包括缩放尺寸、水平或垂直裁剪位图、旋转以及转换位图索引或色彩。

我们从 ST 官网下载的 STemWin 源码里面就有位图转换器这个软件的，按照如下路径打开：[STemWin\\_Library\\_V1.1.2->Libraries->STemWinLibrary522->Software](#)。打开以后如图 7.1.1 所示，其中 BmpCvrt.exe 这个软件就是位图转换器。

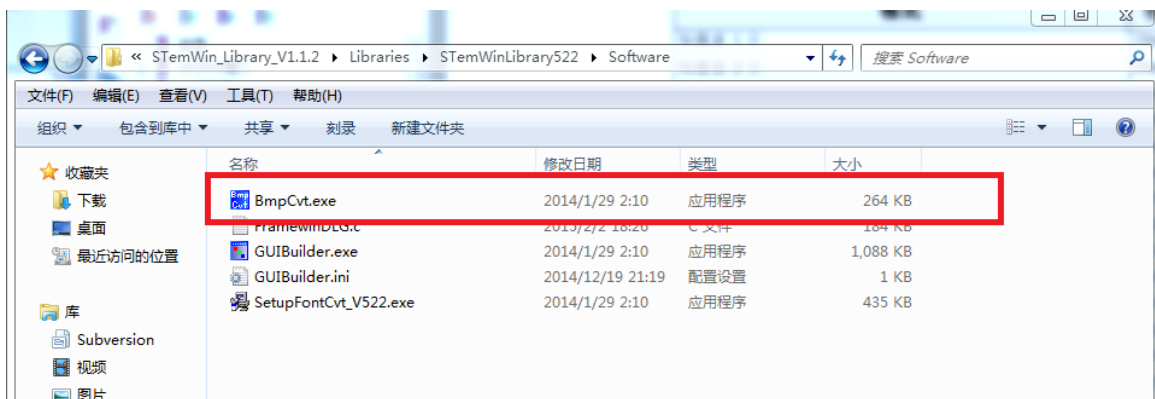


图 7.1.1 位图转换器

### 7.1.2 位图转换器的使用

1、打开位图转换器，如图 7.1.2 所示。

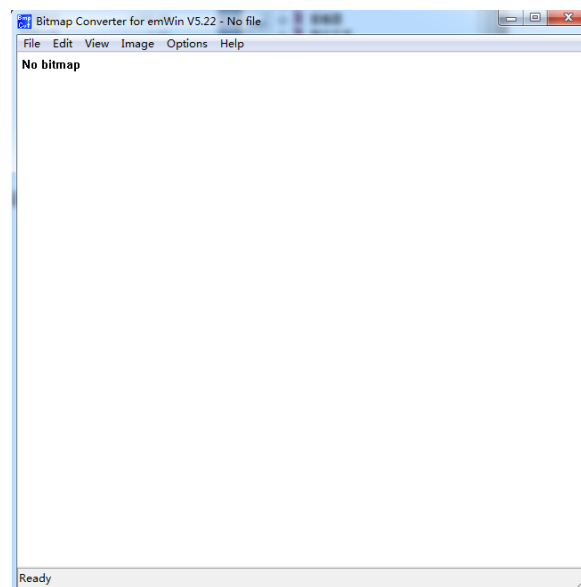


图 7.1.2 位图转换器

2、向位图转换器中加载图片，点击位图转换器的“File->Open”，打开想要转换的图片，这里我打开了 ALIENTEK 的 logo 图片，格式为 PNG 的，前面说过位图转换器可以打开 bmp,gif 和 png 格式的图片，打开以后如图 7.1.3 所示。



图 7.1.3 向位图转换器中加载图片

3、色彩转换，选择所需的调色板，点击“Image->Convert to”，如图 7.1.4 所示，我们可以看到，有很多种格式可以选择，这里我们选择“Best palette”，也就是“最佳调色板”，大家也可以尝试选择其他的格式试一下。

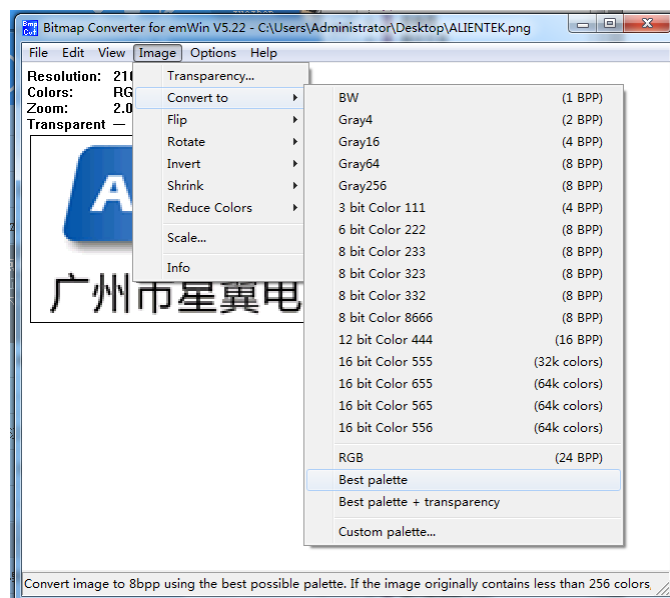


图 7.1.4 设置转换格式

转换位图颜色格式的主要原因是为了减少内存消耗。最常用的方法是使用上买你所示的选项“最佳调色板”，它可定制特定位图的调色板，只包含图像中要用的颜色。对于为了使调色板尽可能小，同时还要完全支持图像的全彩位图，此选项尤其有用。在位图转换器中打开位图文件后，只需从菜单中选择 Image/Convert Into/Best palette。如果需要保持透明性，请选择

Image/Convert Into/Best palette + transparency。对于某些应用,可能使用固定颜色调色板更高效,请在 Image/Convert Into 菜单下选择。例如,假如要在仅支持四种灰度的显示器上显示全彩模式的位图。此时使图像保持为原始格式会浪费内存,因为在显示器上只显示为四种灰度。将全彩位图转换为四灰度、2bpp 位图可获得最高效率。

4、将位图存为 C 文件,点击“File->Save As”,注意这里选择存为.C 文件,如图 7.1.5 所示。注意,设置好文件的保存路劲。

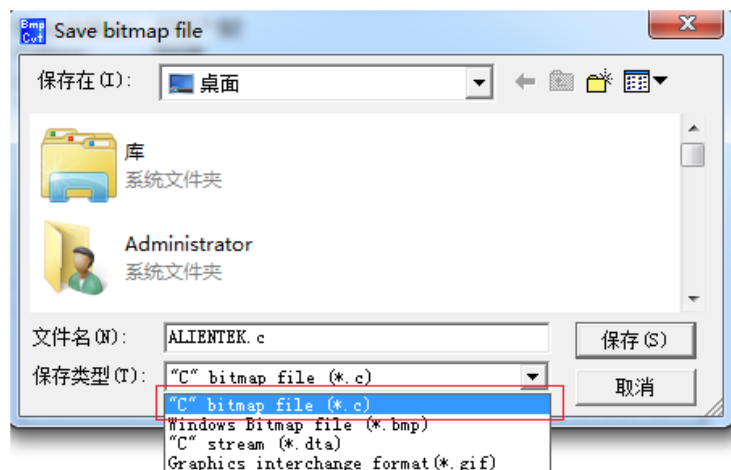


图 7.1.5 保存为.c 文件

5、选择为.C 文件以后,点击保存,会弹出如图 7.1.6 所示画面,让我们选择格式,这里我们选择“High color(565)”。

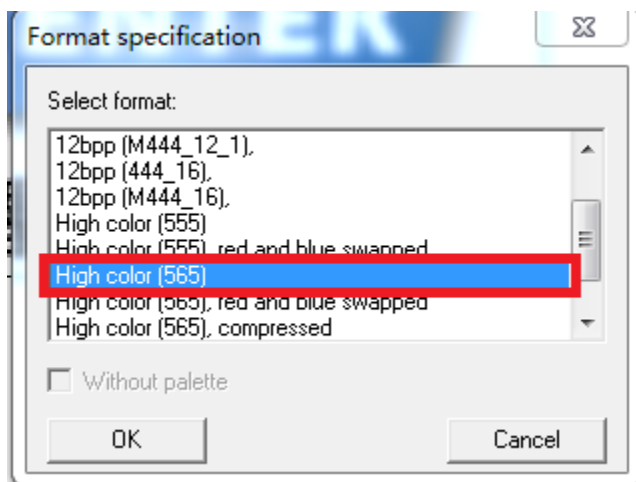
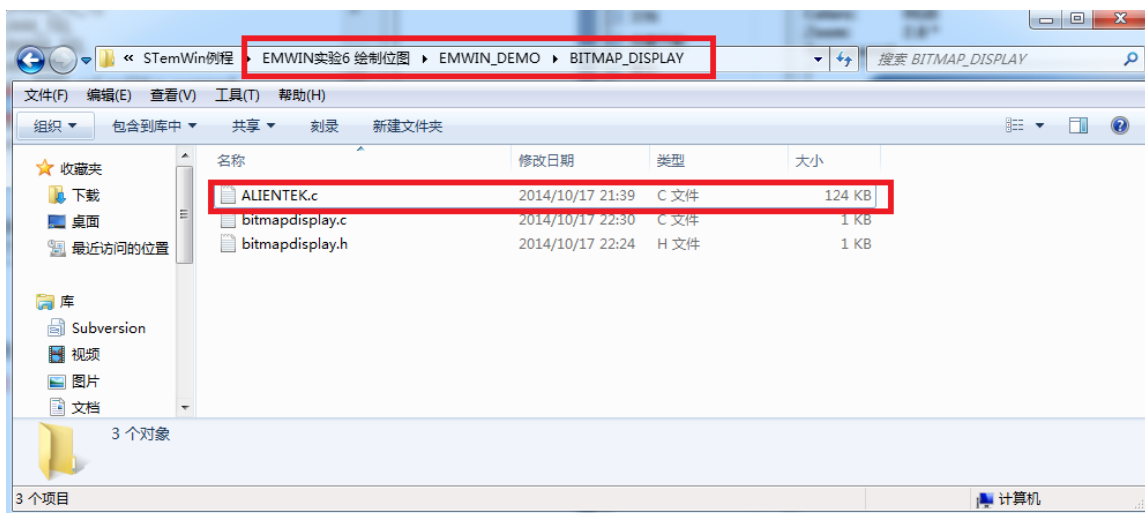


图 7.1.6 选择格式

选择完格式后点击“OK”,这样就在你之前设置好的文件夹下生成了一个.C 文件,将这个 C 文件拷贝到我们的工程中,我们在接下来的实验中就要使用这个 C 文件。按照如下路径打开我们的实验工程: **EMWIN 实验 6 绘制位图->EMWIN\_DEMO->BITMAP\_DISPLAY**, 打开以后如图 7.1.7 所示。



7.1.7 将位图 C 文件放入工程中

## 7.2 位图绘制 API 简介

emWin 中提供了绘制位图的 API 函数，如表 7.2.1 所示，使用这些函数我们就可以将上一小节中生成的 C 文件位图绘制出来。

| 函数                      | 描述                                     |
|-------------------------|----------------------------------------|
| GUI_DrawBitmap()        | 绘制位图。                                  |
| GUI_DrawBitmapEx()      | 绘制缩放的位图。                               |
| GUI_DrawBitmapHWAlpha() | 在具有硬件 Alpha 混合支持的系统上绘制带 Alpha 混合信息的位图。 |
| GUI_DrawBitmapMag()     | 绘制放大的位图。                               |

表 7.2.1 位图绘制 API 函数

我们最常用的就是 GUI\_DrawBitmap() 和 GUI\_DrawBitmapEx() 这两个函数，在下一小节中的演示实例中我们就主要讲解这两个函数的使用。

### 1、GUI\_DrawBitmap()

#### GUI\_DrawBitmap()

##### 描述

在当前窗口中的指定位置绘制位图图像。

##### 原型

```
void GUI_DrawBitmap(const GUI_BITMAP * pBM, int x, int y);
```

| 参数  | 描述               |
|-----|------------------|
| pBM | 指向要显示的位图。        |
| x   | 显示器中位图左上角的 X 位置。 |
| y   | 显示器中位图左上角的 Y 位置。 |

##### 其他信息

图片数据的解释为从第一个字节的最高有效位 (msb) 开始的位流。

新行始终在偶数字节地址开始，就像位图的第 n 行从偏移 n\*BytesPerLine 开始。在客户区的任何位置都能看到位图。

### 2、GUI\_DrawBitmapEx()

## GUI\_DrawBitmapEx()

### 描述

此例程可以在显示器中缩放和 / 或镜像位图。

### 原型

```
void GUI_DrawBitmapEx(const GUI_BITMAP * pBitmap,
                      int x0,      int y0,
                      int xCenter, int yCenter,
                      int xMag,    int yMag);
```

| 参数      | 描述             |
|---------|----------------|
| pBM     | 指向要显示的位图。      |
| x0      | 显示器中定位点的 X 位置。 |
| y0      | 显示器中定位点的 Y 位置。 |
| xCenter | 位图中定位点的 X 位置。  |
| yCenter | 位图中定位点的 Y 位置。  |
| xMag    | X 方向的比例因子。     |
| yMag    | Y 方向的比例因子。     |

### 其他信息

**xMag** 参数为负值将在 X 轴镜像位图，**yMag** 参数为负值将在 Y 轴镜像位图。**xMag** 和 **yMag** 的单位为千分之一。**xCenter** 和 **yCenter** 给定的位置指定在显示器中 **x0/y0** 位置（不考虑比例或镜像）显示的位图的像素。

不能使用此函数绘制 **RLE** 压缩的位图。

在上面两个函数中，位图信息都是用结构体 **GUI\_BITMAP** 来定义的，我们使用位图转换器生成的 C 文件最终就是将图片信息放到结构体里面，包括图片的数据，尺寸，调色板信息等等。

## 7.3 综合演示例程

这里我们编写了一个综合演示例程在开发板上运行演示 emWin 的位图显示功能，实验完整工程为：**EMWIN 实验 6 绘制位图**，打开 **bitmapdisplay.c**，程序源码如下：

```
extern GUI_BITMAP bmALIENTEK; //ALIENTEK 图标
//显示 C 文件格式的位图
void draw_bitmap(void)
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetColor(GUI_YELLOW);
    GUI_Clear();

    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_SetTextMode(GUI_TM_TRANS); //透明显示
    GUI_DispStringHCenterAt("ALIENTEK BITMAP DISPLAY",400,0);
    GUI_DrawBitmap(&bmALIENTEK,295,194); //绘制 ALIENTEK 图标
}
//在显示器中缩放位图
//Xmag:X 方向的比例因子，单位为千分之
//Ymag:Y 方向的比例因子，单位为千分之
void zoom_bitmap(int Xmag,int Ymag)
```



```
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_DrawBitmapEx(&bmALIENTEK,400,240,105,46,Xmag,Ymag); //按照比例绘制位图
}
```

在上面程序中有两个函数 `draw_bitmap()` 和 `zoom_bitmap()`，`draw_bitmap()` 函数主要展示了 `GUI_DrawBitmap()` 函数的使用，`zoom_bitmap()` 主要展示了 `GUI_DrawBitmapEx()` 的使用方法，程序非常简单，我们在打开 `main.c` 文件，`main.c` 中的 `emwin_maintask()` 函数为 `emWin` 的任务函数，`emWin` 的演示 `demo` 都在这个任务中运行。

//EMWINDEMO 任务

```
void emwin_maintask(void *pdata)
{
    int Xmag,Ymag; //缩放因子,单位 千分之
    GUI_CURSOR_Show();
    draw_bitmap(); //绘制位图, 未作缩放处理
    OSTimeDlyHMSM(0,0,2,0); //延时 2s
    while(1)
    {
        Xmag += 100;
        Ymag += 100;
        if(Xmag>=2000)Xmag = 100;
        if(Ymag>=2000)Ymag = 100;
        zoom_bitmap(Xmag,Ymag); //绘制缩放的位图
        GUI_Delay(500);
    }
}
```

上面函数的功能就是，一开始先调用 `draw_bitmap()` 函数显示 `ALIENTEK` 的 logo，延时 2s 以后进入 `while(1)` 循环中，循环调用 `zoom_bitmap()` 函数显示由小到大的 `ALIENTEK` 的 logo，每次 X,Y 的比例因子增加 100%，也就是十分之一。

程序运行结果如图 7.3.1 所示，**注意！**此程序是基于 800\*480 分辨率 LCD 的，如果是在 `ALIENTEK` 的 2.8 或者 3.5 寸屏幕上会显示不完全，稍微改动一下程序就可以了。



图 7.3.1 位图显示

## 第八章 颜色

emWin 支持黑色 / 白色、灰度（具有不同强度的单色）和彩色显示器。同一用户程序可用于任何显示器；只有 LCD 显示器的配置才需要进行更改。emWin 中颜色分为逻辑颜色和物理颜色，逻辑颜色是应用程序处理的颜色，逻辑颜色始终按 RGB 值进行定义，是一个 24 位的值，每种颜色包含 8 位。物理颜色是可用显示器实际显示的颜色，按逻辑颜色映射到物理颜色，本章分为如下几部分：

- 8.1 预定义的基本颜色
- 8.2 调色板
- 8.3 综合演示例程

## 8.1 预定义的基本颜色

在 emWin 中预定义一些标准颜色，这些预定义的颜色都用宏定义来表示，我们可以使用这些预定义的基本颜色来设置背景色和前景色，如果需要其他颜色的话我们就要自己定义了，这些预定义的颜色如表 8.1.1 所示。

| 颜色宏定义            | 颜色 | RGB 值      |
|------------------|----|------------|
| GUI_BLUE         |    | 0x00FF0000 |
| GUI_GREEN        |    | 0x0000FF00 |
| GUI_RED          |    | 0x000000FF |
| GUI_CYAN         |    | 0x00FFFF00 |
| GUI_MAGENTA      |    | 0x00FF00FF |
| GUI_YELLOW       |    | 0x0000FFFF |
| GUI_LIGHTBLUE    |    | 0x00FF8080 |
| GUI_LIGHTGREEN   |    | 0x0080FF80 |
| GUI_LIGHTRED     |    | 0x008080FF |
| GUI_LIGHTCYAN    |    | 0x00FFFF80 |
| GUI_LIGHTMAGENTA |    | 0x00FF80FF |
| GUI_LIGHTYELLOW  |    | 0x0080FFFF |
| GUI_DARKBLUE     |    | 0x00800000 |
| GUI_DARKGREEN    |    | 0x00008000 |
| GUI_DARKRED      |    | 0x00000080 |
| GUI_DARKCYAN     |    | 0x00808000 |
| GUI_DARKMAGENTA  |    | 0x00800080 |
| GUI_DARKYELLOW   |    | 0x00008080 |
| GUI_WHITE        |    | 0x00FFFFFF |
| GUI_LIGHTGRAY    |    | 0x00D3D3D3 |
| GUI_GRAY         |    | 0x00808080 |
| GUI_DARKGRAY     |    | 0x00404040 |
| GUI_BLACK        |    | 0x00000000 |
| GUI_BROWN        |    | 0x002A2AA5 |
| GUI_ORANGE       |    | 0x0000A5FF |

表 8.1.1 emWin 预定义颜色

## 8.2 调色板

emWin 中内置了很多固定的调色板模式，如表 8.2.1 所示。

| 标识符           | 可用的颜色数         | 掩码                            |
|---------------|----------------|-------------------------------|
| GUICC_1       | 2 (黑白)         | 0x01                          |
| GUICC_2       | 4 (灰度)         | 0x03                          |
| GUICC_4       | 16 (灰度)        | 0x0F                          |
| GUICC_5       | 32 (灰度)        | 0x1F                          |
| GUICC_111     | 8              | BGR                           |
| GUICC_M111    | 8              | RGB                           |
| GUICC_222     | 64             | BBGGRR                        |
| GUICC_M222    | 64             | RRGGBB                        |
| GUICC_233     | 256            | BBGGRRR                       |
| GUICC_M233    | 256            | RRGGGBB                       |
| GUICC_323     | 256            | BBBGGRRR                      |
| GUICC_M323    | 256            | RRRGGBB                       |
| GUICC_332     | 256            | BBBGGGRR                      |
| GUICC_M332    | 256            | RRRGGBB                       |
| GUICC_44412   | 4096           | 0000BBBBGGGGRRRR              |
| GUICC_M444_12 | 4096           | 0000RRRRGGGGBBBB              |
| GUICC_444121  | 4096           | BBBBGGGGRRRR0000              |
| GUICC_44416   | 4096           | 0BBBB0GGGG0RRRR0              |
| GUICC_M44416  | 4096           | 0RRRR0GGGG0BBBB0              |
| GUICC_555     | 32768          | 0BBBBBGGGGRRRRR               |
| GUICC_M555    | 32768          | 0RRRRGGGGBBBBBB               |
| GUICC_556     | 65536          | BBBBBGGGGRRRRR                |
| GUICC_M556    | 65536          | RRRRGGGGBBBBBB                |
| GUICC_565     | 65536          | BBBBBGGGGRRRRR                |
| GUICC_M565    | 65536          | RRRRGGGGBBBBBB                |
| GUICC_655     | 65536          | BBBBBGGGGRRRRR                |
| GUICC_M655    | 65536          | RRRRGGGGBBBBBB                |
| GUICC_666     | 262144         | BBBBBGGGGRRRRR                |
| GUICC_M666    | 262144         | RRRRGGGGBBBBBB                |
| GUICC_666_9   | 262144         | 000000BBBBBGGG000000GGGGRRRRR |
| GUICC_M666_9  | 262144         | 000000RRRRRGGG000000GGGBBBBB  |
| GUICC_822216  | 256            | 0xFF                          |
| GUICC_84444   | 240            | 0xFF                          |
| GUICC_8666    | 232            | 0xFF                          |
| GUICC_86661   | 233 (232 + 透明) | 0xFF                          |
| GUICC_888     | 16777216       | BBBBBBBGGGGGGGGRRRRRRR        |

表 8.2.1 emWin 内置调色板

关于表 8.2.1 中固定调色板详细的说明可以参考 EMWIN 中文手册，在前面讲解 STemWin 移植的时候我们详细的讲解了 LCDConf\_FlexColor\_Template.c 文件中的函数 LCD\_X\_Config(), 在这个函数中我们调用了 GUI\_DEVICE\_CreateAndLink() 来设置调色板，我们选择的是 GUICC\_M565 这个调色板。

如果没有固定调色板模式与色彩转换的需求相匹配，则可以使用定制调色板模式，利用此模式就可使用应用程序定义的色彩转换，本教程不讲解此模式，感兴趣的同学可以参考 EMWIN 中文手册中此模式的使用方法。

### 8.3 颜色 API 函数

表 8.3.1 列出了有关颜色的可用的 API 函数。

| 例程                      | 描述           |
|-------------------------|--------------|
| 基本颜色函数                  |              |
| GUI_GetBkColor()        | 返回当前背景色。     |
| GUI_GetBkColorIndex()   | 返回当前背景色的索引。  |
| GUI_GetColor()          | 返回当前前景色。     |
| GUI_GetColorIndex()     | 返回当前前景色的索引。  |
| GUI_SetBkColor()        | 设置当前背景色。     |
| GUI_SetBkColorIndex()   | 设置当前背景色的索引。  |
| GUI_SetColor()          | 设置当前前景色。     |
| GUI_SetColorIndex()     | 设置当前前景色的索引。  |
| 索引和颜色转换                 |              |
| GUI_CalcColorDist()     | 返回           |
| GUI_CalcVisColorError() | 将差值返回给下一可用颜色 |
| GUI_Color2Index()       | 将色彩转换为颜色索引。  |
| GUI_Color2VisColor()    | 返回最接近的可用颜色   |
| GUI_ColorIsAvailable()  | 检查给定的颜色是否可用  |
| GUI_Index2Color()       | 将颜色索引转换为颜色。  |

表 8.3.1 颜色 API 函数

在这些 API 函数中我们最常使用的就是 GUI\_SetColor()和 GUI\_SetBkColor()这两个函数，用来设置前景色和背景色。

### 8.3 综合演示例程

这里我们编写了一个颜色条绘制的例程，实验完整工程为：EMWIN 实验 7 颜色条绘制，打开 color\_bar.c 文件，例程源码如下。

```
#define X_START 60 //颜色条开始 X 坐标
#define Y_START 40 //颜色条开始 Y 坐标

typedef struct {
    int NumBars;
    GUI_COLOR Color;
    const char * s;
} BAR_DATA;
```

```

static const BAR_DATA _aBarData[] = {
    { 2, GUI_RED    , "Red" },
    { 2, GUI_GREEN  , "Green" },
    { 2, GUI_BLUE   , "Blue" },
    { 1, GUI_WHITE  , "Grey" },
    { 2, GUI_YELLOW , "Yellow" },
    { 2, GUI_CYAN   , "Cyan" },
    { 2, GUI_MAGENTA, "Magenta" },
};

static const GUI_COLOR _aColorStart[] = { GUI_BLACK, GUI_WHITE };

//显示颜色条
static void _DemoShowColorBar(void)
{
    GUI_RECT Rect;
    int      yStep;
    int      i;
    int      j;
    int      xSize;
    int      ySize;
    int      NumBars;    //要显示的颜色条数量 2+2+2+1+2+2+2=13
    int      NumColors;  //要显示的颜色数量，这里有 7 个

    //获取显示尺寸
    xSize = LCD_GetXSize();
    ySize = LCD_GetYSize();

    //获取颜色条的数量
    NumColors = GUI_COUNTOF(_aBarData);
    for (i = NumBars = 0, NumBars = 0; i < NumColors; i++) {
        NumBars += _aBarData[i].NumBars;
    }
    yStep = (ySize - Y_START) / NumBars;

    //显示文本
    Rect.x0 = 0;
    Rect.x1 = X_START - 1;
    Rect.y0 = Y_START;
    GUI_SetFont(&GUI_Font8x16);
    for (i = 0; i < NumColors; i++) {
        Rect.y1 = Rect.y0 + yStep * _aBarData[i].NumBars - 1;

```

```
//显示文本
GUI_DispStringInRect(_aBarData[i].s, &Rect, GUI_TA_LEFT | GUI_TA_VCENTER);
Rect.y0 = Rect.y1 + 1;
}

//显示颜色
Rect.x0 = X_START;
Rect.x1 = xSize - 1;
Rect.y0 = Y_START;
for (i = 0; i < NumColors; i++) {
    for (j = 0; j < _aBarData[i].NumBars; j++) {
        Rect.y1 = Rect.y0 + yStep - 1;
        GUI_DrawGradientH(Rect.x0, Rect.y0, Rect.x1, Rect.y1, \
            _aColorStart[j], _aBarData[i].Color); //显示颜色
        Rect.y0 = Rect.y1 + 1;
    }
}
}

//颜色条测试实验
void color_bar_test(void)
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetColor(GUI_YELLOW);
    GUI_Clear();

    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_SetTextMode(GUI_TM_TRANS);    //透明显示
    GUI_DispStringHCenterAt("COLOR_BAR TEST!", 400, 0);

    _DemoShowColorBar();    //显示颜色条
}
```

上面程序比较简单，就是在 LCD 上绘制多条颜色条，程序运行效果如图 8.3.1 所示。





图 8.3.1 颜色条实验

## 第九章 存储设备

存储设备可在各种情况下使用，主要用于防止在绘制重叠项目时出现显示器闪烁。其基本思想很简单，不使用存储设备时，绘制操作直接写入显示器，屏幕在执行绘制操作时随时更新，从而在进行各种更新时使屏幕闪烁。例如，如果要在背景中绘制一个位图，在前景中绘制一些透明文本，应首先绘制位图，然后绘制文本，效果将是文本出现闪烁。但是，如果在此过程中使用存储设备，则所有绘制操作都在存储器中执行。仅在所有操作都完成后才将最终结果显示在屏幕上，其优点是没有闪烁。差别可归纳如下：如果不使用存储设备，则可以看到一步步的绘制操作效果，缺点是会出现显示器闪烁。使用存储设备时，一次可见到所有例程的效果，就象单次操作一样，不能实际看见中间步骤。如上所述，使用存储设备的优点是完全消除了显示器的闪烁，这也是通常所期望的。本章分为如下几部分：

### 9.1 存储设备简介

### 9.2 存储设备基本 API 函数

### 9.3 分段存储设备

### 9.4 自动设备对象

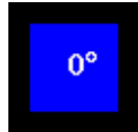
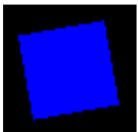
## 9.1 存储设备简介

### 9.1.1 存储设备初窥

要使用存储设备的话首先需要将 GUIConf.h 文件中的宏 GUI\_WINSUPPORT 定义为 1，如下所示：

```
#define GUI_WINSUPPORT (1) //窗口管理
```

使用存储设备的主要原因就是可以防止屏幕闪烁，下面的示例展示了存储设备是如何来消除屏幕闪烁的。

| API 函数                                          | 无存储设备                                                                               | 有存储设备                                                                                 |
|-------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 第 0 步：初始状态                                      |    |    |
| 第 1 步：GUI_Clear()                               |   |   |
| 第 2 步：GUI_DrawPolygon()                         |  |  |
| 第 3 步：GUI_DispString()                          |  |  |
| 第 4 步：<br>GUI_MEMDEV_CopyToLCD()<br>(仅在使用存储设备时) |  |  |

上面的示例展示了一下同一操作在使用和不使用存储设备时的屏幕截图。两种情况下的目的是相同的：旋转工件并分别标志旋转角度(此处为 10 度)。第一种情况下不使用存储设备，屏幕必须清除，然后在新位置重绘多边形，并写入带新标志的字符串。第二种情况下（使用存储设备），在存储器中执行相同的操作，但屏幕此时不更新。仅在调用 GUI\_MEMDEV\_CopyToLCD() 例程时出现更新，并且此更新一次就反映所有操作。

窗口管理器可与存储设备完美搭配，有关窗口管理器会在后面的章节讲解，每个窗口都有一个标记，告诉窗口管理器是否应使用存储设备进行渲染。此标记可以在创建窗口时指定，也可在任何时候进行设置 / 重设。如果为特定窗口设置了存储设备标记，则 WM 在绘制窗口时自动使用存储设备，它会在绘制窗口前创建一个存储设备，然后在绘制操作完成后将其删除。

如果有足够的内存可用，会将整个窗口装入 WM 所创建存储设备的内存中。如果没有足够的内存可用于将整个窗口装入一个存储设备中，则 WM 使用“分段”来绘制窗口，有关“分段”的内容在接下来的小节中说明。用于绘制操作的内存仅在绘制操作期间分配，如果在绘制（重绘）窗口时没有足够的内存可用，则不使用存储设备重绘窗口。

### 9.1.2 内存要求

使用存储设备的时候势必会耗费内存的，而所需要的内存字节数取决于存储设备的色彩深度以及是否需要支持透明。

#### 无透明支持的内存使用情况

无透明支持的存储设备所需内存如表 9.1.1 所示，所需内存取决于色彩深度。

| 存储设备的色彩深度 | 系统色彩深度<br>(LCD_BITSPERPIXEL) | 内存使用                          |
|-----------|------------------------------|-------------------------------|
| 1bpp      | 1bpp                         | 1 字节/像素:<br>(XSIZE+7)/8*YSIZE |
| 8bpp      | 2、4 和 8bpp                   | XSIZE*YSIZE                   |
| 16bpp     | 12 和 16bpp                   | 2 字节/像素<br>XSIZE*YSIZE*2      |
| 32bpp     | 18、24 和 32bpp                | 4 字节/像素:<br>XSIZE*YSIZE*4     |

表 9.1.1 无透明支持需要的内存

加入要创建 X 方向 111 像素、Y 方向 33 像素的存储设备，它还应兼容使用 12bpp 色彩深度的显示器并支持透明。所需的字节数计算如下：

所需字节数 =  $(111 * 2 + (111 + 7) / 8) * 33 = 7788$  字节

#### 有透明支持的内存使用情况

如果存储设备要支持透明性，则每 8 像素需要一个额外字节来进行内部管理，所需内存如下表表 9.1.2 所示。

| 存储设备的色彩深度 | 系统色彩深度<br>(LCD_BITSPERPIXEL) | 内存使用                                              |
|-----------|------------------------------|---------------------------------------------------|
| 1bpp      | 1bpp                         | 2 字节/像素:<br>(XSIZE+7)/8*YSIZE*2                   |
| 8bpp      | 2、4 和 8bpp                   | 1 字节/像素+1 字节/8 像素:<br>(XSIZE+(XSIZE+7)/8)*YSIZE   |
| 16bpp     | 12 和 16bpp                   | 2 字节/像素+1 字节/8 像素:<br>(XSIZE*2+(XSIZE+7)/8)*YSIZE |
| 32bpp     | 18、24 和 32bpp                | 4 字节/像素+1 字节/8 像素:<br>(XSIZE*4+(XSIZE+7)/8)*YSIZE |

表 9.1.2 有透明支持所需的内存

要创建 X 方向 200 像素、Y 方向 50 像素的存储设备，它还应兼容 4bpp 色彩深度的显示器并支持透明。所需的字节数计算如下：

所需字节数 =  $(200 + (200 + 7) / 8) * 50 = 11250$  字节

使用存储设备通常不会显著影响性能。使用存储设备时，驱动的工作就更容易了：只需将位图传输给显示控制器。在具有较慢驱动的系统（例如通过串口连接的显示器），如果使用存储设备性能会更高；在具有快速驱动的系统（如内存映射的显示存储器、GUIDRV\_Lin 以

及其他), 使用存储设备会牺牲某些性能。如果需要“分段”, 则绘制窗口所用的时间将随着分段数增加而增加。存储设备可用的内存越多, 性能也越好。

## 9.2 存储设备基本 API 函数

| 函数                            | 描述                                   |
|-------------------------------|--------------------------------------|
| GUI_MEMDEV_Clear()            | 将存储设备内容标志为未更改。                       |
| GUI_MEMDEV_CopyFromLCD()      | 将 LCD 内容复制到存储设备                      |
| GUI_MEMDEV_CopyToLCD()        | 将存储设备内容复制到 LCD                       |
| GUI_MEMDEV_CopyToLCDAA()      | 复制存储设备内容并抗锯齿处理                       |
| GUI_MEMDEV_CopyToLCDAt()      | 复制存储设备内容到 LCD 的指定位置                  |
| GUI_MEMDEV_Create()           | 创建存储设备(第一步)                          |
| GUI_MEMDEV_CreateEx()         | 创建具有额外创建标记的存储设备                      |
| GUI_MEMDEV_CreateFixed()      | 创建具有给定色彩深度的存储设备                      |
| GUI_MEMDEV_Delete()           | 释放存储设备使用的内存                          |
| GUI_MEMDEV_DrawPerspectiveX() | 将透视变形的给定存储设备绘制到当前选定设备                |
| GUI_MEMDEV_GetDataPtr()       | 返回指向数据直接操作区域的指针                      |
| GUI_MEMDEV_GetXSize()         | 返回存储设备的 X 尺寸(宽度)                     |
| GUI_MEMDEV_GetYSize()         | 返回存储设备的 Y 尺寸(高度)                     |
| GUI_MEMDEV_MarkDirty()        | 将某矩形区域标志为脏污。                         |
| GUI_MEMDEV_ReduceYSize()      | 减小存储设备的 Y 尺寸                         |
| GUI_MEMDEV_Rotate()           | 旋转并缩放存储设备, 并将结果写入使用“最近邻居”法的存储设备      |
| GUI_MEMDEV_RotateHQ()         | 旋转并缩放存储设备, 并将结果写入使用“高品质”法的存储设备       |
| GUI_MEMDEV_Select()           | 选择作为绘制操作目标的存储设备                      |
| GUI_MEMDEV_SetOrg()           | 改变存储设备在 LCD 上的原点                     |
| GUI_MEMDEV_Write()            | 将存储设备的内容写入存储设备                       |
| GUI_MEMDEV_WriteAlpha()       | 将存储设备的内容写入使用 Alpha 混合处理的存储设备         |
| GUI_MEMDEV_WriteAlphaAt()     | 将存储设备的内容写入使用给定位置和 Alpha 混合处理的存储设备    |
| GUI_MEMDEV_WriteAt()          | 将存储设备的内容写入存储设备的给定位置                  |
| GUI_MEMEDV_WriteEx()          | 将存储设备的内容写入使用 Alpha 混合处理和缩放的存储设备      |
| GUI_MEMDEV_WriteExAt()        | 将存储设备的内容写入使用 Alpha 混合处理和缩放的存储设备的给定位置 |
| GUI_SelectLCD()               | 选择作为绘制操作目标的                          |

表 9.2.1 存储设备 API 函数

## 9.2.1 GUI\_MEMDEV\_Clear()

**GUI\_MEMDEV\_Clear()****描述**

将存储设备的整个内容标志为“未更改”。

**原型**

```
void GUI_MEMDEV_Clear(GUI_MEMDEV_Handle hMem);
```

| 参数                | 描述       |
|-------------------|----------|
| <code>hMem</code> | 存储设备的句柄。 |

**其他信息**

使用 `GUI_MEMDEV_CopyToLCD()` 的下一绘制操作仅会写入在 `GUI_MEMDEV_Clear()` 和 `GUI_MEMDEV_CopyToLCD()` 之间修改的字节数。

## 9.2.2 GUI\_MEMDEV\_CopyToLCD()

**GUI\_MEMDEV\_CopyToLCD()****描述**

将存储设备的内容从内存复制到 LCD。

**原型**

```
void GUI_MEMDEV_CopyToLCD(GUI_MEMDEV_Handle hMem);
```

| 参数                | 描述       |
|-------------------|----------|
| <code>hMem</code> | 存储设备的句柄。 |

**其他信息**

请勿在窗口管理器调用的着色回调函数中使用此函数，因为它将禁用窗口管理器的裁剪区域。应使用函数 `GUI_MEMDEV_WriteAt` 代替。

## 9.2.3 GUI\_MEMDEV\_CopyToLCDAt()

**GUI\_MEMDEV\_CopyToLCDAt()****描述**

将存储设备的内容从内存复制到 LCD 的指定位置。

**原型**

```
void GUI_MEMDEV_CopyToLCDAt(GUI_MEMDEV_Handle hMem, int x, int y)
```

| 参数                | 描述       |
|-------------------|----------|
| <code>hMem</code> | 存储设备的句柄。 |
| <code>x</code>    | X 方向位置。  |
| <code>y</code>    | Y 方向位置。  |

## 9.2.4 GUI\_MEMDEV\_Create()

## GUI\_MEMDEV\_CopyToLCDAt()

## 描述

将存储设备的内容从内存复制到 LCD 的指定位置。

## 原型

```
void GUI_MEMDEV_CopyToLCDAt(GUI_MEMDEV_Handle hMem, int x, int y)
```

| 参数   | 描述       |
|------|----------|
| hMem | 存储设备的句柄。 |
| x    | X 方向位置。  |
| y    | Y 方向位置。  |

## 9.2.5 GUI\_MEMDEV\_Delete()

## GUI\_MEMDEV\_Delete()

## 描述

删除存储设备。

## 原型

```
void GUI_MEMDEV_Delete(GUI_MEMDEV_Handle MemDev);
```

| 参数   | 描述       |
|------|----------|
| hMem | 存储设备的句柄。 |

## 返回值

已删除存储设备的句柄。

## 9.2.6 GUI\_MEMDEV\_Select()

## GUI\_MEMDEV\_Select()

## 描述

激活存储设备（如果句柄为 0 则激活 LCD）。

## 原型

```
void GUI_MEMDEV_Select(GUI_MEMDEV_Handle hMem)
```

| 参数   | 描述       |
|------|----------|
| hMem | 存储设备的句柄。 |

## 9.2.7 存储设备演示例程

下面程序延时了一下使用和不使用存储设备的区别，实验完整工程为：[EMWIN 实验 8 存储设备](#)，打开 memdev.c 文件，源码如下：

```
static GUI_RECT Rect = {200, 150, 300, 210};
```

```
//未使用存储设备绘制
```

```
static void _Draw(int Delay)
```

```
{
```

```
    GUI_SetPenSize(5);
```

```

GUI_SetColor(GUI_RED);
GUI_DrawLine(Rect.x0 + 3, Rect.y0 + 3, Rect.x1 - 3, Rect.y1 - 3);
GUI_Delay(Delay);
GUI_SetColor(GUI_GREEN);
GUI_DrawLine(Rect.x0 + 3, Rect.y1 - 3, Rect.x1 - 3, Rect.y0 + 3);
GUI_Delay(Delay);
GUI_SetColor(GUI_WHITE);
GUI_SetFont(&GUI_FontComic24B_ASCII);
GUI_SetTextMode(GUI_TM_TRANS);
GUI_DispStringInRect("Closed", &Rect, GUI_TA_HCENTER | GUI_TA_VCENTER);
GUI_Delay(Delay);
}

//使用存储设备绘制
void _DemoMemDev(void)
{
    GUI_MEMDEV_Handle hMem;
    int i;

    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetColor(GUI_YELLOW);
    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_DispStringHCenterAt("MEMDEV_MemDev - Sample", 400, 0);
    GUI_SetFont(&GUI_Font20_ASCII);
    GUI_DispStringHCenterAt("Shows the advantage of using a\nmemorydevice", 400,50);
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_DispStringHCenterAt("Draws the picture\nwithout a\nmemory device", 250, 100);
    GUI_DispStringHCenterAt("Draws the picture\nusing a\nmemory device", 550, 100);

    //创建存储设备
    hMem = GUI_MEMDEV_Create(Rect.x0, Rect.y0, Rect.x1 - Rect.x0, Rect.y1 - Rect.y0); (1)
    GUI_MEMDEV_Select(hMem); //选择 hMem 作为绘制操作目标的存储设备 (2)
    _Draw(0); //执行要显示的内容复制到存储设备中 (3)
    GUI_MEMDEV_Select(0); //选中 LCD (4)
    while (1)
    {
        for (i = 0; i < 3; i++)
        {
            GUI_Delay(250);
            //清除选中的屏幕
            GUI_ClearRect(LCD_GetXSize() - Rect.x1, Rect.y0, LCD_GetXSize(), Rect.y1);
        }
    }
}

```



```

GUI_Delay(250);
//将存储设备的内容从内存复制到 LCD 的指定位置
GUI_MEMDEV_CopyToLCDAt(hMem, LCD_GetXSize() - Rect.x1, Rect.y0);
}
GUI_Delay(500);

//未使用存储设备
_Draw(400);
GUI_Delay(400);
GUI_ClearRect(0, 150, 799, 210);
}
}

```

(1)、创建一个存储设备，hMem 为创建好的存储设备的句柄。

(2)、选中(1)中创建好的存储设备。

(3)、在(2)中已经选中了存储设备，注意！此时调用函数\_Draw()绘制的画，并不会在 LCD 上显示出来，而是绘制到了存储设备中。

(4)、选中 LCD，此后会有有关 LCD 的操作，将直接体现在 LCD 上，比如如果再次调用函数\_Draw()的话就会在 LCD 上显示出来。

(5)、将存储设备中的内容搬到 LCD 上显示出来。

上面程序中，我们显示通过显示红线、绿线、“Closed”这三个元素来展示使用存储设备的区别，在屏幕的右侧我们使用了存储设备，可以看到红线、绿线、“Closed”这三个元素是同时出来的，左边的是红线、绿线、“Closed”依次出现的。

程序运行效果如图 9.2.1 所示，注意：此程序是在 4.3 寸，800\*480 分辨率上调试的，如果在 2.8 或者 3.5 寸屏幕上会显示不完全，请大家自行稍微修改一下程序。

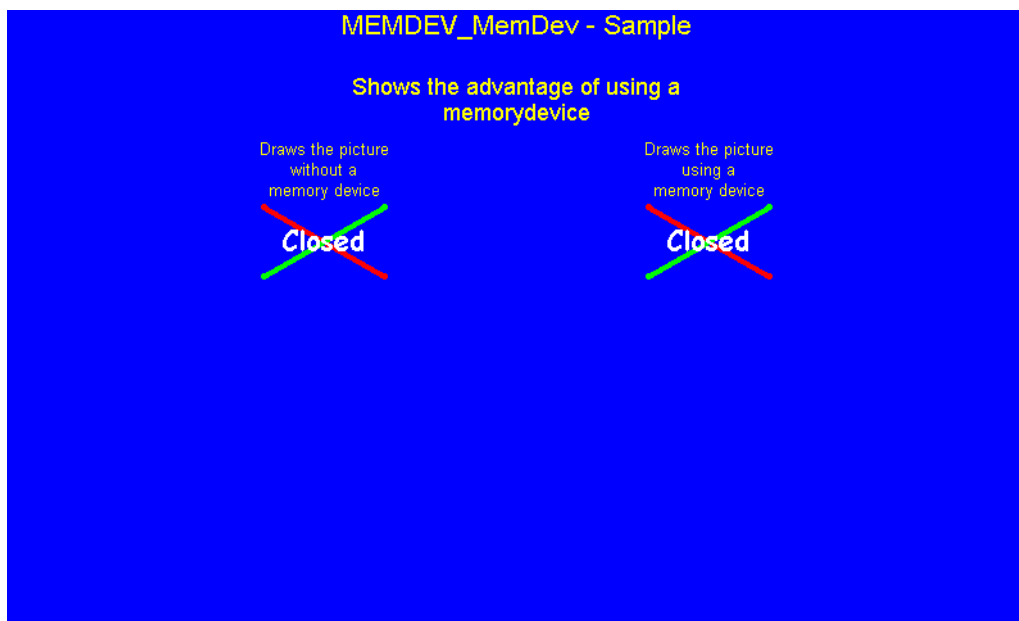


图 9.2.1 存储设备效果图

## 9.3 分段存储设备

### 9.3.1 分段存储简介

前面我们说过，使用存储设备的最大好处就是防止屏幕删除，使用存储设备的时候首先通过执行指定绘制函数来填充存储设备，填充后，将存储设备中的内容绘制到 LCD 中。但是有时候没有用于一次性存储整个输出区域的内存可用，此时分段存储设备就派上用场啦，分段存储设备将绘制区域分成多段，每段包含当前可用内存可包含的最多行。

分段存储设备就一个 API 函数，GUI\_MEMDEV\_Draw()，函数描述如下：

#### GUI\_MEMDEV\_Draw()

##### 描述

基本绘制函数，防止显示器闪烁。

##### 原型

```
int GUI_MEMDEV_Draw( GUI_RECT* pRect,
                     GUI_CALLBACK_VOID_P* pfDraw,
                     void* pData,
                     int NumLines,
                     int Flags)
```

| 参数       | 描述                          |
|----------|-----------------------------|
| pRect    | 所使用 LCD 区域的 GUI_RECT 结构的指针。 |
| pfDraw   | 执行绘制的回调函数的指针。               |
| pData    | 用作回调函数参数的数据结构的指针。           |
| NumLines | 0(推荐)或存储设备的行数               |
| Flags    | 见表 9.3.2                    |

表 9.3.1 函数参数描述表

| 参数 Flags 的允许值                        |                                     |
|--------------------------------------|-------------------------------------|
| GUI_MEMDEV_HASTRANS<br>(recommended) | 默认: 使用透明性标记创建存储设备, 该标记确保正确绘制背景。     |
| GUI_MEMDEV_NOTRANS                   | 创建存储设备, 无透明性。用户必须确保正确绘制背景, 仅应用于优化目的 |

表 9.3.2 Flags 值

##### 返回值

成功时为 0，例程失败则为 1。

##### 其他信息

如果参数 NumLines 为 0，则每段的行数由该函数自动计算。然后函数通过移动存储设备的原点，逐段迭代输出区域。

### 9.3.2 分段存储设备演示例程

以下程序演示了分段存储设备的效果，实验完整工程为：**EMWIN 实验 9 分段存储设备**，打开 banding.c 文件，程序源码如下：

```
#define USE_BANDING_MEMDEV (1) //定义使用分段存储
#define SIZE_OF_ARRAY(Array) (sizeof(Array) / sizeof(Array[0]))
```

//多边形点坐标

```
static const GUI_POINT aPoints[] = {
    {-50, 0},
    {-10, 10},
    { 0, 50},
    { 10, 10},
    { 50, 0},
    { 10,-10},
    { 0,-50},
    {-10,-10}
};
```

//定义一个结构体

```
typedef struct
{
    int XPos_Poly;
    int YPos_Poly;
    int XPos_Text;
    int YPos_Text;
    GUI_POINT aPointsDest[8];
} tDrawItContext;
```

//绘制图形

```
static void _DrawIt(void * pData)
{
    tDrawItContext * pDrawItContext = (tDrawItContext *)pData;
    GUI_Clear();
    GUI_SetFont(&GUI_Font8x8);
    GUI_SetTextMode(GUI_TM_TRANS);

    //绘制背景
    GUI_SetColor(GUI_GREEN);
    GUI_FillRect(pDrawItContext->XPos_Text,
                pDrawItContext->YPos_Text - 25,
                pDrawItContext->XPos_Text + 100,
                pDrawItContext->YPos_Text - 5);

    //绘制多边形
    GUI_SetColor(GUI_BLUE);
    GUI_FillPolygon(pDrawItContext->aPointsDest, SIZE_OF_ARRAY(aPoints), 160, 120);

    //绘制前景
    GUI_SetColor(GUI_RED);
    GUI_FillRect(220 - pDrawItContext->XPos_Text,
```

```

        pDrawItContext->YPos_Text + 5,
        220 - pDrawItContext->XPos_Text + 100,
        pDrawItContext->YPos_Text + 25);
    }

//分段存储演示程序
void _DemoBandingMemdev(void)
{
    tDrawItContext DrawItContext;
    int i, swap=0;
    GUI_SetBkColor(GUI_BLACK);
    GUI_Clear();
    GUI_SetColor(GUI_YELLOW);
    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_DispStringHCenterAt("MEMDEV_Banding - Sample", 160, 5);
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_DispStringHCenterAt("Banding memory device\nwithout flickering", 160, 40);
    DrawItContext.XPos_Poly = 160;
    DrawItContext.YPos_Poly = 120;
    DrawItContext.YPos_Text = 116;
    while (1)
    {
        swap = ~swap;
        for (i = 0; i < 220; i++)
        {
            float angle = i * 3.1415926 / 55;
            DrawItContext.XPos_Text = (swap) ? i : 220 - i;
            //旋转多边形
            GUI_RotatePolygon(DrawItContext.aPointsDest, aPoints,
                              SIZE_OF_ARRAY(aPoints), angle);
            #if USE_BANDING_MEMDEV
            {
                GUI_RECT Rect = {0, 70, 320, 170};
                //使用分段存储绘制
                GUI_MEMDEV_Draw(&Rect, &_DrawIt, &DrawItContext, 0, 0);
            }
            #else
            //如果没有开启分段存储的话就使用普通绘制方式
            _DrawIt((void *)&DrawItContext);
            #endif
        }
    }
}

```

上面程序中有两个重要的函数\_DrawIt()和\_DemoBandingMemdev(), \_DrawIt()函数用来绘制绿色矩形的背景、多边形和红色矩形的前景。\_DemoBandingMemdev()函数里面有一个条件编译, 当宏 USE\_BANDING\_MEMDEV 定义为 1 的话就会使用分段存储设备, 当宏 USE\_BANDING\_MEMDEV 定义为 0 的话就不使用分段存储设备, 大家可以通过修改宏 USE\_BANDING\_MEMDEV 来观察使用和不使用分段存储设备两种情况下的区别。

程序运行效果如图 9.3.1 所示。

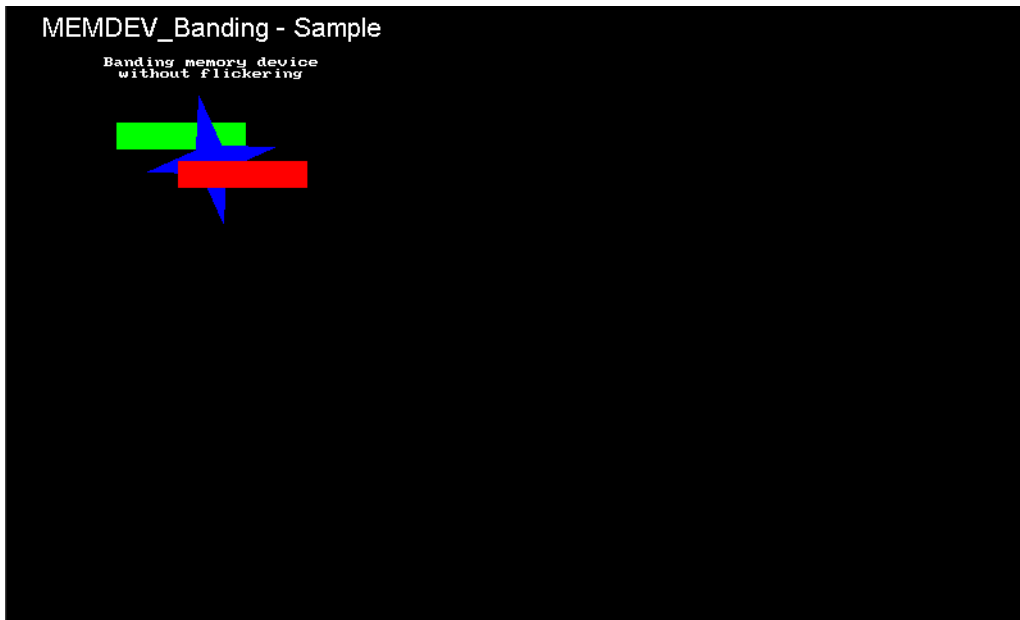


图 9.3.1 分段存储设备运行效果图

## 9.4 自动设备对象

### 9.4.1 自动设备对象简介

有时候我们只需要更新 LCD 的一小部分内容, 如指针, 针对这种情况, emwin 提供了另外一种解决方法——自动设备对象。自动设备对象以分段存储设备为基础, 对于一次仅更新显示器一小部分的应用而言可能更高效。该设备会自动识别包含固定对象的显示器区域, 以及包含必须更新的移动或更改对象的区域。第一次调用绘制函数时, 会绘制所有项目, 每一次后续调用都将仅更新移动或更改对象使用的空间, 实际绘制操作使用分段存储设备, 但仅限在必要空间内。使用自动设备对象的主要优势 (相对于使用分段存储设备而言) 是计算时间减少, 因为它不始终更新整个显示器。

### 9.4.2 自动设备对象 API 函数

自动设备对象 API 函数如表 9.4.1 所示。

| 函数                      | 描述       |
|-------------------------|----------|
| GUI_MEMDEV_CreateAuto() | 创建自动设备对象 |
| GUI_MEMDEV_DeleteAuto() | 删除自动设备对象 |
| GUI_MEMDEV_DrawAuto()   | 使用       |

表 9.4.1 自动设备对象 API 函数

#### 1、GUI\_MEMDEV\_CreateAuto()

## GUI\_MEMDEV\_CreateAuto()

### 描述

创建自动设备对象。

### 原型

```
int GUI_MEMDEV_CreateAuto(GUI_AUTODEV * pAutoDev);
```

| 参数       | 描述                 |
|----------|--------------------|
| pAutoDev | GUI_AUTODEV 对象的指针。 |

### 返回值

目前 0，留待以后使用。

## 2、GUI\_MEMDEV\_DeleteAuto()

### GUI\_MEMDEV\_DeleteAuto()

### 描述

删除自动设备对象。

### 原型

```
void GUI_MEMDEV_DeleteAuto(GUI_AUTODEV * pAutoDev);
```

| 参数       | 描述                 |
|----------|--------------------|
| pAutoDev | GUI_AUTODEV 对象的指针。 |

## 3、GUI\_MEMDEV\_DrawAuto()

### GUI\_MEMDEV\_DrawAuto()

### 描述

使用分段存储设备执行指定绘制例程。

### 原型

```
int GUI_MEMDEV_DrawAuto(GUI_AUTODEV          * pAutoDev,
                        GUI_AUTODEV_INFO      * pAutoDevInfo,
                        GUI_CALLBACK_VOID_P    * pfDraw,
                        void                    * pData);
```

| 参数           | 描述                      |
|--------------|-------------------------|
| pAutoDev     | GUI_AUTODEV 对象的指针。      |
| pAutoDevInfo | GUI_AUTODEV_INFO 对象的指针。 |
| pfDraw       | 要执行的用户定义绘制函数的指针。        |
| pData        | 传递给绘制函数的数据结构体的指针。       |

### 返回值

成功时为 0，例程失败则为 1。

自动设备对象使用方法如下：

1、肯定是要先调用 GUI\_MEMDEV\_CreateAuto()函数创建一个自动设备对象。

2、函数 GUI\_MEMDEV\_DrawAuto()参数 1 就是我们在上一步中创建的自动设备对象，参数 2 是一个结构体 GUI\_AUTODEV\_INFO，这个结构体如下：

```
typedef struct
```

```
{
```

```
    char DrawFixed;
```

```
    char IsMeasurement;
```

```
} GUI_AUTODEV_INFO;
```

如果必须绘制所有项目，则 DrawFixed 设置为 1，如果仅绘制移动或者更改对象时，设置为 0。如果使用此功能的话可以按照如下方式：

//定义一个结构体，此结构体必须包含 GUI\_AUTODEV\_INFO 类型变量，此结构体中也可以添加其他用户需要的参数。

```
typedef struct
```

```
{
```

```
    //结构体 GUI_AUTODEV_INFO 变量,函数 GUI_MEMDEV_DrawAuto()要用到这个变量
```

```
    GUI_AUTODEV_INFO AutoDevInfo;
```

```
    //其他用户所需参数
```

```
    ...
```

```
} PARAM;
```

```
//绘制函数
```

```
static void Draw(void *p)
```

```
{
```

```
    PARAM * pParam = (PARAM*)p;
```

```
    if (pParam->AutoDevInfo.DrawFixed)
```

```
    {
```

```
        //绘制固定的背景
```

```
        ...
```

```
    }
```

```
    //绘制移动的内容，比如鼠标指针等
```

```
    if (pParam->AutoDevInfo.DrawFixed)
```

```
    {
```

```
        //绘制固定的前景
```

```
        ...
```

```
    }
```

```
}
```

```
void main(void)
```

```
{
```

```
    PARAM Param;
```

```
    GUI_AUTODEV AutoDev; //定义一个自动设备对象
```

```
    //设置或者定义其他用户所需内容，比如设置字体，颜色，显示一些提示信息等
```

```
    ...
```

```
    GUI_MEMDEV_CreateAuto(&AutoDev); //创建自动设备对象 AutoDev
```

```
    GUI_MEMDEV_DrawAuto( &AutoDev, //使用分段存储设备执行指定绘制例程
```

```
        &Param.AutoDevInfo,
```

```
        &Draw,
```

```
        &Param);
```

```
    GUI_MEMDEV_DeleteAuto(&AutoDev); //删除自动设备对象
```

```
}
```

在下一小节的自动设备对象演示例程中，大家就可以看到我们基本就是按照这个顺序来写程序的。

### 9.4.3 自动设备对象演示例程

以下程序演示了自动设备对象，实验完整工程为：**EMWIN 实验 10 自动设备对象**，打开 autodev.c 文件，程序源码如下：

```
static const GUI_COLOR ColorsScaleR140[] = {
    //限于篇幅原因，此处省略掉了数组中的内容，至于数组中的具体内容大家可以查阅实验
    //工程源码。
};

//调色板
static const GUI_LOGPALETTE PalScaleR140 = {
    15,    //条目的数量
    0,     //无透明度
    &ColorsScaleR140[0]
};

static const unsigned char acScaleR140[] = {
    //限于篇幅原因，此处省略掉了数组中的内容，至于数组中的具体内容大家可以查阅实验
    //工程源码。
};

//定义位图
static const GUI_BITMAP bmScaleR140 = {
    200,        // XSize
    73,         // YSize
    100,        // BytesPerLine
    4,          // BitsPerPixel
    acScaleR140, // Pointer to picture data (indices)
    &PalScaleR140 // Pointer to palette
};

//多边形的外形
static const GUI_POINT _aNeedle[] = {
    { MAG * ( 0), MAG * ( 0 + 125) },
    { MAG * (-3), MAG * (-15 + 125) },
    { MAG * (-3), MAG * (-65 + 125) },
    { MAG * ( 3), MAG * (-65 + 125) },
    { MAG * ( 3), MAG * (-15 + 125) },
};

//PARAM, 包含绘图所需信息的结构体
```



```

typedef struct
{
    GUI_AUTODEV_INFO AutoDevInfo;    // GUI_AUTODEV_INFO 结构体
    GUI_POINT      aPoints[7];      // 多边形外形数据
    float          Angle;           //角度
} PARAM;

//获取角度,这里只是模拟。在实际使用中这个角度应该是一个实际值
static float _GetAngle(int tDiff) {
    if (tDiff < 15000) {
        return 225 - 0.006 * tDiff ;
    }
    tDiff -= 15000;
    if (tDiff < 7500) {
        return 225 - 90 + 0.012 * tDiff ;
    }
    return 225;
}

//绘图
static void _Draw(void * p)
{
    PARAM * pParam;
    pParam = (PARAM *)p;

    //显示背景
    if (pParam->AutoDevInfo.DrawFixed) {
        GUI_ClearRect (60, 80 + bmScaleR140.YSize, 60 + bmScaleR140.XSize - 1, 180);
        GUI_DrawBitmap(&bmScaleR140, 60, 80);
    }

    //移动指针
    GUI_SetColor(GUI_WHITE);
    GUI_AA_FillPolygon(pParam->aPoints, countof(_aNeedle), MAG * 160, MAG * 220); //绘制
    实心无锯齿多边形

    //显示前景色
    if (pParam->AutoDevInfo.DrawFixed) {
        GUI_SetTextMode(GUI_TM_TRANS);
        GUI_SetColor(GUI_RED);
        GUI_SetFont(&GUI_Font24B_ASCII);
    }
}

```

```

        GUI_DispStringHCenterAt("RPM / 1000", 160, 140);
    }

}

void _DemoScale(void)
{
    GUI_AUTODEV AutoDev;    // 自动设备对象
    PARAM          Param;

    int            Cnt;
    int            tDiff;
    int            t0;

    //显示信息
    GUI_SetBkColor(GUI_BLACK);
    GUI_Clear();
    GUI_SetColor(GUI_WHITE);
    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_DispStringHCenterAt("MEMDEV_AutoDev - Sample", 160, 5);
    GUI_SetFont(&GUI_Font8x16);
    GUI_DispStringHCenterAt("Scale using GUI_AUTODEV-object", 160, 50);

    //使能高分辨率抗锯齿
    GUI_AA_EnableHiRes();
    GUI_AA_SetFactor(MAG);
    while (1)
    {
        t0 = GUI_GetTime();
        GUI_MEMDEV_CreateAuto(&AutoDev); //创建自动设备对象
        for (Cnt = 0; (tDiff = GUI_GetTime() - t0) < 24000; Cnt++)
        {
            Param.Angle = _GetAngle(tDiff)* DEG2RAD;
            //按照指定角度旋转多边形
            GUI_RotatePolygon(Param.aPoints, _aNeedle, countof(_aNeedle), Param.Angle);
            //使用自动设备对象绘制图形
            GUI_MEMDEV_DrawAuto(&AutoDev, &Param.AutoDevInfo, &_Draw, &Param);
        }

        //显示 milliseconds / picture 图片
        GUI_SetColor(GUI_WHITE);
        GUI_SetFont(&GUI_Font8x16);
        GUI_DispStringHCenterAt("Milliseconds / picture:", 160, 200);
    }
}

```

```
GUI_SetTextAlign(GUI_TA_CENTER);
GUI_SetTextMode(GUI_TM_NORMAL);
GUI_DispNextLine();
GUI_GotoX(160);
GUI_DisFloatMin((float)tDiff / (float)Cnt, 2);

//删除自动设备对象
GUI_MEMDEV_DeleteAuto(&AutoDev);
GUI_Delay(3000);
GUI_ClearRect(0, 70, 319, 239);
}
}
```

上面程序基本就是按照我们上一小节中讲到的思路和方法来编写的。一开始我们先定义了一个位图 `bmScaleR140`，因为实验的背景色就是一个汽车的速度表盘，是固定图案，因此这个图案可以按照绘制位图的方式来绘制，通过调用函数 `GUI_DrawBitmap()`来绘制位图，其他部分程序大家就可以按照上一小节讲到的思路分析一下。

程序运行结果如图 9.4.1 所示。

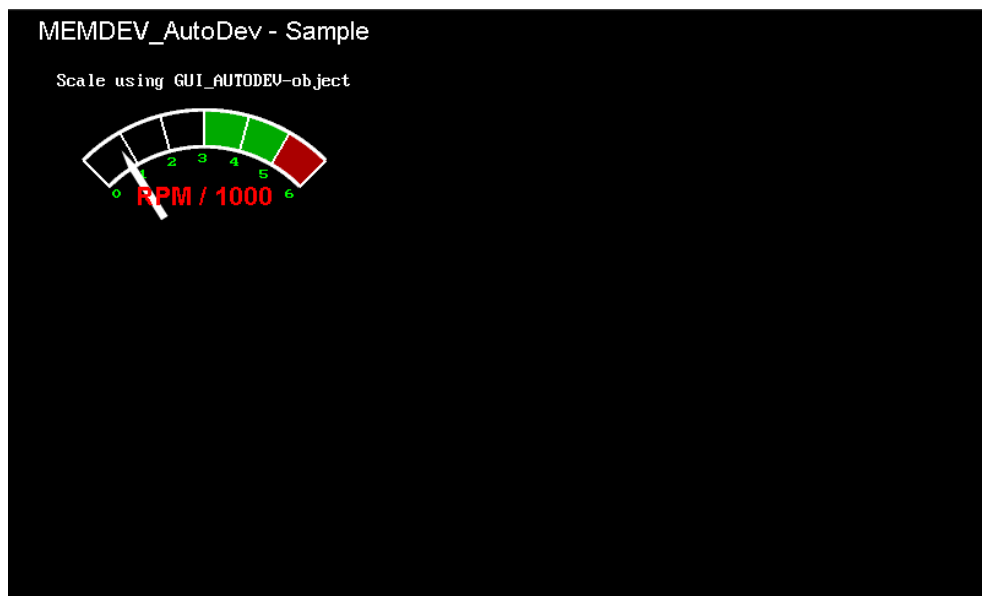


图 9.4.1 自动设备对象运行效果图

## 第十章 窗口管理器

大家在使用电脑的时候对于窗口应该是非常熟悉的了，窗口是用户界面中最重要的部分，它是屏幕上与一个应用程序相对应的矩形区域,包括框架和客户区，是用户与产生该窗口的应用程序之间的可视界面。每当用户开始运行一个应用程序时，应用程序就创建并显示一个窗口；当用户操作窗口中的对象时，程序会作出相应反应。用户通过关闭一个窗口来终止一个程序的运行；通过选择相应的应用程序窗口来选择相应的应用程序。Emwin 同样提供了窗口管理器的功能，本章我们就讲解一下 emWin 的窗口管理，本章分为如下几部分：

- 10.1 窗口术语说明
- 10.2 窗口回调机制
- 10.3 消息
- 10.4 窗口管理器 API 函数
- 10.5 演示例程

## 10.1 窗口术语说明

窗口是有其原点(左上角的 X 和 Y 坐标)以及 X 和 Y 值(分别为宽度和高度)定义的矩形, EmWin 中的窗口有如下特点:

- 1、是矩形的
- 2、具有 Z 位置
- 3、可以隐藏或显示
- 4、可具有有效和无效区域
- 5、可以透明或不透明
- 6、可以具有或不具有回调例程

### 活动窗口

当前正用于绘制操作的窗口称为活动窗口, 不一定就是最上面的窗口。

### 回调例程

回调例程由用户程序定义, 指示在特定事件出现时图形系统调用特定的函数。它们通常用于在窗口内容更改时自动重绘窗口。

### 子窗口/父窗口, 同属窗口

子窗口是相对于其他窗口(称为父窗口)定义的。只要父窗口移动, 其子窗口就会相应移动。子窗口始终完全包含在其父窗口中, 并在必要时会被裁剪。具有相同父窗口的多个子窗口被视为“同属”窗口。

### 客户区

窗口的客户区就是其可用区域。如果窗口包含边框或标题栏, 则客户区是内部的矩形区域。如果没有这种边框, 则客户区的坐标与窗口自身的坐标相同。

### 裁剪, 裁剪区域

裁剪是将输出限制为一个窗口或窗口一部分的过程。窗口的裁剪区域是其可见区域。它是窗口区域减去被更高 Z 轴阶层的同属窗口遮挡的区域, 然后减去没有放入父窗口可见区域的任何部分。

### 坐标

坐标通常是 2 维坐标, 以像素单位表示, 坐标由 2 个值组成。第一个值指定水平分量(也称为 x 坐标), 第二个值指定垂直分量(也称为 y 坐标)。

### 桌面坐标

桌面坐标是桌面窗口的坐标。显示器的左上角位置(原点)为(0,0)。

### 桌面窗口

桌面窗口由窗口管理器自动创建, 并且始终覆盖整个显示区域。它始终是最底层的窗口, 在没有定义其他窗口时, 它是默认(活动)窗口。所有窗口都是桌面窗口的后代窗口(子窗口、孙窗口等)。

### 句柄

创建新窗口后, WM 会分配一个称为句柄的唯一标识符。句柄用于在该特定窗口中执行的其他任何操作。

### 隐藏/显示窗口

隐藏的窗口不可见, 但是仍然存在(有一个句柄)。创建窗口时, 如果不指定创建标记, 默

认情况下是隐藏的。显示窗口使其可见，隐藏窗口则使其不可见。

### 父坐标

父坐标是与父窗口相关的窗口坐标。窗口的左上角位置（原点）为 (0,0)。

### 透明性

具有透明区域的窗口包含不与窗口其余部分一起重绘的区域。这些区域就象其背后窗口“透过”它们显示一样。在此情况下，在有透明区域的窗口之前重绘背后窗口非常重要。WM 自动按正确的顺序进行重绘。

### 有效化/无效化

有效窗口是不需要重绘的完全更新窗口。无效窗口不会反映所有更新，因此需要完全或部分重绘。作出的更改影响了特定窗口时，WM 将该窗口标记为无效。下次窗口重绘时（手动或通过回调例程），将进行验证。

### 窗口坐标

窗口坐标是窗口的坐标。窗口的左上角位置(原点)为 (0,0)。

### Z 位置，底部/顶部

尽管窗口显示在以 X 和 Y 表示的二维屏幕上，但是 WM 也管理所谓的 Z 位置或深度坐标 -- 虚拟的第三维上的位置，该坐标确定从背景到前景的位置。各窗口因此可在其他窗口之上或之下出现。将某窗口设置为底部，会将该窗口置于其所有同属窗口（如果有）的“底部”；设置为顶部，则将其置于其同属窗口的“顶部”。创建窗口时，如果不指定创建标记，默认情况下设置为顶部。

**以上关于窗口的术语非常重要，请大家一定要牢记和理解！**

## 10.2 窗口回调机制

回调机制是 EmWin 的一个重要的特性和功能，WM 可在有或无回调例程（回调函数）时使用，大多情况下最好使用回调。EmWin 为窗口和窗口对象（小工具）提供回调机制的根本概念是一个事件驱动系统。因为在大多数窗口式系统中，其原理是控制流不仅仅是从用户程序到图形系统，还要再通过用户程序提供的回调例程返回用户程序。此机制通常称为“好莱坞原则”（“不要打电话给我们，我们会给你打电话的！”），窗口管理器需要它的主要目的是触发窗口重绘。这与传统编程相反，但是它能利用窗口管理器的无效化逻辑。

### 10.2.1 不使用回调渲染

回调例程不是必须使用的，但是如果这样做，WM 会失去管理窗口重绘（更新）的能力。也可能混合使用，例如让有些窗口使用回调，有些不使用。当然，如果窗口不使用回调机制，则由应用程序负责更新其内容。

### 10.2.2 使用回调渲染

如果要使用回调渲染的话，那么在调用函数 WM\_CreateWindow() 创建窗口之前必须先定义一个回调函数，然后在调用函数 WM\_CreateWindow() 创建窗口的时候将这个回调函数作为参数。

回调函数原型如下：

```
void callback(WM_MESSAGE * pMsg);
```

| 参数   | 描述                      |
|------|-------------------------|
| pMsg | 指向类型 WM_MESSAGE 的数据结构指针 |

回调函数里面具体做什么样的处理取决于接收到的消息（pMsg）的类型，一般情况下我们收到的 pMsg 里面会包含多种类型的消息，那么我们就可以使用 switch 语句来对不同的消息做出不同的处理，pMsg 里面至少会有 WM\_PAINT，下一节中我们就会详细的讲解 pMsg 的消息类型。

### WM\_PAINT 消息

窗口收到 WM\_PAINT 消息时，应重绘自身，将此消息发送到窗口前，WM 确保它已被选定。

#### 非透明窗口（默认!）必须重绘其整个无效区域。

最简单的方式是重新着色窗口的整个区域。WM 的裁剪机制确保了仅重绘无效区域。为了加速绘制过程，仅重绘无效区域非常有用。另外，透明窗口不必重绘整个无效区域；它可让窗口区域部分不受影响。此不受影响的区域会变成透明。WM 发送 WM\_PAINT 消息到透明窗口之前，下面的区域已经重绘（通过发送一条 WM\_PAINT 消息到下面窗口）。

#### 注意：处理 WM\_PAINT 时，不得执行某些操作

处理 WM\_PAINT 消息时，回调例程除了重绘窗口的内容外，不得执行其它操作。处理 WM\_PAINT 事件时，下列函数不能调用：WM\_SelectWindow()、WM\_Paint()、WM\_DeleteWindow() 和 WM\_CreateWindow()。更改窗口属性的其他任何函数也不能调用：WM\_Move()、WM\_Resize()等等。

### 10.2.3 背景窗口重绘和回调

初始化窗口管理器期间，会创建一个包含整个 LCD 区域的窗口作为背景窗口，此窗口的句柄为 WM\_HBKWIN。WM 不会自动重绘背景窗口的区域，因为没有默认的背景颜色，也就是说如果创建了另一个窗口然后将其删除，则删除的窗口仍然可见。需要指定函数 WM\_SetDesktopColor() 以便为重绘背景窗口设置颜色，也可设置回调函数来处理此问题，如果如前所述创建然后删除了一个窗口，则回调例程将触发 WM 来确认背景窗口不再有效，并自动重绘它。

### 10.2.4 无效化

无效化窗口或窗口的一部分会告诉 WM 该窗口的无效区域在下次调用 GUI\_Exec() 或 GUI\_Delay()时应重绘。emWin 的无效化例程不会重绘窗口的无效部分，只管理窗口的无效区域。

#### 窗口的无效区域

对于每个窗口，WM 只使用一个矩形来存储包含整个无效区域的最小矩形。例如，如果左上角的一小部分和右下角的一小部分变为无效，则整个窗口即无效。

#### 使用无效化的原因

使用窗口无效化而非立即重绘每个窗口的优点是只绘制窗口一次，即使其被无效化多次。例如，如果窗口的多个属性需要更改（如背景颜色、字体和窗口大小），与所有属性都更改后重绘一次窗口相比，每个属性更改后立即重绘窗口需要更多的次数。

#### 重绘无效窗口

函数 GUI\_Exec() 重绘所有无效窗口，这通过向每个无效窗口发送一条或多条 WM\_PAINT

消息完成。

### 10.2.5 渲染透明窗口

如果需要绘制透明窗口，则 WM 会自动确保在透明窗口收到 WM\_PAINT 消息前绘制窗口的背景。方法为，在向透明窗口发送 WM\_PAINT 消息前，首先重绘透明窗口无效区域下面的所有窗口区域。为确保窗口管理器能执行透明窗口的重绘，有必要响应 WM\_PAINT 消息并重绘窗口，否则，不能保证透明窗口的外观是正确的。使用透明窗口比使用不透明窗口需要更多消耗 CPU。如果需要考虑性能问题，尝试避免使用透明窗口或许是一个选择。

### 10.2.6 自动使用存储设备

窗口管理器的默认特性是向每个需要重绘的窗口发送一条 WM\_PAINT，这会导致闪变效应，为抑制这些每个窗口的闪变效应，可为重绘操作自动使用存储设备。方法为，在创建窗口时设置标记 WM\_CF\_MEMDEV、使用函数 WM\_SetCreateFlags() 设置默认创建标记，或使用函数 WM\_EnableMemdev()。WM 然后将 WM\_PAINT 消息输出重定向到存储设备中，再复制到显示器中。如果整个窗口的内存不够，会自动使用分段，存储设备只是临时使用，在绘制操作完成后会移除。

**注意：**10.2 节中的这些关于窗口的基础知识非常重要，请大家一定要理解！

## 10.3 消息

上一节讲到在窗口的回调函数中根据不同的消息相应的做出不同的处理，本节我们就讲解一下回调函数中的参数——消息(pMsg)。

### 10.3.1 消息结构

回调例程被调用时，它会收到以其 pMsg 参数指定的消息，此消息实际上是一个 WM\_MESSAGE 数据结构，如下所示：

```
struct WM_MESSAGE
{
    int MsgId;
    WM_HWIN hWin;
    WM_HWIN hWinSrc;
    union
    {
        const void * p;
        int v;
        GUI_COLOR Color;
    } Data;
}
```



结构体 WM\_MESSAGE 各个元素的含义如下表

| 数据类型    | 元素      | 描述   |
|---------|---------|------|
| int     | MsgId   | 消息类型 |
| WM_HWIN | hWin    | 目标窗口 |
| WM_HWIN | hWinSrc | 源窗口  |
| void*   | Data,p  | 数据指针 |
| int     | Data,v  | 数据值  |

表 10.3.1 WM\_MESSAGE 结构体各个元素含义

### 10.3.2 消息类型

Emwin 中窗口的消息类型分为 4 种，见表 10.3.2 所示。

| 消息 ID<br>(MsgId)      | 描述                               |
|-----------------------|----------------------------------|
| 系统定义的消息               |                                  |
| WM_CREATE             | 窗口创建后立即发送，使窗口可以初始化并创建任何子窗口。      |
| WM_DELETE             | 要删除窗口前发送，告诉窗口释放其数据结构(如果有)        |
| WM_GET_ID             | 发送到窗口，请求窗口的 Id                   |
| WM_INIT_DIALOG        | 创建对话框后立即发送到对话框窗口。                |
| WM_KEY                | 按下某个键后发送到当前包含焦点的窗口。              |
| WM_MOVE               | 窗口移动后立即发送到窗口。                    |
| WM_NOTIFY_PARENT      | 告知父窗口，其子窗口中发生了某些改变。              |
| WM_NOTIFY_VIS_CHANGED | 窗口可见性更改时发送到窗口。                   |
| WM_PAINT              | 窗口变为无效并应重绘时，发送到窗口。               |
| WM_POST_PAINT         | 最后一条 WM_PAINT 消息处理后发送到窗口         |
| WM_PRE_PAINT          | 第一条 WM_PAINT 消息发出前发送到窗口          |
| WM_SET_FOCUS          | 获得或失去输入焦点时发送到窗口。                 |
| WM_SET_ID             | 发送到窗口以更改窗口 Id。                   |
| WM_SIZE               | 窗口大小更改后发送到窗口。                    |
| WM_TIMER              | 定时器到期后发送到窗口。                     |
| 指针输入设备(PID)消息         |                                  |
| WM_MOUSEOVER          | 指针输入设备接触到窗口轮廓时发送到窗口，仅在启用鼠标支持时发送。 |
| WM_MOUSEOVER_END      | 指针输入设备已移出窗口轮廓时发送到窗口。仅在启用鼠标支持时发送。 |
| WM_PID_STATE_CHANGED  | 按下状态已更改时，发送到指针输入设备指向的窗口。         |
| WM_TOUCH              | 指针输入设备接触到处于按下状态的窗口轮廓时发送到窗口。      |
| WM_TOUCH_CHILD        | 指针输入设备已接触到子窗口时发送到父窗口。            |

表 10.3.2 消息类型

续表 10.3.2

| 通知代码                            |                                                      |
|---------------------------------|------------------------------------------------------|
| WM_NOTIFICATION_CHILD_DELETED   | 此通知消息将在某窗口被删除前，从该窗口发送到其父窗口。                          |
| WM_NOTIFICATION_CLICKED         | 此通知消息将在点击窗口后发送。                                      |
| WM_NOTIFICATION_LOST_FOCUS      | 此通知消息将在窗口失去焦点时发送。                                    |
| WM_NOTIFICATION_MOVED_OUT       | 此通知消息将在指针移出窗口并点击时发送。                                 |
| WM_NOTIFICATION_RELEASED        | 此通知消息将在被点击的小工具已被释放时发送。                               |
| WM_NOTIFICATION_SCROLL_CHANGED  | 此通知消息将在附加的 SCROLLBAR（滚动条）小工具的滚动位置更改时发送。              |
| WM_NOTIFICATION_SCROLLBAR_ADDED | 此通知消息在将                                              |
| WM_NOTIFICATION_SEL_CHANGED     | 此通知消息将在小工具选择已更改时发送。                                  |
| WM_NOTIFICATION_VALUE_CHANGED   | 此通知消息将在小工具的特定值已更改时发送。                                |
| 用户定义的消息                         |                                                      |
| WM_USER                         | 应用程序可使用 WM_USER 常数来定义私人消息，通常形式为 WM_USER+X，其中 X 为整数值。 |

表 10.3.2 中这些消息在使用窗口小工具的时候会常常被用到，在下一小节中我们就讲解其中一部分消息的使用方法。

## 10.4 窗口管理器 API 函数

表 10.4.1 列出了窗口管理器的 API 函数。

| 例程                       | 描述                  |
|--------------------------|---------------------|
| 基本函数                     |                     |
| WM_Activate()            | 激活窗口管理器。            |
| WM_AttachWindow()        | 将窗口附加到一个新的父窗口。      |
| WM_AttachWindowAt()      | 在给定位置将窗口附加到一个新的父窗口。 |
| WM_BroadcastMessage()    | 发送消息到所有现有窗口。        |
| WM_BringToBottom()       | 将窗口放在其同属窗口后面。       |
| WM_BringToTop()          | 将窗口放在其同属窗口前面。       |
| WM_ClrHasTrans()         | 清除“有透明性”标记          |
| WM_CreateWindow()        | 创建窗口。               |
| WM_CreateWindowAsChild() | 创建子窗口。              |
| WM_Deactivate()          | 去激活窗口管理器。           |
| WM_DefaultProc()         | 句柄消息的默认例程。          |
| WM_DeleteWindow()        | 删除窗口。               |
| WM_DetachWindow()        | 断开窗口与其父窗口的关联。       |
| WM_DisableWindow()       | 将小工具状态设置为禁用。        |

表 10.4.1 窗口管理 API 函数

续表 10.4.1

| 函数                       | 描述                      |
|--------------------------|-------------------------|
| WM_EnableWindow()        | 将窗口状态设置为启用（默认）          |
| WM_Exec()                | 通过执行回调重绘无效窗口（所有工作）      |
| WM_Exec1()               | 通过执行一个回调重绘一个无效窗口（仅一项工作） |
| WM_ForEachDesc()         | 迭代窗口的所有后代。              |
| WM_GetActiveWindow()     | 返回活动窗口的句柄。              |
| WM_GetCallback()         | 返回窗口回调函数的指针。            |
| WM_GetClientRect()       | 返回活动窗口的尺寸。              |
| WM_GetClientRectEx()     | 返回窗口的尺寸。                |
| WM_GetDesktopWindow()    | 返回桌面窗口的窗口句柄             |
| WM_GetDesktopWindowEx()  | 返回指定桌面窗口的窗口句柄           |
| WM_GetDialogItem()       | 返回对话框项目（小工具）的窗口句柄       |
| WM_GetFirstChild()       | 返回窗口第一个子窗口的句柄。          |
| WM_GetFocussedWindow()   | 返回具有输入焦点的窗口的句柄。         |
| WM_GetHasTrans()         | 返回“有透明性”标记的当前值          |
| WM_GetInvalidRect()      | 返回给定窗口的无效矩形。            |
| WM_GetNextSibling()      | 返回指定页面的窗口句柄。            |
| WM_GetOrgX()             | 返回活动窗口 X 方向的原点。         |
| WM_GetOrgY()             | 返回活动窗口 Y 方向的原点。         |
| WM_GetParent()           | 返回窗口的父窗口的句柄。            |
| WM_GetPrevSibling()      | 返回窗口前一同属窗口的句柄。          |
| WM_GetStayOnTop()        | 返回“保持在顶部”标记的当前值         |
| WM_GetUserData()         | 检索窗口的用户数据               |
| WM_GetWindowOrgX()       | 返回窗口 X 方向的原点。           |
| WM_GetWindowOrgY()       | 返回窗口 Y 方向的原点。           |
| WM_GetWindowRect()       | 返回活动窗口的屏幕坐标。            |
| WM_GetWindowRectEx()     | 返回窗口的屏幕坐标。              |
| WM_GetWindowSizeX()      | 返回窗口的水平尺寸（宽度）           |
| WM_GetWindowSizeY()      | 返回窗口的垂直尺寸（高度）           |
| WM_HasCaptured()         | 检查给定窗口是否已捕获鼠标和触摸屏输入。    |
| WM_HasFocus()            | 检查给定窗口是否具有输入焦点。         |
| WM_HideWindow()          | 使窗口不可见。                 |
| WM_InvalidateArea()      | 使显示器的某些部分无效。            |
| WM_InvalidateRect()      | 使部分窗口无效。                |
| WM_InvalidateWindow()    | 使窗口无效。                  |
| WM_IsCompletelyCovered() | 检查窗口是否完全被覆盖。            |
| WM_IsCompletelyVisible() | 检查窗口是否完全可见。             |
| WM_IsEnabled()           | 返回指定页面的启用状态。            |
| WM_IsVisible()           | 返回表示窗口是否可见的值。           |
| WM_IsWindow()            | 确定指定句柄是否是有效的窗口句柄。       |
| WM_MakeModal()           | 将窗口变成“模态”窗口             |

续表 10.4.1

| 函数                        | 描述                                            |
|---------------------------|-----------------------------------------------|
| WM_MoveChildTo()          | 设置窗口在窗口坐标中的位置。                                |
| WM_MoveTo()               | 设置窗口在桌面坐标中的位置。                                |
| WM_MoveWindow()           | 将窗口移动到其他位置。                                   |
| WM_NotifyParent()         | 发送 WM_NOTIFY_PARENT 消息到给定窗口的父窗口               |
| WM_Paint()                | 立即绘制或重绘窗口。                                    |
| WM_PaintWindowAndDescs()  | 立即绘制给定窗口和所有后代窗口。                              |
| WM_ReleaseCapture()       | 停止捕获鼠标和触摸屏输入。                                 |
| WM_ResizeWindow()         | 改变窗口尺寸。                                       |
| WM_SelectWindow()         | 设置要用于绘制操作的活动窗口。                               |
| WM_SendMessage()          | 向窗口发送消息。                                      |
| WM_SendMessageNoPara()    | 向窗口发送无参数的消息。                                  |
| WM_SendToParent()         | 将给定消息发送到给定窗口的父窗口。                             |
| WM_SetDesktopColor()      | 设置桌面窗口颜色。                                     |
| WM_SetDesktopColorEx()    | 设置给定桌面的桌面窗口颜色。                                |
| WM_SetCallback()          | 设置窗口的回调例程。                                    |
| WM_SetCapture()           | 开始捕获鼠标和触摸屏输入。                                 |
| WM_SetCreateFlags()       | 设置在创建新窗口时用作默认的标记。                             |
| WM_SetFocus()             | 将输入焦点设置到指定窗口。                                 |
| WM_SetHasTrans()          | 设置有透明性标记。                                     |
| WM_SetId()                | 将 WM_SET_ID 消息发送到给定窗口                         |
| WM_SetpfPollPID()         | 设置 WM 轮询 PID 而要调用的函数。                         |
| WM_SetSize()              | 设置窗口的尺寸。                                      |
| WM_SetWindowPos()         | 设置窗口的尺寸和位置。                                   |
| WM_SetXSize()             | 设置窗口的新的 X 尺寸。                                 |
| WM_SetYSize()             | 设置窗口的新的 Y 尺寸。                                 |
| WM_SetStayOnTop()         | 设置保持在顶部标记。                                    |
| WM_SetTransState()        | 设置或清除 WM_CF_HASTRANS 和 WM_CF_CONST_OUTLINE 标记 |
| WM_SetUserClipRect()      | 临时缩小裁剪区域。                                     |
| WM_SetUserData()          | 设置给定窗口的用户数据。                                  |
| WM_ShowWindow()           | 使窗口可见。                                        |
| WM_Update()               | 绘制给定窗口的无效部分。                                  |
| WM_UpdateWindowAndDescs() | 绘制给定窗口的无效部分和所有后代窗口的无效部分。                      |
| WM_ValidateRect()         | 有效化部分窗口。                                      |
| WM_ValidateWindow()       | 有效化窗口。                                        |
| 存储设备支持(可选)                |                                               |
| WM_DisableMemdev()        | 禁用存储设备用于重绘。                                   |
| WM_EnableMemdev()         | 启用存储设备用于重绘。                                   |
| 定时器相关                     |                                               |
| WM_CreateTimer()          | 创建向窗口发送 WM_TIMER 消息的定时器                       |

续表 10.4.1

| 函数                   | 描述              |
|----------------------|-----------------|
| WM_DeleteTimer()     | 删除定时器。          |
| WM_GetTimerId()      | 获取给定定时器的 ID     |
| WM_RestartTimer()    | 重启定时器。          |
| 与小工具相关的函数            |                 |
| WM_GetClientWindow() | 返回客户端窗口的句柄。     |
| WM_GetId()           | 返回小工具的 ID。      |
| WM_GetInsideRect()   | 返回活动窗口减去边界后的尺寸。 |
| WM_GetInsideRectEx() | 返回窗口减去边界后的尺寸。   |
| WM_GetScrollPosH()   | 返回窗口水平滚动条的滚动位置。 |
| WM_GetScrollPosV()   | 返回窗口垂直滚动条的滚动位置。 |
| WM_GetScrollState()  | 获取滚动条小工具的状态。    |
| WM_SetScrollPosH()   | 设置窗口水平滚动条的滚动位置。 |
| WM_SetScrollPosV()   | 设置窗口垂直滚动条的滚动位置。 |
| WM_SetScrollState()  | 设置滚动条小工具的状态。    |

表 10.4.1 窗口管理 API 函数

**WM** 的许多函数都有窗口句柄为参数，使用句柄时请遵守以下规则：

- 1、窗口句柄可以为 0。在此情况下，函数立即返回，除非函数的文档另有规定。
- 2、如果窗口句柄为 != 0，则应是一个有效句柄。**WM** 不会检查给定的句柄是否有效。如果向函数提供了无效句柄，则函数会失败，甚至崩溃。

## 10.5 演示例程

### 10.5.1 窗口重绘

以下程序演示了窗口管理的基本使用方法，实验完整工程为：**EMWIN 实验 11 WM 重绘背景**，打开 redraw.c 文件，程序源码如下：

```
//背景窗口回调函数
static void _cbBkWindow(WM_MESSAGE *pMsg) (1)
{
    switch(pMsg->MsgId) //根据不同的消息类型处理
    {
        case WM_PAINT: //WM_PAINT 消息，窗口重绘自身 (2)
            GUI_ClearRect(0,50,319,239);
            break;
        default:
            WM_DefaultProc(pMsg); //此函数可处理未处理的消息
    }
}

//窗口回调函数
static void _cbWindow(WM_MESSAGE *pMsg) (3)
{
```

```

GUI_RECT Rect;
switch(pMsg->MsgId)
{
    case WM_PAINT:
        WM_GetInsideRect(&Rect); //返回活动窗口减去边界后的尺寸
        GUI_SetBkColor(GUI_RED); //红色背景色
        GUI_SetColor(GUI_YELLOW); //黄色
        GUI_ClearRectEx(&Rect); //清除 Rect 区域,既绘制红色区域
        GUI_DrawRectEx(&Rect); //绘制 Rect 区域,既绘制黄色框框
        GUI_SetColor(GUI_BLACK); //黑色
        GUI_SetFont(&GUI_Font16_ASCII); //设置字体
        GUI_DispStringHCenterAt("Foreground Window",75,40);
        break;
    default:
        WM_DefaultProc(pMsg);
}
}

//移动窗口
static void _MoveWindow(const char* pText)
{
    WM_HWIN hWnd;
    int i;
    //创建前景窗口
    hWnd = WM_CreateWindow(10,50,150,100,WM_CF_SHOW,_cbWindow,0); (4)
    GUI_Delay(500);

    //移动前景窗口
    for(i=0;i<40;i++)
    {
        WM_MoveWindow(hWnd,2,2); //移动窗口 (5)
        GUI_Delay(10);
    }

    //删除窗口之前显示文本
    if(pText)
    {
        GUI_DispStringAt(pText,5,50);
        GUI_Delay(2500);
    }

    //删除前景窗口

```

```

WM_DeleteWindow(hWnd); //删除窗口 (6)
WM_Invalidate(WM_HBKWIN); //背景窗口无效 (7)
GUI_Exec(); (8)
}

//窗口演示
void _DemoRedraw(void)
{
    WM_CALLBACK *_cbOldBK;
    GUI_SetBkColor(GUI_BLACK); //设置背景颜色
    GUI_Clear();
    GUI_SetColor(GUI_WHITE); //白色
    GUI_SetFont(&GUI_Font24_ASCII); //设置字体
    GUI_DispStringHCenterAt("WM_Redraw-Sample",160,5);
    GUI_SetFont(&GUI_Font16B_ASCII); //设置字体
    while(1)
    {
        _MoveWindow("Background has not been redraw"); //移动窗口 (9)

        //清除背景
        GUI_ClearRect(0,50,319,239);
        GUI_Delay(1000);

        //为背景窗口设置回调函数
        _cbOldBK = WM_SetCallback(WM_HBKWIN,_cbBkWindow); (10)
        _MoveWindow("Background has been redraw"); //移动窗口
        //窗口 WM_HBKWIN 的回调函数重新设置为上一个回调函数
        WM_SetCallback(WM_HBKWIN,_cbOldBK);
    }
}

```

(1)、前面讲过在初始化窗口管理器期间，会创建一个包含整个 LCD 区域的窗口作为背景窗口，这个窗口的句柄为 WM\_HBKWIN，函数\_cbBkWindow()就是这个背景窗口的回调函数。

(2)、WM\_PAINT 消息，实现重绘背景窗口，这里支持清屏操作。

(3)、新建窗口的回调函数。

(4)、调用函数 WM\_CreateWindow()创建一个窗口 hWnd，窗口的回调函数为\_cbWindow()，窗口属性为可见的。

(5)、移动窗口，每次 X,Y 分别移动 2 个像素。

(6)、删除窗口 hWnd。

(7)、使背景窗口 WM\_HBKWIN 无效，这样下一次调用 GUI\_Exec()函数的时候就会重绘背景窗口 WM\_HBKWIN。

(8)、调用 GUI\_Exec()函数重绘所有无效窗口。

(9)、移动窗口 hWnd，注意此时背景窗口 WM\_HBKWIN 并没有设置回调函数，因此会看到



当 hWnd 移动的时候会有拖影。

(10)、调用函数 WM\_SetCallback() 为背景窗口设置回调函数 \_cbBkWindow(), 这样当移动窗口 hWnd 的时候就会调用 \_cbBkWindow() 重绘背景窗口, 这样就看不到拖影了。

本例程给我们展示了一下如何使用回调函数重绘窗口, 大家一定要学会回调函数重绘窗口的方法, 这个是使用窗口的基础, 程序效果如图 10.5.1 和 10.5.2 所示, 图 10.5.1 展示的是背景窗口 WM\_HBKWIN 未使用回调函数重绘的效果图, 10.5.2 展示的是背景窗口 WM\_HBKWIN 使用了回调函数重绘的效果图。

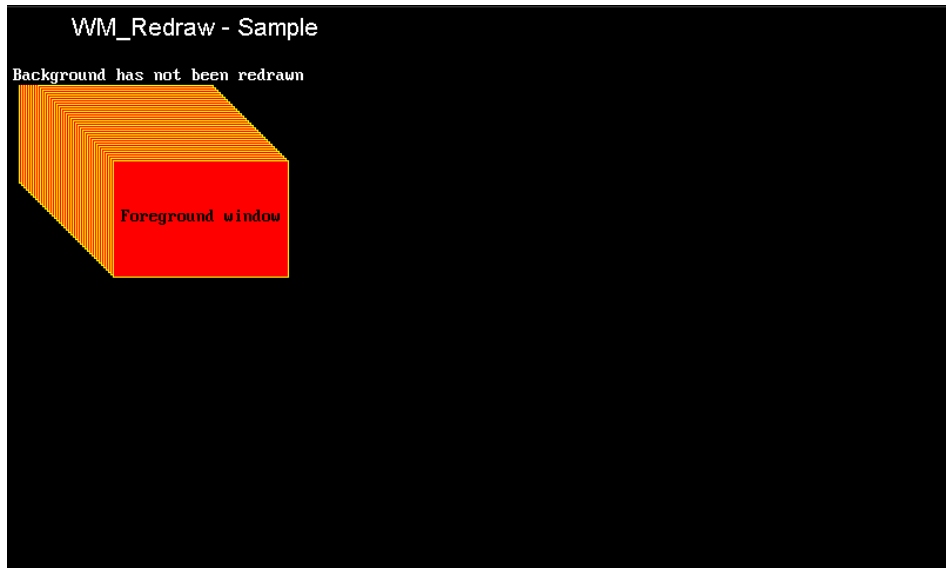


图 10.5.1 窗口 WM\_HBKWIN 未使用回调函数重绘



图 10.5.2 窗口 WM\_HBKWIN 使用回调函数重绘

### 10.5.1 窗口管理 API 函数使用

在 10.4 节中的表 10.4.1 中讲解了很多关于窗口的 API 函数, 本小节的程序就演示一下部分 API 函数的使用, 实验完整工程为: **EMWIN 实验 12 WM 函数使用**, 打开 sample.c 文件, 由于本例程代码比较多, 下面我们只是截取了其中几个比较重要的给大家讲解一下。

```
#define MSG_CHANGE_TEXT (WM_USER + 0) //用户自定义的消息类型 (1)
```



```
//窗口句柄
```

```
static WM_HWIN _hWindow1; //窗口 1
static WM_HWIN _hWindow2; //窗口 2
static WM_HWIN _hChild;   //子窗口
```

```
//向背景窗口发送一个消息并使背景窗口无效，
//背景窗口的回调函数就会显示发送的消息
```

```
static void _ChangeInfoText(char *pStr) (2)
```

```
{
    WM_MESSAGE Message;
    Message.MsgId = MSG_CHANGE_TEXT;
    Message.Data.p = pStr;
    WM_SendMessage(WM_HBKWIN,&Message); //向背景窗口发送消息 (3)
    WM_InvalidateWindow(WM_HBKWIN);     //使背景窗口失效 (4)
}
```

```
//背景窗口的回调函数
```

```
static void _cbBkWindow(WM_MESSAGE *pMsg)
```

```
{
    switch(pMsg->MsgId)
    {
        case MSG_CHANGE_TEXT: (5)
            strcpy(_acInfoText,(char const *)pMsg->Data.p);
        case WM_PAINT: (6)
            GUI_SetBkColor(GUI_BLACK);
            GUI_Clear();
            GUI_SetColor(GUI_WHITE);
            GUI_SetFont(&GUI_Font24_ASCII);
            GUI_DispStringHCenterAt("WindowManager - Sample",160,5);
            GUI_SetFont(&GUI_Font8x16);
            GUI_DispStringAt(_acInfoText,5,40);
            break;
        default:
            WM_DefaultProc(pMsg);
    }
}
```

```
//窗口 1 回调函数
```

```
static void _cbWindow1(WM_MESSAGE *pMsg)
```

```
{
    GUI_RECT Rect;
    int x;
    int y;
```

```
switch(pMsg->MsgId)
{
    case WM_PAINT:
        WM_GetInsideRect(&Rect); //返回客户区的坐标
        GUI_SetBkColor(_WindowColor1);
        GUI_SetColor(_FrameColor1);
        GUI_ClearRectEx(&Rect);      //填充一个矩形区域
        GUI_DrawRectEx(&Rect);      //绘制一个矩形
        GUI_SetColor(GUI_WHITE);
        GUI_SetFont(&GUI_Font24_ASCII);
        x=WM_GetWindowSizeX(pMsg->hWin); //获取窗口的 X 轴大小
        y=WM_GetWindowSizeY(pMsg->hWin); //获取窗口的 Y 轴大小
        GUI_DispStringHCenterAt("Window 1",x/2,(y/2)-12);
        break;
    default:
        WM_DefaultProc(pMsg);
}
}

//窗口 2 回调函数
static void _cbWindow2(WM_MESSAGE *pMsg)
{
    GUI_RECT Rect;
    int x;
    int y;

    switch(pMsg->MsgId)
    {
        case WM_PAINT:
            WM_GetInsideRect(&Rect); //返回客户区的坐标
            GUI_SetBkColor(_WindowColor2);
            GUI_SetColor(_FrameColor2);
            GUI_ClearRectEx(&Rect);      //填充一个矩形区域
            GUI_DrawRectEx(&Rect);      //绘制一个矩形
            GUI_SetColor(GUI_WHITE);
            GUI_SetFont(&GUI_Font24_ASCII);
            x = WM_GetWindowSizeX(pMsg->hWin); //获取窗口的 X 轴大小
            y = WM_GetWindowSizeY(pMsg->hWin); //获取窗口的 Y 轴大小
            GUI_DispStringHCenterAt("Window 2", x / 2, (y / 4) - 12);
            break;
```

```
        default:
            WM_DefaultProc(pMsg);
    }
}

//子窗口的回调函数
static void _cbChild(WM_MESSAGE *pMsg)
{
    GUI_RECT Rect;
    int x;
    int y;

    switch(pMsg->MsgId)
    {
        case WM_PAINT:
            WM_GetInsideRect(&Rect);
            GUI_SetBkColor(_ChildColor);
            GUI_SetColor(_ChildFrame);
            GUI_ClearRectEx(&Rect);
            GUI_DrawRectEx(&Rect);
            GUI_SetColor(GUI_RED);
            GUI_SetFont(&GUI_Font24_ASCII);
            x = WM_GetWindowSizeX(pMsg->hWin);
            y = WM_GetWindowSizeY(pMsg->hWin);
            GUI_DispStringHCenterAt("Child window", x / 2, (y / 2) - 12);
            break;
        default:
            WM_DefaultProc(pMsg);
    }
}

//窗口 1 的另一个回调函数
static void _cbDemoCallback1(WM_MESSAGE *pMsg)
{
    int x,y;
    switch(pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_GREEN);
            GUI_Clear();
            GUI_SetColor(GUI_RED);
            GUI_SetFont(&GUI_FontComic18B_1);
            x=WM_GetWindowSizeX(pMsg->hWin);
```

```

        y = WM_GetWindowSizeY(pMsg->hWin);
        GUI_DispStringHCenterAt("Window 1\nanother Callback", x / 2, (y / 2) - 18);
        break;
    default:
        WM_DefaultProc(pMsg);
    }
}

//窗口 2 的另一个回调函数
static void _cbDemoCallback2(WM_MESSAGE *pMsg)
{
    int x,y;
    switch(pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_MAGENTA);
            GUI_Clear();
            GUI_SetColor(GUI_YELLOW);
            GUI_SetFont(&GUI_FontComic18B_1);
            x = WM_GetWindowSizeX(pMsg->hWin);
            y = WM_GetWindowSizeY(pMsg->hWin);
            GUI_DispStringHCenterAt("Window 2\nanother Callback", x / 2, (y / 4) - 18);
            break;
        default:
            WM_DefaultProc(pMsg);
    }
}
}

```

(1)、用户自定义了一个消息 MSG\_CHANGE\_TEXT。

(2)、向背景窗口 WM\_HBKWIN 发送一条消息。

(3)、这里调用的是函数 WM\_SendMessage()向 WM\_HBKWIN 发送消息 Message，此函数的第二个参数为 WM\_MESSAGE 类型的，所以需要打包一个 WM\_MESSAGE 的变量。另一个向指定窗口发送消息的函数是 WM\_SendMessageNoPara()，此函数的第二个参数为 int 类型，我们在使用 WM\_SendMessageNoPara()函数发送消息的时候可以直接发送消息 ID。

(4)、调用函数 WM\_InvalidateWindow()使背景窗口 WM\_HBKWIN 无效，只有使窗口无效化了，才能在下次调用 GUI\_Exec()函数或者包含 GUI\_Exec()的其他函数的时候才会重绘窗口。

(5)、MSG\_CHANGE\_TEXT 类型的消息，将消息中的数据 Data.p 拷贝到数组\_acInfoText 中去

(6)、WM\_PAINT 消息。

这个演示例程展示了窗口 API 函数的使用，大家一定要仔细阅读，观察不同 API 函数的作用和效果，演示例程效果如图 10.5.1 所示。注意，此例程可以在 2.8, 3.5, 4.3, 7 寸屏幕上显示。

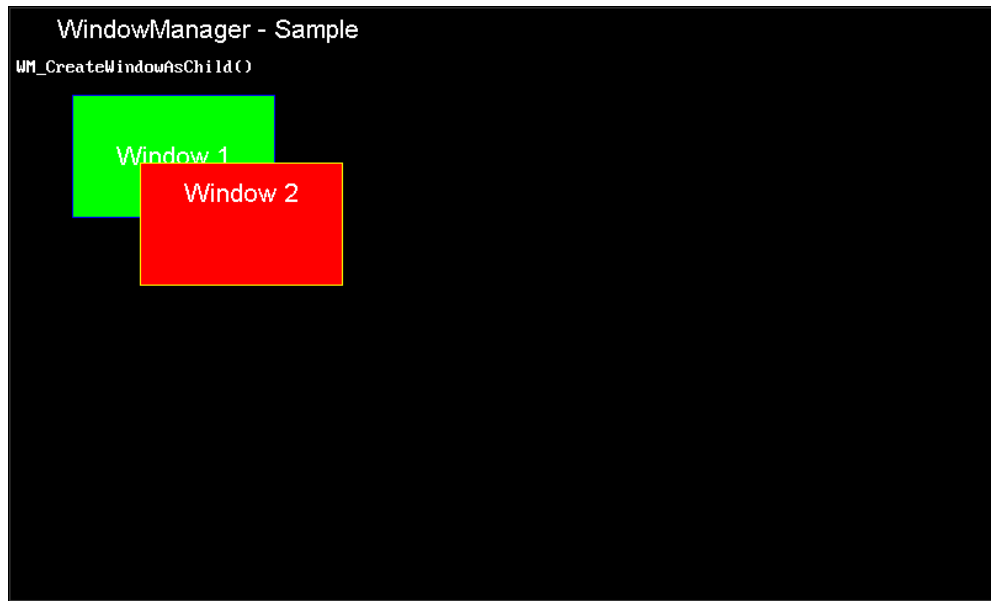


图 10.5.3 窗口 API 函数展示效果图

### 10.5.3 窗口裁剪

下面的例程也是 segger 官方的例程，为 WM\_LateClipping.c，这个例程中展示了窗口的裁剪，在这个例程中使用到了 BUTTON 和 FRAMEWIN 两个控件，我们可以先不用管这两个东西，后面章节中我们会讲到这两个控件的使用方法，实验完整工程为：[EMWIN 实验 13 WM 裁剪](#)。

```
static WM_HWIN _hWin1;
static WM_HWIN _hWin2;
static WM_HWIN _hBut1;
static WM_HWIN _hBut2;
static int      _PaintCount1;
static int      _PaintCount2;

static GUI_COLOR _aColors[]={
GUI_BLUE, GUI_YELLOW, GUI_RED, GUI_LIGHTCYAN, GUI_DARKRED, GUI_DARKRED
};

//背景窗口回调函数
static void _cbBkWin(WM_MESSAGE *pMsg)
{
    switch(pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_BLACK);
            GUI_Clear();
            GUI_SetColor(0x0060FF);
            GUI_DispStringAt("PaintCount (Early):",0,0);
            GUI_DispDecAt(_PaintCount1,120,0,5);
    }
}
```

(1)

```

    GUI_SetColor(0x00FFC0);
    GUI_DispStringAt("PaintCount (Late):",0,12);
    GUI_DispDecAt(_PaintCount2,120,12,5);
    break;
case WM_NOTIFY_PARENT:
    if(pMsg->Data.v == WM_NOTIFICATION_RELEASED) //按钮被释放时 (3)
    {
        if(pMsg->hWinSrc == _hBut1) //button1 被释放
        {
            WM_InvalidateWindow(_hWin1); //窗口 1 失效 (4)
            WM_InvalidateWindow(_hWin2); //窗口 2 失效 (5)
        }else if(pMsg->hWinSrc == _hBut2) //button2 被释放
        {
            _PaintCount1 = 0; (6)
            _PaintCount2 = 0; (7)
            WM_InvalidateWindow(pMsg->hWin); //主窗口失效 (8)
        }
    }
    break;
default:
    WM_DefaultProc(pMsg);
}
}

//顶层窗口回调函数
static void _cbTop(WM_MESSAGE *pMsg) (9)
{
    switch(pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_MAGENTA);
            GUI_Clear();
            break;
        default:
            WM_DefaultProc(pMsg);
    }
}

//框架窗口 1 回调函数
static void _cbFrameWin1(WM_MESSAGE *pMsg) (10)
{
    switch(pMsg->MsgId)

```

```

{
    case WM_PAINT:
        GUI_SetBkColor(_aColors[_PaintCount1 % 6]); //设置背景颜色
        GUI_Clear();
        GUI_SetColor(0x0060FF);
        GUI_FillCircle(25,25,15);
        GUI_SetColor(GUI_BLACK);
        GUI_DrawCircle(25,25,15);
        _PaintCount1++;
        WM_InvalidateWindow(WM_HBKWIN); //背景窗口失效
        break;
    default:
        WM_DefaultProc(pMsg);
}
}

```

//框架窗口 2 回调函数

static void \_cbFrameWin2(WM\_MESSAGE \*pMsg) (11)

```

{
    switch(pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(_aColors[_PaintCount2 % 6]);
            GUI_Clear();
            GUI_SetColor(0x00FFC0);
            GUI_FillCircle(25,25,15);
            GUI_SetColor(GUI_BLACK);
            GUI_DrawCircle(25,25,15);
            _PaintCount2++;
            WM_InvalidateWindow(WM_HBKWIN);
            break;
        default:
            WM_DefaultProc(pMsg);
    }
}

```

//测试函数

void \_ShowDemo(void) (12)

```

{
    WM_HWIN hWin0;
    WM_HWIN hWin1;
    WM_HWIN hWin2;
    WM_HWIN hFrame1;
}

```

```

WM_HWIN hFrame2;
WM_HWIN hClient1;
WM_HWIN hClient2;

WM_SetCallback(WM_HBKWIN,_cbBkWin);
//创建框架窗口 1,可移动
hFrame1 = FRAMEWIN_CreateEx(10,30,140,140,0,WM_CF_SHOW,\
    FRAMEWIN_CF_MOVEABLE,0,"Early Clipping",_cbFrameWin1);
//创建框架窗口 2,可移动
hFrame2 = FRAMEWIN_CreateEx(170,30,140,140,0,WM_CF_SHOW,\
    FRAMEWIN_CF_MOVEABLE,0,"Late Clipping",_cbFrameWin2);
hClient1= WM_GetClientWindow(hFrame1);    //返回框架 1 的客户端窗口句柄
hClient2= WM_GetClientWindow(hFrame2);    //返回框架 2 的客户端窗口句柄
_hWin1 = WM_CreateWindowAsChild(0,0,\
    WM_GetWindowSizeX(hClient1),WM_GetWindowSizeY(hClient2),\
    hClient1,WM_CF_SHOW,_cbFrameWin1,0);
_hWin2 = WM_CreateWindowAsChild(0,0,\
    WM_GetWindowSizeX(hClient2),WM_GetWindowSizeY(hClient2),\
    hClient2,WM_CF_SHOW|WM_CF_LATE_CLIP,_cbFrameWin2,0);
_hBut1 = BUTTON_CreateEx(10,210,140,20,0,WM_CF_SHOW,0,1);    //创建按钮 1
_hBut2 = BUTTON_CreateEx(170,210,140,20,0,WM_CF_SHOW,0,2);    //创建按钮 2
hWin0 = FRAMEWIN_CreateEx(60,80,40,40, 0, WM_CF_SHOW|\
    WM_CF_STAYONTOP,FRAMEWIN_CF_MOVEABLE,0,"Top 0",_cbTop);
hWin1 = FRAMEWIN_CreateEx(220,80,40,40, 0, WM_CF_SHOW|\
    WM_CF_STAYONTOP,FRAMEWIN_CF_MOVEABLE,0,"Top 1",_cbTop);
hWin2 = FRAMEWIN_CreateEx(140,170,40,40,0, WM_CF_SHOW|\
    WM_CF_STAYONTOP,FRAMEWIN_CF_MOVEABLE,0,"Top 2",_cbTop);

FRAMEWIN_SetResizable(hWin0,1);    //窗口设置为可缩放
FRAMEWIN_SetResizable(hWin1,1);    //窗口设置为可缩放
FRAMEWIN_SetResizable(hWin2,1);    //窗口设置为可缩放

BUTTON_SetText(_hBut1,"Invalidate");    //设置按钮的名字
BUTTON_SetText(_hBut2,"Reset counts");
while(1)
{
    GUI_Delay(50);
}
}

```

(1)、背景窗口 WM\_HBKWIN 的回调函数。

(2)、消息 WM\_NOTIFY\_PARENT，小工具与父窗口进行通信的时候就是用此消息，两个按钮\_hBut1 和\_hBut2 作为背景窗口 WM\_HBKWIN 的子窗口，因此当这两个按钮被按下或释放的时候就会向 WM\_HBKWIN 发送消息 WM\_NOTIFY\_PARENT。



(3)、通过判断 pMsg 的 Data.v 来判断是什么样的事件，这里如果是 WM\_NOTIFICATION\_RELEASED 事件，也就是按键被释放的话就执行相应的操作。

(4)、窗口 1 无效化，下一次调用 GUI\_Exec()的话就重绘窗口 1。

(5)、窗口 2 无效化，下一次调用 GUI\_Exec()的话就重绘窗口 2。

(6)、窗口 1 的颜色索引清零。

(7)、窗口 2 的颜色索引清零。

(8)、背景窗口 WM\_HBKWIN 无效化，下一次调用 GUI\_Exec()的话就重绘背景窗口 WM\_HBKWIN。

(9)、3 个顶层窗口的回调函数，这三个窗口分别为:hWin0, hWin1, hWin2。

(10)、框架窗口 1 的回调函数。

(11)、框架窗口 2 的回调函数。

(12)、在函数\_ShowDemo()中设置了背景窗口 WM\_HBKWIN 的回调函数，创建了 2 个框架窗口: hFrame1 和 hFrame2; 两个窗口\_hWin1 和\_hWin2 分别作为框架窗口 hFrame1 和 hFrame2 的子窗口; 两个按键\_hBut1 和\_hBut2; 3 个窗口 hWin0、hWin1 和 hWin2，这三个窗口始终位于最顶层。

这个例程我们重点学习回调函数的使用和消息的处理!关于 FRAMEWIN 和 BUTTON 控件的创建、使用可以先不用管，例程运行结果如图 10.5.4 所示。

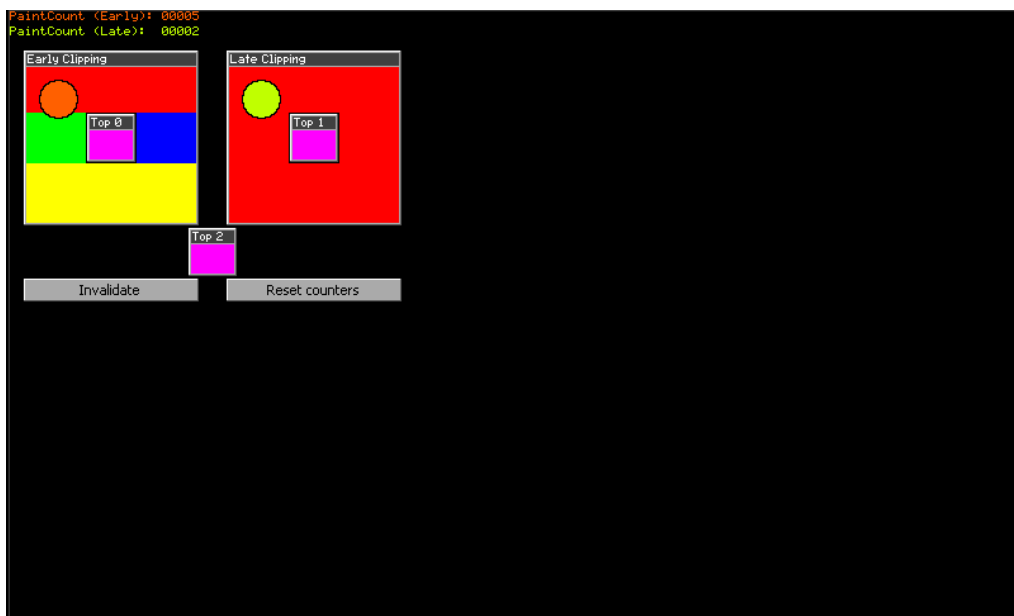


图 10.5.4 例程运行效果图

## 第十一章 GUIBulider 的使用

本章我们介绍一个使用 emwin 做界面的时候常使用到的一款“神器”：GUIBulider，使用这款软件就不需要我们自己用 C 语言编写界面了，可以在 GUIBulider 中设计好界面，然后导出 C 程序，十分的方便。GUIBulider 是 emwin 官方出的软件，每个版本的 emwin 都有其对应版本的 GUIBulider 软件，控件非常齐全，熟练使用 GUIBulider 在使用 emWin 设计 GUI 界面的时候会起到事半功倍的效果，本章分为如下几部分：

11.1 GUIBulider 简介

11.2 GUIBulider 使用步骤

11.3 GUIBulider 输出

11.4 修改 C 文件

11.5 LCD 显示结果

## 11.1 GUIBulider 简介

我们在 segger 官网或者 ST 官网下载 emwin 或 STemWin 的时候里面就已经包含了 GUIBulider 这个软件，这里以 STemWin 为例，按照如下路径打开 GUIBulider，STemWin\_Library\_V1.1.2->Libraries->STemWinLibrary522->Software->GUIBulider.exe，位置如图 11.1.1 所示，注意路径

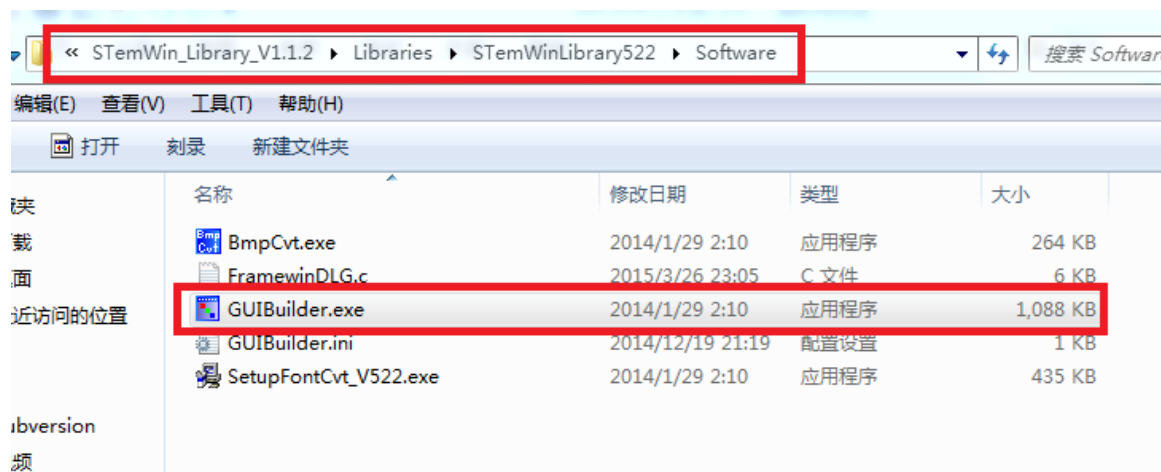
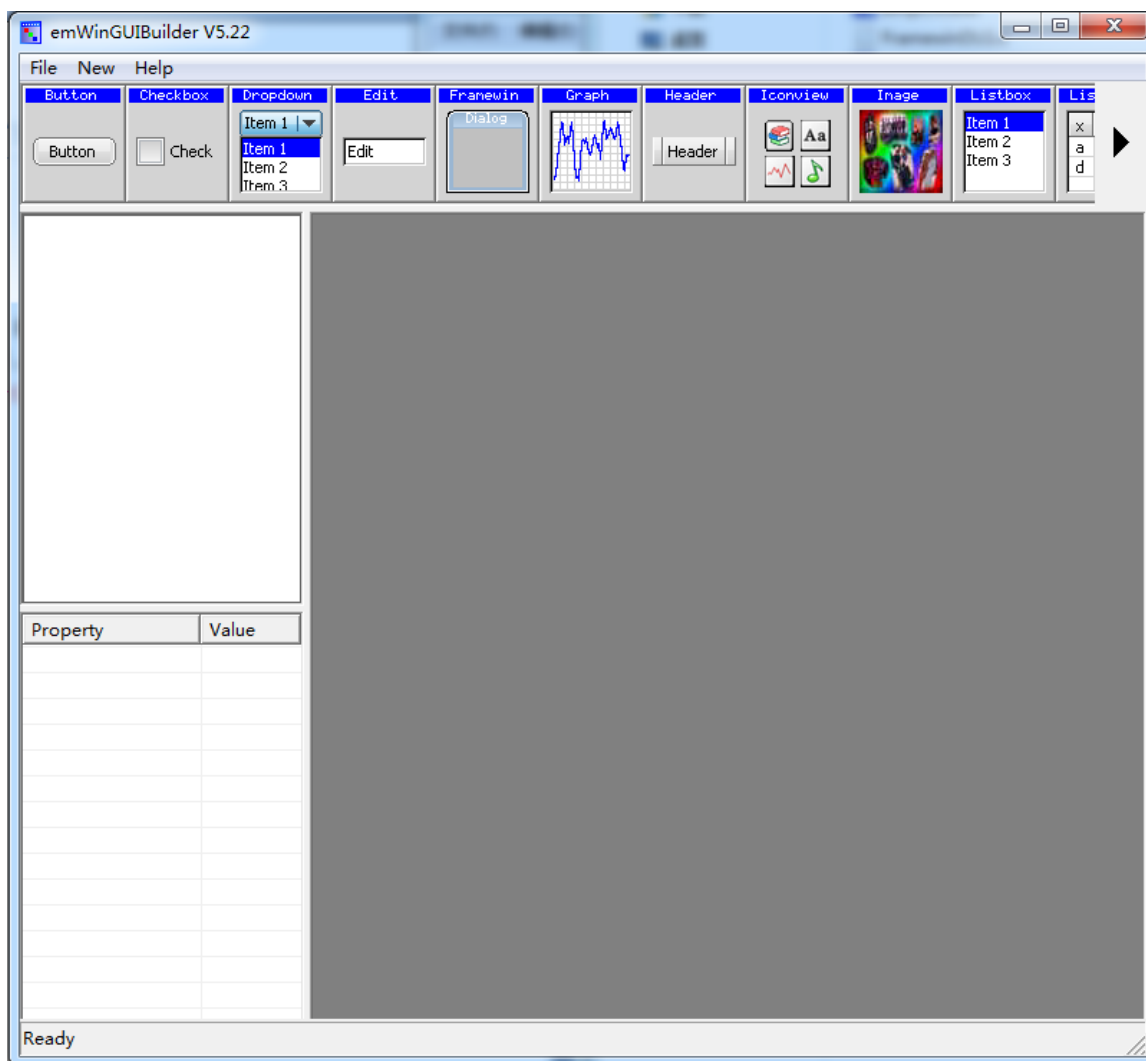


图 11.1.1 GUIBuilder 位置

打开以后如图 11.1.2 所示。



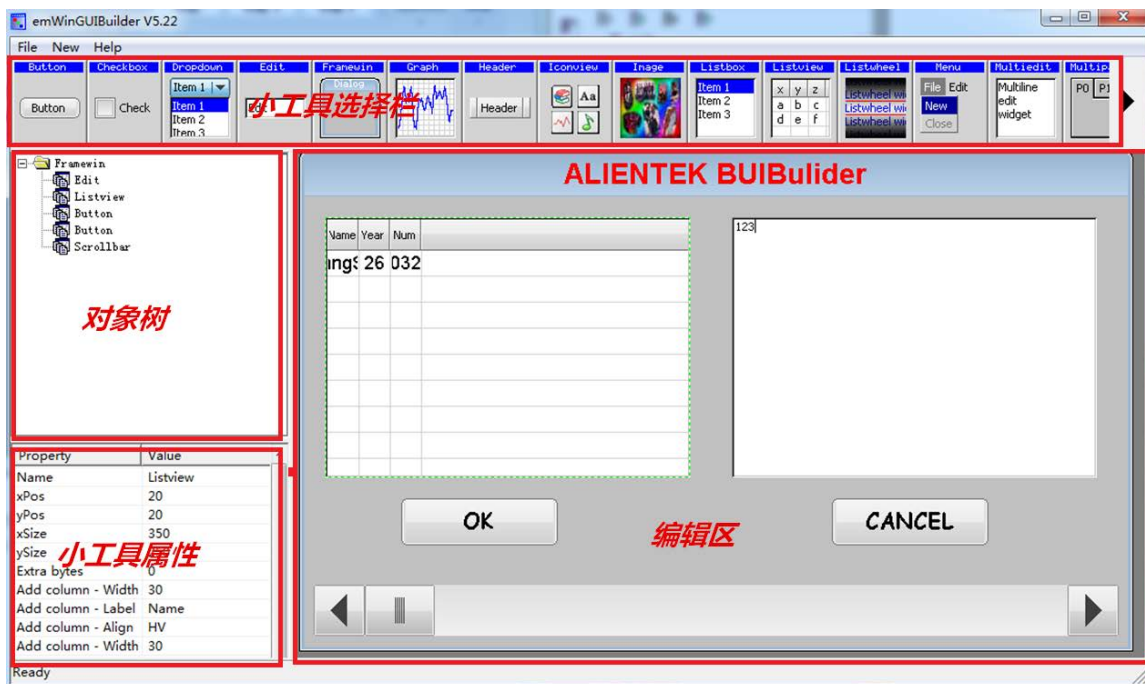


图 11.1.3 GUIBulider 用于界面各要素

### 小工具选择栏

该选择栏包含 GUIBulider 所有可用的小工具，只要单击所需小工具上的选择栏或将其拖入编辑器区域，即可进行添加。

### 对象树

该区域显示所有当前加载的对话框及其子小工具，只要单击相应条目，即可选择小工具。

### 小工具属性

它显示每个小工具的属性，并可以用于编辑属性。

### 编辑器

编辑器窗口显示当前所选的对话框，他可用于放置对话框及其小工具，并调整他们的大小。

## 11.2 GUIBulider 使用步骤

1、创建对话框，每个对话框都需要一个有效的父小工具，目前能够做父小工具有两个:Framework(框架窗口小工具)和 Window(窗口小工具)，更多情况下我们都是创建的 Framework 小工具，点击如图 11.2.1 中的控件，创建一个 Framework。

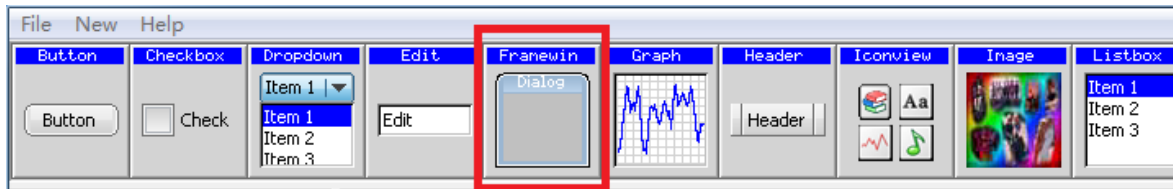


图 11.2.1 选择 Framework

添加完成 Framework 以后 GUIBulider 如图 11.2.1 所示。

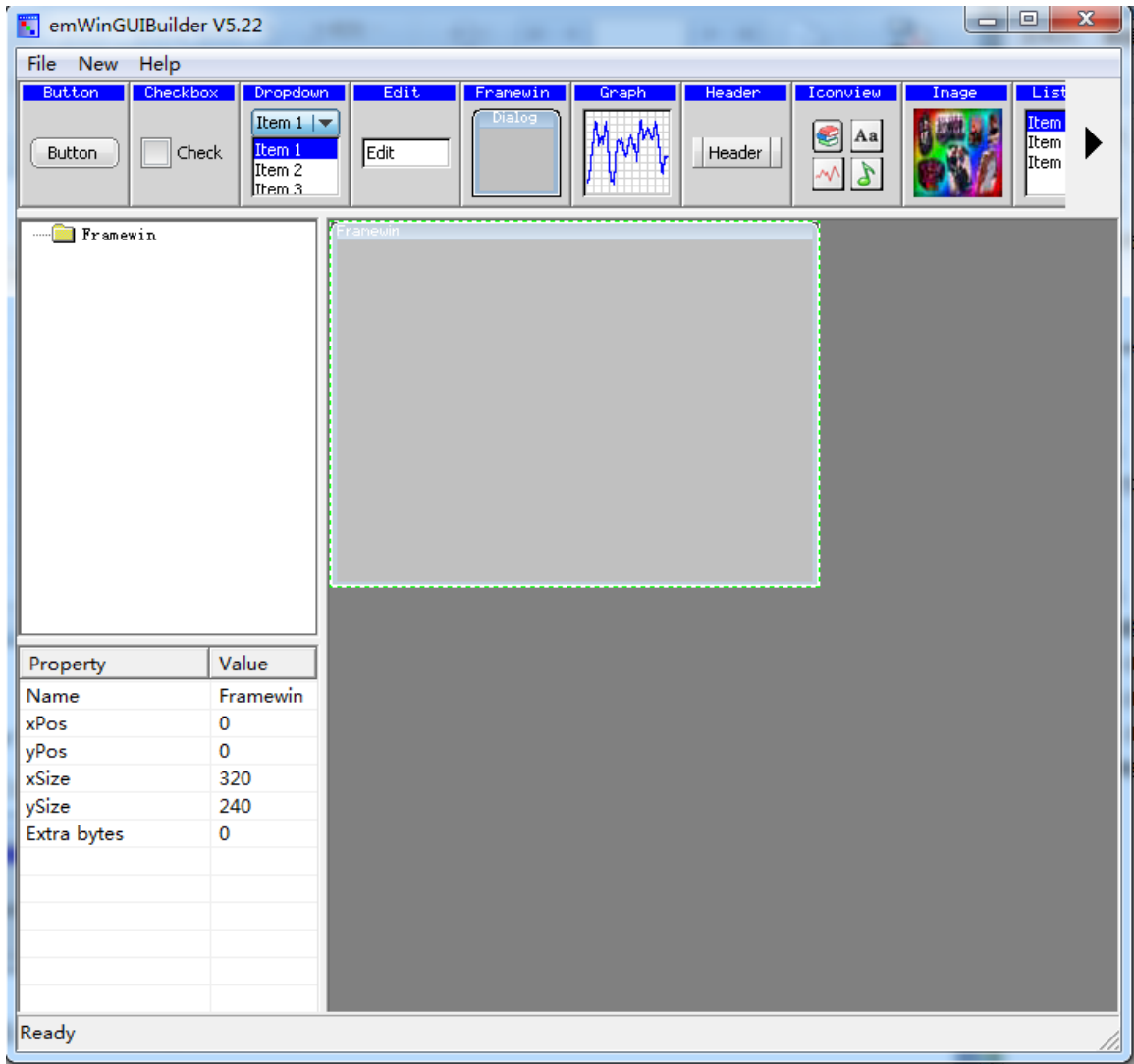


图 11.2.1 添加完成 Framewin

2、设置 Framewin 参数，参数如下：

(1)、Framewin 控件坐标为(0,0)，即空间左上角在 LCD 上的坐标为(0,0)。

(2)、控件 X 轴大小为 800，Y 轴大小为 480(注意此处是按照 800\*480 分辨率来设置的，大家如果使用的其他分辨率的屏请按照自己的屏幕尺寸设置控件大小，不要超过屏幕尺寸就可以。这里设置为 800\*480，将来显示控件的时候就会全屏显示)。

(3)、设置标题栏高度为 40，标题栏名字设置为“ALIENTEK GUIBulider”，字体设置为“GUI\_FONT\_32B\_ASCII”，名字的对齐方式为水平和垂直居中，字体为红色。

OK，现在我们就按照上面的参数要求设置 Framewin 控件，首先设置控件在 LCD 中的位置，如图 11.2.2 所示，也可以通过用鼠标点住控件，通过移动鼠标来改变控件的位置。

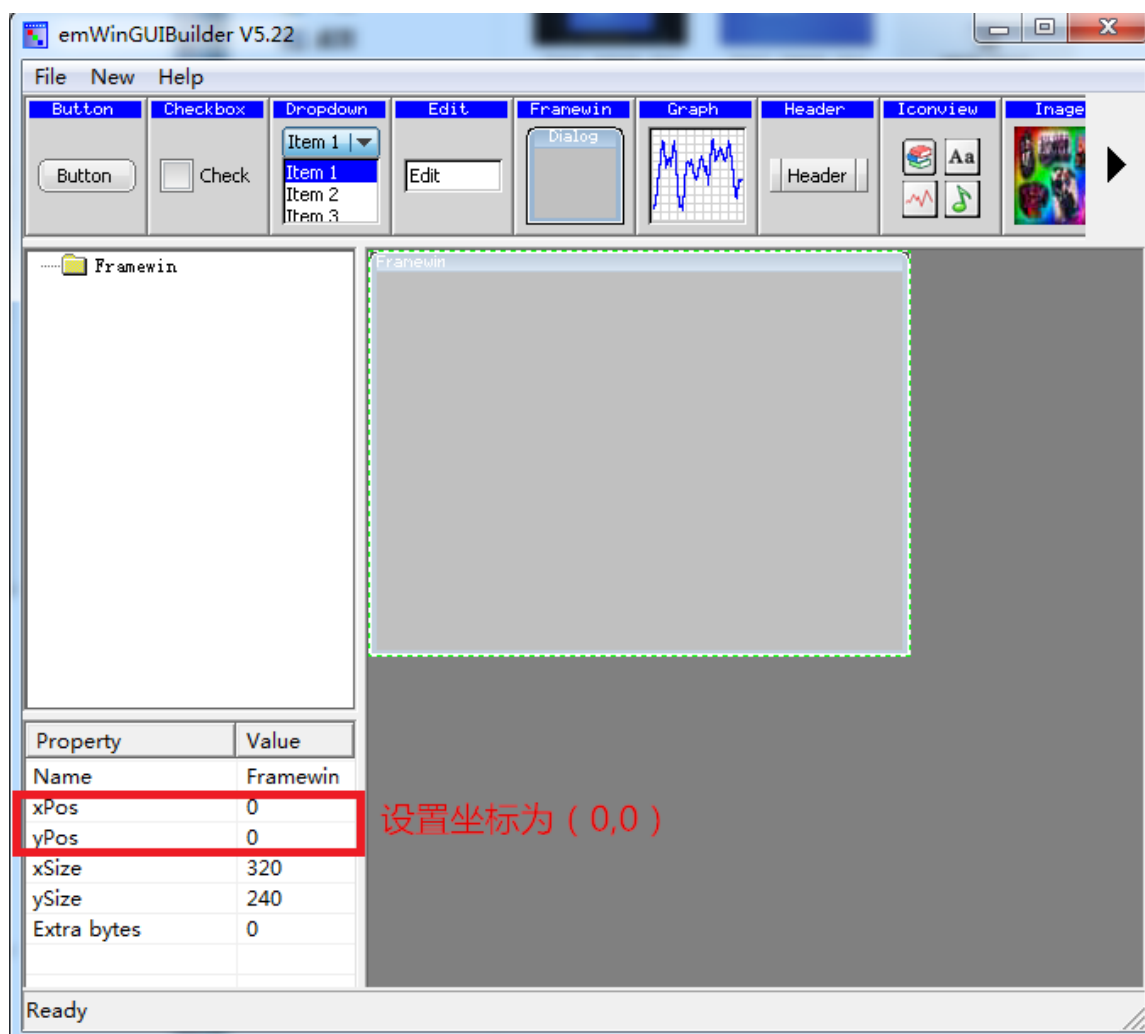


图 11.2.2 设置控件坐标

改变控件大小，按照图 11.2.3 所示修改。

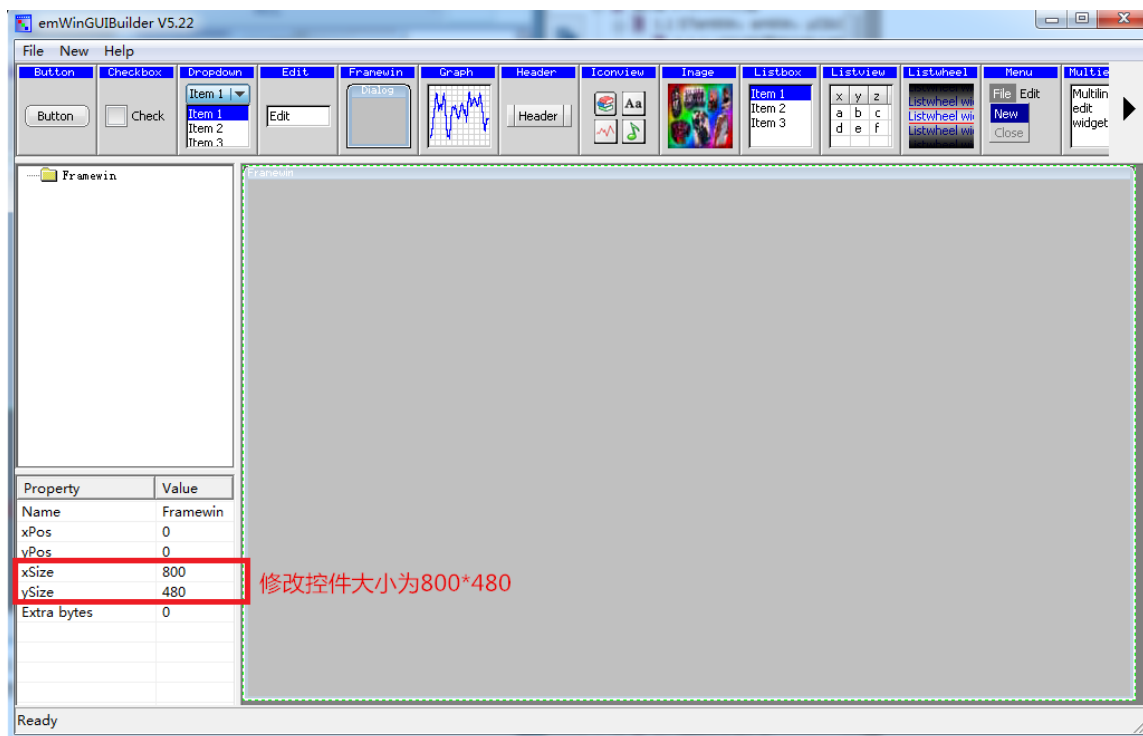


图 11.2.3 修改控件大小

设置标题栏的高度，在控件上点击鼠标右键，会出现很多设置选项，如图 11.2.4 所示，这里我们选择“Set title height”。



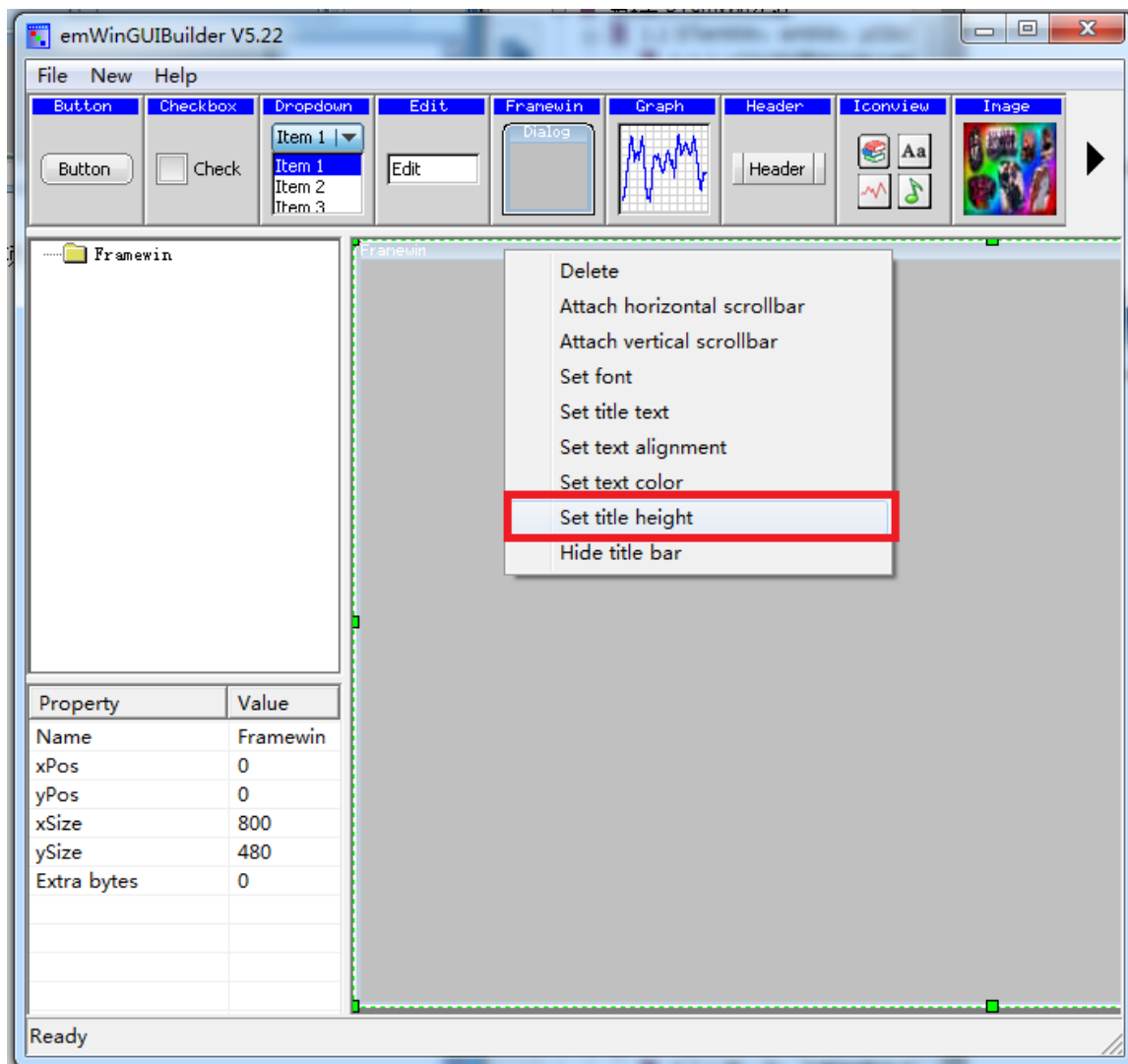


图 11.2.4 设置标题栏高度

当我们选择完成后就会在 GUIBuilder 左下方的小工具属性栏出现“Title height”设置选项，默认为 14，这里我们修改为 40，修改完成以后如图 11.2.5 所示。

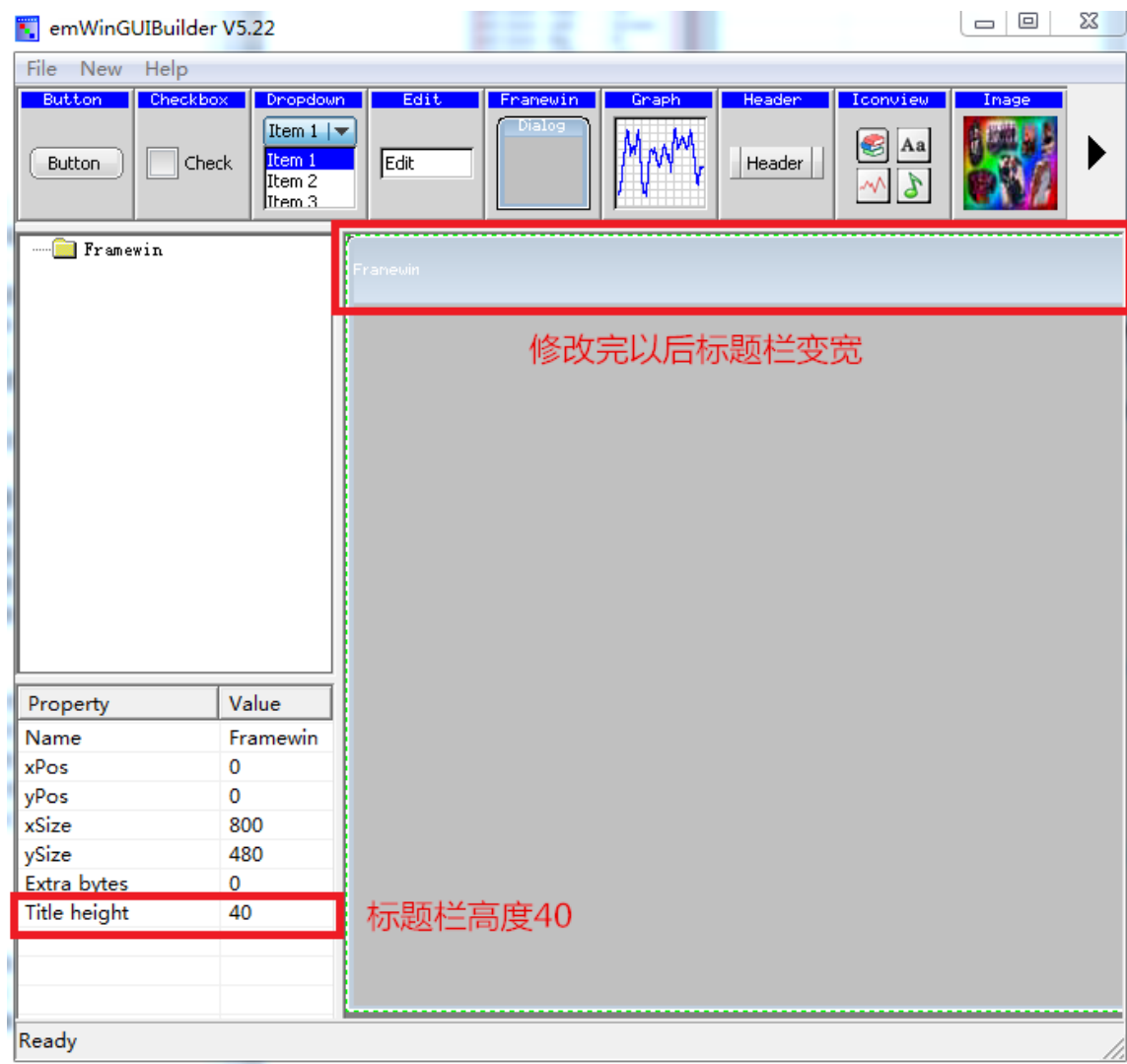


图 11.2.5 修改完标题栏高度

设置标题栏名字，在控件上点击右键并选择“Set title text”，设置标题栏名字为“ALIENTEK GUIBulider”，如图 11.2.6 所示。

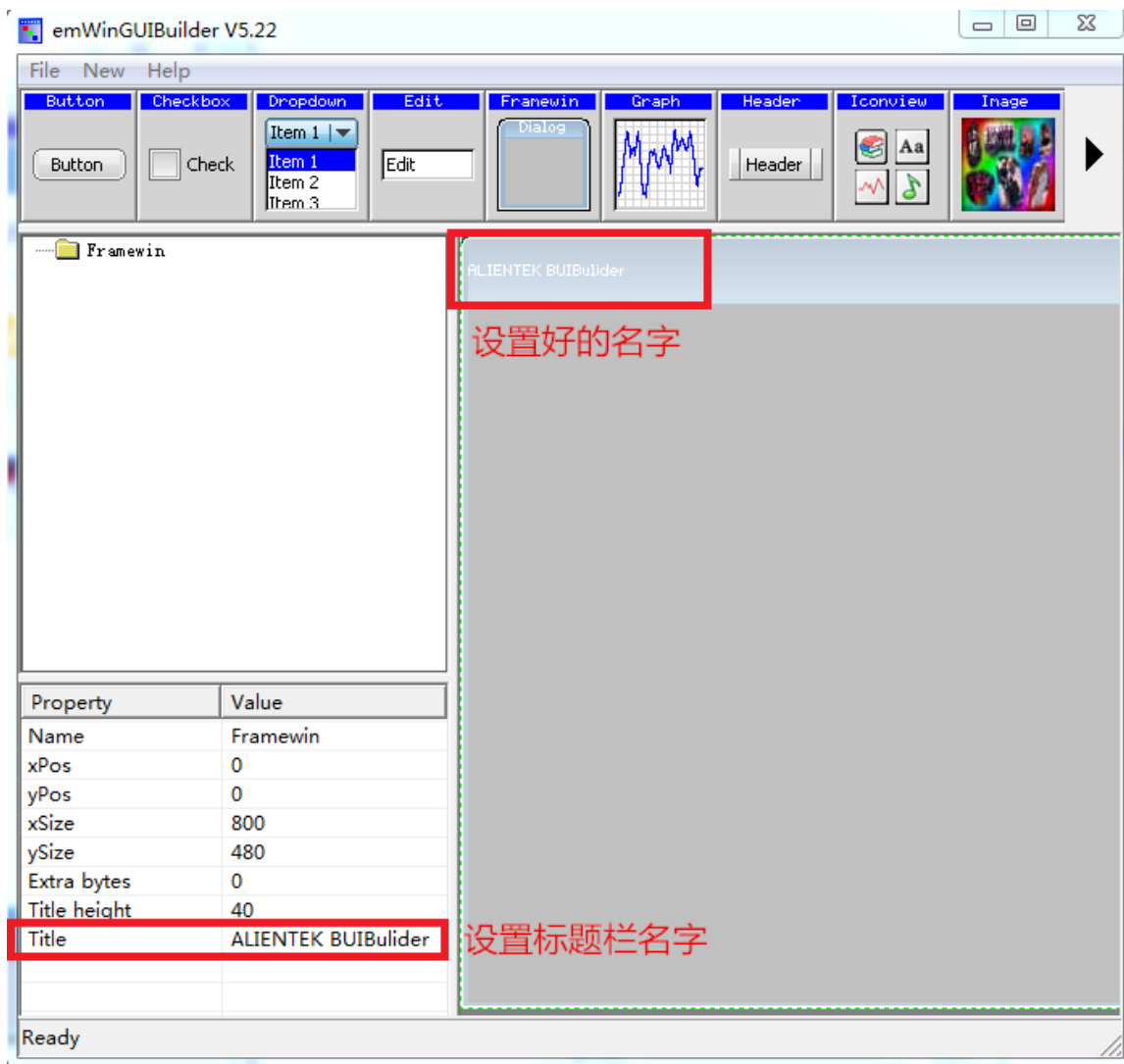


图 11.2.6 设置标题栏名字

设置标题栏名字字体为“GUI\_FONT\_32B\_ASCII”，控件上点击鼠标右键，选择“Set font”，出现如图 11.2.7 所示对话框，选择 GUI\_FONT\_32B\_ASCII。

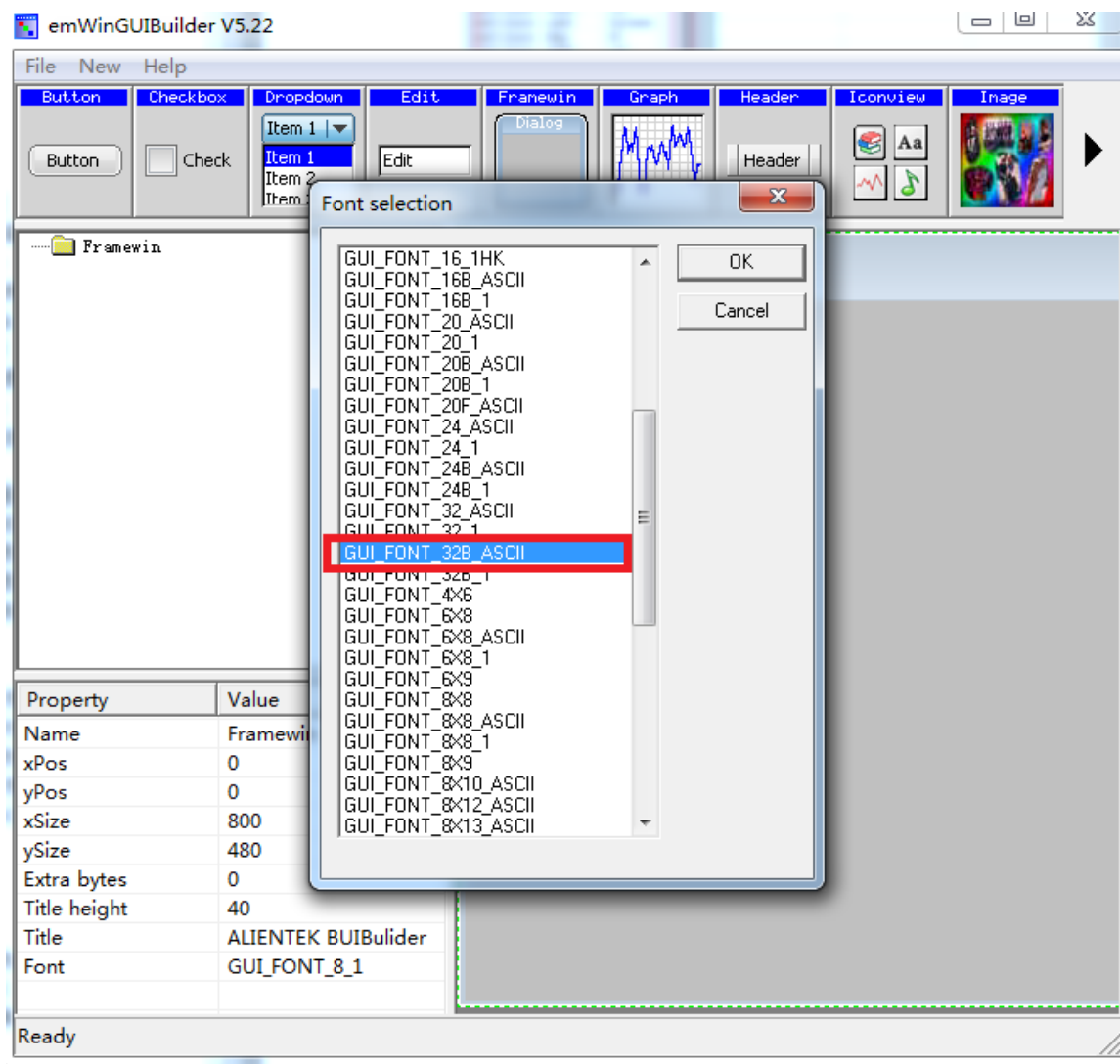


图 11.2.7 设置字体

设置标题栏名字的颜色为红色，控件上点击鼠标右键选择“Set text color”，选择红色，设置完成以后如图 11.2.8 所示。

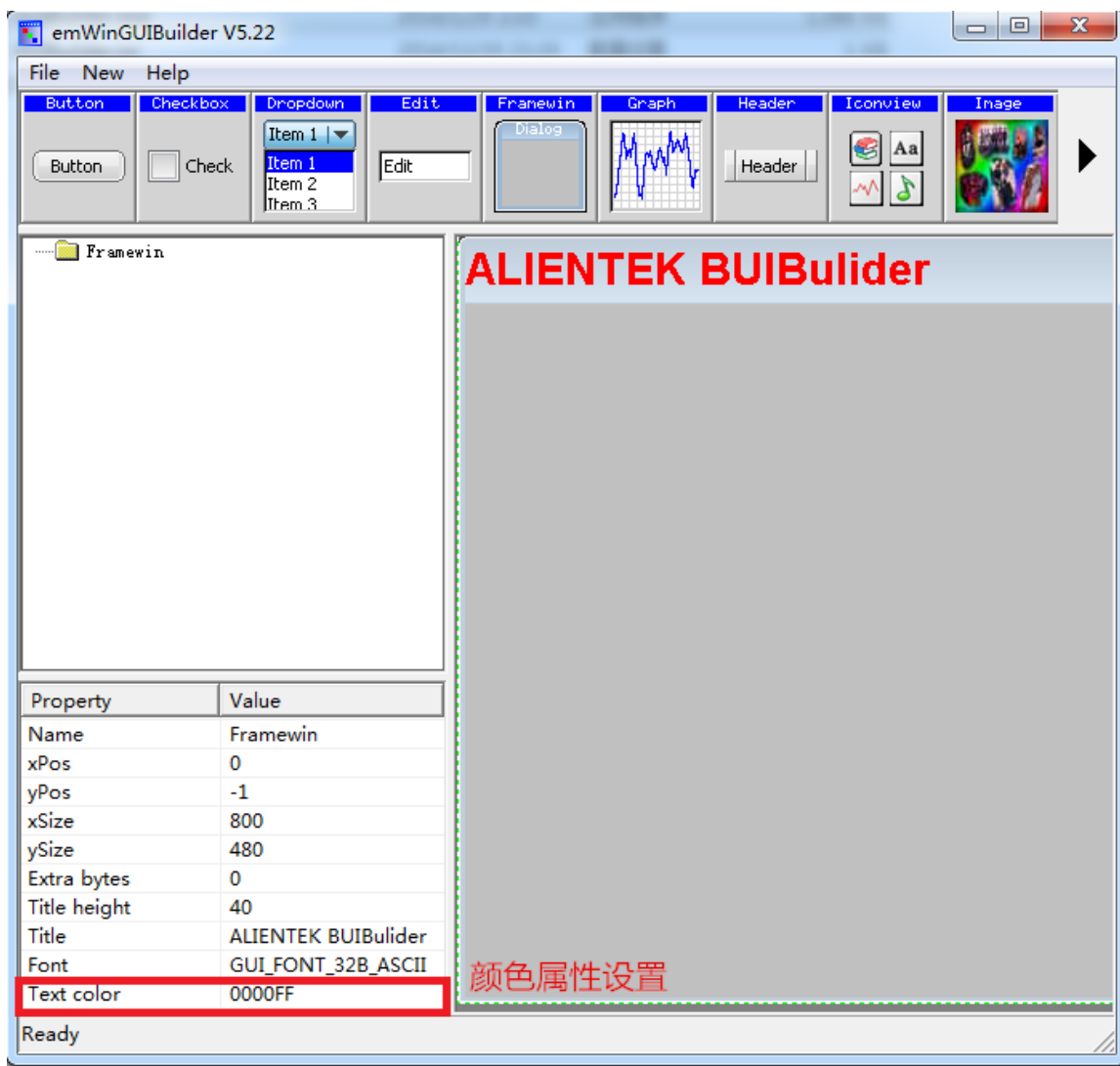


图 11.2.8 设置颜色

设置标题栏名字的对齐方式为水平和垂直居中，鼠标右键选择“Set text alignment”，在出现的对话框中选择自己想要的对齐方式，这里我们选择水平和居中对齐，如图 11.2.9 所示。

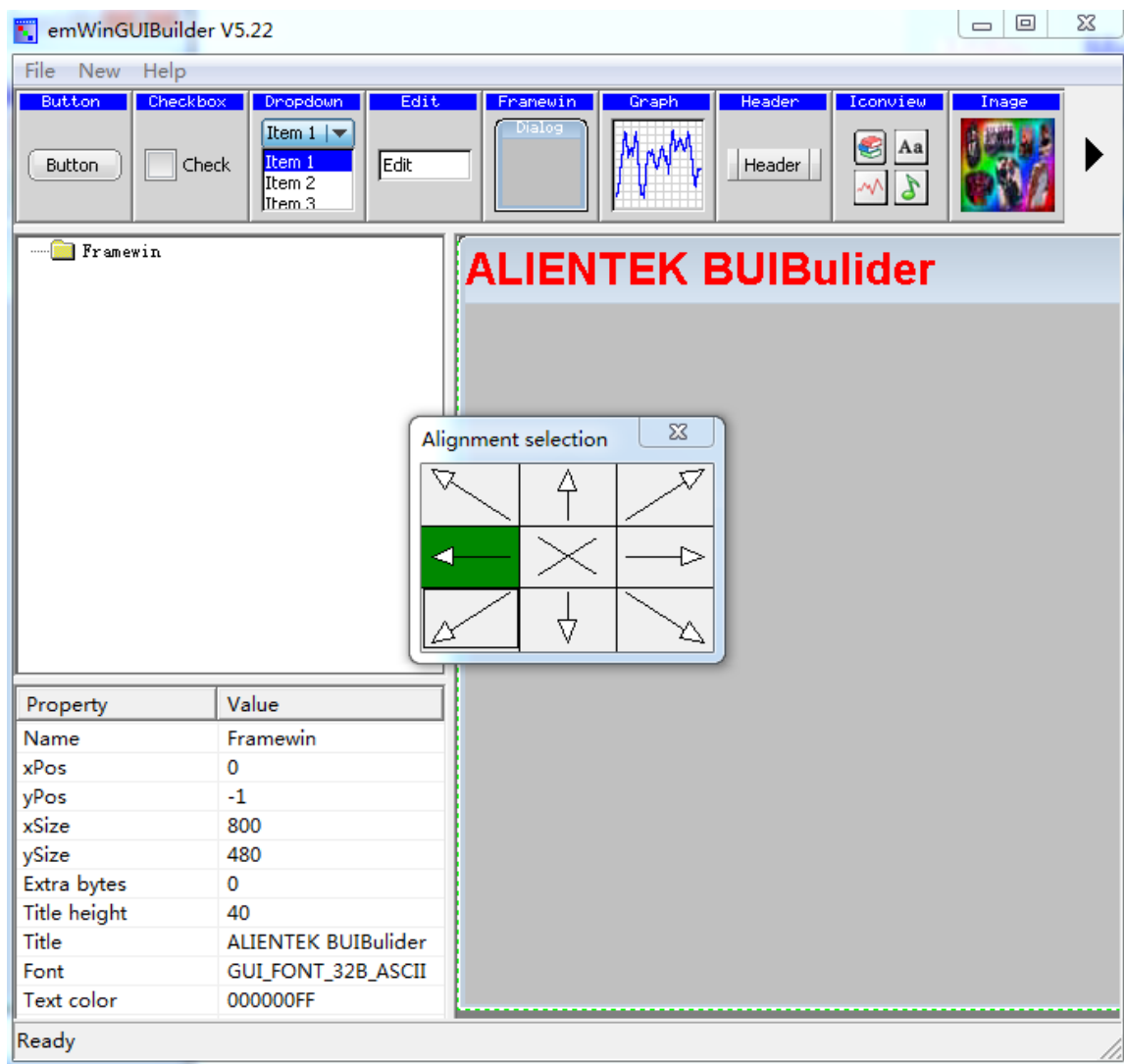


图 11.2.9 选择对齐方式

全部都设置完成以后 Framewin 控件就如图 11.2.10 所示。

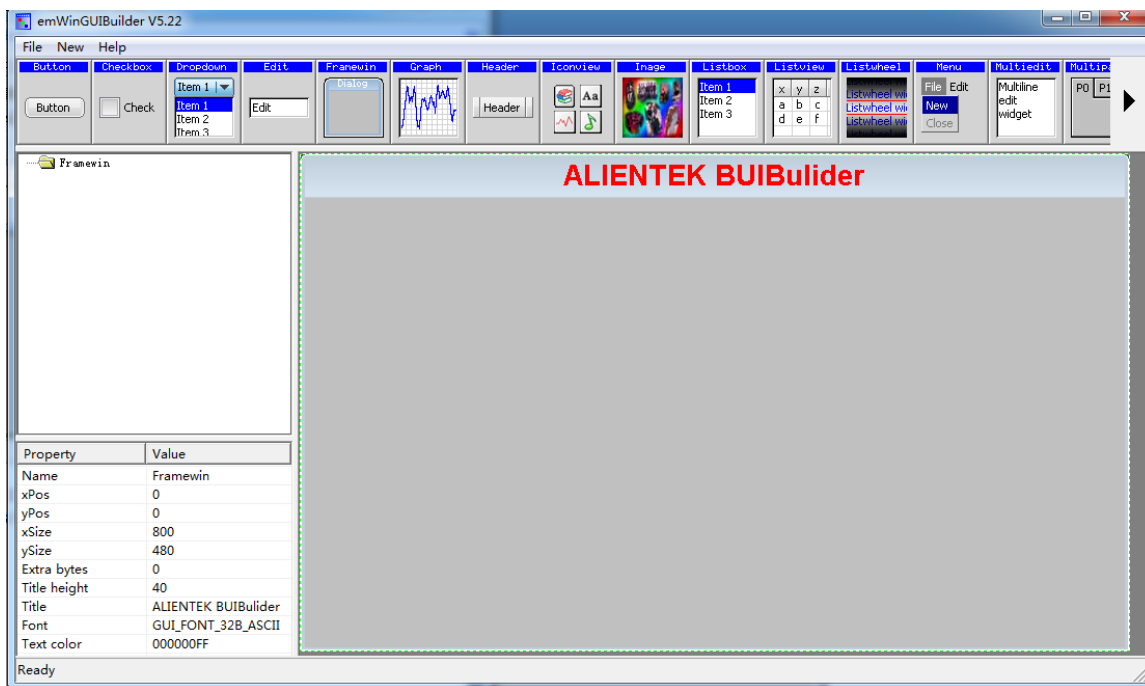


图 11.2.10 设置完成

2、我们再来在 FrameWin 中添加一个 Listview 控件，一个 Edit 控件，两个 Button 控件和一个 Scrollbar 控件，添加完成后如图 11.2.11 所示。

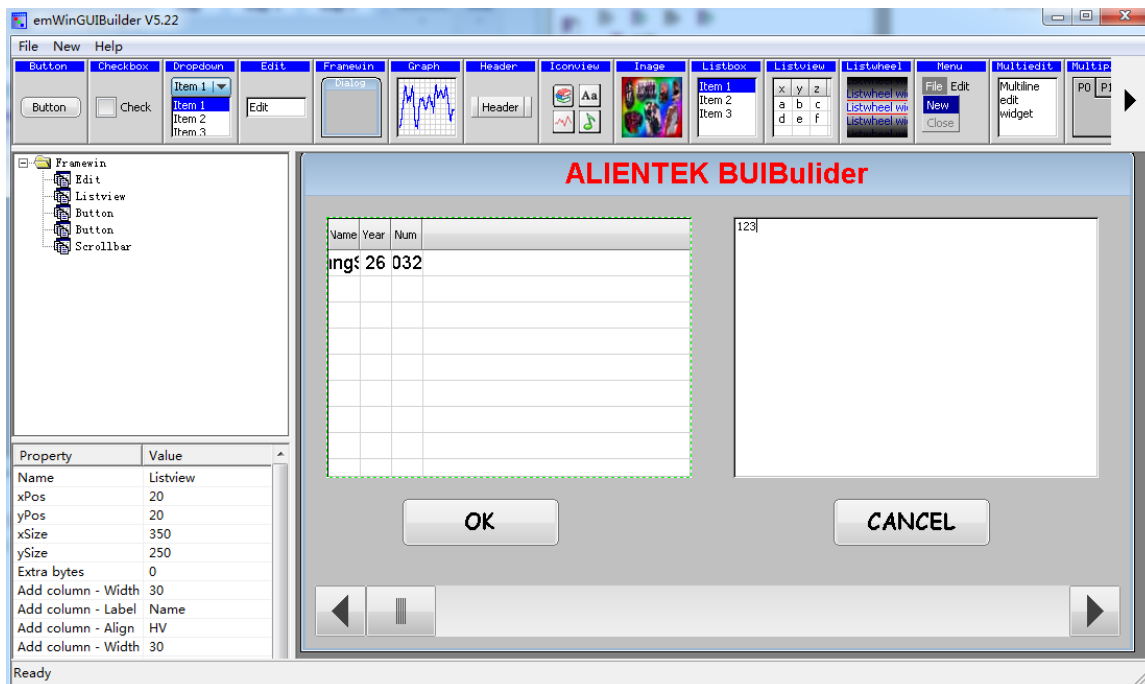


图 11.2.11 设计完成

3、保存设计，生成C文件，点击 File->Save，出现提示对话框，点击“确定”，这样在 GUIBulider 所在的文件夹下就会有一个.C 文件生成，如图 11.2.13 所示，FramewinDLG.c 就是我们通过 GUIBulider 设计并生成的界面。

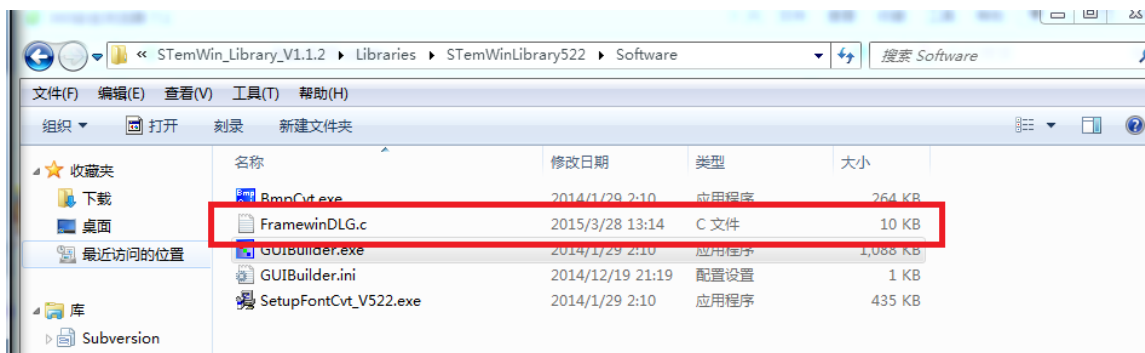


图 11.2.13 生成 C 文件

### 11.3 GUIBulider 输出

上一小节我们已经提到过了，GUIBulider 最终输出的就是 C 语言，一旦关闭了 GUIBulider 软件就再也不能打开我们以前的设计修改了，只能通过修改 C 文件修改以前的设计，这也是 GUIBulider 的一大缺点！将 FrameWinDLG.c 文件拷贝到工程中，本实验的对应工程路径为：**EMWIN\_DEMO->GUIBulder**，如图 11.3.1 所示，实验完整工程为：**EMWIN 实验 14 GUIBulder 使用**。

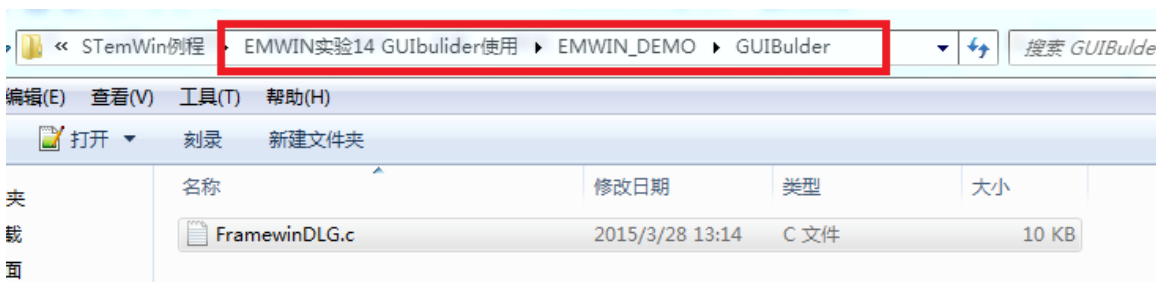


图 11.3.1 GUIBulider 生成的 C 文件拷贝到工程中

生成的 C 文件如下，限于篇幅原因，这里删除了生成的 C 文件中的注释性语言。

```
#include "DIALOG.h"

#define ID_FRAMEWIN_0 (GUI_ID_USER + 0x01)
#define ID_EDIT_0 (GUI_ID_USER + 0x02)
#define ID_LISTVIEW_0 (GUI_ID_USER + 0x03)
#define ID_BUTTON_0 (GUI_ID_USER + 0x04)
#define ID_BUTTON_1 (GUI_ID_USER + 0x05)
#define ID_SCROLLBAR_0 (GUI_ID_USER + 0x06)

static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 3, -2, 800, 480, 0, 0x64, 0 },
    { EDIT_CreateIndirect, "Edit", ID_EDIT_0, 410, 20, 350, 250, 0, 0x64, 0 },
    { LISTVIEW_CreateIndirect, "Listview", ID_LISTVIEW_0, 20, 20, 350, 250, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 93, 290, 150, 45, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_1, 506, 290, 150, 45, 0, 0x0, 0 },
    { SCROLLBAR_CreateIndirect, "Scrollbar", ID_SCROLLBAR_0, 9, 373, 772, 49, 0, 0x0, 0 },
}
```



```

};
//对话框回调函数
static void _cbDialog(WM_MESSAGE * pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;

    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:
            //初始化 Framewin 小工具
            hItem = pMsg->hWin;
            FRAMEWIN_SetTitleHeight(hItem, 40);
            FRAMEWIN_SetText(hItem, "ALIENTEK BUIBulider");
            FRAMEWIN_SetFont(hItem, GUI_FONT_32B_ASCII);
            FRAMEWIN_SetTextColor(hItem, 0x000000FF);
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
            //初始化 Edit 小工具
            hItem = WM_GetDialogItem(pMsg->hWin, ID_EDIT_0);
            EDIT_SetText(hItem, "123");
            //初始化 Listview 小工具
            hItem = WM_GetDialogItem(pMsg->hWin, ID_LISTVIEW_0);
            LISTVIEW_AddColumn( hItem, 30, "Name", GUI_TA_HCENTER | \
                                GUI_TA_VCENTER);
            LISTVIEW_AddColumn( hItem, 30, "Year", GUI_TA_HCENTER | \
                                GUI_TA_VCENTER);
            LISTVIEW_AddColumn( hItem, 30, "Num", GUI_TA_HCENTER | \
                                GUI_TA_VCENTER);
            LISTVIEW_AddRow(hItem, NULL);
            LISTVIEW_SetGridVis(hItem, 1);
            LISTVIEW_SetFont(hItem, GUI_FONT_20_ASCII);
            LISTVIEW_SetItemText(hItem, 0, 0, "ZhangSan");
            LISTVIEW_SetAutoScrollH(hItem, 1);
            LISTVIEW_SetAutoScrollV(hItem, 1);
            LISTVIEW_SetHeaderHeight(hItem, 30);
            LISTVIEW_SetItemText(hItem, 1, 0, "26");
            LISTVIEW_SetItemText(hItem, 2, 0, "20150328001");
            LISTVIEW_SetRowHeight(hItem, 25);
            //初始化 BUTTON0 小工具
            hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
            BUTTON_SetText(hItem, "OK");
            BUTTON_SetFont(hItem, GUI_FONT_COMIC24B_1);
    }
}

```

```
//初始化 BUTTON1 小工具
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
BUTTON_SetText(hItem, "CANCEL");
BUTTON_SetFont(hItem, GUI_FONT_COMIC24B_1);
break;
case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;
    switch(Id)
    {
        case ID_EDIT_0: // Edit 发送的通知消息
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    break;
                case WM_NOTIFICATION_VALUE_CHANGED:
                    break;
            }
            break;
        case ID_LISTVIEW_0: // Listview 发送的通知消息
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    break;
                case WM_NOTIFICATION_SEL_CHANGED:
                    break;
            }
            break;
        case ID_BUTTON_0: // Button0 发送的通知消息
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    break;
            }
            break;
        case ID_BUTTON_1: // Button1 发送的通知消息
            switch(NCode)
```

```

        {
            case WM_NOTIFICATION_CLICKED:
                break;
            case WM_NOTIFICATION_RELEASED:
                break;
        }
        break;
    case ID_SCROLLBAR_0: // Scrollbar 发送的通知消息
        switch(NCode)
        {
            case WM_NOTIFICATION_CLICKED:
                break;
            case WM_NOTIFICATION_RELEASED:
                break;
            case WM_NOTIFICATION_VALUE_CHANGED:
                break;
        }
        break;
    }
    break;
default:
    WM_DefaultProc(pMsg);
    break;
}
}

```

```
WM_HWIN CreateFramewin(void);
```

```
WM_HWIN CreateFramewin(void)
```

```

{
    WM_HWIN hWin;
    hWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate), \
                                _cbDialog, WM_HBKWIN, 0, 0);
    return hWin;
}

```

从以上代码可以看出 GUIBulider 生成的只是一个初始化的代码，仅仅只是一个程序框架，至于这些控件的具体功能需要我们自己编写程序的！至于这些控件的具体作用不是本节的内容，会在以后章节讲解的。

## 11.4 修改 C 文件

生成的 C 文件中包含了很多自定义代码区域，如下所示：

```

// USER START (选择性插入自己的代码)
// USER END

```

我们可以在这些自定义代码区域加上自己的代码，比如初始化控件，换肤，定义控件的功能等，为了给大家提前展示一下 emwin 强大的换肤功能，开启 emwin 的换肤功能很简单，只要在 main.c 中的 emwin 任务函数 emwin\_maintask（）中加入了一下程序就可以了。

```
//更换皮肤
```

```
BUTTON_SetDefaultSkin(BUTTON_SKIN_FLEX);
CHECKBOX_SetDefaultSkin(CHECKBOX_SKIN_FLEX);
DROPDOWN_SetDefaultSkin(DROPDOWN_SKIN_FLEX);
FRAMEWIN_SetDefaultSkin(FRAMEWIN_SKIN_FLEX);
HEADER_SetDefaultSkin(HEADER_SKIN_FLEX);
MENU_SetDefaultSkin(MENU_SKIN_FLEX);
MULTIPAGE_SetDefaultSkin(MULTIPAGE_SKIN_FLEX);
PROGBAR_SetDefaultSkin(PROGBAR_SKIN_FLEX);
RADIO_SetDefaultSkin(RADIO_SKIN_FLEX);
SCROLLBAR_SetDefaultSkin(SCROLLBAR_SKIN_FLEX);
SLIDER_SetDefaultSkin(SLIDER_SKIN_FLEX);
SPINBOX_SetDefaultSkin(SPINBOX_SKIN_FLEX);
```

新建一个 FramewinDLG.h 文件，在这个 h 文件里面定义一下函数 CreateFramewin()，这样我们就可以在其他地方调用了。

修改 main.c 中 emwin 的任务函数 emwin\_maintask（）如下，编译完成下载到开发板中，显示结果如图 11.5.1 所示。

```
//EMWINDEMO 任务
```

```
void emwin_maintask(void *pdata)
{
    CreateFramewin();
    while(1)
    {
        GUI_Delay(100);
    }
}
```

图 11.5.1 是使用换肤以后的效果图，注意这是在 800\*480 分辨率上显示的，如果在 320\*240 分辨率的 LCD 上显示会显示不完全！实验完整工程为：[EMWIN 实验 14 GUIBulider 使用](#)。

## 11.5 LCD 显示结果

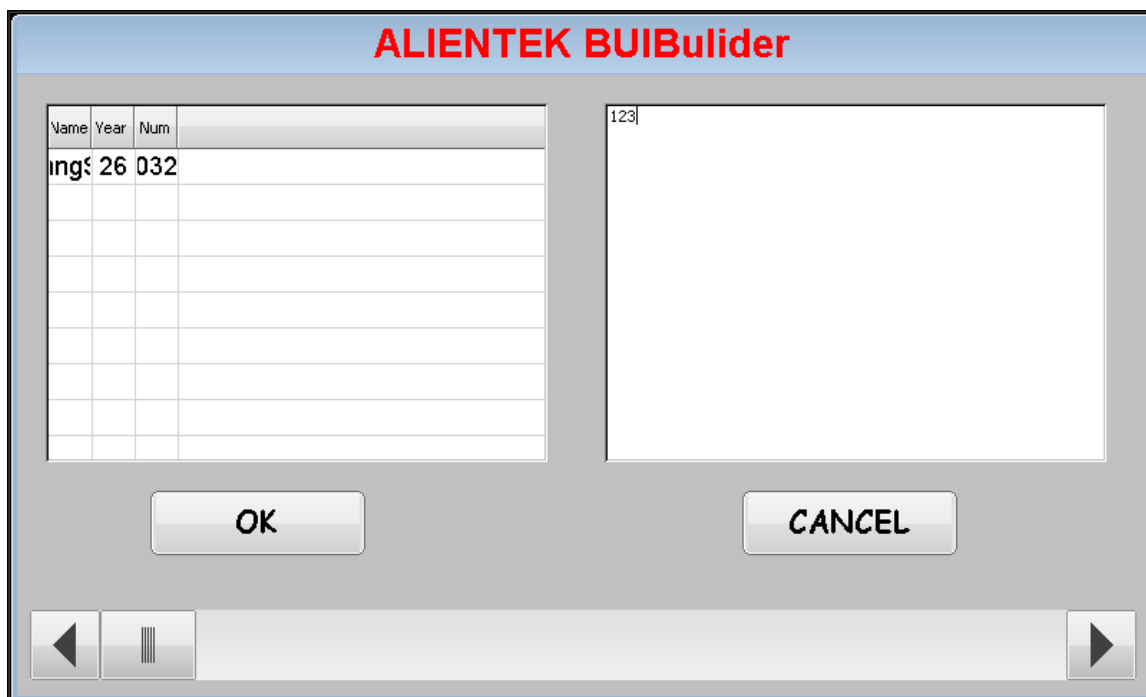


图 11.5.1 换肤后的 LCD 显示效果图

图 11.1.18 是换肤后的显示效果图，类似于 WIN7 的效果，我们可注释掉换肤部分的程序，看一下如果不换肤的话会显示成什么样子的，关闭换肤功能后的显示效果如图 11.5.2 所示。

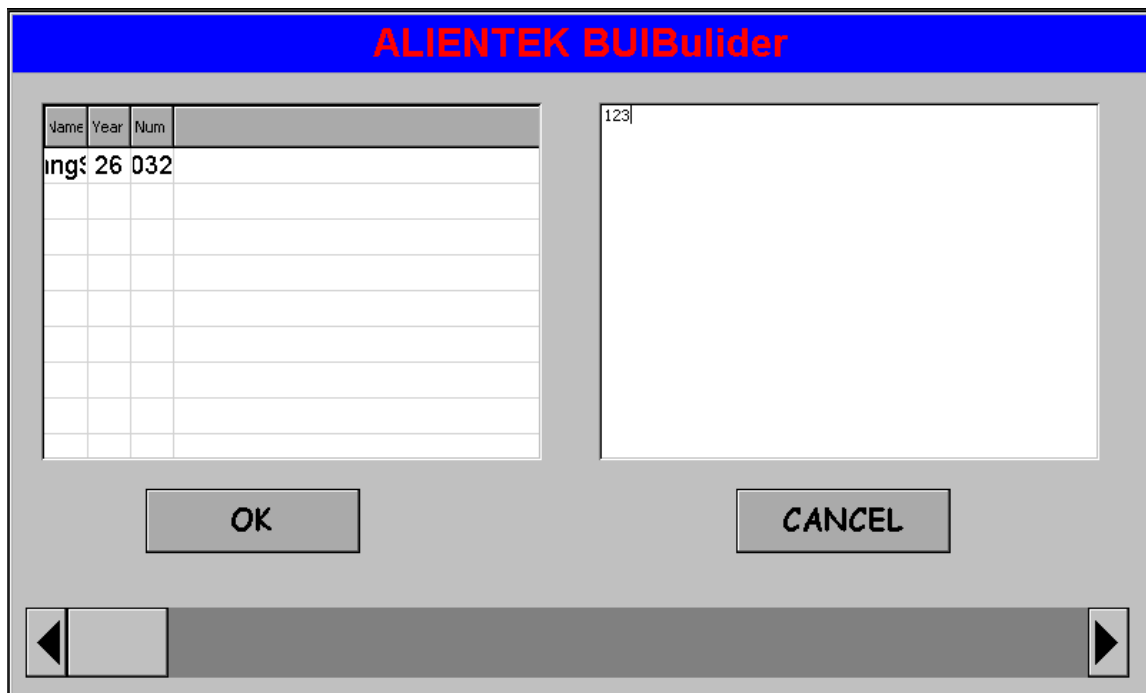


图 11.5.2 未换肤的显示效果

## 第十二章 对话框

小工具可以创建并独立使用，因为它们本质上就是窗口，但是，通常需要使用对话框，对话框是一个包含一个或多个小工具的窗口。对话框通常是一个窗口，它在出现时会要求用户输入信息，它可能包含多个小工具，要求用户根据各种选择来提供信息，或者以消息框的形式，仅提供信息（比如向用户提供注意事项或警告）和一个“确定”按钮。其实在上一章讲解 GUIBuilder 的时候就使用到了对话框，使用 GUIBuilder 的制作 GUI 界面的时候基本都是用了对话框，本章我们就详细的讲解一下什么是对话框，本章分为如下几个部分：

### 12.1 对话框基本原理

### 12.2 创建对话框

### 12.3 定义小工具功能

### 12.4 对话框 API 函数

## 12.1 对话框基本原理

### 输入焦点

窗口管理器能记住用户使用触摸屏、鼠标、键盘或用其他方式最终所选择的窗口或窗口对象。该窗口会收到键盘输入消息，即具有输入焦点。追踪输入焦点的主要原因是为了确定键盘命令的发送目的地。具有输入焦点的窗口会接收由键盘所生成的事件。如果要将对话框内的输入焦点移至下一个焦点对话框项目，可以使用 GUI\_KEY\_TAB 键。如果要向后移动，则可以使用 GUI\_KEY\_BACKTAB。

### 阻塞式和非阻塞式对话框

对话框窗口可以分为阻塞式和非阻塞式，阻塞式对话框会阻塞执行的线程，默认情况下，它有输入焦点，用户必须先关闭它，线程才能继续执行。阻塞式对话框不会同时禁用所显示的其他对话框，换言之，阻塞式对话框并非模式对话框。如果对话框为阻塞式，则表示只有在对话框关闭后，所使用的函数(GUI\_ExecDialogBox() 或 GUI\_ExecCreatedDialog())才会返回值。而非阻塞式对话框则不会阻塞调用的线程——在它为可见时，可允许任务继续运行。创建对话框后，函数会立即返回值。

**需要注意的是，切勿从回调函数中调用阻塞式函数，否则可能会导致应用程序故障！**

### 对话框过程函数

对话框就是一个窗口，它接收消息的方式与系统中其他所有窗口一样。大多数消息由对话框的窗口回调程序自动处理，而其他消息则传递到建立对话框时所指定的回调程序，这便称为对话框过程函数。

### 对话框消息

发送到对话框过程函数的两种附加消息为：WM\_INIT\_DIALOG 和 WM\_NOTIFY\_PARENT。在显示对话框前，WM\_INIT\_DIALOG 消息会立即发送到对话框过程函数，对话框过程函数通常使用该消息来初始化小工具，并执行其他任何会影响对话框外观的初始化任务。WM\_NOTIFY\_PARENT 消息则通过对话框的子窗口发送到对话框，通知任何事件的父窗口以确保同步化。通过子窗口发送的事件取决于其类型，并针对每个类型的小工具单独记录。

## 12.2 创建对话框

创建对话框需要两个基本要素：资源表和对话框过程；前者定义所要包括的小工具，后者定义小工具的初始值及其行为。一旦具备这两个要素，则只需进行单个函数调用(GUI\_CreateDialogBox() 或 GUI\_ExecDialogBox())就能创建对话框。

### 12.2.1 对话框资源表

对话框可以基于阻塞(使用 GUI\_ExecDialogBox() )或非阻塞(使用 GUI\_CreateDialogBox() )方式创建。必须首先定义一个资源表，以指定在对话框中所要包括的所有小工具，一般我们使用 GUIBulider 设计界面的时候选择了对话框的话就会在最终生成的 C 文件中自动生成资源表，下面的示例定义了一个资源表：

```
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Dialog", 0,30, 5,260, 230, FRAMEWIN_CF_MOVEABLE,0 },
    { BUTTON_CreateIndirect, "OK", GUI_ID_OK, 100, 5, 60, 20 },
```

```

{ BUTTON_CreateIndirect,"Cancel", GUI_ID_CANCEL,100, 30, 60, 20 },
{ TEXT_CreateIndirect,"LText",0,10, 55, 48, 15, TEXT_CF_LEFT },
{ TEXT_CreateIndirect,"RText", 0, 10, 80, 48, 15, TEXT_CF_RIGHT },
{ EDIT_CreateIndirect, NULL, GUI_ID_EDIT0, 60, 55, 100, 15, 0, 50 },
{ EDIT_CreateIndirect, NULL, GUI_ID_EDIT1, 60, 80, 100, 15, 0, 50 },
{ TEXT_CreateIndirect,"Hex", 0, 10, 100, 48, 15, TEXT_CF_RIGHT },
{ EDIT_CreateIndirect, NULL, GUI_ID_EDIT2, 60, 100, 100, 15, 0, 6 },
{ TEXT_CreateIndirect, "Bin", 0, 10, 120, 48, 15, TEXT_CF_RIGHT },
{ EDIT_CreateIndirect, NULL, GUI_ID_EDIT3, 60, 120, 100, 15 },
{ LISTBOX_CreateIndirect, NULL,GUI_ID_LISTBOX0,10, 10, 60, 40 },
{ CHECKBOX_CreateIndirect, NULL, GUI_ID_CHECK0, 10, 140, 0, 0 },
{ CHECKBOX_CreateIndirect, NULL, GUI_ID_CHECK1, 30, 140, 0, 0 },
{ SLIDER_CreateIndirect, NULL, GUI_ID_SLIDER0, 60, 140, 180, 20 },
{ SLIDER_CreateIndirect, NULL, GUI_ID_SLIDER1, 10, 170, 230, 30 },
{ DROPDOWN_CreateIndirect,NULL, GUI_ID_DROPDOWN0,170, 10, 80, 60,0,3 },
{ DROPDOWN_CreateIndirect, NULL,GUI_ID_DROPDOWN1,170, 60, 80, 60,0,3 }
};

```

对对话框中所要包括的任何小工具都必须使用 <WIDGET>\_CreateIndirect() 函数来间接创建。

### 12.2.2 对话框回调函数

前面说过了对话框也是窗口的，创建对话框也是需要回调函数，通过回调函数来初始化对话框中其他小工具和定义这些小工具的行为，回调函数如下所示：

```

//对话框回调函数
static void _cbDialog(WM_MESSAGE * pMsg)
{
    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:

            //其他消息...

        default:
            WM_DefaultProc(pMsg);
    }
}

```

在使用函数 GUI\_CreateDialogBox() 或 GUI\_ExecDialogBox()创建对话框的时候需要将回调函数作为参数。使用上面资源表创建完成以后的对话框如图 12.2.1 所示：



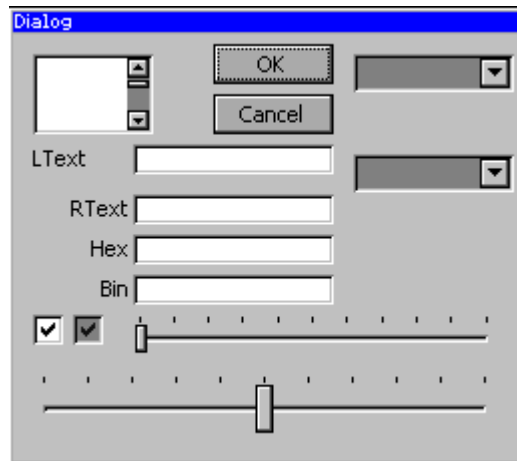


图 12.2.1 创建好的对话框

创建对话框后，所有资源表中的小工具都将可见。尽管这些小工具在上面的屏幕截图中可见，但它们是以“空”的形式出现的。这是因为对话框过程函数尚未包含初始化单个元素的代码。小工具的初始值、由它们所引起的行为以及它们之间的交互作用都需要在对话框过程中进行定义。

### 12.2.3 初始化对话框

图 12.2.1 中只展示的只是创建好的对话框，但是对话框还没有设置初始化值，接下来我们就要使用各个小工具的初始值对其进行初始化，接收到 WM\_INIT\_DIALOG 消息时做小工具的初始化，下面的程序展示了这个过程：

```
static void _cbCallback(WM_MESSAGE * pMsg)
{
    WM_HWIN hEdit0;
    WM_HWIN hEdit1;
    WM_HWIN hEdit2;
    WM_HWIN hEdit3;
    WM_HWIN hListBox;
    WM_HWIN hDropd0;
    WM_HWIN hDropd1;
    WM_HWIN hWin= pMsg->hWin;

    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG: //WM_INIT_DIALOG 消息，初始化所有小控件
            hEdit0  = WM_GetDialogItem(hWin, GUI_ID_EDIT0);
            hEdit1  = WM_GetDialogItem(hWin, GUI_ID_EDIT1);
            hEdit2  = WM_GetDialogItem(hWin, GUI_ID_EDIT2);
            hEdit3  = WM_GetDialogItem(hWin, GUI_ID_EDIT3);
            hListBox = WM_GetDialogItem(hWin, GUI_ID_LISTBOX0);
            hDropd0 = WM_GetDialogItem(hWin, GUI_ID_DROPDOWN0);
            hDropd1 = WM_GetDialogItem(hWin, GUI_ID_DROPDOWN1);
```

```

//初始化所有小控件
EDIT_SetText(hEdit0, "EDIT widget 0");
EDIT_SetText(hEdit1, "EDIT widget 1");
EDIT_SetTextAlign(hEdit1, GUI_TA_LEFT);
EDIT_SetHexMode(hEdit2, 0x1234, 0, 0xffffffff);
EDIT_SetBinMode(hEdit3, 0x1234, 0, 0xffff);
LISTBOX_SetText(hListBox, _apListBox);
WM_DisableWindow(WM_GetDialogItem(hWin, GUI_ID_CHECK1));
CHECKBOX_Check(WM_GetDialogItem(hWin, GUI_ID_CHECK0));
CHECKBOX_Check(WM_GetDialogItem(hWin, GUI_ID_CHECK1));
SLIDER_SetWidth(WM_GetDialogItem(hWin, GUI_ID_SLIDER0), 5);
SLIDER_SetValue(WM_GetDialogItem(hWin, GUI_ID_SLIDER1), 50);
SCROLLBAR_CreateAttached(hListBox, SCROLLBAR_CF_VERTICAL);
DROPDOWN_AddString(hDropd0, "Item 0");
DROPDOWN_AddString(hDropd0, "Item 1");
DROPDOWN_AddString(hDropd0, "Item 2");
DROPDOWN_AddString(hDropd1, "Item 0");
DROPDOWN_AddString(hDropd1, "Item 1");
DROPDOWN_AddString(hDropd1, "Item 2");
DROPDOWN_AddString(hDropd1, "Item 3");
DROPDOWN_AddString(hDropd1, "Item 4");
break;

default:
    WM_DefaultProc(pMsg);
}
}

```

上述程序中红色字体部分就是对话框回调函数收到消息 WM\_INIT\_DIALOG 后初始化对话框中其他的小工具，至于具体调用的初始化函数我们这里不用管，在以后讲解各个控件的时候会详细的讲解这些 API 函数，初始化完以后的对话框如图 12.2.2 所示。

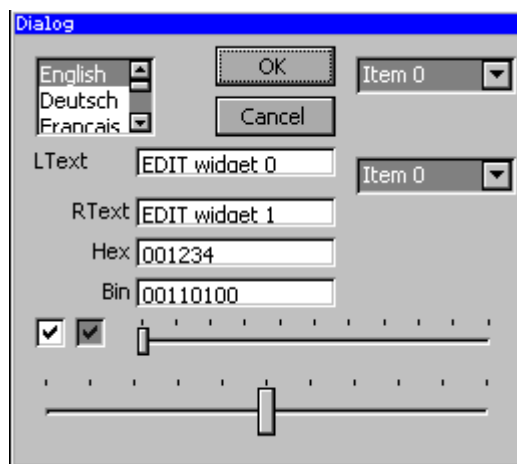


图 12.2.2 初始化对话框小工具

### 12.3 定义小工具功能

一旦对话框得到初始化，则剩下的所有工作便是向对话框过程函数添加代码，这样可定义小工具为，从而使其能充分操作。继续同一个示例，最终的对话框过程函数如下所示：

```
static void _cbCallback(WM_MESSAGE * pMsg)
{
    WM_HWIN hEdit0;
    WM_HWIN hEdit1;
    WM_HWIN hEdit2;
    WM_HWIN hEdit3;
    WM_HWIN hListBox;
    WM_HWIN hDropd0;
    WM_HWIN hDropd1;
    WM_HWIN hWin;

    hWin = pMsg->hWin;
    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG: //WM_INT_DIALOG 消息，用于初始化小工具
            hEdit0 = WM_GetDialogItem(hWin, GUI_ID_EDIT0);
            hEdit1 = WM_GetDialogItem(hWin, GUI_ID_EDIT1);
            hEdit2 = WM_GetDialogItem(hWin, GUI_ID_EDIT2);
            hEdit3 = WM_GetDialogItem(hWin, GUI_ID_EDIT3);
            hListBox = WM_GetDialogItem(hWin, GUI_ID_LISTBOX0);
            hDropd0 = WM_GetDialogItem(hWin, GUI_ID_DROPDOWN0);
            hDropd1 = WM_GetDialogItem(hWin, GUI_ID_DROPDOWN1);

            //初始化小工具
            EDIT_SetText(hEdit0, "EDIT widget 0");
            EDIT_SetText(hEdit1, "EDIT widget 1");
            EDIT_SetTextAlign(hEdit1, GUI_TA_LEFT);
            EDIT_SetHexMode(hEdit2, 0x1234, 0, 0xffffffff);
            EDIT_SetBinMode(hEdit3, 0x1234, 0, 0xffff);
            LISTBOX_SetText(hListBox, _apListBox);
            WM_DisableWindow(WM_GetDialogItem(hWin, GUI_ID_CHECK1));
            CHECKBOX_Check(WM_GetDialogItem(hWin, GUI_ID_CHECK0));
            CHECKBOX_Check(WM_GetDialogItem(hWin, GUI_ID_CHECK1));
            SLIDER_SetWidth(WM_GetDialogItem(hWin, GUI_ID_SLIDER0), 5);
            SLIDER_SetValue(WM_GetDialogItem(hWin, GUI_ID_SLIDER1), 50);
            SCROLLBAR_CreateAttached(hListBox, SCROLLBAR_CF_VERTICAL);
            DROPDOWN_AddString(hDropd0, "Item 0");
            DROPDOWN_AddString(hDropd0, "Item 1");
            DROPDOWN_AddString(hDropd0, "Item 2");
            DROPDOWN_AddString(hDropd1, "Item 0");
```

```

        DROPDOWN_AddString(hDropd1, "Item 1");
        DROPDOWN_AddString(hDropd1, "Item 2");
        DROPDOWN_AddString(hDropd1, "Item 3");
        DROPDOWN_AddString(hDropd1, "Item 4");
        break;
    case WM_KEY:
        switch (((WM_KEY_INFO*)(pMsg->Data.p))>Key)
        {
            case GUI_KEY_ESCAPE:
                GUI_EndDialog(hWin, 1);
                break;
            case GUI_KEY_ENTER:
                GUI_EndDialog(hWin, 0);
                break;
        }
        break;
    case WM_NOTIFY_PARENT:
        Id= WM_GetId(pMsg->hWinSrc);    //获取小控件 ID 号
        NCode = pMsg->Data.v;           //消息类型
        switch (NCode)
        {
            case WM_NOTIFICATION_RELEASED: //按键被释放
                if (Id == GUI_ID_OK)
                {
                    GUI_EndDialog(hWin, 0);
                }
                if (Id == GUI_ID_CANCEL)
                {
                    GUI_EndDialog(hWin, 1);
                }
                break;
        }
        break;
    default:
        WM_DefaultProc(pMsg);
    }
}

```

- (1)、WM\_NOTIFY\_PARENT 消息，当子控件发生改变的时候就会向父控件发送此消息。
- (2)、获取小控件的 ID 号
- (3)、获取小控件发来的消息类型。
- (4)、消息类型为 WM\_NOTIFICATION\_RELEASED, 即当按键被释放以后就会发送此消息。
- (5)、控件 ID 为 GUI\_ID\_OK, 也就是 OK 按键被释放了
- (6)、空间 ID 为 GUI\_ID\_CANCEL, 也就是 CANCEL 按键被释放了。

## 12.4 对话框 API 函数

表 12.4.1 列出了有关对话框的 API 函数：

| 程序                      | 描述         |
|-------------------------|------------|
| GUI_CreateDialogBox()   | 创建非阻塞式对话框。 |
| GUI_ExecCreatedDialog() | 执行已创建的对话框。 |
| GUI_ExecDialogBox()     | 创建并执行对话框。  |
| GUI_EndDialog()         | 结束对话框。     |

表 12.4.1 对话框 API 函数

### 12.4.1 GUI\_CreateDialogBox()

#### GUI\_CreateDialogBox()

##### 描述

创建非阻塞式对话框。

##### 原型

```
WM_HWIN GUI_CreateDialogBox(const GUI_WIDGET_CREATE_INFO * paWidget,
                             int NumWidgets,
                             WM_CALLBACK * cb,
                             WM_HWIN hParent,
                             int x0,
                             int y0);
```

| 参数         | 描述                      |
|------------|-------------------------|
| paWidget   | 定义对话框中所要包含的小工具的资源表的指针。  |
| NumWidgets | 对话框中所包含的小工具的总数。         |
| cb         | 应用程序特定回调函数（对话框过程函数）的指针。 |
| hParent    | 父窗口的句柄（0 表示没有父窗口）。      |
| x0         | 对话框相对于父窗口的 X 轴位置。       |
| y0         | 对话框相对于父窗口的 Y 轴位置。       |

### 12.4.2 GUI\_ExecCreatedDialog()

#### GUI\_ExecCreatedDialog()

##### 描述

执行已创建的对话框。

##### 原型

```
int GUI_ExecCreatedDialog(WM_HWIN hDialog);
```

| 参数      | 描述      |
|---------|---------|
| hDialog | 对话框的句柄。 |

##### 返回值

从 GUI\_EndDialog 返回的值。

### 12.4.3 GUI\_ExecDialogBox()

#### GUI\_ExecDialogBox()

##### 描述

创建并执行对话框

**原型**

```
int GUI_ExecDialogBox(const GUI_WIDGET_CREATE_INFO * paWidget,
                    int NumWidgets,
                    WM_CALLBACK * cb,
                    WM_HWIN hParent,
                    int x0,
                    int y0);
```

| 参数                      | 描述                      |
|-------------------------|-------------------------|
| <code>paWidget</code>   | 定义对话框中所要包含的小工具的资源表的指针。  |
| <code>NumWidgets</code> | 对话框中所包含的小工具的总数。         |
| <code>cb</code>         | 应用程序特定回调函数（对话框过程函数）的指针。 |
| <code>hParent</code>    | 父窗口的句柄（0 表示没有父窗口）。      |
| <code>x0</code>         | 对话框相对于父窗口的 X 轴位置。       |
| <code>y0</code>         | 对话框相对于父窗口的 Y 轴位置。       |

**返回值**

从 `GUI_EndDialog` 返回的值。

**12.4.4 GUI\_EndDialog()****GUI\_EndDialog()****描述**

结束（关闭）对话框。

**原型**

```
void GUI_EndDialog(WM_HWIN hDialog, int r);
```

| 参数                   | 描述                                     |
|----------------------|----------------------------------------|
| <code>hDialog</code> | 对话框的句柄。                                |
| <code>r</code>       | 由 <code>GUI_ExecDialogBox</code> 返回的值。 |

**返回值**

指定从创建对话框的函数返回至调用线程的值（通常只与 `GUI_ExecDialogBox()` 相关）。对于非阻塞式对话框，没有等待的应用程序线程，并将忽略返回值。

**其他信息**

需要注意的是，在调用该函数后，句柄 `hDialog` 便不再有效。如上所述，如果该函数结束对话框，则存储器中的该对话框将被删除。对于指定窗口的所有子窗口来说，这也同样适用。

关于对话框我们就讲解到这里，本章的内容非常重要，在我们制作好的 UI 界面的时候几乎都要用到对话框的，尤其是用 `GUIBuilder` 的时候，所以请大家务必掌握对话框的使用方法。

## 第十三章 窗口小工具(控件)

小工具是具有对象类型属性的各种窗口，在窗口界被称为控件，是组成用户界面的元素。它们可自动对某些事件作出反应，例如，按下某按钮后，它可以不同状态显示。小工具需要创建，具有可在存续期间随时更改的属性，并通常在不再需要时被删除。正如窗口一样，小工具通过其创建函数返回的句柄进行引用，小工具要求使用窗口管理器，小工具创建后，可像其他任何窗口一样处理它。**WM** 确保了在必要时随时正确显示（并重绘）它，小工具不是编写应用程序或用户界面所必需的，但它们可大大简化编程，本章分为如下几个内容：

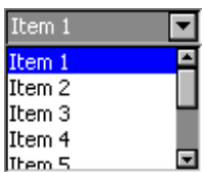
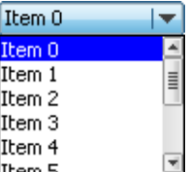
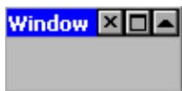
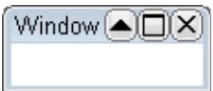
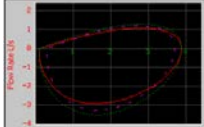
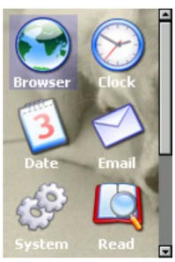
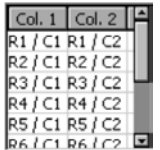
### 13.1 小工具基础知识

### 13.2 通用控件 API 函数

## 13.1 小工具基础知识

### 13.1.1 可用小工具

下表列出了目前 5.22 版本 emwin 支持的控件：

| 控件名字     | 屏幕截图<br>(经典)                                                                        | 屏幕截图<br>(皮肤设置)                                                                      | 描述                    |
|----------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-----------------------|
| BUTTON   |    |    | 可按下的按钮，按钮上可显示文本或者位图。  |
| CHECKBOX |    |    | 可被选中或取消选中的复选框。        |
| DROPDOWN |    |    | 下来列表，按下后会打开一个列表框。     |
| EDIT     |    |                                                                                     | 单行编辑字段，提示用户键入数字或文本。   |
| FRAMEWIN |   |   | 框架窗口，创建典型的 GUI 外观。    |
| GRAPH    |  |                                                                                     | 图形小工具，用于显示曲线或测量值。     |
| HEADER   |  |  | 标题控件，用于管理各栏。          |
| ICONVIEW |  |                                                                                     | 图标视图小工具，对基于图标的平台非常有用。 |
| IMAGE    |  |                                                                                     | 图像控件，自动显示各种格式的图片      |
| LISTBOX  |  |                                                                                     | 突出显示用户选择项目的列表框。       |
| LISTVIEW |  |                                                                                     | Listview 小工具用于创建表。    |



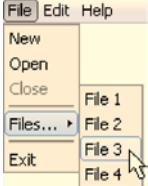
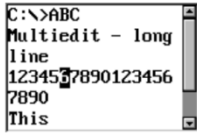
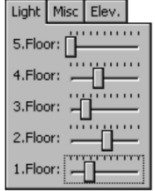
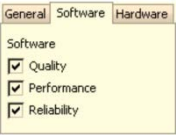
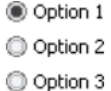
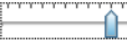
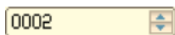
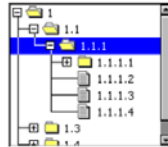
|           |                                                                                     |                                                                                     |                                 |
|-----------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------|
| LISTWHEEL |    |                                                                                     | Listwheel 小工具，可使用指针输入设备移动或加速数据。 |
| MENU      |    |    | 菜单小工具可用于创建水平或垂直菜单。              |
| MULTIEDIT |    |                                                                                     | Multedit 小工具可用于编辑多行文本。          |
| MULTIPAGE |    |    | Multipage 小工具用于创建具有多个页面的对话框。    |
| PROGBAR   |   |   | 用于可视化的进度条。                      |
| RADIO     |  |  | 可以选择的单选按钮，一次只能选择一个按钮。           |
| SCROLLBAR |  |  | 水平或者垂直的滚动条。                     |
| SLIDER    |  |  | 用于更改值的滚动条                       |
| SPINBOX   |  |  | 用于改变一个数值。                       |
| TEXT      |  |                                                                                     | 静态文本控件，一般用于对话框。                 |
| TREEVIEW  |  |                                                                                     | 用于管理分层列表的树形图形小工具。               |

表 13.1.1 可用控件

### 13.1.2 了解重绘机制

小工具根据其属性绘制自身，此操作在调用 WM\_Exec()、GUI\_Exec() 或 GUI\_Delay() 时执行。在多任务环境中，通常由后台任务来调用 WM\_Exec() 并更新小工具（以及所有其他具有回调函数的窗口）。小工具的属性更改后，该小工具的窗口（或部分窗口）会被标记为无效，但不会立即重绘，因此，代码段的执行速度会非常快。重绘由 WM 在稍后执行，或者通过为该小工具调用 WM\_Paint() 强制执行（或直到重绘所有窗口时调用 WM\_Exec()）。

### 13.1.3 如何使用小工具

假设我们要显示一个进度条，所需的只是以下代码：

```
PROGBAR_Handle hProgBar; (1)
GUI_DispStringAt("Progress bar", 100, 20);
hProgBar = PROGBAR_Create(100, 40, 100, 20, WM_CF_SHOW); (2)
```

(1)、定义一个进度条句柄。

(2)、调用 PROGBAR\_Create()函数创建一个进度条，创建完成以后窗口管理器会在调用单 WM\_Exec() 时自动绘制此小工具。每种类型的小工具都有若干能够修改其外观的成员函数，小工具创建后，可通过调用其成员函数之一更改其属性，这些函数使用小工具的句柄作为第一个自变量。要使上述创建的进度条显示 45% 并将条颜色从默认设置（深灰 / 浅灰）更改为绿色 / 红色，可使用以下代码段：

```
PROGBAR_SetBarColor(hProgBar, 0, GUI_GREEN);
PROGBAR_SetBarColor(hProgBar, 1, GUI_RED);
PROGBAR_SetValue(hProgBar, 45);
```

修改完以后的进度条属性如图 13.1.1 所示：

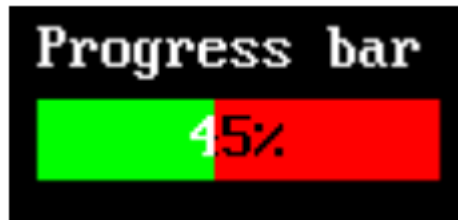


图 13.1.1 修改进度条属性

#### 默认配置

所有小工具都还具有一个或多个配置宏，它们定义各种默认设置（如所使用的字体和颜色）。在后续章节中详细讲解控件的时候就会列出了每个控件的可用配置选项

#### 小工具如何通信

小工具通常作为子窗口创建，父窗口可为任何窗口类型，甚至是另一种小工具，为了确保同步，无论何时父窗口的任何子项有任何事件发生，通常都应通知父窗口。有事件发生时，子窗口小工具通过发送 WM\_NOTIFY\_PARENT 消息与其父窗口通信，作为消息一部分发送的通知代码取决于事件。大多数小工具都有定义不同类型事件的一个或多个通知代码，每种小工具可用的通知代码（如果有）在其各自章节中列出。

#### 皮肤设置

小工具的外观可使用其各自的成员函数进行修改，其中一些支持皮肤设置。如果对某小工具使用了皮肤设置，则“皮肤”将决定该小工具的外观，且一些成员函数将失效。

#### 小工具的动态内存使用

在嵌入式应用中，由于分裂效应，通常都不太希望使用动态内存，可使用许多不同策略来避免此情况，但只要内存区域被应用程序中的指针引用，这些策略的使用都有局限性。因此，emWin 使用不同的方法：所有对象（以及运行时存储的所有数据）都存储在被句柄引用的内存区域中。这使它能够在运行时重新分配已分配的内存区域，因此避免了使用指针时会发生的长期分配问题。所以所有小工具都使用句柄引用。

## 13.2 通用控件 API 函数

### 13.2.1 用于控件的 WM 函数

由于小工具本质上就是窗口，所以它们与任何窗口管理器 API 例程都兼容。小工具的句柄用作 hWin 参数，且小工具的处理方式与其他任何窗口都相同。最常用于控件的 WM 函数如表 13.2.1 所示：

| 函数                    | 描述            |
|-----------------------|---------------|
| WM_DeleteWindow()     | 删除窗口。         |
| WM_DisableMemdev()    | 禁止使用存储设备进行重绘。 |
| WM_EnableMemdev()     | 启用存储设备用于重绘。   |
| WM_InvalidateWindow() | 使窗口无效。        |
| WM_Paint()            | 立即绘制或重绘窗口。    |

表 13.2.1 用于控件的常用 WM 函数

### 13.2.2 常用 API 函数

表 13.2.2 按字母顺序列出了可用的小工具相关函数，这些函数对于所有控件都是通用的，在此列出以避免重复，每个小工具可用的额外成员函数在稍后各章节中介绍。

| 函数                        | 描述                             |
|---------------------------|--------------------------------|
| <WIDGET>_Callback()       | 默认回调函数。                        |
| <WIDGET>_CreateIndirect() | 用于对话框中的自动创建。                   |
| <WIDGET>_CreateUser()     | 使用额外字节作为用户数据创建小工具。             |
| <WIDGET>_GetUserData()    | 检索用<WIDGET>_SetUserData 设置的数据。 |
| <WIDGET>_SetUserData()    | 设置小工具的额外数据。                    |
| WIDGET_GetDefaultEffect() | 返回用于小工具的默认效果。                  |
| WIDGET_SetDefaultEffect() | 设置用于小工具的默认效果。                  |
| WIDGET_SetEffect()        | 设置用于给定小工具的效果。                  |

表 13.2.2 控件通用 API 函数

<WIDGET>表示的是小工具的名称，其中函数<WIDGET>\_CreateIndirect()尤为重要，因为在使用对话框的时候就是使用的这个函数来创建各个小工具的。

#### <WIDGET>\_CreateIndirect()

##### 描述

创建要在对话框中使用的小工具。

##### 原型

```
<WIDGET>_Handle <WIDGET>_CreateIndirect(
    const GUI_WIDGET_CREATE_INFO * pCreateInfo,
    WM_HWIN hParent,
    int x0,
    int y0,
    WM_CALLBACK * cb);
```

##### 其他信息

任何小工具都可使用适宜的前缀间接创建。例如：BUTTON\_CreateIndirect() 可间接创建一

个按钮小工具，CHECKBOX\_CreateIndirect() 可间接创建一个复选框小工具，等等。仅在小工具要被包含在对话框中时，才需要间接创建它，否则可使用 <WIDGET>\_Create() 函数直接创建它。

### 资源表

GUI\_WIDGET\_CREATE\_INFO 数据结构在对话框资源表中定义如下：

```
typedef struct
{
    GUI_WIDGET_CREATE_FUNC * pfCreateIndirect; // Create function
    const char * pName;                        // Text (not used for all widgets)
    I16 Id;                                    // Window ID of the widget
    I16 x0, y0, xSize, ySize;                  // Size and position of the widget
    I16 Flags;                                // Widget-specific flags (or 0)
    I32 Para;                                  // Widget-specific parameter (or 0)
    U32 NumExtraBytes;                          // Number of extra bytes usable
                                              // with <WIDGET>_SetUserData &
                                              // <WIDGET>_GetUserData
} GUI_WIDGET_CREATE_INFO;
```

小工具标记和参数是可选的，并随小工具类型不同而不同。每个小工具可用的标记和参数（如果有）将在后续章节中讲解。

### 13.2.3 用户绘制控件

一些小工具支持自画功能，例如 LISTBOX 控件小工具的用户绘制模式激活后，将调用 WIDGET\_DRAW\_ITEM\_FUNC 类型的应用程序定义函数来绘制该控件（项目）。应用程序定义的自画函数的原型应按如下定义：

#### 原型：

```
int WIDGET_DRAW_ITEM_FUNC(const WIDGET_ITEM_DRAW_INFO * pDrawItemInfo)
```

| 参数            | 描述                             |
|---------------|--------------------------------|
| pDrawItemInfo | 指向 WIDGET_ITEM_DRAW_INFO 结构的指针 |

#### WIDGET\_ITEM\_DRAW\_INFO 的元素

| 数据类型    | 元素        | 描述               |
|---------|-----------|------------------|
| WM_HWIN | hWin      | 小工具的句柄           |
| int     | Cmd       | （参见下表）           |
| int     | ItemIndex | 要绘制项目的以零为基准的索引   |
| int     | x0        | 绘制项目时要使用的在窗口坐标中的 |
| int     | y0        | 绘制项目时要使用的在窗口坐标中的 |

## 元素 Cmd 的允许值

| 元素 Cmd 的允许值            |                                         |
|------------------------|-----------------------------------------|
| WIDGET_ITEM_GET_XSIZE  | 该函数返回给定项目的                              |
| WIDGET_ITEM_GET_YSIZE  | 该函数返回给定项目的                              |
| WIDGET_ITEM_DRAW       | 函数在指定位置绘制指定项目。                          |
| WIDGET_DRAW_BACKGROUND | 要绘制的小工具的背景。                             |
| WIDGET_DRAW_OVERLAY    | 此命令在所有其他绘制操作都完成后发出，目的是可以在小工具之上绘制一些覆盖项目。 |

## 返回值

取决于给定命令

## 对命令的反应

该函数必须对在 WIDGET\_ITEM\_DRAW\_INFO 结构中给定的命令作出反应，以下 2 种方式之一执行：

- 1、调用特定控件提供的适当默认函数（例如，LISTBOX\_OwnerDraw()）
- 2、提供作出相应反应的代码

## 命令

该函数支持并应对以下所列命令作出反应，如上所描述，应为所有未处理函数调用默认自画函数，这可减小代码大小（例如，如果高度与默认高度相同），并确保代码与将来的软件新版本和引入的其他命令保持兼容。

**WIDGET\_ITEM\_GET\_XSIZE**

必须返回给定项目的 X 尺寸，以像素为单位。

**WIDGET\_ITEM\_GET\_YSIZE**

必须返回给定项目的 Y 尺寸（高度），以像素为单位。

**WIDGET\_ITEM\_DRAW**

必须绘制给定项目，WIDGET\_ITEM\_DRAW\_INFO 结构的 x0 和 y0 指定项目在窗口坐标中的位置。该项目必须填充其整个矩形，该矩形由提供给函数的起始位置 x0、y0 以及函数反应命令 WIDGET\_ITEM\_GET\_YSIZE、WIDGET\_ITEM\_GET\_XSIZE 返回的尺寸定义。此矩形区域的任何部分都不得留空，它不能绘制到此矩形外，因为在调用该函数之前已设置了裁剪矩。

## 第十四章 BUTTON 控件

从本章开始我们就讲解 STemWin 中各个控件，首先讲解的就是最常用的按钮控件，按钮控件通常用作触摸屏的主要用户界面元素，如果按钮有输入焦点，它还会对键 GUI\_KEY\_SPACE 和 GUI\_KEY\_ENTER 作出反应，本章分为如下几部分：

- 14.1 BUTTON 控件简介
- 14.2 通知代码
- 14.3 键盘反应
- 14.4 常用 API 函数
- 14.5 BUTTON 控件基础例程
- 14.6 BUTTON 控件显示位图

## 14.1 BUTTON 控件简介

按钮是常用的一种控件，可以在按钮上显示文本或者位图，默认情况下按钮显示如图 14.1.1 所示。



图 14.1.1 默认按钮显示效果

我们可以通过设置按钮的皮肤改变按钮的显示效果，设置按钮皮肤以后显示如图 14.1.2 所示。



图 14.1.2 设置皮肤后的按钮显示效果

## 14.2 通知代码

以下事件是按钮小工具作为 WM\_NOTIFY\_PARENT 消息的一部分发送给其父窗口的，我们通常根据以下通知代码来编写按钮的功能的，尤其是 GUIBulider 生成的代码中，表 14.2.1 列出了可能的通知代码。

| 消息                        | 描述                     |
|---------------------------|------------------------|
| WM_NOTIFICATION_CLICKED   | 按钮已被点击。                |
| WM_NOTIFICATION_RELEASED  | 按钮已被释放。                |
| WM_NOTIFICATION_MOVED_OUT | 按钮已被点击，且指针已移出按钮并且没有释放。 |

表 14.2.1 BUTTON 通知代码

## 14.3 键盘反应

如果小工具具有输入焦点，则它将对下列各键做出反应，表 14.3.1 列出了按键反应：

| 按键            | 反应                                 |
|---------------|------------------------------------|
| GUI_KEY_ENTER | 此类键按下后，按钮认为它被按下并立即释放，然后作出相应反应。     |
| GUI_KEY_SPACE | 此类键按下后，按钮状态改变为已按下。键释放后，按钮状态改变为未按下。 |

表 14.3.1 BUTTON 的按键反应

## 14.4 常用 API 函数

表 14.4.1 列举出了 BUTTON 的 API 函数：

| 函数                                      | 描述                 |
|-----------------------------------------|--------------------|
| <a href="#">BUTTON_Create()</a>         | 创建 BUTTON 小工具      |
| <a href="#">BUTTON_CreateAsChild()</a>  | 将 BUTTON 小工具创建为子窗口 |
| <a href="#">BUTTON_CreateEx()</a>       | 创建 BUTTON 小工具      |
| <a href="#">BUTTON_CreateIndirect()</a> | 从资源表项创建 BUTTON 小工具 |

表 14.4.1 BUTTON 控件 API 函数



续表 14.4.1

| 函数                           | 描述                            |
|------------------------------|-------------------------------|
| BUTTON_CreateUser()          | 使用额外字节作为用户数据创建 BUTTON 小工具。    |
| BUTTON_GetBitmap()           | 返回指向 BUTTON 位图的指针。            |
| BUTTON_GetBkColor()          | 返回 BUTTON 的背景色。               |
| BUTTON_GetDefaultBkColor()   | 返回 BUTTON 小工具的默认背景色。          |
| BUTTON_GetDefaultFont()      | 返回 BUTTON 小工具的默认字体。           |
| BUTTON_GetDefaultTextAlign() | 返回 BUTTON 小工具的默认文本对齐方式。       |
| BUTTON_GetDefaultTextColor() | 返回 BUTTON 小工具的默认文本颜色。         |
| BUTTON_GetFont()             | 返回 BUTTON 小工具字体的指针。           |
| BUTTON_GetText()             | 检索指定 BUTTON 的文本。              |
| BUTTON_GetTextAlign()        | 返回 BUTTON 文本的对齐方式。            |
| BUTTON_GetTextColor()        | 返回指定 BUTTON 的文本颜色。            |
| BUTTON_GetUserData()         | 检索用 BUTTON_SetUserData 设置的数据。 |
| BUTTON_IsPressed()           | 返回表示按钮是否按下的值。                 |
| BUTTON_SetBitmap()           | 设置显示 BUTTON 时使用的位图。           |
| BUTTON_SetBitmapEx()         | 设置显示 BUTTON 时使用的位图。           |
| BUTTON_SetBkColor()          | 设置按钮的背景色。                     |
| BUTTON_SetBMP()              | 设置显示 BUTTON 时使用的位图。           |
| BUTTON_SetBMPEx()            | 设置显示 BUTTON 时使用的位图。           |
| BUTTON_SetDefaultBkColor()   | 设置 BUTTON 小工具的默认背景色。          |
| BUTTON_SetDefaultFont()      | 设置 BUTTON 小工具的默认字体。           |
| BUTTON_SetDefaultTextAlign() | 设置 BUTTON 小工具的默认文本对齐方式。       |
| BUTTON_SetDefaultTextColor() | 设置 BUTTON 小工具的默认文本颜色          |
| BUTTON_SetFocussable()       | 设置接收输入焦点的能力。                  |
| BUTTON_SetFont()             | 选择文本的字体。                      |
| BUTTON_SetPressed()          | 将按钮的状态设置为按下或未按下。              |
| BUTTON_SetReactOnLevel()     | 将所有 BUTTON 小工具设置为对水平做出反应。     |
| BUTTON_SetStreamedBitmap()   | 设置显示 BUTTON 小工具时使用的位图。        |
| BUTTON_SetStreamedBitmapEx() | 设置显示 BUTTON 小工具时使用的位图。        |
| BUTTON_SetText()             | 设置文本。                         |
| BUTTON_SetTextAlign()        | 设置 BUTTON 文本的对齐方式。            |
| BUTTON_SetTextColor()        | 设置文本的颜色。                      |
| BUTTON_SetTextOffset()       | 根据当前的文本对齐方式设置，调节按钮文本的位置。      |
| BUTTON_SetUserData()         | 设置 BUTTON 小工具的额外数据。           |

表 14.4.1 BUTTON 控件 API 函数。

## 14.5 BUTTON 控件基础例程

### 14.5.1 程序设计

我们用 GUIBulider 创建一个如图 14.5.1 所示的 GUI 界面，此界面包括一个 Frame，两个 BUTTON，两个 BUTTON 用来控制开发板上的 LED1 和蜂鸣器的开关，界面分辨率为 800\*480，



大家也可以根据自己所使用的 LCD 来设置合适的界面大小，实验完整工程为：[EMWIN 实验 15 BUTTON 使用](#)，打开 ButtonUse.c 文件，源码如下：

```
#include "DIALOG.h"
#include "ButtonUse.h"
#include "led.h"           //包含 LED 和 BEEP 头文件
#include "beep.h"

#define ID_FRAMEWIN_0 (GUI_ID_USER + 0x00)
#define ID_BUTTON_0 (GUI_ID_USER + 0x01)
#define ID_BUTTON_1 (GUI_ID_USER + 0x02)

//对话框资源表
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 800, 480, 0, 0x64, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 300, 122, 150, 50, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_1, 300, 251, 150, 50, 0, 0x0, 0 },
};

//对话框回调函数
static void _cbDialog(WM_MESSAGE * pMsg)
{
    WM_HWIN hItem;
    int     NCode;
    int     Id;

    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:                                     (1)
            //初始化对话框
            hItem = pMsg->hWin;
            FRAMEWIN_SetTitleHeight(hItem, 30);
            FRAMEWIN_SetText(hItem, "ALIENTE BUTTON USE");
            FRAMEWIN_SetFont(hItem, GUI_FONT_24_ASCII);
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
            FRAMEWIN_SetTextColor(hItem, 0x0000FFFF);

            //初始化 BUTTON0
            hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
            BUTTON_SetFont(hItem, GUI_FONT_24_ASCII);
            BUTTON_SetText(hItem, "LED1");

            //初始化 BUTTON1
```

```

        hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
        BUTTON_SetText(hItem, "BEEP");
        BUTTON_SetFont(hItem, GUI_FONT_24_ASCII);
        break;
    case WM_NOTIFY_PARENT:
        Id = WM_GetId(pMsg->hWinSrc);
        NCode = pMsg->Data.v;
        switch(Id)
        {
            case ID_BUTTON_0: //BUTTON_0 的通知代码, 控制 LED1
                switch(NCode)
                {
                    case WM_NOTIFICATION_CLICKED:
                        break;
                    case WM_NOTIFICATION_RELEASED: //按钮被按下并释放 (4)
                        LED1=~LED1; //LED1 反转 (5)
                        break;
                }
                break;
            case ID_BUTTON_1: //BUTTON_1 的通知代码, 控制 BEEP
                switch(NCode)
                {
                    case WM_NOTIFICATION_CLICKED:
                        break;
                    case WM_NOTIFICATION_RELEASED:
                        BEEP=~BEEP; //蜂鸣器反转 (6)
                        break;
                }
                break;
        }
        break;
    default:
        WM_DefaultProc(pMsg);
        break;
    }
}

//创建一个对话框
WM_HWIN CreateFramewin(void)
{
    WM_HWIN hWin;

    hWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate), _cbDialog,

```

```
WM_HBKWIN, 0, 0);
    return hWin;
}

//BUTTO 演示函数
void ButtonUse_Demo(void)
{
    WM_HWIN hWin;
    hWin=CreateFrameWin();
    while(1)
    {
        GUI_Delay(100);
    }
}
```

(1)、WM\_INIT\_DIALOG 消息，用来初始化对话框中的小工具。

(2)、子控件向父控件发送的通知消息，在这个消息中我们根据不同控件的不同消息类型来做出不同的处理。

(3)、小控件的通知代码被放到结构体 WM\_MESSAGE 的变量 Data.v 中，这里我们获取到 BUTTON 按钮的通知代码，通过分析通知代码来判断 BUTTO 是被按下还是被释放了的。

(4)、BUTTON 按钮被释放了。

(5)、控制 LED 开关。

(6)、控制蜂鸣器开关。

#### 14.5.2 下载验证

编译并下载到开发板中，结果显示如图 14.5.1 所示，图 14.5.1 是设置了皮肤的，请大家注意了。

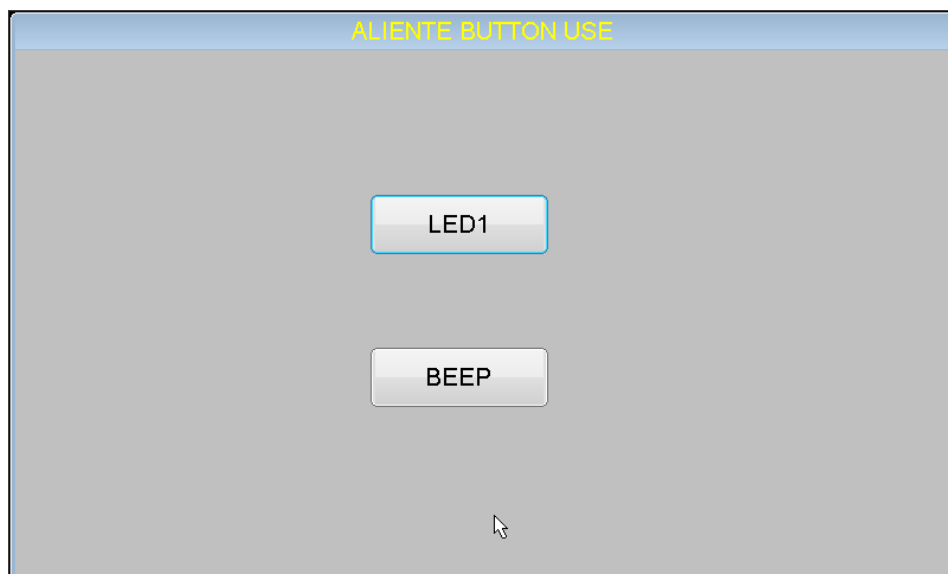


图 14.5.1 按钮显示效果图

通过按下 LED1 和 BEEP 按钮来控制开发板上 LED1 和蜂鸣器的打开和关闭。

## 14.6 BUTTON 控件显示位图

上一小节我们讲解了如何使用 emwin 的 BUTTION 控件，有时候我们希望在按钮上显示一些图片以区分打开和关闭，比如上一小节中打开 LED，使用 emwin 自带的 BUTTON 控件的时候打开和关闭 LED 时 BUTTON 的状态都是一样的，不能很只管的判断出此时 LED 灯的状态，需要我们去查看 LED 的亮灭才能判断。emwin 提供了在 BUTTON 上显示位图的 API 函数，那么我们就可以在打开和关闭的时候通过显示不同的位图来区分开、关这两种状态，本节就讲解一下如何在 BUTTON 上显示位图。

### 14.6.1 制作位图 C 文件

首先我们肯定需要制作两个位图文件，当关闭的时候我们在 BUTTON 上显示图 14.6.1 所示图片，打开的时候显示图 14.6.2 所示图片，这两种图片的格式均为 BMP。



图 14.6.1 OFF



图 14.6.2 ON

在第七章我们就已经讲过了位图转换器的使用方法，本节我们在回顾一下，首先打开 OFF 图片，打开后如图 14.6.3 所示。

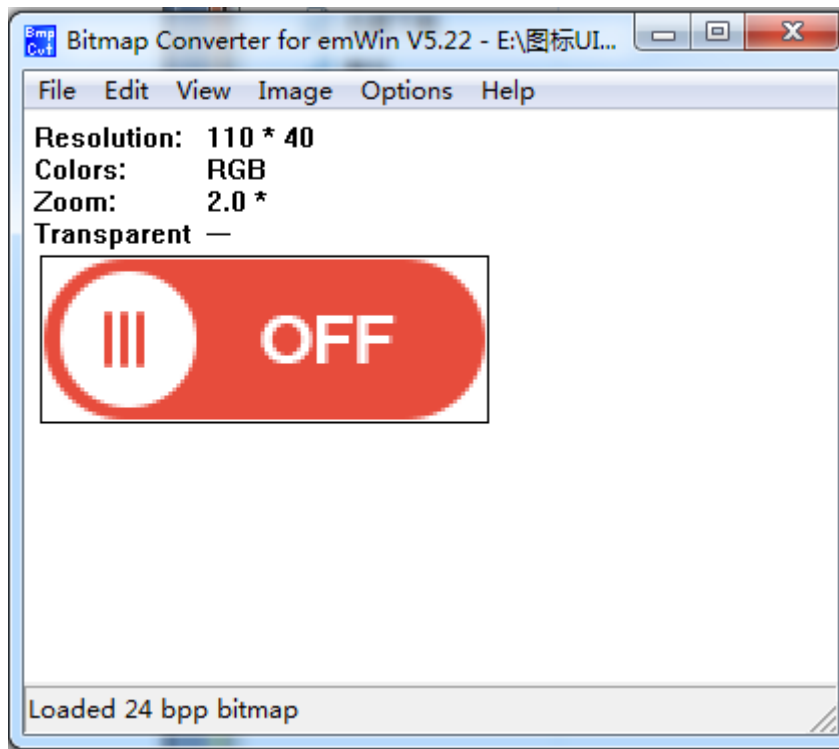


图 14.6.3 使用位图转换器打开 OFF 图片

设置色彩转换，点击 Image->Convert to->Best palette，设置完成以后如图 14.6.4 所示。

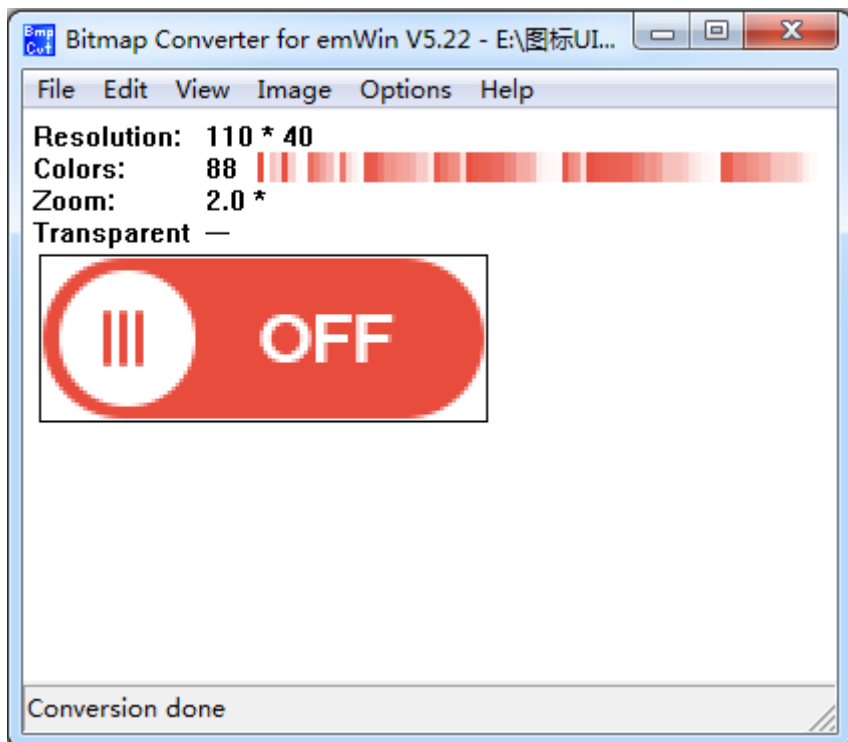


图 14.6.4 设置色彩转换

设置为色彩转换格式以后就可以保存为 C 文件了，点击 File->Save As，提示如图 14.6.5 所示对话框，设置好保存的文件类型和保存路径，我这里是保存在桌面上，注意图 14.6.5 红框中的保存类型！

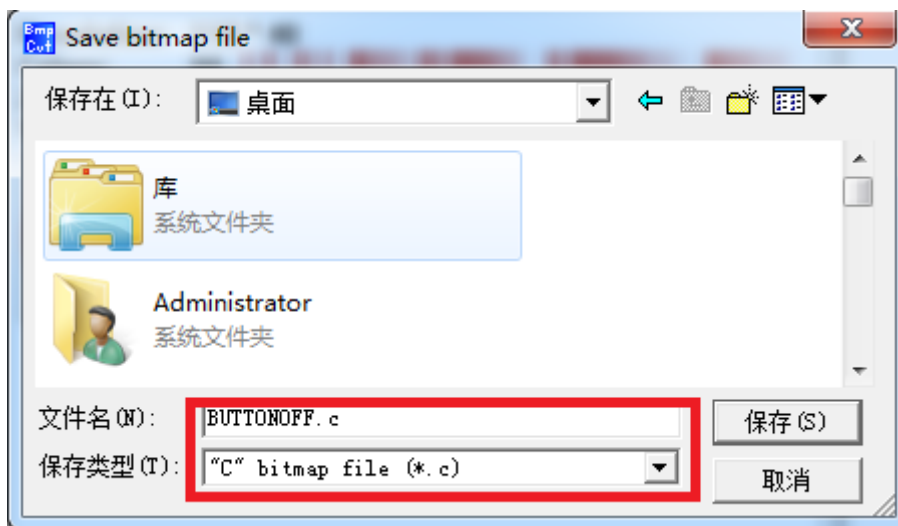


图 14.6.5 保存 C 文件

点击图 14.6.5 中的保存以后又会跳出图 14.6.6 所示对话框，用来设置生成的格式，这里选择 “High color(565)”。

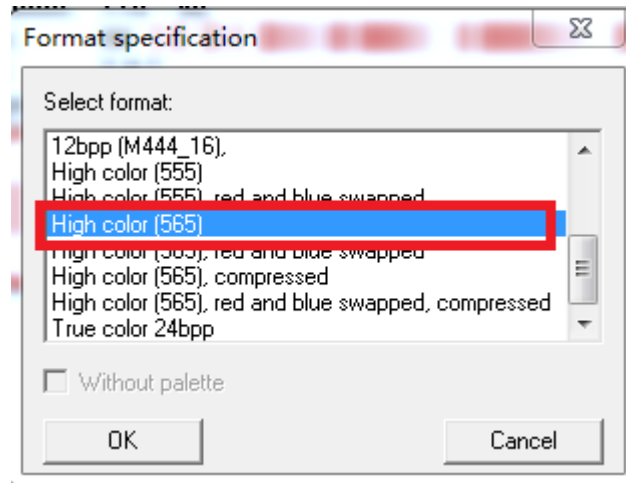


图 14.6.6 设置格式

设置完图 14.6.6 中的格式以后点击“OK”按钮，这样就会在刚刚设置的保存路径下生成一个 C 文件，这个 C 文件就是位图转换器生成的位图文件，我们一会要用这个位图文件，用同样的方法着 ON 图片的位图文件。

### 14.6.2 GUIBulider 设计界面

用 GUIBulider 设计如图 14.6.7 所示界面。

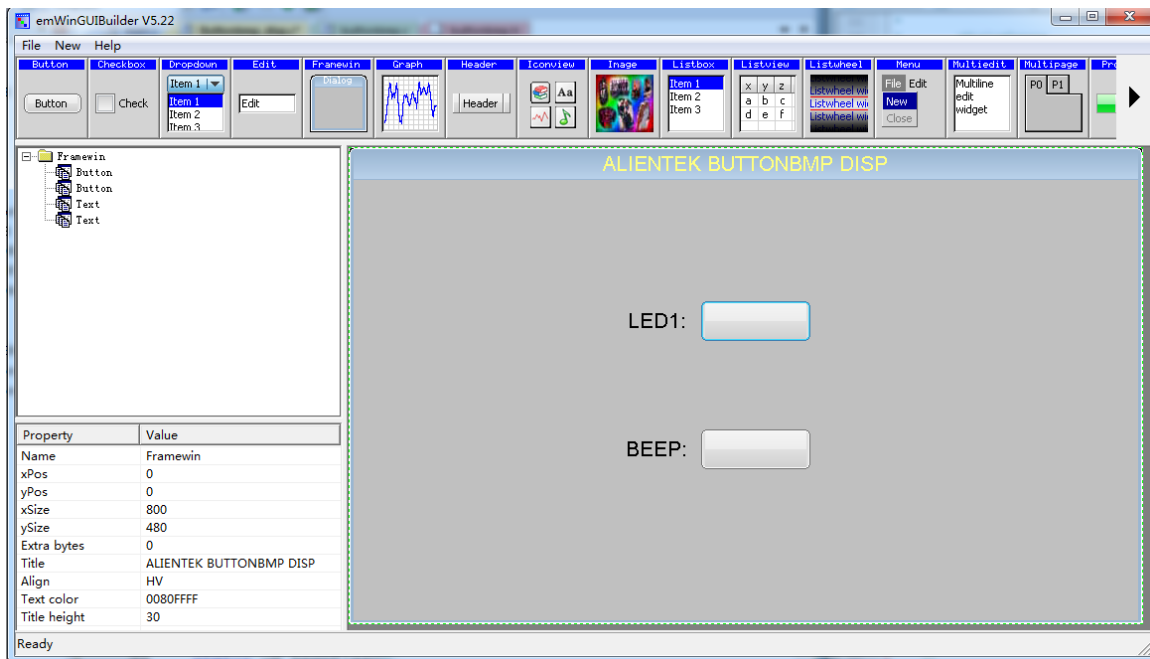


图 14.6.7 界面设计

图 14.6.7 用 GUIBulider 设计出的界面中使用了一个我们还没有讲解过的控件：TEXT，我们用 TEXT 控件显示“LED1:”和“BEEP:”这两个提示信息。图中的按钮还是使用的 emwin 默认的按钮类型，要想修改为位图就需要修改生成的 C 文件，注意：BUTTON 控件上没有显示字符，GUIBulider 只是生成一个大体的程序框架而已，生成的 C 程序如下所示，为了节省篇幅，这里删除了部分注释行。

```
#include "DIALOG.h"
```

```
/******
```

```

*      Defines
*****

*/

#define ID_FRAMEWIN_0 (GUI_ID_USER + 0x00)
#define ID_BUTTON_0 (GUI_ID_USER + 0x01)
#define ID_BUTTON_1 (GUI_ID_USER + 0x02)
#define ID_TEXT_0 (GUI_ID_USER + 0x03)
#define ID_TEXT_1 (GUI_ID_USER + 0x04)

/*****

*

*      _aDialogCreate
*/

static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] = {
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 800, 480, 0, 0x64, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 350, 122, 110, 40, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_1, 350, 251, 110, 40, 0, 0x0, 0 },
    { TEXT_CreateIndirect, "Text", ID_TEXT_0, 266, 132, 80, 20, 0, 0x64, 0 },
    { TEXT_CreateIndirect, "Text", ID_TEXT_1, 266, 261, 80, 20, 0, 0x64, 0 },
};

/*****

*

*      _cbDialog
*/

static void _cbDialog(WM_MESSAGE * pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;

    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:
            //
            // Initialization of 'Framewin'
            //
            hItem = pMsg->hWin;
            FRAMEWIN_SetTitleHeight(hItem, 30);
            FRAMEWIN_SetText(hItem, "ALIENTEK BUTTONBMP DISP");
            FRAMEWIN_SetFont(hItem, GUI_FONT_24_ASCII);
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
            FRAMEWIN_SetTextColor(hItem, 0x0000FFFF);
    }
}

```

```

//
// Initialization of 'Button'
//
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
BUTTON_SetText(hItem, "");
//
// Initialization of 'Button'
//
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
BUTTON_SetText(hItem, "");
//
// Initialization of 'Text'
//
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
TEXT_SetFont(hItem, GUI_FONT_32_ASCII);
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetText(hItem, "LED1:");
//
// Initialization of 'Text'
//
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_1);
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_32_ASCII);
TEXT_SetText(hItem, "BEEP:");
break;
case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;
    switch(Id)
    {
        case ID_BUTTON_0: // Notifications sent by 'Button'
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    break;
            }
            break;
        case ID_BUTTON_1: // Notifications sent by 'Button'
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:

```



```

        break;
    case WM_NOTIFICATION_RELEASED:
        break;
    }
    break;
}
break;
default:
    WM_DefaultProc(pMsg);
    break;
}
}

/*****
*
*      CreateFramewin
*/
WM_HWIN CreateFramewin(void);
WM_HWIN CreateFramewin(void)
{
    WM_HWIN hWin;
    hWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate)\
        , _cbDialog, WM_HBKWIN, 0, 0);

    return hWin;
}

```

上面程序就是用 GUIBulider 生成的模板文件，接下来我们就要在这个模板文件中修修改改、缝缝补补的完成本次实验。新建 buttonbmp.c 文件将生成的位图文件都放进去，整理好的位图文件如下所示：

```

#include "buttonbmp.h"
#include <stdlib.h>

static GUI_CONST_STORAGE unsigned short _acBUTTONOFF[] =
{

    //数组数据

};

static GUI_CONST_STORAGE unsigned short _acBUTTONON[] =
{

    //数组数据

```

```

};

GUI_CONST_STORAGE GUI_BITMAP bmBUTTONOFF = {
    110, // xSize
    40, // ySize
    220, // BytesPerLine
    16, // BitsPerPixel
    (unsigned char *)_acBUTTONOFF, // Pointer to picture data
    NULL, // Pointer to palette
    GUI_DRAW_BMP565
};

GUI_CONST_STORAGE GUI_BITMAP bmBUTTONON = {
    110, // xSize
    40, // ySize
    220, // BytesPerLine
    16, // BitsPerPixel
    (unsigned char *)_acBUTTONON, // Pointer to picture data
    NULL, // Pointer to palette
    GUI_DRAW_BMP565
};

```

在 buttonbmp.c 文件就是将前面生成的那两个位图 C 文件复制到一个 C 文件里面,很简单,这里就不讲解了,上面为了篇幅原因没贴出数组 \_acBUTTONOFF[] 和 \_acBUTTONON 中的数据,新建 buttonbmp.h 文件,添加如下代码:

```

#ifndef _BUTTONBMP_H
#define _BUTTONBMP_H
#include "sys.h"
#include "GUI.h"
#ifndef GUI_CONST_STORAGE
#define GUI_CONST_STORAGE const
#endif

extern GUI_CONST_STORAGE GUI_BITMAP bmBUTTONOFF;
extern GUI_CONST_STORAGE GUI_BITMAP bmBUTTONON;
#endif

```

buttonbmp.h 文件主要是声明了两个 GUI\_BITMAP 结构体变量: bmBUTTONOFF 和 bmBUTTONON,这样我们就可以在其他文件中调用这两个结构体变量,可以简单的理解为这两个结构体变量就是图片 OFF 和 ON。

### 14.6.3 程序设计

本实验完整工程为: **EMWIN 实验 16 BUTTON 位图显示**,打开 buttonbmpdisp.c 文件,这个文件就是在 GUIBulider 生成的 C 文件基础上修改的,打开以后程序源码如下:

```

#define ID_FRAMEWIN_0 (GUI_ID_USER + 0x00)

```

```

#define ID_BUTTON_0 (GUI_ID_USER + 0x01)
#define ID_BUTTON_1 (GUI_ID_USER + 0x02)
#define ID_TEXT_0 (GUI_ID_USER + 0x03)
#define ID_TEXT_1 (GUI_ID_USER + 0x04)

GUI_BITMAP buttonbmp_tab[2];

//对话框资源表
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 800, 480, 0, 0x64, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 350, 122, 110, 40, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_1, 350, 251, 110, 40, 0, 0x0, 0 },
    { TEXT_CreateIndirect, "Text", ID_TEXT_0, 266, 132, 80, 20, 0, 0x64, 0 },
    { TEXT_CreateIndirect, "Text", ID_TEXT_1, 266, 261, 80, 20, 0, 0x64, 0 },
};

//对话框回调函数
static void _cbDialog(WM_MESSAGE * pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;
    static u8 ledflag=0;
    static u8 beepflag=0;

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_WHITE);
            GUI_Clear();
            break;
        case WM_INIT_DIALOG:
            //初始化对话框
            hItem = pMsg->hWin;
            FRAMEWIN_SetTitleHeight(hItem, 30);
            FRAMEWIN_SetText(hItem, "ALIENTEK BUTTONBMP DISP");
            FRAMEWIN_SetFont(hItem, GUI_FONT_24_ASCII);
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
            FRAMEWIN_SetTextColor(hItem, 0x0000FFFF);

            //初始化 BUTTON0
            hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);

```

(1)

```

BUTTON_SetBitmapEx(hItem,0,&buttonbmp_tab[0],0,0);
BUTTON_SetText(hItem, "");

//初始化 BUTTON1
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
BUTTON_SetBitmapEx(hItem,0,&buttonbmp_tab[0],0,0);
BUTTON_SetText(hItem, "");

//初始化 TEXT0
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_32_ASCII);
TEXT_SetText(hItem, "LED1:");

//初始化 TEXT1
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_1);
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_32_ASCII);
TEXT_SetText(hItem, "BEEP:");
break;
case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;
    switch(Id)
    {
        case ID_BUTTON_0: //BUTTON_0 的通知代码，控制 LED1
            hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED: //按钮被按下并释放
                    LED1=~LED1; //LED1 反转
                    ledflag=~ledflag;
                    BUTTON_SetBitmapEx(hItem,0,ledflag?
                        &buttonbmp_tab[1]:&buttonbmp_tab[0],0,0);
                    break;
            }
            break;
        case ID_BUTTON_1: //BUTTON_1 的通知代码，控制 BEEP
            hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
            switch(NCode)
            {

```

```

        case WM_NOTIFICATION_CLICKED:
            break;
        case WM_NOTIFICATION_RELEASED:
            BEEP=~BEEP;           //蜂鸣器反转
            beepflag=~beepflag;
            BUTTON_SetBitmapEx(hItem,0,beepflag?\           (5)
                               &buttonbmp_tab[1]:&buttonbmp_tab[0],0,0);
            break;
    }
    break;
}
break;
default:
    WM_DefaultProc(pMsg);
    break;
}
}

//创建一个对话框
WM_HWIN CreateFramewin(void)
{
    WM_HWIN hWin;
    buttonbmp_tab[0]=bmBUTTONOFF;
    buttonbmp_tab[1]=bmBUTTONON;
    hWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate)\
                               , _cbDialog, WM_HBKWIN, 0, 0);

    return hWin;
}

//BUTTON 按钮上显示位图
void Buttonbmp_Demo(void)
{
    WM_HWIN hWin;
    hWin=CreateFramewin();
    while(1)
    {
        GUI_Delay(100);
    }
}

```

(1)、WM\_PAINT 消息，使用 GUIBulider 生成的 C 文件中没有这个消息，这里需要我们手动添加进去，当接收到 WM\_PAINT 消息的时候就将对话框的背景色刷新为白色。

(2)、初始化 BUTTON0 的时候调用函数 BUTTON\_SetBitmapEx()在 BUTTON 上显示位图 OFF。

(3)、初始化 BUTTON1 的时候调用函数 `BUTTON_SetBitmapEx()`在 BUTTON 上显示位图 OFF。

(4)、根据 `ledflag` 的不同调用函数 `BUTTON_SetBitmapEx()`在 BUTTON 上显示 OFF 或者 ON。

(5)、根据 `beepflag` 的不同调用函数 `BUTTON_SetBitmapEx()`在 BUTTON 上显示 OFF 或者 ON。

注意：本例程要设置换肤！

#### 14.6.4 下载验证

编译并下载到开发板中，显示如图 14.6.8 所示。

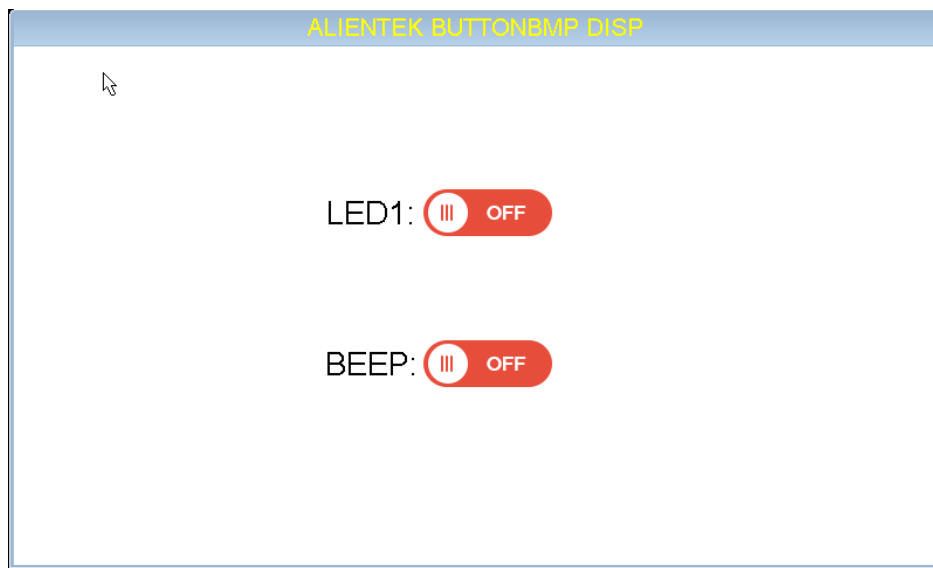


图 14.6.8 显示效果图

点击 LED1 和 BEEP 的开关，显示如图 14.6.9 所示，并且 LED1 点亮，蜂鸣器响。

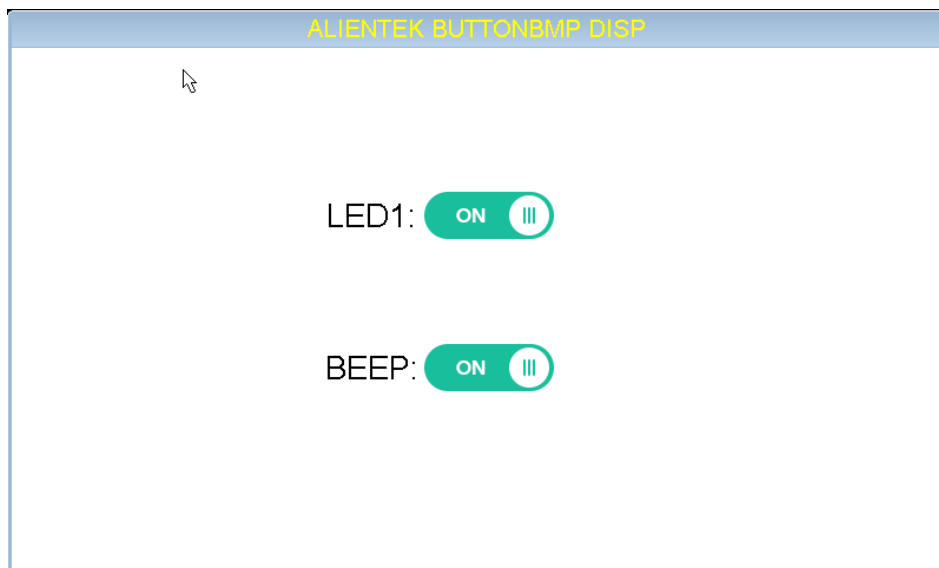


图 14.6.9 打开 LED1 和 BEEP

本例程所使用的 LCD 屏幕分辨率为 800\*480，如果在其他低于此分辨率的 LCD 上运行此例程的话可能导致显示不完全，请自行修改控件大小和位置以适应所使用的 LCD 屏。

## 第十五章 CHECKBOX 控件

复选框是选择各种不同选项的最常用小工具之一，用户可选中或取消选中复选框，并且可一次选中任意个框。使用键盘界面时，被聚焦的复选框的状态可使用 <SPACE> 键切换，禁用的框显示为灰色，本章就讲解一下 emwin 的复选框。

15.1 通知代码

15.2 键盘反应

15.3 CHECKBOX 控件 API 函数

15.4 CHECKBOX 控件演示例程

## 15.1 通知代码

表 15.1.1 列举出了复选框小工具作为 WM\_NOTIFY\_PARENT 消息的一部分发送给其父窗口的消息类型：

| 消息                            | 描述                     |
|-------------------------------|------------------------|
| WM_NOTIFICATION_CLICKED       | 复选框已被点击。               |
| WM_NOTIFICATION_RELEASED      | 复选框已被释放。               |
| WM_NOTIFICATION_MOVED_OUT     | 复选框已被点击，且指针已移出框并且没有释放。 |
| WM_NOTIFICATION_VALUE_CHANGED | 复选框的状态已改变。             |

表 15.1.CHECKBOX 通知代码

## 15.2 键盘反应

如果 CHECKBOX 控件具有输入焦点，则它将对表 15.2.1 所列的各键做出反应：

| 按键            | 反应          |
|---------------|-------------|
| GUI_KEY_SPACE | 切换小工具的选中状态。 |

表 15.2.1 CHECKBOX 的键盘反应

## 15.3 CHECKBOX 控件 API 函数

表 15.3.1 列举出了 CHECKBOX 控件 API 函数

| 函数                              | 描述                                |
|---------------------------------|-----------------------------------|
| CHECKBOX_Check()                | 将复选框的状态设置为选中。(弃用)                 |
| CHECKBOX_Create()               | 创建 CHECKBOX 小工具。(弃用)              |
| CHECKBOX_CreateEx()             | 创建 CHECKBOX 小工具。                  |
| CHECKBOX_CreateIndirect()       | 从资源表项创建 CHECKBOX 小工具。             |
| CHECKBOX_CreateUser()           | 使用额外字节作为用户数据创建 CHECKBOX 小工具。      |
| CHECKBOX_GetDefaultBkColor()    | 返回 CHECKBOX 小工具文本的默认背景            |
| CHECKBOX_GetDefaultFont()       | 返回用于显示 CHECKBOX 小工具文本的默认字体。       |
| CHECKBOX_GetDefaultSpacing()    | 返回 CHECKBOX 小工具文本和框之间的默认间距。       |
| CHECKBOX_GetDefaultTextAlign()  | 返回用于显示 CHECKBOX 小工具文本的默认对齐方式。     |
| CHECKBOX_GetDefaultTextColor()  | 返回用于显示 CHECKBOX 小工具文本的默认文本颜色。     |
| CHECKBOX_GetState()             | 返回复选框的当前状态。                       |
| CHECKBOX_GetText()              | 返回复选框的文本。                         |
| CHECKBOX_GetUserData()          | 检索使用 CHECKBOX_SetUserData()设置的数据。 |
| CHECKBOX_IsChecked()            | 返回复选框的当前状态(选中或未选中)                |
| CHECKBOX_SetBkColor()           | 设置给定 CHECKBOX 小工具的背景色。            |
| CHECKBOX_SetBoxBkColor()        | 设置复选框区域的背景色。                      |
| CHECKBOX_SetDefaultBkColor()    | 设置 CHECKBOX 小工具的默认背景色。            |
| CHECKBOX_SetDefaultFocusColor() | 设置 CHECKBOX 小工具的默认焦点矩形颜色。         |
| CHECKBOX_SetDefaultFont()       | 设置用于显示 CHECKBOX 小工具文本的默认字体。       |
| CHECKBOX_SetDefaultImage()      | 设置复选框被选中后要显示的默认图像。                |
| CHECKBOX_SetDefaultSpacing()    | 设置 CHECKBOX 小工具文本和框之间的默认间距。       |

表 15.3.1 CHECKBOX 控件 API 函数



续表 15.3.1

| 函数                             | 描述                            |
|--------------------------------|-------------------------------|
| CHECKBOX_SetDefaultTextAlign() | 设置用于显示复选框文本的默认对齐方式。           |
| CHECKBOX_SetDefaultTextColor() | 设置用于显示 CHECKBOX 小工具文本的默认文本颜色。 |
| CHECKBOX_SetFocusColor()       | 设置聚焦框的颜色。                     |
| CHECKBOX_SetFont()             | 设置复选框字体。                      |
| CHECKBOX_SetImage()            | 设置复选框被选中后要显示的图像。              |
| CHECKBOX_SetNumStates()        | 设置复选框的可能状态的数量(2 或 3)          |
| CHECKBOX_SetSpacing()          | 设置框和复选框文本之间的间距。               |
| CHECKBOX_SetState()            | 设置 CHECKBOX 小工具的状态。           |
| CHECKBOX_SetText()             | 设置 CHECKBOX 小工具的文本。           |
| CHECKBOX_SetTextAlign()        | 设置用于显示 CHECKBOX 小工具文本的对齐方式。   |
| CHECKBOX_SetTextColor()        | 设置用于显示 CHECKBOX 小工具文本的颜色。     |
| CHECKBOX_SetUserData()         | 设置 CHECKBOX 小工具的额外数据          |
| CHECKBOX_Uncheck()             | 将复选框的状态设置为未选中。(弃用)            |

表 15.3.1 CHECKBOX 控件 API 函数

## 15.4 CHECKBOX 控件演示例程

### 15.4.1 程序设计

Segger 官方提供了一个关于 CHECKBOX 控件的演示例程，这里我们将这个例程移植到开发板中，学习一下 CHECKBOX 控件的使用方法，本实验完整工程为：**EMWIN 实验 17 CHECKBOX 小工具**。在 EMWIN\_DEMO 下的 CHECKBOXDEMO 文件夹里面有 5 个文件，分别为：BitmapBar.c、BitmapCheckXL.c、BitmapCheckXXL.c、checkboxdemo.c 和 checkboxdemo.h。其中前三个里面定义了本实验需要用到的位图，checkboxdemo.c 文件是我们本次实验的重点，打开 checkboxdemo.c 文件，程序源码如下：

```
#include "checkboxdemo.h"
#include "GUI.h"
#include "DIALOG.h"

//对话框资源表
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] = {
    { FRAMEWIN_CreateIndirect, "Check box sample", 0, 10, 10, 300, 220, \
      FRAMEWIN_CF_MOVEABLE },
    { TEXT_CreateIndirect, "Enabled:", 0, 5, 10, 120, 0 },
    { CHECKBOX_CreateIndirect, 0, GUI_ID_CHECK0, 5, 30, 120, 0 },
    { CHECKBOX_CreateIndirect, 0, GUI_ID_CHECK1, 5, 60, 120, 0 },
    { CHECKBOX_CreateIndirect, 0, GUI_ID_CHECK2, 5, 90, 120, 20 },
    { CHECKBOX_CreateIndirect, 0, GUI_ID_CHECK3, 5, 125, 120, 26 },
    { TEXT_CreateIndirect, "Disabled:", 0, 150, 10, 120, 0 },
    { CHECKBOX_CreateIndirect, 0, GUI_ID_CHECK4, 150, 30, 120, 0 },
    { CHECKBOX_CreateIndirect, 0, GUI_ID_CHECK5, 150, 60, 120, 0 },
    { CHECKBOX_CreateIndirect, 0, GUI_ID_CHECK6, 150, 90, 120, 26 },
}
```

```

{ CHECKBOX_CreateIndirect, 0,          GUI_ID_CHECK7, 150, 125, 120, 26 },
{ BUTTON_CreateIndirect,  "OK",        GUI_ID_OK, 10, 165, 80, 30 },
{ BUTTON_CreateIndirect,  "Cancel", GUI_ID_CANCEL, 200, 165, 80, 30 },
};

//复选框文本
static const char* _apLabel[]={
    "Default",
    "3 States",
    "Box XL",
    "Box XXL"
};

//回调函数
static void _cbCallback(WM_MESSAGE *pMsg)
{
    int i;
    int NCode,Id;
    WM_HWIN hDlg,hItem;
    hDlg = pMsg->hWin;
    switch(pMsg->MsgId)
    {
        case WM_INIT_DIALOG:
            //获取 CHECKBOX 的句柄
            hItem = WM_GetDialogItem(hDlg,GUI_ID_CHECK0);
            for(i=0;i<8;i++) (1)
            {
                int Index=i%4;
                //获取 CHECKBOX 的句柄
                hItem = WM_GetDialogItem(hDlg,GUI_ID_CHECK0+i); (2)
                CHECKBOX_SetText(hItem,_apLabel[Index]);
                switch(Index)
                {
                    case 1: (3)
                        CHECKBOX_SetNumStates(hItem,3); (4)
                        CHECKBOX_SetImage(hItem,&_abmBar[0],\ (5)
                                           CHECKBOX_BI_INACTIV_3STATE);
                        CHECKBOX_SetImage(hItem,&_abmBar[1],\ (6)
                                           CHECKBOX_BI_ACTIV_3STATE);
                        CHECKBOX_SetState(hItem,2); (7)
                        break;
                    case 2: (8)
                        CHECKBOX_SetState(hItem,1);

```

```

        CHECKBOX_SetImage(hItem,&_abmXL[0],\
                           CHECKBOX_BI_INACTIV_CHECKED);
        CHECKBOX_SetImage(hItem,&_abmXL[1],\
                           CHECKBOX_BI_ACTIV_CHECKED);
        CHECKBOX_SetFont(hItem,&GUI_FontComic18B_ASCII);
        break;
    case 3:
        CHECKBOX_SetState(hItem,1);
        CHECKBOX_SetImage(hItem,&_abmXXL[0],\
                           CHECKBOX_BI_INACTIV_CHECKED);
        CHECKBOX_SetImage(hItem,&_abmXXL[1],\
                           CHECKBOX_BI_ACTIV_CHECKED);
        CHECKBOX_SetFont(hItem,&GUI_FontComic24B_ASCII);
        break;
    }
    //失能对话框右边的所有 CHECK 小工具
    if(i>=4)
    {
        WM_DisableWindow(hItem);
    }
}
break;
case WM_NOTIFY_PARENT:
    Id=WM_GetId(pMsg->hWinSrc); //小工具 ID
    NCode=pMsg->Data.v;          //通知代码
    switch(NCode)
    {
        case WM_NOTIFICATION_RELEASED:
            if(Id==GUI_ID_OK)
            {
                GUI_EndDialog(hDlg,0);    //关闭对话框
            }
            if(Id==GUI_ID_CANCEL)
            {
                GUI_EndDialog(hDlg,1);    //关闭对话框
            }
            break;
        }
    }
break;
default:
    WM_DefaultProc(pMsg);
}
}

```

```
//CHECKBOX 小工具演示 Demo
```

```
void CheckBoxDemo(void)
```

```
{
    GUI_CURSOR_Show();
    #if GUI_SUPPORT_MEMDEV
        WM_SetCreateFlags(WM_CF_MEMDEV);
    #endif
    WM_SetDesktopColor(GUI_BLACK);
    while(1)
    {
        GUI_ExecDialogBox(_aDialogCreate,GUI_COUNTOF(_aDialogCreate),\
                        &_cbCallback,0,0,0);

        GUI_Delay(500);
    }
}
```

(1)、WM\_INIT\_DIALOG 消息下，用来初始化 CHECKBOX 控件，因为一共有 8 个 CHECKBOX,所以要循环设置 8 次。

(2)、根据 i 的不同获取各个 CHECKBOX 控件的句柄。

(3)、一共有四排 CHECKBOX 小工具，每排两个，8 个 CHECKBOX 的排序是按照从上到下，从左到右。

(4)、调用 CHECKBOX\_SetNumStates()函数设置复选框的可选状态的数量，默认情况下复选框有 2 种状态，选中(1)和取消选中(0)，使用函数 CHECKBOX\_SetNumStates()可将可能状态增加到 3，如果复选框处于第三种状态，则该函数返回 2。

(5)、调用函数 CHECKBOX\_SetImage() 设置复选框被选中后显示的图像，CHECKBOX\_SetImage()函数的第三个参数含义见表 15.4.1。

| 参数允许值                         |                        |
|-------------------------------|------------------------|
| CHECKBOX_BI_INACTIV_UNCHECKED | 设置复选框被取消选中并禁用时显示的位图。   |
| CHECKBOX_BI_ACTIV_UNCHECKED   | 设置复选框被取消选中并启用时显示的位图。   |
| CHECKBOX_BI_INACTIV_CHECKED   | 设置复选框被选中并禁用时显示的位图。     |
| CHECKBOX_BI_ACTIV_CHECKED     | 设置复选框被选中并启用时显示的位图。     |
| CHECKBOX_BI_INACTIV_3STATE    | 设置复选框处于第三种状态并禁用时显示的位图。 |
| CHECKBOX_BI_ACTIV_3STATE      | 设置复选框处于第三种状态并启用时显示的位图。 |

表 15.4.1 函数参数允许值

这里我们设置的是第二排复选框处于第三种状态并且在禁用时的显示位图。

(6)、同上，设置第二排复选框处于第三种状态并且在启用时显示的位图。

(7)、调用函数 CHECKBOX\_SetState()设置复选框控件的状态，这里设置为 2，即第三种状态，函数 CHECKBOX\_SetState()第二个参数允许设置的值见表 15.4.2。

| 参数允许值 |       |
|-------|-------|
| 0     | 取消选中  |
| 1     | 选中    |
| 2     | 第三种状态 |

(8)、设置第二排两个复选框的属性。

(9)、设置第三排两个复选框的属性。

#### 15.4.2 下载验证

编译并下载到开发板中，LCD 显示如图 14.5.1 所示。

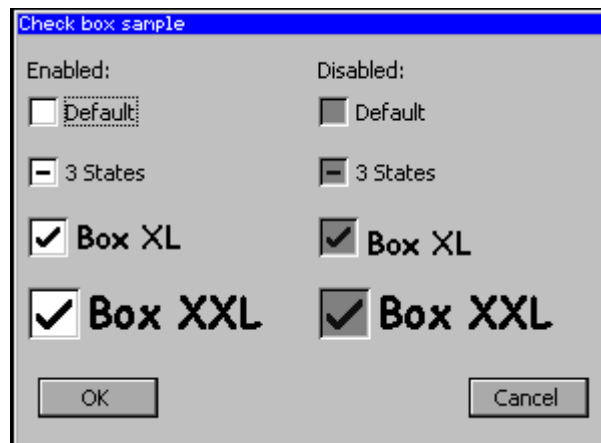


图 14.5.1 LCD 显示结果

点按 LCD 触摸屏上复选框，观察结果和我们设置的是否一致。

## 第十六章 DROPDOWN 控件

DROPDOWN 小工具用于从具有若干栏的列表中选择一个元素，它以非打开状态显示当前选择的项目，如果用户打开 DROPDOWN 小工具，就会出现一个选择新项目的 LISTBOX，本章分为如下几部分：

16.1 DROPDOWN 控件简介

16.2 通知代码

16.3 键盘反应

16.4 DROPDOWN 控件 API 函数

16.5 DROPDOWN 控件演示例程

## 16.1 DROPDOWN 控件简介

DROPDOWN 控件在打开和关闭情况下分别如图 16.1.1 和 16.1.2 所示。



图 16.1.1 DROPDOWN 关闭状态

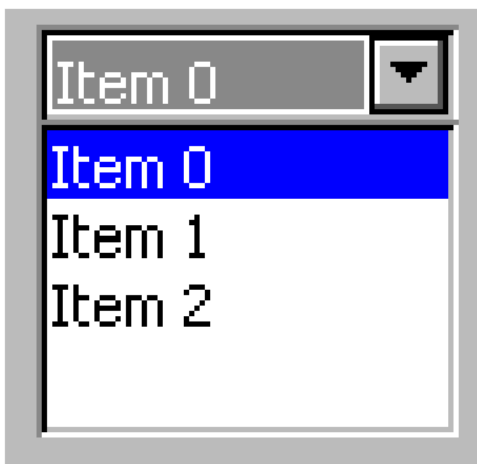


图 16.1.2 DROPDOWN 打开状态

DROPDOWN 可以换肤，换肤以后如图 16.1.3 所示：

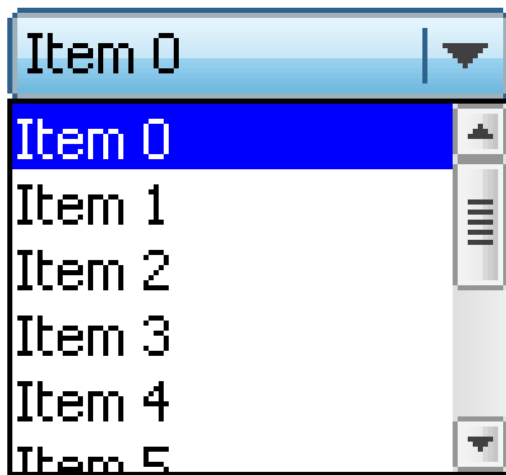


图 16.1.3 换肤后的 DROPDOWN 控件

## 16.2 通知代码

表 16.2.1 列举出了 DROPDOWN 小工具作为 WM\_NOTIFY\_PARENT 消息的一部分发送给其父窗口的消息类型：

| 消息                             | 描述                         |
|--------------------------------|----------------------------|
| WM_NOTIFICATION_CLICKED        | 已单击小工具。                    |
| WM_NOTIFICATION_RELEASED       | 已释放小工具。                    |
| WM_NOTIFICATION_MOVED_OUT      | 已单击小工具，并且指针已移出小工具，但没有释放。   |
| WM_NOTIFICATION_SCROLL_CHANGED | 已打开的下拉列表小工具的可选滚动条的滚动位置已改变。 |
| WM_NOTIFICATION_SEL_CHANGED    | 下拉列表的选择已更改。                |

表 16.2.1 DROPDOWN 通知代码

### 16.3 键盘反应

如果 DROPDOWN 控件具有输入焦点，则它将对表 16.3.1 所列的各键做出反应：

| 按键            | 反应                    |
|---------------|-----------------------|
| GUI_KEY_ENTER | 从打开的下拉列表中选择项目，然后关闭列表。 |
| GUI_KEY_SPACE | 打开下拉列表。               |

表 16.3.1 DROPDOWN 键盘反应

### 16.4 DROPDOWN 控件 API 函数

表 16.4.1 列举出了 DROPDOWN 控件 API 函数

| 函数                                         | 描述                                |
|--------------------------------------------|-----------------------------------|
| <a href="#">DROPDOWN_AddString()</a>       | 给 DROPDOWN 列表添加元素。                |
| <a href="#">DROPDOWN_Collapse()</a>        | 关闭下拉列表。                           |
| <a href="#">DROPDOWN_Create()</a>          | 创建 DROPDOWN 小工具。                  |
| <a href="#">DROPDOWN_CreateEx()</a>        | 创建 DROPDOWN 小工具。                  |
| <a href="#">DROPDOWN_CreateIndirect()</a>  | 从资源表项创建 DROPDOWN 小工具。             |
| <a href="#">DROPDOWN_CreateUser()</a>      | 使用额外字节作为用户数据创建 DROPDOWN 小工具。      |
| <a href="#">DROPDOWN_DecSel()</a>          | 减小选定范围。                           |
| <a href="#">DROPDOWN_DecSelExp()</a>       | 递减处于展开状态的选取项。                     |
| <a href="#">DROPDOWN_DeleteItem()</a>      | 删除 DROPDOWN 列表的项目。                |
| <a href="#">DROPDOWN_Expand()</a>          | 打开下拉列表。                           |
| <a href="#">DROPDOWN_GetDefaultFont()</a>  | 返回用于创建 DROPDOWN 小工具的默认字体。         |
| <a href="#">DROPDOWN_GetItemDisabled()</a> | 返回给定项目的状态。                        |
| <a href="#">DROPDOWN_GetListbox()</a>      | 返回处于展开状态的附加 DROPDOWN 的句柄          |
| <a href="#">DROPDOWN_GetNumItems()</a>     | 返回下拉列表中项目的数量。                     |
| <a href="#">DROPDOWN_GetSel()</a>          | 返回当前选定元素的数量。                      |
| <a href="#">DROPDOWN_GetSelExp()</a>       | 返回处于展开状态的当前选定元素的数量。               |
| <a href="#">DROPDOWN_GetUserData()</a>     | 检索使用 DROPDOWN_SetUserData() 设置的数据 |
| <a href="#">DROPDOWN_IncSel()</a>          | 增加选定范围。                           |
| <a href="#">DROPDOWN_IncSelExp()</a>       | 递增处于展开状态的选取项。                     |
| <a href="#">DROPDOWN_SetAutoScroll()</a>   | 启用在下拉列表中自动使用滚动条。                  |
| <a href="#">DROPDOWN_SetBkColor()</a>      | 设置背景颜色。                           |

表 16.4.1 DROPDOWN 控件 API 函数



续表 16.4.1

| 函数                                  | 描述                          |
|-------------------------------------|-----------------------------|
| DROPDOWN_SetColor()                 | 设置 DROPDOWN 小工具的箭头和按钮的颜色。   |
| DROPDOWN_SetDefaultColor()          | 设置 DROPDOWN 小工具的箭头和按钮的默认颜色。 |
| DROPDOWN_SetDefaultFont()           | 设置 DROPDOWN 小工具的默认字体。       |
| DROPDOWN_SetDefaultScrollbarColor() | 设置下拉列表中可选滚动条的默认颜色。          |
| DROPDOWN_SetFont()                  | 设置给定 DROPDOWN 小工具的字体。       |
| DROPDOWN_SetItemDisabled()          | 设置给定项目的状态。                  |
| DROPDOWN_SetItemSpacing()           | 设置下拉列表中各项目之间的间距。            |
| DROPDOWN_SetScrollbarColor()        | 设置下拉列表中滚动条的颜色。              |
| DROPDOWN_SetSel()                   | 设置当前选定内容。                   |
| DROPDOWN_SetSelExp()                | 设置处于展开状态的当前选择。              |
| DROPDOWN_SetTextAlign()             | 设置用于显示关闭状态下拉文本的对齐方式。        |
| DROPDOWN_SetTextColor()             | 设置给定 DROPDOWN 小工具的文本颜色。     |
| DROPDOWN_SetTextHeight()            | 设置用于显示关闭状态下拉文本的矩形的高度。       |
| DROPDOWN_SetUserData()              | 设置 DROPDOWN 小工具的额外数据。       |

表 16.4.1 DROPDOWN 控件 API 函数

## 16.5 DROPDOWN 控件演示例程

### 16.5.1 程序设计

本实验完整工程为：**EMWIN 实验 18 DROPDOWN 小工具**，打开 dropdowndemo.c，程序源码如下：

```
#include "dropdowndemo.h"
#include "DIALOG.h"

#define ID_FRAMEWIN_0 (GUI_ID_USER + 0x02)
#define ID_DROPDOWN_0 (GUI_ID_USER + 0x04)

//对话框资源表
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 320, 240,
        FRAMEWIN_CF_MOVEABLE, 0x64, 0 },
    { DROPDOWN_CreateIndirect, "Dropdown", ID_DROPDOWN_0, 88, 24, 109, 60, 0, 0x0, 0 },
};

//对话框回调函数
static void _cbDialog(WM_MESSAGE * pMsg)
{
    WM_HWIN hItem;
    int NCode;
    int Id;
```

```

switch (pMsg->MsgId)
{
    case WM_INIT_DIALOG:
        hItem = pMsg->hWin;
        //初始化 FRAMEWIN
        FRAMEWIN_SetText(hItem, "DROPDOWN ");
        FRAMEWIN_SetFont(hItem, GUI_FONT_16_ASCII);
        FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
        //初始化 DROPDOWN
        hItem = WM_GetDialogItem(pMsg->hWin, ID_DROPDOWN_0);
        DROPDOWN_SetFont(hItem, GUI_FONT_13H_ASCII);           (1)
        DROPDOWN_SetAutoScroll(hItem,1);//启用自动使用滚动条    (2)
        DROPDOWN_AddString(hItem, "China");                     (3)
        DROPDOWN_AddString(hItem, "Cambodia");
        DROPDOWN_AddString(hItem, "Japan");
        DROPDOWN_AddString(hItem, "Australien");
        DROPDOWN_AddString(hItem, "Canada");
        break;
    case WM_NOTIFY_PARENT:
        Id = WM_GetId(pMsg->hWinSrc);
        NCode = pMsg->Data.v;
        switch(Id)
        {
            case ID_DROPDOWN_0: //DROPDOWN 通知代码
                switch(NCode)
                {
                    case WM_NOTIFICATION_CLICKED:
                        break;
                    case WM_NOTIFICATION_RELEASED:
                        break;
                    case WM_NOTIFICATION_SEL_CHANGED:
                        break;
                }
                break;
        }
        break;
    default:
        WM_DefaultProc(pMsg);
        break;
}
}

```

//创建对话框

```
void dropdown_demo(void)
```

```
{
```

```
    WM_HWIN hWin;
```

```
    hWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate), _cbDialog,
```

```
WM_HBKWIN, 0, 0);
```

```
    while(1)
```

```
    {
```

```
        GUI_Delay(100);
```

```
    }
```

```
}
```

(1)、设置 DROPDOWN 控件的字体。

(2)、启用 DROPDOWN 控件自动使用滚动条的功能，这样当控件中条目过多的时候就会有滚动条出现。

(3)、向 DROPDOWN 控件中添加新元素。

本例程比较简单，Segger 官方提供了一个 DROPDOWN 的例程：WIDGET\_Dropdown.c，大家可以使用模拟器仿真运行一个这个例程，这个例程中详细的展示了 DROPDOWN 的 API 函数的使用。

### 16.5.2 下载验证

编译并下载到开发板中，如图 16.5.1 所示：



图 16.5.1 LCD 显示结果

**注意：FRAMEWIN 大小为 320\*240！**

## 第十七章 EDIT 控件

EDIT 控件通常用来作为输入文本的主要用户界面，也可使用编辑字段以二进制、十进制或十六进制模式输入值，本章分为如下几部分：

17.1 EDIT 控件简介

17.2 通知代码

17.3 键盘反应

17.4 EDIT 空间 API 函数

17.5 EDIT 控件演示例程

## 17.1 EDIT 控件简介

EDIT 控件如图 17.1.1 所示



图 17.10.1 EDIT 控件

## 17.2 通知代码

表 17.2.1 列举出了 EDIT 小工具作为 WM\_NOTIFY\_PARENT 消息的一部分发送给其父窗口的消息类型：

| 消息                            | 描述                       |
|-------------------------------|--------------------------|
| WM_NOTIFICATION_CLICKED       | 已单击小工具。                  |
| WM_NOTIFICATION_RELEASED      | 已释放小工具。                  |
| WM_NOTIFICATION_MOVED_OUT     | 已单击小工具，并且指针已移出小工具，但没有释放。 |
| WM_NOTIFICATION_VALUE_CHANGED | 编辑小工具的值(内容)已更改           |

表 17.2.1 EDIT 控件通知代码

## 17.3 键盘反应

如果小工具具有输入焦点，则它将对下列各键做出反应：

| 按键                | 反应                                         |
|-------------------|--------------------------------------------|
| GUI_KEY_UP        | 上升当前字符。例如，如果当前字符(光标下的字符)为“A”，则变为“B”        |
| GUI_KEY_DOWN      | 下降当前字符。例如，如果当前字符为为“B”，则变为“A”               |
| GUI_KEY_RIGHT     | 将光标向右移动一个字符。                               |
| GUI_KEY_LEFT      | 将光标向左移动一个字符。                               |
| GUI_KEY_BACKSPACE | 如果小工具在文本模式下操作，则删除光标前的字符。                   |
| GUI_KEY_DELETE    | 如果小工具在文本模式下操作，则删除当前字符。                     |
| GUI_KEY_INSERT    | 如果小工具在文本模式下操作，则此键在 GUI_EDIT_MODE_OVERWRITE |

表 17.3.1 EDIT 控件按键反应

## 17.4 EDIT 控件 API 函数

表 17.4.1 按字母顺序列出了可用的 EDIT 控件 相关例程：

| 函数                    | 描述                 |
|-----------------------|--------------------|
| EDIT_AddKey()         | 按键输入例程。            |
| EDIT_Create()         | 创建 EDIT 小工具。       |
| EDIT_CreateAsChild()  | 创建 EDIT 小工具,作为子窗口。 |
| EDIT_CreateEx()       | 创建 EDIT 小工具。       |
| EDIT_CreateIndirect() | 从资源表项创建 EDIT 小工具。  |

表 17.4.1 EDIT 控件 API 函数

续表 17.4.1

| 函数                         | 描述                           |
|----------------------------|------------------------------|
| EDIT_CreateUser()          | 使用额外字节作为用户数据创建 EDIT 小工具。     |
| EDIT_EnaBleBlink()         | 启用/禁用闪烁光标                    |
| EDIT_GetCursorCharPos()    | 返回光标位置的字符编号。                 |
| EDIT_GetCursorPixelPos()   | 返回光标的像素位置。                   |
| EDIT_GetDefaultBkColor()   | 返回默认背景颜色。                    |
| EDIT_GetDefaultFont()      | 返回默认的字体的。                    |
| EDIT_GetDefaultTextAlign() | 返回默认的文本对齐方式。                 |
| EDIT_GetDefaultTextColor() | 返回默认文本颜色。                    |
| EDIT_GetFloatValue()       | 按浮点值返回当前值。                   |
| EDIT_GetNumChars()         | 返回给定编辑小工具的字符数。               |
| EDIT_GetText()             | 获取用户输入。                      |
| EDIT_GetUserData()         | 检索用 EDIT_SetUserData()设置的数据。 |
| EDIT_GetValue()            | 返回当前值。                       |
| EDIT_SetBinMode()          | 启用二进制编辑模式。                   |
| EDIT_SetBkColor()          | 设置编辑字段的背景色。                  |
| EDIT_SetCursorAtChar()     | 将编辑小工具光标设置到指定的字符位置。          |
| EDIT_SetCursorAtPixel()    | 将编辑小工具光标设置到指定的像素位置。          |
| EDIT_SetDecMode()          | 启用十进制编辑模式。                   |
| EDIT_SetDefaultBkColor()   | 设置默认背景色。                     |
| EDIT_SetDefaultFont()      | 设置用于编辑字段的默认字体。               |
| EDIT_SetDefaultTextAlign() | 设置编辑字段的默认文本对齐方式。             |
| EDIT_SetDefaultTextColor() | 设置默认文本颜色。                    |
| EDIT_SetFloatMode()        | 启用浮点编辑模式。                    |
| EDIT_SetFloatValue()       | 设置使用浮点编辑模式时的浮点值。             |
| EDIT_SetFont()             | 选择文本的字体。                     |
| EDIT_SetHexMode()          | 启用十六进制编辑模式。                  |
| EDIT_SetInsertMode()       | 启用或禁用插入模式。                   |
| EDIT_SetMaxLen()           | 设置编辑字段的最大字符数。                |
| EDIT_SetpfAddKeyEx()       | 设置添加字符时要调用的函数。               |
| EDIT_SetSel()              | 设置当前选定内容。                    |
| EDIT_SetText()             | 设置文本。                        |
| EDIT_SetTextAlign()        | 设置编辑字段的文本对齐方式。               |
| EDIT_SetTextColor()        | 设置文本的颜色。                     |
| EDIT_SetTextMode()         | 将小工具的编辑模式设置回文本模式。            |
| EDIT_SetValue()            | 设置当前值。                       |
| EDIT_SetUlongMode()        | 启用不带符号的长十进制编辑模式。             |
| EDIT_SetUserData()         | 设置 EDIT 小工具的额外数据。            |
| GUI_EditBin()              | 编辑当前光标位置处的二进制值。              |

续表 17.4.1

| 函数               | 描述               |
|------------------|------------------|
| GUI_EditDec()    | 编辑当前光标位置处的十进制值。  |
| GUI_EditHex()    | 编辑当前光标位置处的十六进制值。 |
| GUI_EditString() | 编辑当前光标位置处的字符串。   |

表 17.4.1 EDIT 控件 API 函数

## 17.5 EDIT 控件演示例程

### 17.5.1 程序设计

本实验完整工程为：EMWIN 实验 19 EDIT 小工具，打开 editwindemo.c，程序源码如下：

```
#include "editwinmode.h"
#include "GUI.h"
#include "WM.h"
#include "DIALOG.h"

#define WM_APP_SHOW_TEXT(WM_USER +0) (1)
#define TEXT_MAXLEN 40

WM_HWIN DialoghWin;

//对话框资源表
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Framewin", 0, 55, 90, 200, 110,
      FRAMEWIN_CF_MOVEABLE},
    { EDIT_CreateIndirect, "Edit", GUI_ID_EDIT0, 24, 10, 145, 25, 0,15},
    { BUTTON_CreateIndirect,"Ok", GUI_ID_OK,65, 50, 70, 30}
};

//背景窗口的回调函数
static void _BkCallback(WM_MESSAGE *pMsg) (2)
{
    static WM_HWIN hWin;
    static WM_HWIN hEdit;
    static U8 text;
    char buffer[TEXT_MAXLEN];
    switch(pMsg->MsgId)
    {
        case WM_PAINT: (3)
            GUI_SetBkColor(GUI_DARKGRAY);
            GUI_Clear();
            GUI_SetFont(&GUI_Font24_ASCII);
```

```

GUI_DispStringHCenterAt("WIDGET_Edit - Sample", 160, 5);
GUI_SetFont(&GUI_Font8x16);
if (text)
{
    hEdit=WM_GetDialogItem(DialoghWin,GUI_ID_EDIT0);
    GUI_DispStringHCenterAt("The string you have modified is:", 160, 90);
    EDIT_GetText(hEdit, buffer, TEXT_MAXLEN);           (4)
    GUI_DispStringHCenterAt(buffer, 160, 110);
}
else
{
    GUI_DispStringHCenterAt("Use keyboard to modify string...", 160, 90);
}
break;
case WM_APP_SHOW_TEXT:                               (5)
    hWin=pMsg->hWinSrc;
    WM_HideWindow(hWin);                               //隐藏对话框 (6)
    text=1;
    WM_InvalidateWindow(WM_HBKWIN); //背景窗口无效 (7)
    WM_CreateTimer(WM_HBKWIN,0,3000,0); (8)
    break;
case WM_TIMER:                                       (9)
    text=0;
    WM_InvalidateWindow(WM_HBKWIN); (10)
    WM_ShowWindow(hWin); (11)
    break;
default:
    WM_DefaultProc(pMsg);
    break;
}
}

//对话框对调函数
static void _cbDialog(WM_MESSAGE * pMsg)
{
    WM_HWIN hItem;
    int     NCode;
    int     Id;
    WM_MESSAGE Msg;

    switch (pMsg->MsgId)
    {

```



```

case WM_INIT_DIALOG:
    //初始化 FRAMEWIN
    hItem = pMsg->hWin;
    FRAMEWIN_SetText(hItem, "EDIT USER");
    FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
    FRAMEWIN_SetFont(hItem, GUI_FONT_16B_ASCII);
    //初始化 EDIT
    hItem = WM_GetDialogItem(pMsg->hWin, GUI_ID_EDIT0);
    EDIT_EnableBlink(hItem, 500, 1); (12)
    EDIT_SetText(hItem, "Hello Word!"); (13)
    EDIT_SetFont(hItem, &GUI_Font20_ASCII); (14)
    EDIT_SetTextAlign(hItem, GUI_TA_LEFT | GUI_TA_VCENTER); (15)
    break;
case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;
    switch(Id)
    {
        case GUI_ID_EDIT0: //EDIT 控件通知消息
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    break;
                case WM_NOTIFICATION_VALUE_CHANGED:
                    break;
            }
            break;
        case GUI_ID_OK:
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    Msg.MsgId=WM_APP_SHOW_TEXT;
                    Msg.hWinSrc=pMsg->hWin;
                    WM_SendMessage(WM_HBKWIN,&Msg); (16)
                    break;
            }
    }
    break;

```

```

default:
    WM_DefaultProc(pMsg);
    break;
}
}

void Editwinmode_Demo(void)
{
    WM_SetCallback(WM_HBKWIN, _BkCallback);
    DialoghWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate), \
                                     _cbDialog, WM_HBKWIN, 0, 0);

    while(1)
    {
        GUI_Delay(10);
    }
}

```

- (1)、自定义消息了类型。
- (2)、桌面窗口回调函数。
- (3)、WM\_PAINT 消息，重绘背景窗口 WM\_HBKWIN，当 text 为 1 的时候说明 EDIT 控件中输入了信息，那么就在 WM\_HBKWIN 窗口中显示 EDIT 控件中的字符串。
- (4)、获取编辑框中的字符串，用于显示。
- (5)、WM\_APP\_SHOW\_TEXT 消息，当向 EDIT 控件输入完成并且点击对话框中的“OK”按钮以后就会向背景窗口 WM\_HBKWIN 发送消息 WM\_APP\_SHOW\_TEXT，在收到这个消息以后隐藏对话框、开启一个定时器并且将 text 置 1，标记接收到文本。
- (6)、隐藏对话框。
- (7)、使背景窗口无效。
- (8)、创建一个定时器，周期为 3000ms 的，定时时间到的话就会向窗口 WM\_HBKWIN 发送消息：WM\_TIMER。
- (9)、WM\_TIMER 消息，当上面创建的定时器定时时间到了以后就会向窗口 WM\_HBKWIN 发送 WM\_TIMER，接收到消息 WM\_TIMER 以后使背景窗口 WM\_HBKWIN 无效化，并且显示对话框。
- (10)、使背景窗口 WM\_HBKWIN 无效。
- (11)、开启 EDIT 的光标闪烁功能。
- (12)、在 EDIT 控件上显示字符串“Hello Word!”
- (13)、设置 EDIT 的字体。
- (14)、设置 EDIT 的字符对齐方式。
- (15)、向背景窗口 WM\_HBKWIN 发送消息：WM\_APP\_SHOW\_TEXT。

### 17.5.2 下载验证

编译并下载到开发板中，如图 17.5.1 所示：



图 17.5.1 运行结果

注意！这里我们没有做输入法，所以没办法在开发板上输入字符串，大家自行将本例程可以移植到模拟器中，在 PC 上模拟，通过键盘输入字符串，然后点击 OK 键，那么刚刚输入的字符串就会显示在背景窗口中，如图 17.5.2 所示。



图 17.5.2 背景窗口中显示 EDIT 中的字符串

## 第十八章 FRAMEWIN 控件

框架窗口为我们的应用提供一个类似 PC 应用程序的窗口外观，这些窗口由周围框架、标题栏和用户区组成，标题栏的颜色改变以显示窗口是否激活，本章讲解一下 STemWin 的框架小工具的使用，本章分为如下几部分：

18.1 FRAMEWIN 控件简介

18.2 FRAMEWIN 控件 API 函数

18.3 FRAMEWIN 控件演示例程

## 18.1 FRAMEWIN 控件简介

FRAMEWIN 控件的外观如图 18.1.1 所示：



图 18.1.1 FrameWin 控件

我们也可以将预定义的按钮附加到标题栏中，比如最大化、最小化和关闭按钮，如图 18.1.2 所示。

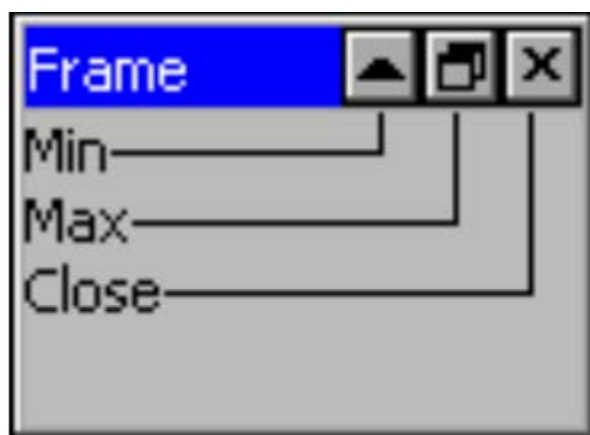


图 18.1.2 FrameWin 控件添加预定义按钮

FrameWin 框架的结构如图 18.1.3 所示：

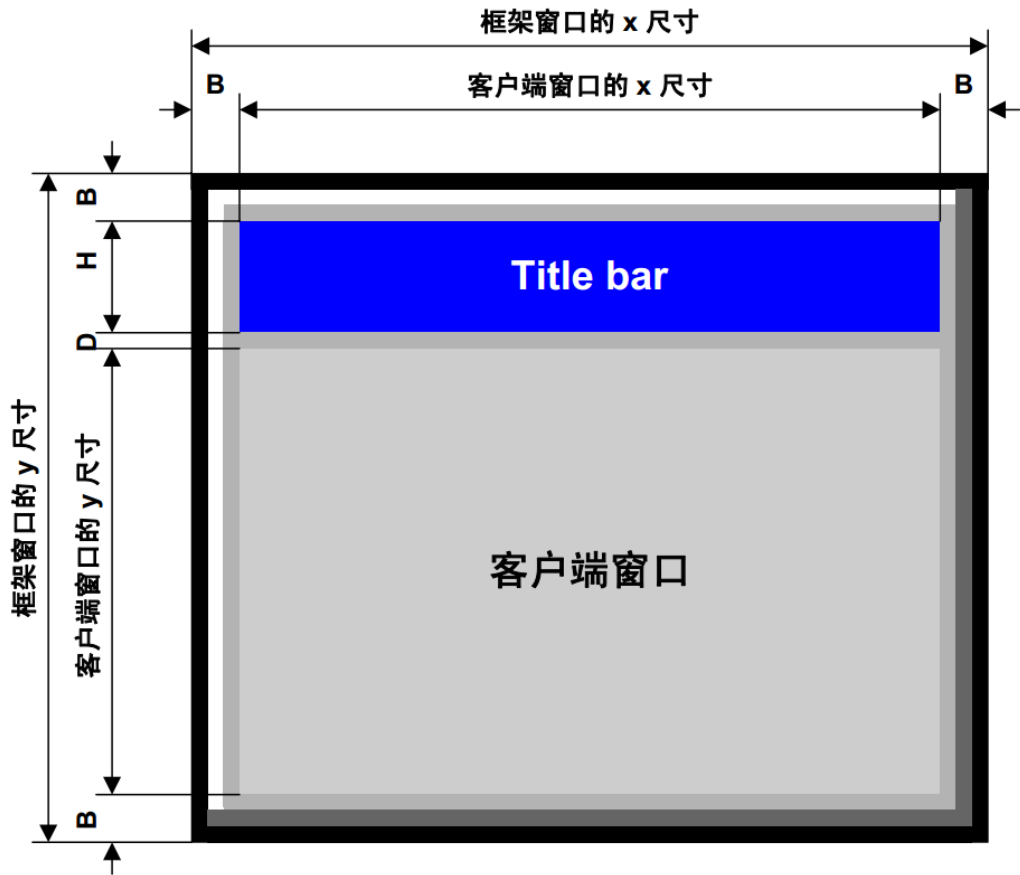


图 18.1.3 FrameWin 框架

框架窗口实际上由 2 种窗口组成：主窗口和子窗口，子窗口称为客户端窗口，处理回调函数时意识到此点非常重要：存在具有 2 种不同回调函数的 2 种窗口。创建子窗口时，这些子窗口通常作为客户端窗口的子窗口而创建；它们的父窗口因此是客户端窗口。图 18.1.3 中各个参数的含义如表 18.1.1 所示：

| 参数    | 描述                          |
|-------|-----------------------------|
| B     | 框架窗口的边框尺寸。默认的边框尺寸为          |
| H     | 标题栏的高度。取决于标题所用字体的尺寸。        |
| D     | 标题栏和客户端窗口的间距。（              |
| 标题栏   | 标题栏是框架窗口的一部分，不是单独的窗口。       |
| 客户端窗口 | 客户端窗口是一个单独的窗口，作为框架窗口的子窗口创建。 |

表 18.1.1 FrameWin 窗口参数说明

## 18.2 FRAMEWIN 控件 API 函数

| 函数                                        | 描述                |
|-------------------------------------------|-------------------|
| <a href="#">FRAMEWIN_AddButton()</a>      | 在标题栏中添加一个按钮。      |
| <a href="#">FRAMEWIN_AddCloseButton()</a> | 在标题栏中添加一个“关闭”按钮。  |
| <a href="#">FRAMEWIN_AddMaxButton()</a>   | 在标题栏中添加一个“最大化”按钮。 |
| <a href="#">FRAMEWIN_AddMenu()</a>        | 向框架窗口添加一个“菜单”按钮。  |
| <a href="#">FRAMEWIN_AddMinButton()</a>   | 在标题栏中添加一个“最小化”按钮。 |

表 18.2.1 FRAMEWIN 控件 API 函数

续表 18.2.1

| 函数                               | 描述                                |
|----------------------------------|-----------------------------------|
| FRAMEWIN_Create()                | 创建一个 FRAMEWIN 小工具。                |
| FRAMEWIN_CreateAsChild()         | 创建一个 FRAMEWIN 小工具作为子窗口。           |
| FRAMEWIN_CreateEx()              | 创建一个 FRAMEWIN 小工具。                |
| FRAMEWIN_CreateIndirect()        | 从资源表项创建 FRAMEWIN 小工具。             |
| FRAMEWIN_CreateUser()            | 使用额外字节作为用户数据创建一个 FRAMEWIN 小工具     |
| FRAMEWIN_GetActive()             | 返回框架窗口是否处于激活状态。                   |
| FRAMEWIN_GetBarColor()           | 返回标题栏的颜色。                         |
| FRAMEWIN_GetBorderSize()         | 返回边框的尺寸。                          |
| FRAMEWIN_GetDefaultBarColor()    | 返回标题栏的默认颜色。                       |
| FRAMEWIN_GetDefaultBorderSize()  | 返回默认的边框尺寸。                        |
| FRAMEWIN_GetDefaultClientColor() | 返回客户区的默认颜色。                       |
| FRAMEWIN_GetDefaultFont()        | 返回标题栏使用的默认字体                      |
| FRAMEWIN_GetDefaultTextColor()   | 返回标题的默认文本颜色。                      |
| FRAMEWIN_GetDefaultTitleHeight() | 返回标题栏的默认尺寸。                       |
| FRAMEWIN_GetFont()               | 返回标题文本所用的字体。                      |
| FRAMEWIN_GetText()               | 返回标题文本。                           |
| FRAMEWIN_GetTextAlign()          | 返回标题文本的对齐方式。                      |
| FRAMEWIN_GetTitleHeight()        | 返回标题栏的高度。                         |
| FRAMEWIN_GetUserData()           | 检索用 FRAMEWIN_SetUserData() 设置的数据。 |
| FRAMEWIN_IsMinimized()           | 返回表示框架窗口是否最小化的值。                  |
| FRAMEWIN_IsMaximized()           | 返回表示框架窗口是否最大化的值。                  |
| FRAMEWIN_Maximize()              | 将框架窗口放大到父窗口的尺寸。                   |
| FRAMEWIN_Minimize()              | 隐藏框架窗口的客户区。                       |
| FRAMEWIN_OwnerDraw()             | 用于绘制标题栏的默认函数。                     |
| FRAMEWIN_Restore()               | 恢复最小化或最大化的框架窗口。                   |
| FRAMEWIN_SetActive()             | 设置框架窗口的状态。(弃用)                    |
| FRAMEWIN_SetBarColor()           | 设置标题栏的颜色。                         |
| FRAMEWIN_SetBorderSize()         | 设置框架窗口的边框尺寸。                      |
| FRAMEWIN_SetClientColor()        | 设置客户区的颜色。                         |
| FRAMEWIN_SetDefaultBarColor()    | 设置标题栏的默认颜色。                       |
| FRAMEWIN_SetDefaultBorderSize()  | 设置默认的边框尺寸。                        |
| FRAMEWIN_SetDefaultClientColor() | 设置客户区的默认颜色。                       |
| FRAMEWIN_SetDefaultFont()        | 设置标题栏使用的默认字体。                     |
| FRAMEWIN_SetDefaultTextColor()   | 设置标题的默认文本颜色。                      |
| FRAMEWIN_SetDefaultTitleHeight() | 设置标题栏的默认高度。                       |
| FRAMEWIN_SetFont()               | 选择标题文本所用的字体。                      |
| FRAMEWIN_SetMoveable()           | 将框架窗口设置为可移动/不可移动状态。               |
| FRAMEWIN_SetOwnerDraw()          | 启用要自画的框架窗口。                       |
| FRAMEWIN_SetResizable()          | 将框架窗口设置为可缩放状态。                    |

续表 18.2.1

| 函数                        | 描述                    |
|---------------------------|-----------------------|
| FRAMEWIN_SetText()        | 设置标题文本。               |
| FRAMEWIN_SetTextAlign()   | 设置标题文本的对齐方式。          |
| FRAMEWIN_SetTextColor()   | 设置标题文本的颜色。            |
| FRAMEWIN_SetTextColorEx() | 设置标题文本的颜色。            |
| FRAMEWIN_SetTitleHeight() | 设置标题栏的高度。             |
| FRAMEWIN_SetTitleVis()    | 设置标题栏的可见性标记。          |
| FRAMEWIN_SetUserData()    | 设置 FRAMEWIN 小工具的额外数据。 |

表 18.2.1 FRAMEWIN 控件 API 函数。

### 18.3 FRAMEWIN 控件演示例程

#### 18.3.1 程序设计

本实验完整工程为：EMWIN 实验 20 FRAMEWIN 小工具，本例程移植的 Segger 官方的例程：打开 editwindemo.c，程序源码如下：

```
#include "myframewin.h"
#include "GUI.h"
#include "FRAMEWIN.h"
#include "string.h"

#define SPEED 1200
#define MSG_CHANGE_MAIN_TEXT (WM_USER+0) (1)

static FRAMEWIN_Handle hFrame;
static WM_CALLBACK* _pcbOldFrame;
static char _acMainText[100];
static int LockClose=1;

//改变背景窗口中显示的字符
static void _ChangeMainText(char* pStr,int Delay) (2)
{
    WM_MESSAGE Message;
    Message.MsgId=MSG_CHANGE_MAIN_TEXT; (3)
    Message.Data.p=pStr;
    GUI_Delay(Delay);
    WM_SendMessage(WM_HBKWIN,&Message); (4)
    WM_InvalidateWindow(WM_HBKWIN);
    GUI_Delay(Delay/3);
}

//子窗口回调函数
```



```
static void _cbChild(WM_MESSAGE *pMsg) (5)
{
    WM_HWIN hWin=(FRAMEWIN_Handle)(pMsg->hWin);
    switch(pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_WHITE);
            GUI_SetColor(GUI_BLACK);
            GUI_SetFont(&GUI_FontComic24B_ASCII);
            GUI_SetTextAlign(GUI_TA_HCENTER|GUI_TA_VCENTER);
            GUI_Clear();
            GUI_DispStringHCenterAt("Clientwindow",WM_GetWindowSizeX(hWin)/2,\
                                   WM_GetWindowSizeY(hWin)/2);

            break;
        default:
            WM_DefaultProc(pMsg);
    }
}
```

//FrameWin 控件回调函数

```
static void _cbFrame(WM_MESSAGE *pMsg) (6)
{
    switch(pMsg->MsgId)
    {
        case WM_NOTIFY_PARENT:
            if(pMsg->Data.v==WM_NOTIFICATION_RELEASED)
            {
                int Id=WM_GetId(pMsg->hWinSrc);
                if(Id==GUI_ID_CLOSE) (7)
                {
                    if(_LockClose) return; (8)
                    _hFrame = 0;`
                }
            }
            break;
    }
    if(_pcbOldFrame) (*_pcbOldFrame)(pMsg);
}
```

//背景窗口 WM\_HBKWIN 回调函数

```
static void _cbBkWindow(WM_MESSAGE* pMsg)
{
    switch(pMsg->MsgId)
```

```

{
    case MSG_CHANGE_MAIN_TEXT:
        strcpy(_acMainText,pMsg->Data.p);
        WM_InvalidateWindow(pMsg->hWin);
        break;
    case WM_PAINT:
        GUI_SetBkColor(GUI_BLACK);
        GUI_Clear();
        GUI_SetColor(GUI_WHITE);
        GUI_SetFont(&GUI_Font24_ASCII);
        GUI_DispStringHCenterAt("WIDGET_FrameWin - Sample",160,5);
        GUI_SetFont(&GUI_Font8x16);
        GUI_DispStringHCenterAt(_acMainText,160,40);
        GUI_SetFont(&GUI_Font6x8);
        GUI_DispStringHCenterAt("The function FRAMEWIN_Create creates both the\n"
                                "frame window and the client window.", 160, 190);
        break;
    default:
        WM_DefaultProc(pMsg);
}
}

static void _DemoFramewin(void)
{
    int i;
    char acInfoText[]="-- sec to play with window";
    WM_HWIN hChild;
    //为背景设置回调函数
    WM_SetCallback(WM_HBKWIN,_cbBkWindow);
    //创建一个框架窗口
    _ChangeMainText("FRAMEWIN_Create",SPEED);
    _hFrame = FRAMEWIN_Create("Frame window",0,WM_CF_SHOW,30,60,260,125);
    FRAMEWIN_SetTitleHeight(_hFrame,20);
    FRAMEWIN_SetFont(_hFrame,&GUI_Font16B_ASCII);
    //为框架窗口设置回调函数
    _pcbOldFrame=WM_SetCallback(_hFrame,_cbFrame);
    //获取子窗口的句柄
    hChild=WM_GetClientWindow(_hFrame);
    //设置子窗口的回调函数
    WM_SetCallback(hChild,_cbChild);
    //设置框架窗口为可移动的
    FRAMEWIN_SetMoveable(_hFrame,1);
    //给框架窗口创建几个按钮

```

```

FRAMEWIN_AddCloseButton(_hFrame,FRAMEWIN_BUTTON_RIGHT,0);
FRAMEWIN_AddMaxButton(_hFrame,FRAMEWIN_BUTTON_RIGHT,2);
FRAMEWIN_AddMinButton(_hFrame,FRAMEWIN_BUTTON_RIGHT,2);

//修改框架窗口的属性
_ChangeMainText("FRAMEWIN_SetActive",SPEED); (13)
FRAMEWIN_SetActive(_hFrame,1); (14)
_ChangeMainText("FRAMEWIN_SetFont",SPEED);
FRAMEWIN_SetFont(_hFrame,&GUI_Font24_ASCII); (15)
FRAMEWIN_SetTitleHeight(_hFrame,25); (16)
_ChangeMainText("FRAMEWIN_SetTextColor",SPEED);
FRAMEWIN_SetTextColor(_hFrame,GUI_YELLOW); (17)
_ChangeMainText("FRAMEWIN_SetTextAlign",SPEED);
FRAMEWIN_SetTextAlign(_hFrame,GUI_TA_HCENTER); (18)
_ChangeMainText("FRAMEWIN_Minimize",SPEED);
FRAMEWIN_Minimize(_hFrame); (19)
_ChangeMainText("FRAMEWIN_Maximize",SPEED);
FRAMEWIN_Maximize(_hFrame); (20)
_ChangeMainText("FRAMEWIN_Restore",SPEED);
FRAMEWIN_Restore(_hFrame);
_ChangeMainText("FRAMEWIN_SetTitleVis",SPEED); (21)
for(i=0;i<5;i++)
{
    FRAMEWIN_SetTitleVis(_hFrame,0); (22)
    GUI_Delay(200);
    FRAMEWIN_SetTitleVis(_hFrame,1); (23)
    GUI_Delay(200);
}

_LockClose=0;
for(i=250;(i>0)&& _hFrame;i--)
{
    acInfoText[0]='0'+((i+9)/100);
    acInfoText[1]='0'+(((i+9)/10)%10);
    _ChangeMainText(acInfoText,0);
    GUI_Delay(100);
}
if(_hFrame) (24)
{
    _ChangeMainText("FRAMEWIN_Delete",SPEED);
    FRAMEWIN_Delete(_hFrame); (25)
}else
{

```

```

        _ChangeMainText("",50);
    }
}

void FrameWin_Demo(void)
{
    WM_EnableMemdev(WM_HBKWIN);
    while(1)
    {
        _DemoFramewin();
    }
}

```

- (1)、MSG\_CHANGE\_MAIN\_TEXT 为用户自定义消息，用于改变要显示的字符串。
- (2)、函数\_ChangeMainText()主要是组建消息 MSG\_CHANGE\_MAIN\_TEXT，并且将其发送给背景窗口 WM\_HBKWIN。
- (3)、消息 ID 为 MSG\_CHANGE\_MAIN\_TEXT，并且将要传递的消息的首地址传递给 Data.p。
- (4)、向背景窗口 WM\_HBKWIN 发送消息 MSG\_CHANGE\_MAIN\_TEXT。
- (5)、FrameWin 控件子窗口的回调函数。
- (6)、FrameWin 控件回调函数。
- (7)、点击了 FrameWin 的“关闭”按钮。
- (8)、如果 FrameWin 已经关闭了的话就直接返回，否则的话将\_hFrame 置 0，标记框架已经关闭了。
- (9)、背景窗口 WM\_HBKWIN 收到消息 MSG\_CHANGE\_MAIN\_TEXT。
- (10)、将消息 MSG\_CHANGE\_MAIN\_TEXT 中要显示的字符串拷贝到数组\_acMainText 中。
- (11)、使目标窗口无效化，这里也就是 WM\_HBKWIN。
- (12)、显示数组\_acMainText 中的字符串。
- (13)、调用函数\_ChangeMainText()，目的就是在背景窗口中显示指定的字符串。
- (14)、调用函数 FRAMEWIN\_SetActive()激活框架\_hFrame。
- (15)、调用函数 FRAMEWIN\_SetFont()设置\_hFrame 的字体为 GUI\_Font24\_ASCII。
- (16)、调用函数 FRAMEWIN\_SetTitleHeight()设置框架标题栏的高度为 25。
- (17)、调用函数 FRAMEWIN\_SetTextColor()设置框架的文本颜色为 GUI\_YELLOW。
- (18)、调用函数 FRAMEWIN\_SetTextAlign()设置框架文本的对齐方式。
- (19)、调用函数 FRAMEWIN\_Minimize()设置框架最小化。
- (20)、调用函数 FRAMEWIN\_Maximize()设置框架最大化。
- (21)、调用函数 FRAMEWIN\_Restore()恢复到原来的尺寸和位置。
- (22)、调用函数 FRAMEWIN\_SetTitleVis()设置框架标题栏不可见。
- (23)、调用函数 FRAMEWIN\_SetTitleVis()设置框架标题栏可见。
- (24)、如果\_hFrame 为 1 的话说明没有框架，那么在这里就删除掉框架窗口。
- (25)、调用函数 FRAMEWIN\_Delete()关闭框架。

### 18.3.2 下载验证

编译并下载到开发板中，如图 18.3.1 所示：

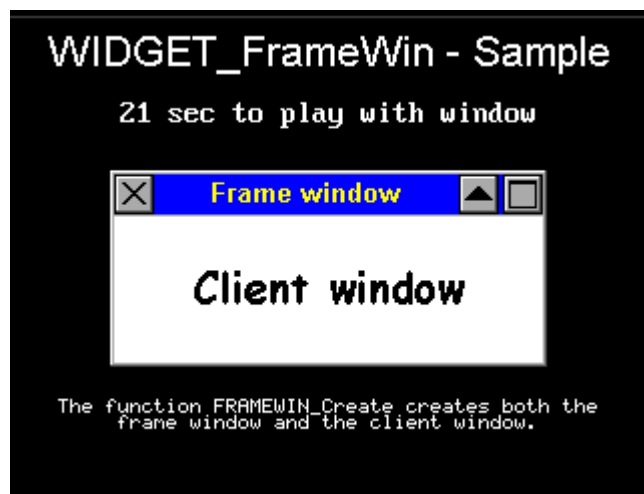


图 18.3.1 演示结果

大家可以下载到开发板中点击框架中的“关闭”，“最大化”和“最小化”按钮来观察框架的变化。

## 第十九章 字体

在前面的所有章节中都没有使用汉字，在接下来的几章将会详细的讲解如何在 emwin 中使用汉字，在此之前我们先简单了解一下有关 emwin 字体的基础知识，本章分为如下几部分：

- 19.1 字体简介
- 19.2 字体类型
- 19.3 字体格式
- 19.4 声明自定义字体
- 19.5 选择字体
- 19.6 字体常用 API 函数

## 19.1 字体简介

字体支持的第一种方法就是使用字体定义中含每个字符的 1 bpp 像素信息位图的 C 文件。这种字体支持仅限于使用应用编译的字体。随着时间推移,字体支持在字体质量、ROM 要求、性能、可扩展性和运行时添加更多字体的能力方面进行了改进。同时,emWin 字体包含抗锯齿、复合字符的图样(例如泰语中的要求)、位于外部非可寻址介质中的字体以及 TrueType 支持。除 TrueType 字体(矢量字体)格式外,所有其他字体均为位图字体。

## 19.2 字体类型

emWin 支持自行定义的内部类型不同的字体以及常用的 TrueType 字体。

### 等宽位图字体

等宽位图字体的每个字符大小相同。在比例字体中,每个字符有自己的宽度,而等宽字体的宽度只需定义一次。像素信息保存为 1 bpp,涵盖整个字符区域。

### 比例位图字体

比例位图字体的每个字符高度相同、宽度可能不同。像素信息保存为 1 bpp,涵盖整个字符区域。

### 抗锯齿字体, 包含 2 bpp 抗锯齿信息

每个字符高度相同、宽度可能不同。像素信息保存为 2 bpp 抗锯齿信息,涵盖整个字符区域。

### 抗锯齿字体, 包含 4 bpp 抗锯齿信息

每个字符高度相同、宽度可能不同。像素信息保存为 4 bpp 抗锯齿信息,涵盖整个字符区域。

### 扩展比例位图字体

扩展比例位图字体的每个字符拥有自己的高度和宽度。像素信息保存为 1 bpp,仅涵盖字形位图区域。

### 扩展比例位图字体, 包含 2 bpp 抗锯齿信息

每个字符高度相同、宽度可能不同。像素信息保存为 2 bpp 抗锯齿信息,仅涵盖字形位图区域。

### 扩展比例位图字体, 包含 4 bpp 抗锯齿信息

每个字符高度相同、宽度可能不同。像素信息保存为 4 bpp 抗锯齿信息,仅涵盖字形位图区域。

### 扩展比例位图字体, 带边框

在某些情况下(例如编译时背景颜色未知),可以使用带边框的字体。带边框的字体始终在透明模式下绘制,与当前设置无关。字符像素按当前所选的前景颜色绘制,边框按背景颜色绘制。前景颜色和背景颜色之间明显的对比可以确保能够阅读文本,而与背景无关。请注意,这种类型的字体不适用于复合字符,例如泰语,另外,也不适用于 Arabic 字体。

### TrueType 矢量字体

emWin 对于矢量字体是支持的,后面章节中会讲解到矢量字体的使用,图 19.2.1 列出了各字体之间的差异。

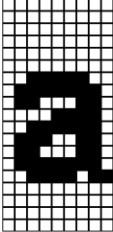
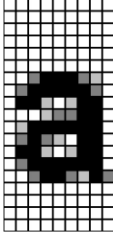
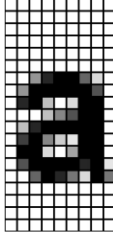
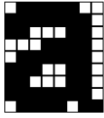
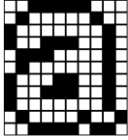
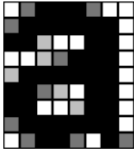
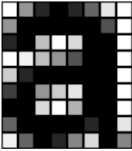
| 比例位图字体                                                                            | 比例位图字体，<br>AA2                                                                    | 比例位图字体，<br>AA2                                                                    | 扩展比例位图<br>字体                                                                       | 扩展比例位图<br>字体，带边框                                                                    |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  |  |  |  |  |
| 扩展比例位图<br>字体， AA2                                                                 | 扩展比例位图<br>字体， AA4                                                                 |                                                                                   |                                                                                    |                                                                                     |
|  |  |                                                                                   |                                                                                    |                                                                                     |

图 19.2.1 各字体类型之间的差异

### 19.3 字体格式

#### 19.3.1 C 文件格式

这是最常见的字体使用方式。使用 C 文件格式的字体时，建议作为库模块来编译所有可用的字体和链接它们，或将所有字体对象文件置于可以链接到应用的库中。这样，可确保实际只链接了您的应用所需的字体。

##### 何时使用

在编译时字体已知以及对于字体数据拥有足够的可寻址内存的情况下，可以使用这种格式。

##### 要求

为了能够在应用中使用字体 C 文件，必须满足以下要求：

- 1、字体文件为与 emWin 兼容的 C 文件、对象文件或库格式。
- 2、字体文件与您的应用链接。
- 3、应用中包含字体声明。

##### 格式说明

字体 C 文件首先包含字体所含全部字符的像素信息，之后，是有关每个字符大小信息的字符信息表。此表之后是字体文件中所含各个字符相邻区域的范围信息结构，而每个结构均指向下一个结构。请注意，如果使用许多单独的字符，这种方法可以大幅扩展字体文件。在范围信息结构之后，是 GUI\_FONT 结构，其中包含字体的类型、像素大小等主要信息。

#### 19.3.2 系统独立字体(SIF)格式

系统独立字体是包含字体信息的二进制数据块，可以使用字体转换器来创建系统独立字体。



后面章节会对字体转换器进行简单的介绍。

#### 何时使用

在编译时字体未知以及对于字体数据拥有足够的可寻址内存的情况下,可以使用这种格式。

#### 要求

为了能够在应用中使用 SIF 字体文件,需要在可寻址存储器 (ROM 或 RAM) 中保存整个文件。

#### 格式说明

SIF 文件的结构与 C 文件大致相同,它在二进制格式中包含相同的信息。文件要素的顺序反之亦然: 常规字体信息,之后是范围信息结构、字符信息表和所有字符的最小像素信息。

### 19.3.3 外部位图字体(XBF)格式

同 SIF 字体一样, XBF 字体也是包含字体信息的二进制数据块,并且可以使用字体转换器来创建 XBF 文件。与其他字体不同的是, XBF 字体在使用时不必位于存储器中,而 emWin 的所有其他字体均需要完全驻存在存储器中。XBF 字体文件在使用时可保存在任意外部介质中,通过 “GetData” 回调函数可以进行数据访问,而所有其他字体都需要驻存在可寻址存储器 (RAM 或 ROM) 中。XBF 字体的优势在于,可以在内存很小的系统上使用非常大的字体。

#### 何时使用

如果对于字体数据没有足够的可寻址内存,并且可以使用任何其他种类的外部介质来存储字体的情况下,可以使用这种格式。

#### 要求

为了能够在应用中使用 XBF 字体,需要使用 “GetData” 回调函数来负责获取字体数据。

#### 格式说明

这种格式总体上与 SIF 和 C 文件格式有所不同。首先,它包含小块常规字体信息,其中包含最低和最高的字符代码。之后,是包含最低和最高字符代码之间每个字符的偏移和数据大小信息的访问表。如果不存在字符,则相应字符的此信息为零。访问表之后是所有字符的字符信息,包含像素数据和字符大小信息。

### 19.3.4 TrueType 字体(TTF)格式

TrueType 是 Apple Computer 开发的轮廓字体标准。它为字体开发人员提供对在各种字体高度下字体显示方式的高度控制。与位图字体 (基于每个字符的位图) 不同, TrueType 字体基于矢量图形。矢量表示的优势在于无损的可扩展性。

这意味着,每个字符在绘制前需要光栅化为位图。为避免每次绘制字符时都进行光栅化,通常用字体引擎缓存位图数据。这要求 CPU 速度快、RAM 足够。默认的emWin源码中不包含 TTF 包。该内容可在 [www.segger.com/link/emwin\\_freetype.zip](http://www.segger.com/link/emwin_freetype.zip) 下免费获得。

#### 许可

emWin 对 TTF 支持的实施基于来自 David Turner、Robert Wilhelm 和 Werner Lembergr 的 FreeType 字体库,该库可在 [www.freetype.org](http://www.freetype.org) 下免费获得。emWin 对该库的使用符合 GUITrueTypeFTL.txt 下的 FreeType 许可。emWin 对该库进行了少许改编,添加了带有 GUI 函数的 “粘贴” 层。

#### 何时使用

如果在运行时需要对字体进行缩放，可使用这种格式。

#### 要求

1、CPU：TTF 支持仅适用于 32 位 CPU，emWin 对 32 位 CPU 的定义为:sizeof(int) = 4。

2、ROM：TTF 引擎的 ROM 要求大约为 250 K，确切大小取决于 CPU、编译器以及编译器的优化水平。

3、RAM：该库的 RAM 要求主要取决于使用的字体，TTF 引擎的基本 RAM 要求大约为 50 K。在使用 GUI\_TTF\_CreateFont() 创建 GUI 字体时，字体引擎会加载生成字符所需的 TTF 文件中定义的所有字体表，不同字体之间的表大小有所差异。创建字体额外要求的 RAM 量可能介于几个 KB 到 1 MB 以上之间，一般字体需要 80-300 kb。取决于使用的字体文件需要多少 RAM，至少，TTF 引擎需要位图缓存。默认情况下，引擎使用 200 K 的缓存就足够大多数应用使用。TTF 引擎通过非 emWin 函数 malloc() 和 free() 分配内存。使用 TTF 引擎之前，必须确保能够运行这些函数。

#### 格式说明

有关 TTF 格式的详细信息，请参阅 [www.apple.com](http://www.apple.com) 下可用的信息。

## 19.4 声明自定义字体

声明自定义字体原型最推荐的方法是将它们置于应用定义的头文件中。使用这些字体的每个应用源文件中均应包含此内容。如以下示例所示：

```
#include "GUI.h"
extern GUI_CONST_STORAGE GUI_FONT GUI_FontApp1;
extern GUI_CONST_STORAGE GUI_FONT GUI_FontApp2;
```

请注意，如果这些字体要用于 BUTTON\_FONT\_DEFAULT 或类似的 emWin 配置宏，这种声明原型不起作用。在这种情况下，字体需要在配置文件 GUIConf.h 中声明。这种情况下的声明如以下示例所示：

```
typedef struct GUI_FONT GUI_FONT;
extern const GUI_FONT GUI_FontApp1;
#define BUTTON_FONT_DEFAULT &GUI_FontApp1
#define EDIT_FONT_DEFAULT &GUI_FontApp1
```

由于前面 emWin 包含 GUIConf.h 的位置未定义 GUI\_FONT 结构，所以需要类型定义符 (typedef)。

## 19.5 选择字体

emWin 提供不同的字体，但是始终选择其中之一。通过调用函数 GUI\_SetFont() 或 GUI\_XXX\_CreateFont()函数之一可更改此选项，它们将选择用于当前任务所有文本输出的字体。如果应用总没有选择任何字体，则将使用默认字体，默认字体在 GUIConf.h 中配置，且可以更改。因此应该确保默认字体是在应用中实际使用的字体。

## 19.6 字体常用 API 函数

表 19.6.1 列出了有关字体的常用 API 函数。

| 函数                         | 描述                      |
|----------------------------|-------------------------|
| GUI_SetDefaultFont()       | 设置默认字体。                 |
| GUI_SetFont()              | 设置当前字体。                 |
| GUI_GetCharDistX()         | 返回当前字体中指定字符的宽度像素(X 大小)。 |
| GUI_GetFont()              | 返回当前选择的字体的指针。           |
| GUI_GetFontDistY()         | 返回当前字体的 Y 间距。           |
| GUI_GetFontInfo()          | 返回包含字体信息的结构。            |
| GUI_GetFontSizeY()         | 返回当前字体的高度像素(Y 大小)       |
| GUI_GetLeadingBlankCols()  | 返回给定字符的前导空格像素列数。        |
| GUI_GetStringDistX()       | 返回使用当前字体的文本的 X 大小       |
| GUI_GetTextExtend()        | 评估使用当前字体的文本的大小。         |
| GUI_GetTrailingBlankCols() | 返回给定字符的后导空格像素列数。        |
| GUI_GetYDistOfFont()       | 返回特定字体的 Y 间距。           |
| GUI_GetYSizeOfFont()       | 返回特定字体的 Y 大小。           |
| GUI_IsInFont()             | 评估特定字体中是否存在指定的字符。       |
| GUI_SetDefaultFont()       | 设置 GUI_Init()之后使用的默认字体。 |

表 19.6.1 字体常用 API 函数

## 第二十章 XBF 格式字体显示

在 LCD 上显示汉字是很重要的功能，emwin 支持多种显示汉字的方法，本章讲解一下如何在 emWin 中使用 XBF 格式的汉字，本章分为如下几部分：

20.1 XBF 格式字库制作

20.2 XBF 字体相关 API 函数

20.3 程序设计

20.4 下载验证

## 20.1 XBF 格式字库制作

首先我们要用字体转换器制作所需的 XBF 字体，我们下载的 STemWin 源码中已经包含了字体转换器，和 GUIBulider 在同一目录下，如图 20.1.1 所示。字体转换器需要安装，不像 GUIBulider 直接打开就可以用。

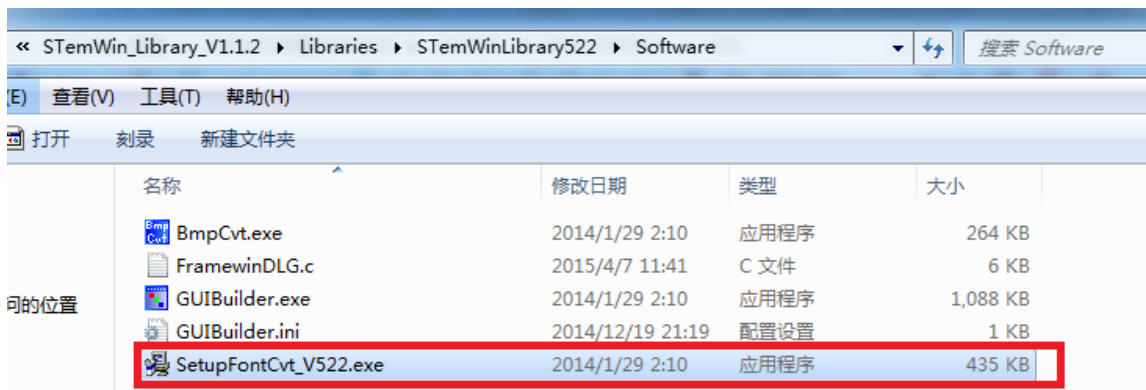


图 20.1.1 字体转换器安装文件

安装完成以后打开字体转换器如图 20.1.2 所示。

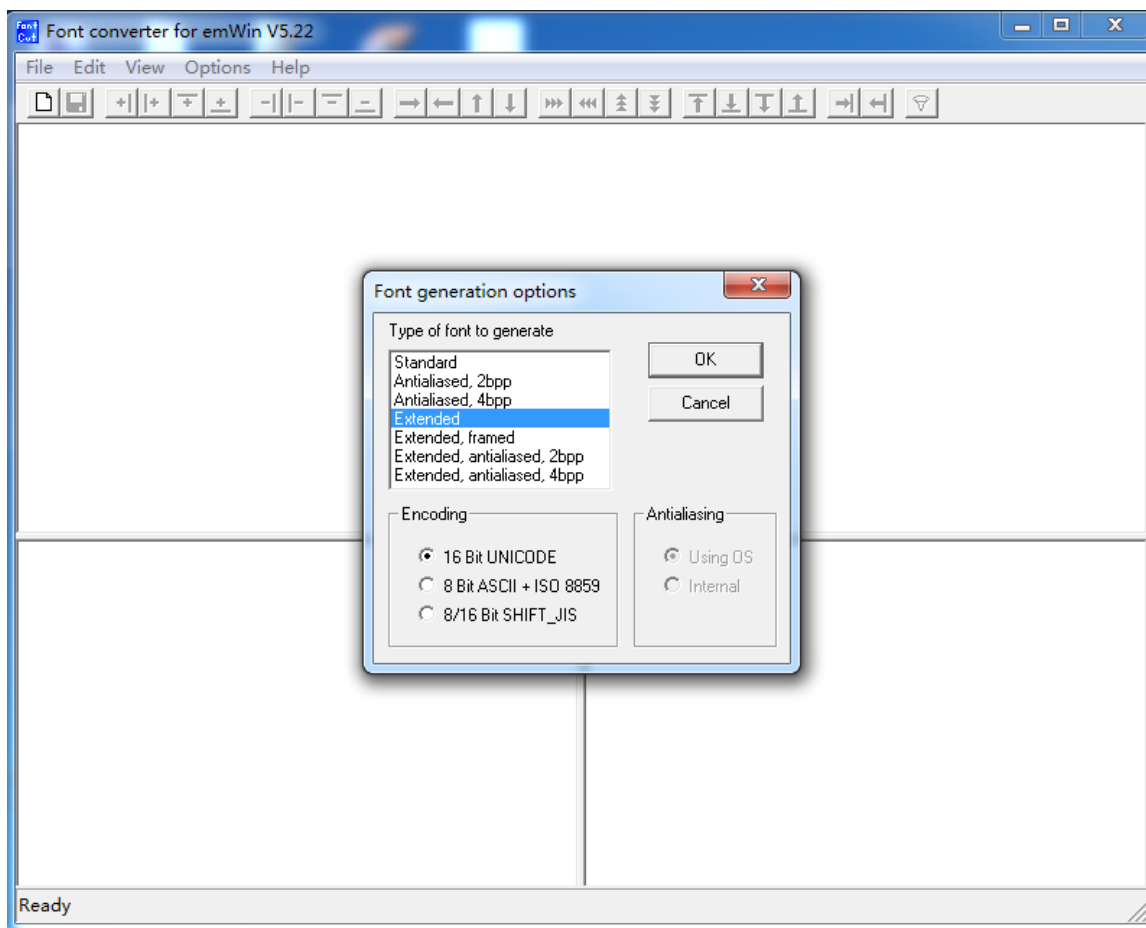


图 20.1.2 字体转换器

接下来我们制作 4 个 XBF 字库，字体都选择为宋体，大小分别为 12,16,24 和 36，字体类型为“Extended,antialiased,2bpp”，也就是扩展比例位图字体，包含 2bpp 抗锯齿。

1、选择字体类型，如图 20.1.3 所示。

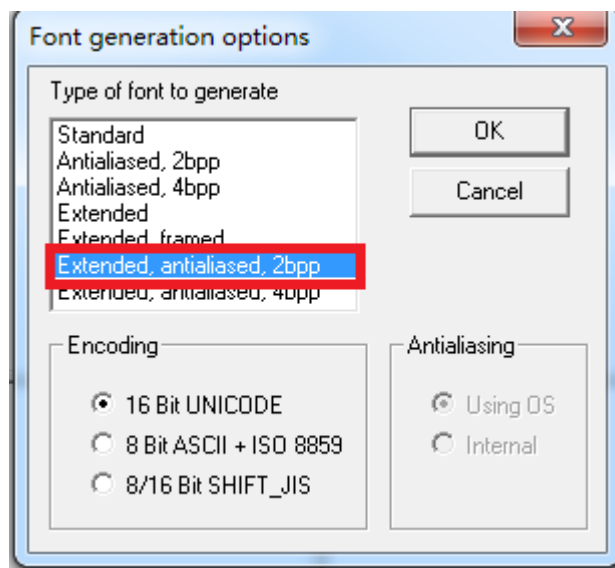


图 20.1.3 字体类型为“Extended,antialiased,2bpp”

2、点击图 20.1.3 中的“OK”以后，出现图 20.1.4 所示对话框，选择字体为“宋体”，大小为“12”，如图 20.1.4 所示。

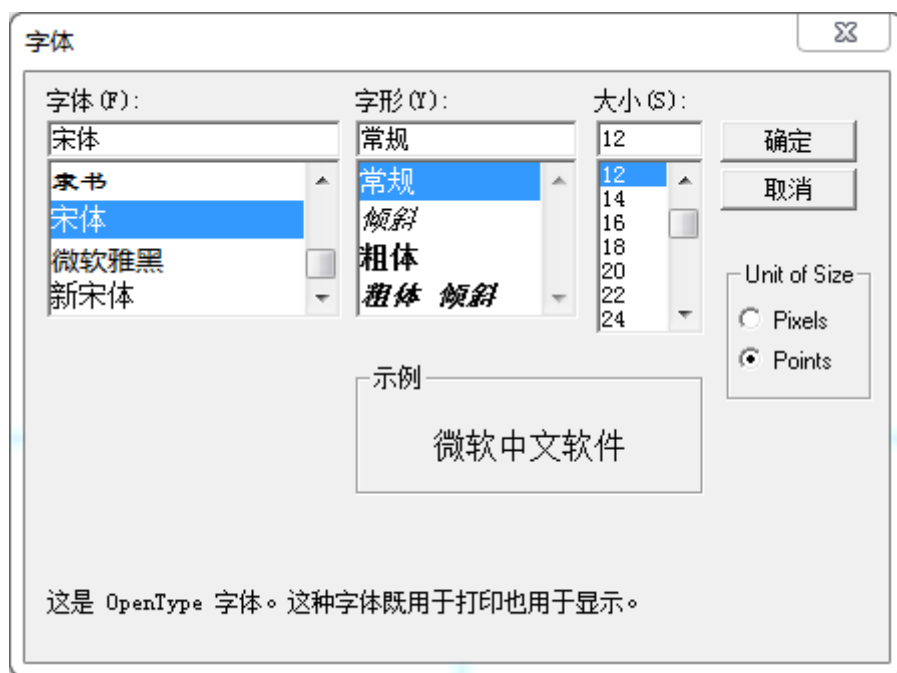


图 20.1.4 设置字体格式

设置完成以后点击“确定”，字体转换器如图 20.1.5 所示。

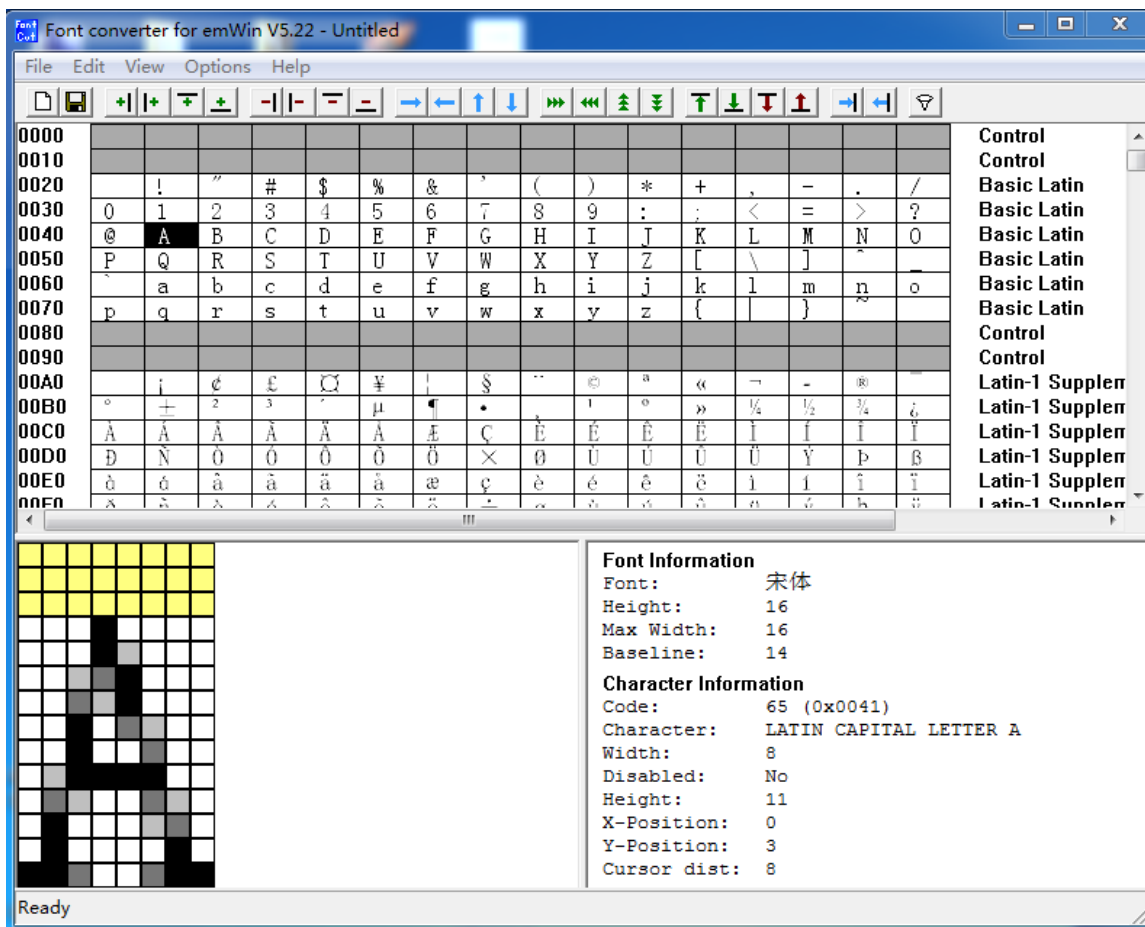


图 20.1.5 设置完字体类型和格式以后的字体转换器

3、保存字体，点击：File->Save As，出现如图 20.1.6 所示对话框，选择保存类型为.xbf，文件名设置为 XBF12.xbf。

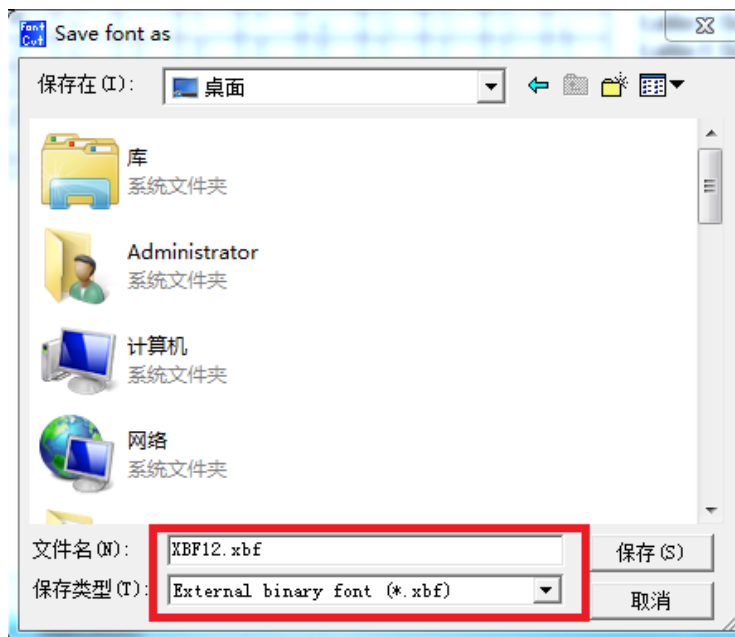


图 20.1.6 设置保存格式

4、点击图 20.1.6 中的保存以后，等待字库制作完成，使用同样的方法制作其他 16,24 和 36 号字。

5、将制作好的 XBF 字体拷贝到 SD 卡指定文件目录中：**SYSTEM->EMWINFONT->XBF**，如图 20.1.7 所示。

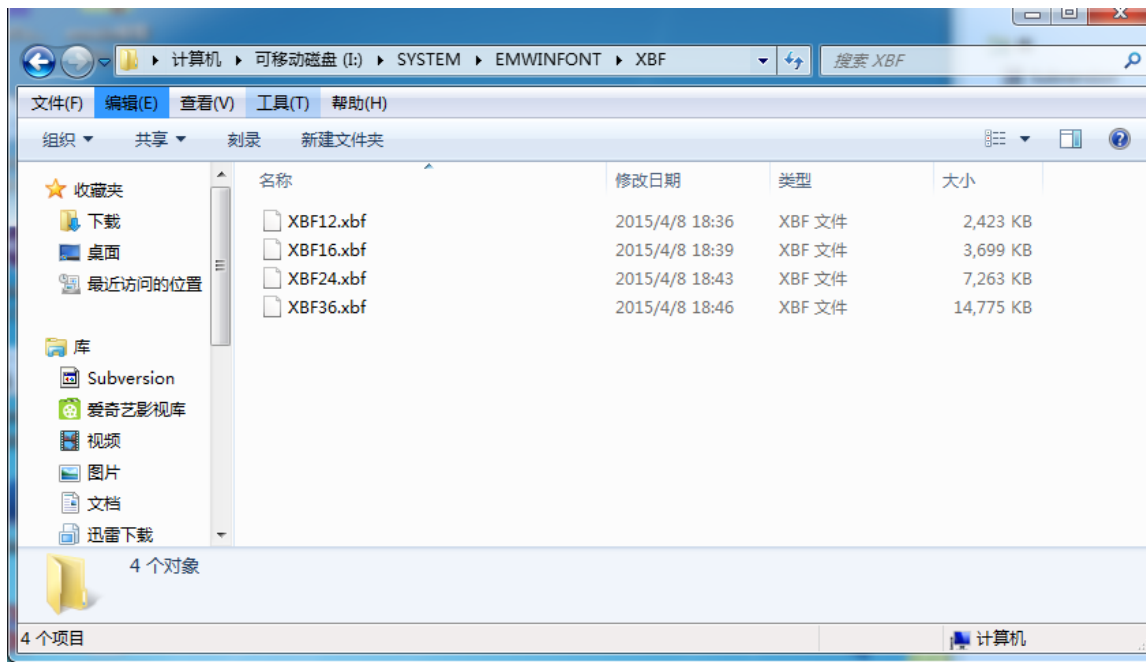


图 20.1.7 将制作好的字体复制到 SD 卡中

我们已经制作好了 12 号、16 号、24 号和 36 号共四种不同大小的字体，大家可以直接复制到 SD 卡中。在后面的章节中我们还会用到 SD 卡，实验中需要用到所有文件 ALIENTEK 都已经准备好了放在文件夹“**EMWIN 实验所需 SD 卡文件**”中，如图 20.1.8 所示，大家可以把其中的所有文件拷贝到 SD 卡根目录下。



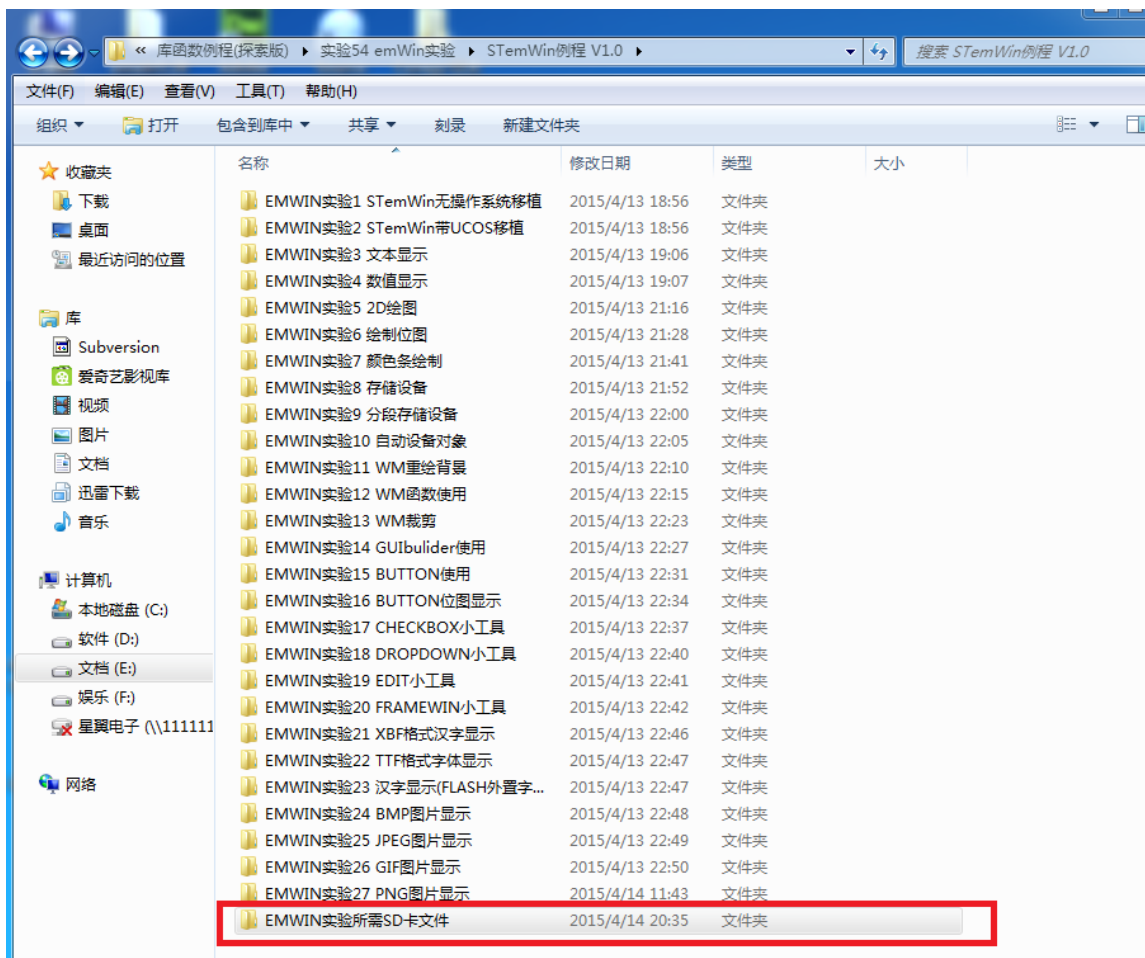


图 20.1.8 实验所需 SD 卡文件 1

打开图 20.1.8 文件夹，如图 20.1.9 所示，共有三个文件夹：PICTURE、SCREENSHORT 和 SYSTEM，将这三个文件拷贝到 SD 卡根目录中！

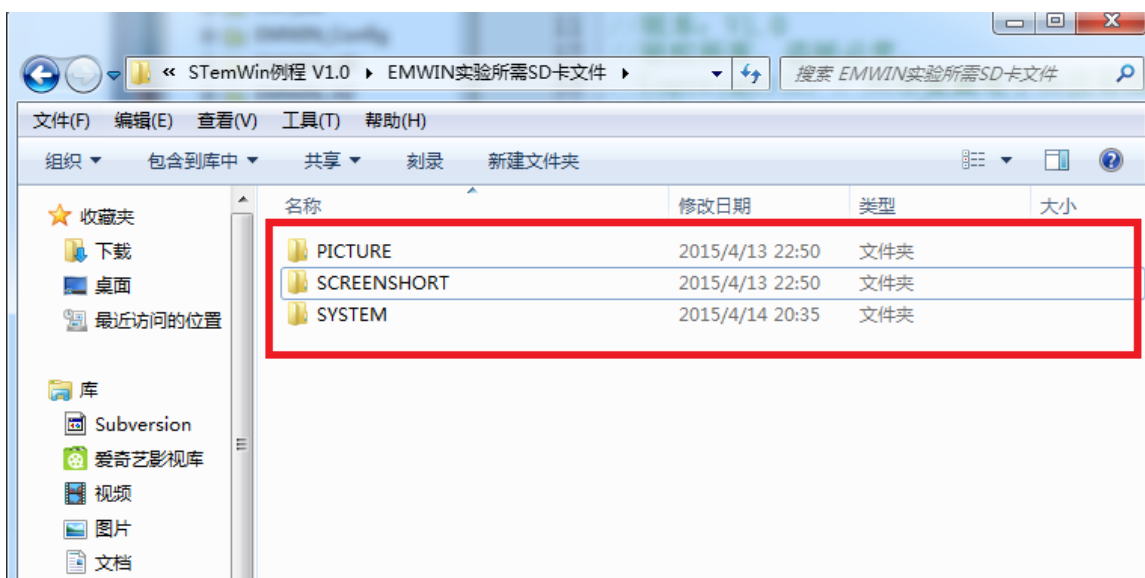


图 20.1.9 实验所需 SD 卡文件 2

## 20.2 XBF 字体相关 API 函数

与 XBF 字体相关的函数有两个：GUI\_XBF\_CreateFont()和 GUI\_XBF\_DeleteFont()。

### 20.2.1 GUI\_XBF\_CreateFont()函数

#### 描述

通过将指针传递至负责从 XBF 字体文件获取数据的回调函数创建和选择字体。

#### 原型

```
int GUI_XBF_CreateFont( GUI_FONT          *pFont,
                        GUI_XBF_DATA      *pXBF_Data,
                        const GUI_XBF_TYPE *pFontType,
                        GUI_XBF_GET_DATA_FUNC *pfGetData,
                        void                *pVoid );
```

函数参数含义如表 20.2.1 所示。

| 参数        | 描述                               |
|-----------|----------------------------------|
| pFont     | 指向 RAM 中由该函数填充的 GUI_FONT 结构。     |
| pXBF_Data | 指向 RAM 中由该函数填充的 GUI_XBF_DATA 结构。 |
| pFontType | 请参见下表。                           |
| pfGetData | 指向负责从字体文件获取数据的回调函数，请参见以下原型。      |
| pVoid     | 传递到“GetData”回调函数的应用定义的指针。        |

表 20.2.1 函数参数描述

函数 GUI\_XBF\_CreateFont()的参数 pFontType 的允许值见表 20.2.2。

| 元素 pFontType 的允许值         |                                     |
|---------------------------|-------------------------------------|
| GUI_XBF_TYPE_PROP         | 在参数 pFont 指向比例字体时使用。                |
| GUI_XBF_TYPE_PROP_EXT     | 在参数 pFont 指向扩展比例字体时使用。              |
| GUI_XBF_TYPE_PROP_FRM     | 在参数 pFont 指向扩展带边框比例字体时使用。           |
| GUI_XBF_TYPE_PROP_AA2_EXT | 在参数 pFont 指向使用 2 bpp 抗锯齿的扩展比例字体时使用。 |
| GUI_XBF_TYPE_PROP_AA4_EXT | 在参数 pFont 指向使用 4 bpp 抗锯齿的扩展比例字体时使用。 |

表 20.2.2 参数 pFontType 允许值

#### GUI\_XBF\_GET\_DATA\_FUNC 描述

```
int GUI_XBF_GET_DATA_FUNC( U32 Off, U16 NumBytes,
                           void *pVoid, void pBuffer);
```

该函数应设置 pBuffer，指向请求数据所在的位置，比如下面演示的“GetData”函数：

```
int GetData(U32 off, U16 NumBytes,void * pVoid,void *pBuffer)
{
    HANDLE * phFile;
    DWORD NumBytesRead;
    phFile = (HANDLE *)pVoid; //要读取的文件
    //检查缓冲区大小
    if (NumBytesReq > sizeof(_acBuffer)) NumBytesReq = sizeof(_acBuffer);
```

```
//移动文件指针到需要读取的数据处
SetFilePointer(*phFile, Off, 0, FILE_BEGIN);
//将数据从存储器中读取到缓冲区_acBuffer 中
ReadFile(phFile, _pBuffer, NumBytes, NumBytes, NULL, (UINT *)&bread);
return 0;
}
```

上面函数 GetData() 只是一个演示函数, 在实际使用过程中需要根据自己所使用的文件系统做出相应的修改。

### 其他信息

参数 pfGetData 应指向负责从字体文件获取数据的应用定义的回调函数, 在请求字体数据时, 参数 pVoid 将传递给回调函数, 例如, 可使用它将文件句柄传递给回调函数。该函数要求指向 GUI\_FONT 结构和 GUI\_XBF\_DATA 结构, 该函数将使用字体信息填充这些结构, 这些结构的内容在字体使用期间必须保持有效。该函数不知道必须要创建的 XBF 字体类型, 因此必须使用参数 pFontType 告知该函数要创建的字体类型, 这样, 可避免链接不必要的代码。默认情况下, 每个字符的数据字节最大值限制为 200, 这应该满足大部分要求, 如果加载使用更多字节的字符, 在调试版本中将生成警告, 通过向文件 GUIConf.h 添加以下定义可增加默认值:

```
#define GUI_MAX_XBF_BYTES 500 //设置最大字节数为 500
```

## 20.2.2 GUI\_XBF\_DeleteFont()

### 描述

删除参数 pFont 指示的 XBF 字体。

### 原型

```
void GUI_XBF_DeleteFont(GUI_FONT * pFont);
```

### 参数说明

pFont: 指向要删除的字体。

### 其他信息

使用 GUI\_XBF\_CreateFont() 创建的字体后, 如果不再使用该字体, 应将其删除。

## 20.3 程序设计

在上一节中我们已经制作好了 XBF 字库, 但是此时 emwin 还不能使用, 我们还没有制作适合 emwin 调用的字体, 本章要使用到 SD 卡和文件系统 FATFS, 关于 SD 卡和 FATFS 的使用不属于本教程范围, 请参考 ALIENTEK 的《STM32F4 开发指南》中相关章节。本实验完整工程为: **EMWIN 实验 21 XBF 格式汉字显示**。

### 20.3.1 制作字体

打开本章对应实验工程中的 xbffontcreate.c 文件, 程序源码如下:

```
#include "xbffontcreate.h"
#include "ff.h"

#if SYSTEM_SUPPORT_UCOS
```

```

#include "includes.h" //ucos 使用
#endif

//定义字体
GUI_FONT XBF12_Font;
GUI_FONT XBF16_Font;
GUI_FONT XBF24_Font;
GUI_FONT XBF36_Font;

GUI_XBF_DATA XBF12_Data;
GUI_XBF_DATA XBF16_Data;
GUI_XBF_DATA XBF24_Data;
GUI_XBF_DATA XBF36_Data;

FIL XBF16FontFile;
FIL XBF12FontFile;
FIL XBF24FontFile;
FIL XBF36FontFile;

//回调函数，用来获取字体数据
//参数： Off:      在 XBF 中偏移(位置)
//      NumBytes:  要读取的字节数
//      pVoid:     要读取的文件
//      pBuff:     读取到的数据的缓冲区
//返回值:0 成功，1 失败
static int _cbGetData(U32 Off, U16 NumBytes, void *pVoid, void *pBuffer) (1)
{
    int result;
    u16 bread;
    FIL *hFile;

    #if SYSTEM_SUPPORT_UCOS
        OS_CPU_SR cpu_sr;
    #endif

    hFile = (FIL*)pVoid; (2)

    //设置在文件中的偏移(位置)
    result = f_lseek(hFile,Off); (3)
    if(result != FR_OK) return 1; //返回错误

    //读取字体数据
    #if SYSTEM_SUPPORT_UCOS

```

```

        OS_ENTER_CRITICAL(); //临界区
    #endif

    result = f_read(hFile,pBuffer,NumBytes,(UINT *)&bread); //读取数据 (4)

    #if SYSTEM_SUPPORT_UCOS
        OS_EXIT_CRITICAL(); //退出临界区
    #endif

    if(result != FR_OK) return 1; //返回错误
    return 0;
}

//创建 XBF12 字体，共 EMWIN 使用
//fxpath:XBF 字体文件路径
//返回值:0，成功；1，失败
u8 Create_XBF12(u8 *fxpath) (5)
{
    int result;
    result = f_open(&XBF12FontFile,(const TCHAR*)fxpath,FA_READ); //打开字库文件

    if(result != FR_OK) return 1;
    //创建 XBF16 字体
    GUI_XBF_CreateFont( &XBF12_Font, //指向 GUI_FONT 结构 (6)
                        &XBF12_Data, //指向 GUI_XBF_DATA 结构
                        GUI_XBF_TYPE_PROP_AA2_EXT, //要创建的字体类型 (7)
                        _cbGetData, //回调函数
                        &XBF12FontFile); //窗体给回调函数_cbGetData 的参数

    return 0;
}

//创建 XBF16 字体，共 EMWIN 使用
//fxpath:XBF 字体文件路径
//返回值:0，成功；1，失败
u8 Create_XBF16(u8 *fxpath) (8)
{
    int result;
    result = f_open(&XBF16FontFile,(const TCHAR*)fxpath,FA_READ); //打开字库文件

    if(result != FR_OK) return 1;
    //创建 XBF16 字体
    GUI_XBF_CreateFont( &XBF16_Font, //指向 GUI_FONT 结构
                        &XBF16_Data, //指向 GUI_XBF_DATA 结构

```

```

        GUI_XBF_TYPE_PROP_AA2_EXT, //要创建的字体类型
        _cbGetData,                //回调函数
        &XBF16FontFile); //窗体给回调函数_cbGetData 的参数

    return 0;
}

//创建 XBF24 字体，共 EMWIN 使用
//fxpath:XBF 字体文件路径
//返回值:0，成功；1，失败
u8 Create_XBF24(u8 *fxpath) (9)
{
    int result;
    result = f_open(&XBF24FontFile, (const TCHAR*)fxpath, FA_READ); //打开字库文件
    if (result != FR_OK) return 1;
    //创建 XBF16 字体
    GUI_XBF_CreateFont( &XBF24_Font, //指向 GUI_FONT 结构
                        &XBF24_Data, //指向 GUI_XBF_DATA 结构
                        GUI_XBF_TYPE_PROP_AA2_EXT, //要创建的字体类型
                        _cbGetData, //回调函数
                        &XBF24FontFile); //窗体给回调函数_cbGetData 的参数

    return 0;
}

//创建 XBF36 字体，共 EMWIN 使用
//fxpath:XBF 字体文件路径
//返回值:0，成功；1，失败
u8 Create_XBF36(u8 *fxpath) (10)
{
    int result;
    result = f_open(&XBF36FontFile, (const TCHAR*)fxpath, FA_READ); //打开字库文件
    if (result != FR_OK) return 1;
    //创建 XBF16 字体
    GUI_XBF_CreateFont( &XBF36_Font, //指向 GUI_FONT 结构
                        &XBF36_Data, //指向 GUI_XBF_DATA 结构
                        GUI_XBF_TYPE_PROP_AA2_EXT, //要创建的字体类型
                        _cbGetData, //回调函数
                        &XBF36FontFile); //窗体给回调函数_cbGetData 的参数

    return 0;
}

```

(1)、函数 GUI\_XBF\_CreateFont() 需要使用到的回调函数，用于从存储器（如,SD 卡等）中获取 XBF 字库信息。

(2)、获取文件路径，即在 SD 卡中存放 XBF 字体的路径。

(3)、移动文件指针到指定位置。

- (4)、从 SD 卡中读取 XBF 字库数据。
- (5)、创建大小为 12 号的 XBF 字体。
- (6)、调用函数 GUI\_XBF\_CreateFont() 创建 XBF 字体。
- (7)、这里设置创建的字体类型，前面我们制作字库的时候选择的字体类型是“Extended,antialiased,2bpp”，也就是扩展比例位图字体，包含 2bpp 抗锯齿，所以这里的参数要设置为 GUI\_XBF\_TYPE\_PROP\_AA2\_EXT。
- (8)、创建大小为 16 号的 XBF 字体。
- (9)、创建大小为 24 号的 XBF 字体。
- (10)、创建大小为 36 号的 XBF 字体。

### 20.3.2 演示例程

上一小节我们已将创建了 XBF 字体，本节我们就编写一个 demo 测试一下字体创建是否成功，打开 xbffontdemo.c 文件，里面是我们创建的测试 demo，代码如下：

```
#include "xbffontdemo.h"
#include "xbffontcreate.h"
#include "DIALOG.h"

#define ID_FRAMEWIN_0      (GUI_ID_USER + 0)
#define ID_MULTIEDIT_0     (GUI_ID_USER + 1)
#define ID_BUTTON_0        (GUI_ID_USER + 2)
#define ID_BUTTON_1        (GUI_ID_USER + 3)
#define ID_CHECKBOX_0      (GUI_ID_USER + 4)
#define ID_CHECKBOX_1      (GUI_ID_USER + 5)
#define ID_CHECKBOX_2      (GUI_ID_USER + 6)
#define ID_CHECKBOX_3      (GUI_ID_USER + 7)
#define ID_RADIO_0         (GUI_ID_USER + 8)
#define ID_LISTVIEW_0      (GUI_ID_USER + 9)
#define ID_DROPDOWN_0      (GUI_ID_USER + 10)
#define ID_TEXT_0          (GUI_ID_USER + 11)
#define ID_TEXT_1          (GUI_ID_USER + 12)
#define ID_TEXT_2          (GUI_ID_USER + 13)
#define ID_TEXT_3          (GUI_ID_USER + 14)
#define ID_SCROLLBAR_0     (GUI_ID_USER + 15)
#define ID_SLIDER_0        (GUI_ID_USER + 16)

WM_HWIN hDialog;

static const char *_ListViewTable[][4]={
    {"张三", "20141226001","97","A"},
    {"李四", "20141226002","84","B"},
    {"刘翔", "20141226003","73","C"},
    {"赵子龙", "20141226004","85","B"},
    {"刘备", "20141226005","45","E"},
}
```

```

{"欧阳夏雨", "20141226006","98","A"},
{"王子奇", "20141226007","84","B"},
{"孙菲菲", "20141226008","65","D"},
};

char *MultiEdit_String =
"STM32F407 探索者开发板\r\n      \
CPU:STM32F407ZET6\r\n      \
FLASH:1M\r\n      \
RAM      :192K+64K\r\n      \
板载:10M/100M 以太网 PHY 芯片\r\n\
      WM8978 音频 DAC  ";

//对话框资源列表
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] = {
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 800, 480, 0, \
      FRAMEWIN_CF_MOVEABLE, 0 },
    { MULTIEDIT_CreateIndirect, "Multiedit", ID_MULTIEDIT_0, 20, 20, 300, 160, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "BUTTON",ID_BUTTON_0, 398,370, 140, 50, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "BUTTON", ID_BUTTON_1, 599, 370, 140, 50, 0, 0x0, 0 },
    { CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_0, 530, 90, 120, 30, 0, 0x0, 0 },
    { CHECKBOX_CreateIndirect, "Checkbox",ID_CHECKBOX_1,530, 130, 120, 30, 0, 0x0, 0 },
    { CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_2,530, 170,120, 30, 0, 0x0, 0 },
    { CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_3, 530, 210, 120, 30, 0, 0x0, 0 },
    { RADIO_CreateIndirect, "Radio", ID_RADIO_0, 650, 119, 80, 87, 0, 0x1404, 0 },
    { LISTVIEW_CreateIndirect, "Listview", ID_LISTVIEW_0, 20, 200, 300, 194, 0, 0x0, 0 },
    { DROPDOWN_CreateIndirect, "Dropdown", ID_DROPDOWN_0, 324, 22, 150, 100, 0, 0x0, 0 },
    { TEXT_CreateIndirect, "Text0", ID_TEXT_0, 327, 180, 150, 20, 0, 0x64, 0 },
    { TEXT_CreateIndirect, "Text1", ID_TEXT_1, 327, 200, 150, 20, 0, 0x64, 0 },
    { TEXT_CreateIndirect, "Text2", ID_TEXT_2, 327, 220, 200, 30, 0, 0x64, 0 },
    { TEXT_CreateIndirect, "Text3", ID_TEXT_3, 327, 250, 300, 50, 0, 0x64, 0 },
    { SCROLLBAR_CreateIndirect, "Scrollbar", ID_SCROLLBAR_0, 482, 310, 260, 40, 0, 0x0, 0 },
    { SLIDER_CreateIndirect, "Slider", ID_SLIDER_0, 513, 18, 259, 38, 0, 0x0, 0 },
};

//对话框窗口回调函数
static void _cbDialog(WM_MESSAGE * pMsg)
{
    WM_HWIN hItem;
    WM_HWIN hHeader;
    int      NCode;
    int      Id;
    int      i;

```



```
switch (pMsg->MsgId)
{
    case WM_INIT_DIALOG:

        //初始化 FRAMEWIN
        hItem = pMsg->hWin;
        FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
        FRAMEWIN_SetTitleHeight(hItem, 30);
        FRAMEWIN_SetFont(hItem,&XBF16_Font);
        FRAMEWIN_SetTextColor(hItem, GUI_RED);
        FRAMEWIN_SetText(hItem, "ALIENTEK XBF 汉字显示");

        //初始化 MULTIEDIT
        hItem = WM_GetDialogItem(pMsg->hWin, ID_MULTIEDIT_0);
        MULTIEDIT_SetTextColor(hItem,MULTIEDIT_CI_EDIT,GUI_BLACK);
        MULTIEDIT_SetBkColor(hItem,MULTIEDIT_CI_EDIT,GUI_LIGHTCYAN);
        MULTIEDIT_SetFont(hItem,&XBF16_Font);
        MULTIEDIT_SetText(hItem,MultiEdit_String);

        //初始化 BUTTON
        hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
        BUTTON_SetFont(hItem,&XBF16_Font);
        BUTTON_SetText(hItem,"确定");
        hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
        BUTTON_SetFont(hItem,&XBF16_Font);
        BUTTON_SetText(hItem,"取消");

        //初始化 CHECKBOX
        hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_0);
        CHECKBOX_SetFont(hItem,&XBF12_Font);
        CHECKBOX_SetText(hItem, "复选框 1");
        hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_1);
        CHECKBOX_SetFont(hItem,&XBF12_Font);
        CHECKBOX_SetText(hItem, "复选框 2");
        hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_2);
        CHECKBOX_SetFont(hItem,&XBF12_Font);
        CHECKBOX_SetText(hItem, "复选框 3");
        hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_3);
        CHECKBOX_SetFont(hItem,&XBF12_Font);
        CHECKBOX_SetText(hItem, "复选框 4");

        //初始化 RADIO
        hItem = WM_GetDialogItem(pMsg->hWin, ID_RADIO_0);
```

```
RADIO_SetFont(hItem,&XBF12_Font);
RADIO_SetText(hItem, "选项 1", 0);
RADIO_SetText(hItem, "选项 2", 1);
RADIO_SetText(hItem, "选项 3", 2);
RADIO_SetText(hItem, "选项 4", 3);

//初始化 LISTVIEW
hItem = WM_GetDialogItem(pMsg->hWin, ID_LISTVIEW_0);
hHeader = LISTVIEW_GetHeader(hItem);
LISTVIEW_SetHeaderHeight(hItem, 30);
HEADER_SetFont(hHeader,&XBF12_Font);
LISTVIEW_AddColumn(hItem, 40, "名字", GUI_TA_HCENTER | \
    GUI_TA_VCENTER);
LISTVIEW_AddColumn(hItem,40,"学号", GUI_TA_HCENTER | \
    GUI_TA_VCENTER);
LISTVIEW_AddColumn(hItem, 40, "成绩", GUI_TA_HCENTER | \
    GUI_TA_VCENTER);
LISTVIEW_AddColumn(hItem, 40, "等级", GUI_TA_HCENTER | \
    GUI_TA_VCENTER);
LISTVIEW_SetGridVis(hItem, 1);
LISTVIEW_SetAutoScrollH(hItem, 1);
LISTVIEW_SetAutoScrollV(hItem, 1);
LISTVIEW_SetRowHeight(hItem, 20);
LISTVIEW_SetFont(hItem,&XBF16_Font);
LISTVIEW_SetBkColor(hItem,LISTVIEW_CI_UNSEL,GUI_ORANGE);
for(i=0;i<GUI_COUNTOF(_ListViewTable);i++)
{
    LISTVIEW_AddRow(hItem,_ListViewTable[i]);
}

//初始化 DROPDOWN
hItem = WM_GetDialogItem(pMsg->hWin, ID_DROPDOWN_0);
DROPDOWN_SetAutoScroll(hItem,1);//开启自动滚动条
DROPDOWN_SetFont(hItem,&XBF16_Font);
DROPDOWN_AddString(hItem, "汉语");
DROPDOWN_AddString(hItem, "英语");
DROPDOWN_AddString(hItem, "韩语");
DROPDOWN_AddString(hItem, "马来西亚语");
DROPDOWN_AddString(hItem, "西班牙语");
DROPDOWN_AddString(hItem, "其他语言.....");

//初始化 TEXT
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
```

```

TEXT_SetFont(hItem,&XBF12_Font);
TEXT_SetTextColor(hItem,GUI_YELLOW);
TEXT_SetText(hItem, "广州星翼电子");
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_1);
TEXT_SetFont(hItem,&XBF16_Font);
TEXT_SetTextColor(hItem,GUI_MAGENTA);
TEXT_SetText(hItem, "广州星翼电子");
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_2);
TEXT_SetFont(hItem,&XBF24_Font);
TEXT_SetTextColor(hItem,GUI_TRANSPARENT);
TEXT_SetText(hItem, "广州星翼电子");
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_3);
TEXT_SetFont(hItem,&XBF36_Font);
TEXT_SetTextColor(hItem,GUI_BLUE);
TEXT_SetText(hItem, "广州星翼电子");
break;
case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;
    switch(Id)
    {
        case ID_MULTIEDIT_0:
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    break;
                case WM_NOTIFICATION_VALUE_CHANGED:
                    break;
            }
            break;
        case ID_BUTTON_0:
            switch(NCode)
            {
                case WM_NOTIFICATION_RELEASED:
                    break;
            }
            break;
        case ID_BUTTON_1:
            switch(NCode)
            {
                case WM_NOTIFICATION_RELEASED:

```

```

        break;
    }
    break;
}
break;
default:
    WM_DefaultProc(pMsg);
    break;
}
}

void XBFFontDemo(void)
{
    //更换皮肤
    BUTTON_SetDefaultSkin(BUTTON_SKIN_FLEX);
    CHECKBOX_SetDefaultSkin(CHECKBOX_SKIN_FLEX);
    DROPDOWN_SetDefaultSkin(DROPDOWN_SKIN_FLEX);
    FRAMEWIN_SetDefaultSkin(FRAMEWIN_SKIN_FLEX);
    HEADER_SetDefaultSkin(HEADER_SKIN_FLEX);
    MENU_SetDefaultSkin(MENU_SKIN_FLEX);
    MULTIPAGE_SetDefaultSkin(MULTIPAGE_SKIN_FLEX);
    PROGBAR_SetDefaultSkin(PROGBAR_SKIN_FLEX);
    RADIO_SetDefaultSkin(RADIO_SKIN_FLEX);
    SCROLLBAR_SetDefaultSkin(SCROLLBAR_SKIN_FLEX);
    SLIDER_SetDefaultSkin(SLIDER_SKIN_FLEX);
    SPINBOX_SetDefaultSkin(SPINBOX_SKIN_FLEX);

    GUI_CURSOR_Show();
    #if GUI_SUPPORT_MEMDEV
        WM_SetCreateFlags(WM_CF_MEMDEV);
    #endif

    hDialog = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate), \
        _cbDialog, WM_HBKWIN, 0, 0);

    GUI_Exec();
    while(1)
    {
        GUI_Delay(100);
    }
}

```

上面就是我们的测试 demo，就是用 GUIBulider 创建的一个对话框，对话框里面有一些控件，很简单，前面章节都已经讲过了这些控件的使用，这里主要是展示一下如何在控件中使用 XBF 格式的汉字。

### 20.3.3 启用 UTF8 编码

使用 XBF 格式字体的话需要使能 emWin 的 UTF8 编码格式，我们在 main.c 中的任务函数 emwin\_maintask() 中使能 emWin 的 UTF8 编码格式，emwin\_maintask() 函数源码如下：

void emwin\_maintask(void \*pdata)

```
{
    GUI_CURSOR_Show();
    Create_XBF12("0:/SYSTEM/EMWINFONT/XBF/XBF12.xbf");           (1)
    Create_XBF16("0:/SYSTEM/EMWINFONT/XBF/XBF16.xbf");           (2)
    Create_XBF24("0:/SYSTEM/EMWINFONT/XBF/XBF24.xbf");           (3)
    Create_XBF36("0:/SYSTEM/EMWINFONT/XBF/XBF36.xbf");           (4)
    GUI_UC_SetEncodeUTF8();                                       (5)
    while(1)
    {
        XBFFontDemo();
    }
}
```

(1)、创建大小为 12 号的 XBF 字体。

(2)、创建大小为 16 号的 XBF 字体。

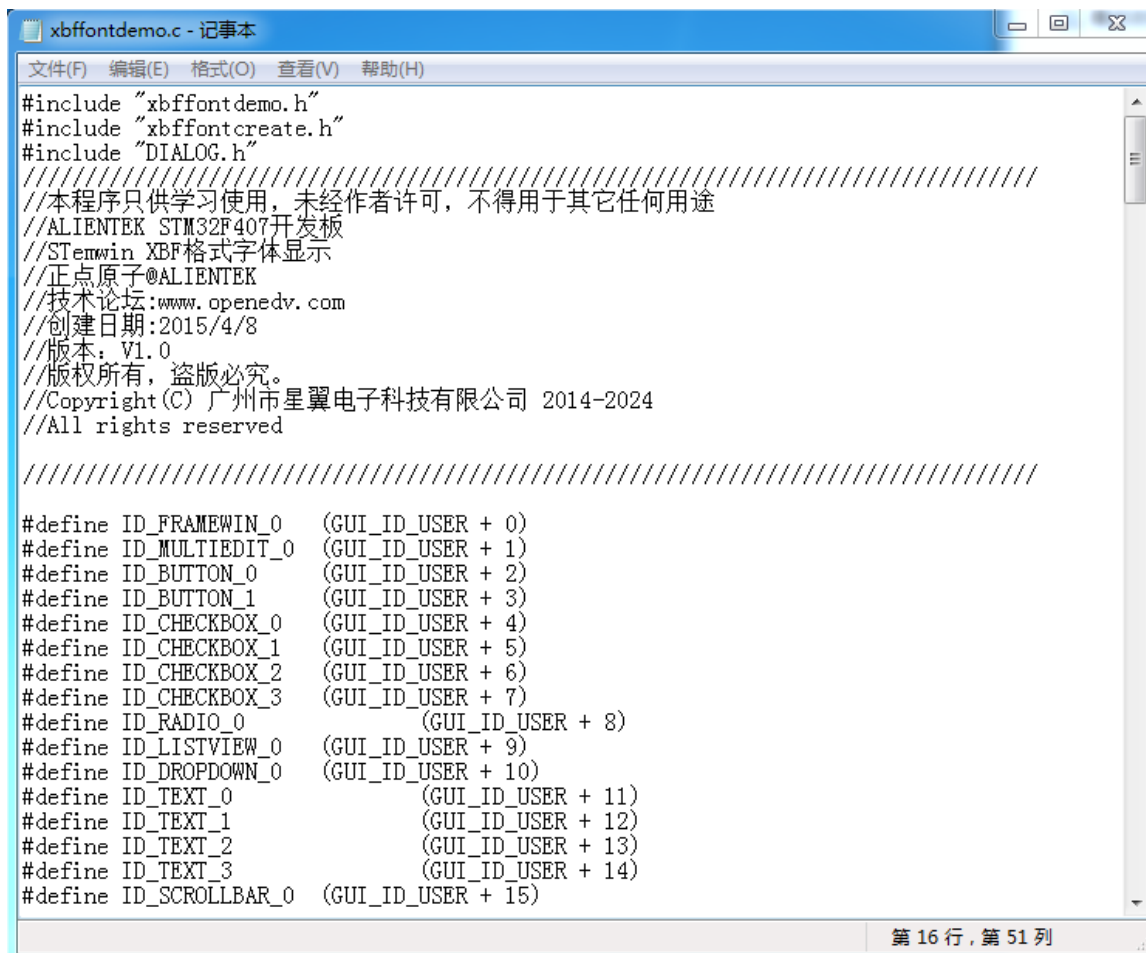
(3)、创建大小为 24 号的 XBF 字体。

(4)、创建大小为 36 号的 XBF 字体。

(5)、调用函数 GUI\_UC\_SetEncodeUTF8() 启用 UTF8 编码。

启用 UTF8 编码以后还需要将使用到 XBF 字体的文件的编码方式改为 UTF8，本例程里面只有 xbffontdemo.c 使用到了我们前面创建的 XBF 字体，所以只需要将 xbffontdemo.c 文件的编码方式更改为 UTF8，按如下方式修改。

1、用记事本打开 xbffontdemo.c 文件，如图 20.3.1 所示。



```
xbffontdemo.c - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

#include "xbffontdemo.h"
#include "xbffontcreate.h"
#include "DIALOG.h"

/////////////////////////////////////////////////////////////////
//本程序只供学习使用，未经作者许可，不得用于其它任何用途
//ALIENTEK STM32F407开发板
//STemwin XBF格式字体显示
//正点原子@ALIENTEK
//技术论坛:www.openedv.com
//创建日期:2015/4/8
//版本: V1.0
//版权所有，盗版必究。
//Copyright(C) 广州市星翼电子科技有限公司 2014-2024
//All rights reserved

/////////////////////////////////////////////////////////////////

#define ID_FRAMEWIN_0 (GUI_ID_USER + 0)
#define ID_MULTIEDIT_0 (GUI_ID_USER + 1)
#define ID_BUTTON_0 (GUI_ID_USER + 2)
#define ID_BUTTON_1 (GUI_ID_USER + 3)
#define ID_CHECKBOX_0 (GUI_ID_USER + 4)
#define ID_CHECKBOX_1 (GUI_ID_USER + 5)
#define ID_CHECKBOX_2 (GUI_ID_USER + 6)
#define ID_CHECKBOX_3 (GUI_ID_USER + 7)
#define ID_RADIO_0 (GUI_ID_USER + 8)
#define ID_LISTVIEW_0 (GUI_ID_USER + 9)
#define ID_DROPDOWN_0 (GUI_ID_USER + 10)
#define ID_TEXT_0 (GUI_ID_USER + 11)
#define ID_TEXT_1 (GUI_ID_USER + 12)
#define ID_TEXT_2 (GUI_ID_USER + 13)
#define ID_TEXT_3 (GUI_ID_USER + 14)
#define ID_SCROLLBAR_0 (GUI_ID_USER + 15)
```

第 16 行, 第 51 列

图 20.3.1 记事本打开文件

2、点击：文件->另存为，出现如图 20.3.2 所示对话框，修改编码方式为 UTF-8。

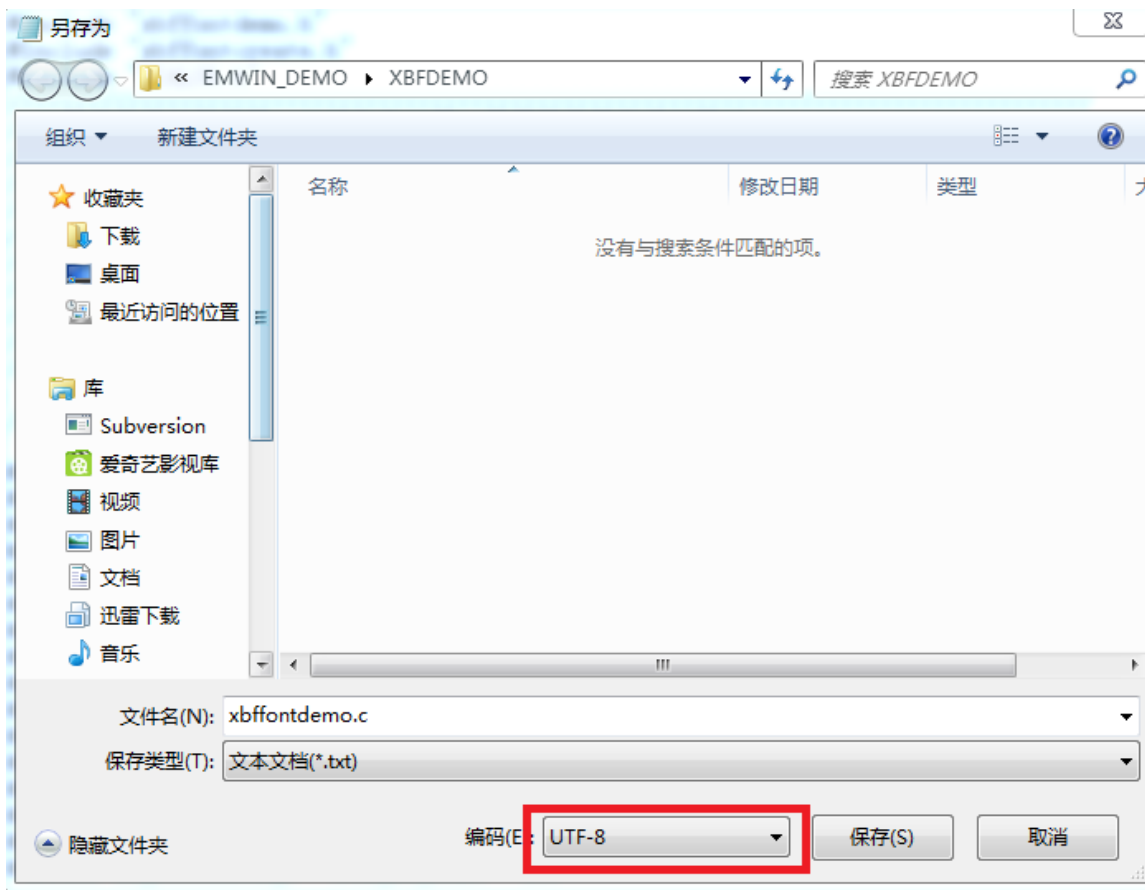


图 20.3.2 修改编码方式

3、修改完成以后点击保存就会替换到原来的文件，新文件的编码方式就改为了 UTF-8 了。

## 20.4 下载验证

编译工程并下载到开发板中，注意字库一定要放在 SD 卡中！下载完成以后 LCD 显示如图 20.4.1 所示，各个控件中汉字显示正常。

**注意！这里使用的是 4.3 寸 800\*480 分辨率的屏幕，如果使用其他小于 800\*480 的屏幕的话会导致显示不完全！**

因为显示汉字的时候需要从 SD 卡中读取字库信息，所以看起来显示的很慢，这个也是 XBF 格式字体的缺点。



图 20.4.1 LCD 显示效果



## 第二十一章 TTF 格式字体显示

TTF 是 True TypeFont 的缩写，也就是矢量字体，emWin 是支持矢量字体的，本章我们就讲解一下如何使用矢量字体，本章分为如下几部分：

21.1 TTF 字体简介

21.2 TTF 字体相关 API 函数

21.3 程序设计

21.4 下载验证

## 21.1 TTF 字体简介

我们平时使用 PDF 阅读文件的时候可以将文档方到很大,但是文档里面的内容确没有变的模糊,相反确很清晰,这是因为 PDF 使用的就是矢量字体, TTF 字体是由 Apple 和 Microsoft 两个公司推出的字体格式。

矢量字体的每个字符都是通过数学曲线描述的,在显示的时候通过读取指定字符的数学矢量并且对其进行一定的数学计算然后将其显示出来,正如前面所说的,这类字体的大小可以任意的缩放,改变颜色。

有三种主流的矢量字体: Type1, TrueType 和 OpenType, Type1 全称 PostScript Type1, 是由 Adobe 公司提出的矢量字体标准, Type1 是不免费的,使用 Type1 的话需要向 Adobe 交付使用费。TrueType 是由 Apple 公司与 Microsoft 公司联合提出另一套矢量字标准, Type1 使用三次贝塞尔曲线来描述字形, TrueType 则使用二次贝塞尔曲线来描述字形,所以 TrueType 没有 Type1 的字体美观。但是实际中用的最广泛的是 TrueType, 这是因为 Type1 字体方程的复杂,所以在屏幕上显示的时候花费的时间多,另一个原因是 Type1 需要支付高昂的使用费用。OpenType 则是 Type1 与 TrueType 之争的最终产物。是由 Adobe 和 Microsoft 合作开发出来的一种兼容 Type1 和 TrueType, 并且真正支持 Unicode 的字体, OpenType 可以嵌入 Type1 和 TrueType, 这样就兼有了二者的特点。

emWin 源码中默认不带 TTF 包的,需要我们自行去下载,下载地址为: [www.segger.com/link/emwin\\_freetype.zip](http://www.segger.com/link/emwin_freetype.zip)。注意这里下载下来的是 TTF 字体的渲染引擎,并不是 TTF 字库, TTF 字库需要我们自行下载,渲染引擎对 TTF 字库进行渲染并且显示在 LCD 上。

## 21.2 TTF 字体相关 API 函数

与 TTF 字体相关的函数有两个: GUI\_TTF\_CreateFont()和 GUI\_TTF\_DeleteFont()。

### 21.2.1 GUI\_TTF\_CreateFont()函数

#### 描述

使用 TTF 字体文件创建和选择 emWin 字体。

#### 原型

```
int GUI_TTF_CreateFont(GUI_FONT * pFont, GUI_TTF_CS * pCS);
```

#### 参数

函数参数描述见表 21.2.1 所示。

| 参数    | 描述                           |
|-------|------------------------------|
| pFont | 指向 RAM 中由该函数填充的 GUI_FONT 结构。 |
| pCS   | 指向包含创建参数的 GUI_TTF_CS 结构。     |

表 21.2.1 函数参数描述

#### 返回值

0 表示成功, 1 表示错误。

GUI\_TTF\_CS 为一个结构体, 如下:

```
typedef struct {
    GUI_TTF_DATA * pTTF;
    U32          aImageTypeBuffer[4];
    int           PixelHeight;
    int           FaceIndex;
```

```
} GUI_TTF_CS;
```

GUI\_TTF\_CS 结构体中各个元素的含义如表 21.2.2 所示：

| 数据类型          | 元素                 | 描述                                                                                        |
|---------------|--------------------|-------------------------------------------------------------------------------------------|
| GUI_TTF_DATA* | pTTF               | 指向包含要使用的字体文件位置和大小 GUI_TTF_DATA 结构。                                                        |
| U32           | aImageTypeBuffer[] | 图片类型结构体缓冲区。                                                                               |
| int           | PixelHeight        | 新字体的像素高度，表示字形 “g” 和 “f” 之间周围矩形的高度。请注意，这并非两行文本之间的距离。换句话说，GUI_GetFontSizeY() 返回的值与此值并不完全相同。 |
| int           | FaceIndex          | 有些字体文件可能包含多种字体风格，对于多种风格，此索引指定使用基于零的风格索引来创建字体，通常为 0。                                       |

表 21.2.2 GUI\_TTF\_CS 结构体元素

GUI\_TTF\_DATA 的元素如表 21.2.3 所示：

| 数据类型        | 元素       | 描述                    |
|-------------|----------|-----------------------|
| const void* | pData    | 指向可寻址存储区域中的 TTF 字体文件。 |
| NumBytes    | NumBytes | 文件的大小（字节）。            |

表 21.2.3 GUI\_TTF\_DATA 结构体元素

#### 其他信息

第一次使用该函数时，它会对 TTF 引擎和内部缓存系统进行初始化。如果缓存需要使用默认定义的其他值，则需要在首次调用此函数之前调用函数 GUI\_TTF\_SetCacheSize() 来进行配置。内部数据缓存管理创建字体和缓存位图数据的整个机制，字体风格在缓存中按参数 pTTF 和 FaceIndex 中给定的地址进行唯一标识，通常为 0。例如，如果应使用相同的字体文件来创建不同大小的字体，则参数 pTTF 应指向 GUI\_TTF\_DATA 结构的相同位置。参数 PixelHeight 指定字形 “g” 和 “f” 之间周围矩形的高度。请注意 PixelHeight 的值与一行文本到另一行文本之间的偏移量不同。

### 21.2.2 GUI\_TTF\_SetCacheSize()函数

#### 描述

设置第一次调用 GUI\_TTF\_CreateFont() 时用于创建缓存的大小参数。

#### 原型

```
void GUI_TTF_SetCacheSize(unsigned MaxFaces,unsigned MaxSizes, U32 MaxBytes);
```

函数参数描述见表 21.2.4。

| 参数       | 描述                         |
|----------|----------------------------|
| MaxFaces | 缓存应能够同时处理的最大字体风格数，0 选择默认值。 |
| MaxSizes | 缓存应能够同时处理的最大字体对象数，0 选择默认值。 |
| MaxBytes | 位图缓存使用的最大字节数，0 选择默认值。      |

表 21.2.4 函数参数描述表

#### 其他信息

例如，如果应使用 3 种字体风格，每种有 2 种大小，则缓存应能够处理 6 种字体对象。TTF 引擎使用的默认值是：2 种风格、4 种大小对象和 200 K 位图数据缓存。

## 21.3 程序设计

### 21.3.1 移植 5.26 版本 EMWIN

在下载好的 TTF 包 emwin\_freetype 中我们使用最新的 V5.26 版本，所以 EMWIN 版本也要相应的更换为 5.26 版本，但是 STemWin 没有对应的 5.26 版本，所以这里需要移植 5.26 版本的 EMWIN，我们从 Segger 官网下载相应版本的 EMWIN，我们已经下载好放到光盘中了，路径：[6、软件资料->3、EMWIN 学习资料->emWin5.26 源码](#)。移植过程和前面讲解的 STemWin 移植过程基本一致，只是这里要选择的 EMWIN 库为：GUI\_CM4F\_L.lib，如图 21.3.1 所示。

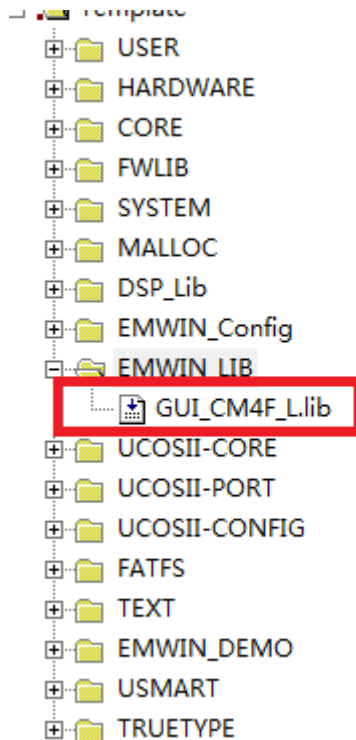


图 21.3.1 EMWIN5.26 移植

### 21.3.2 移植 Freetype 库

将下载下来的 Freetype 库移植到工程中，Freetype 我们已经下载下来放到光盘，路径：[6、软件资料->3、EMWIN 学习资料->emwin\\_freetype](#)，新建 TRUETYPE 分组，将 Freetype 文件添加进来，添加完成以后如图 21.3.2 所示。

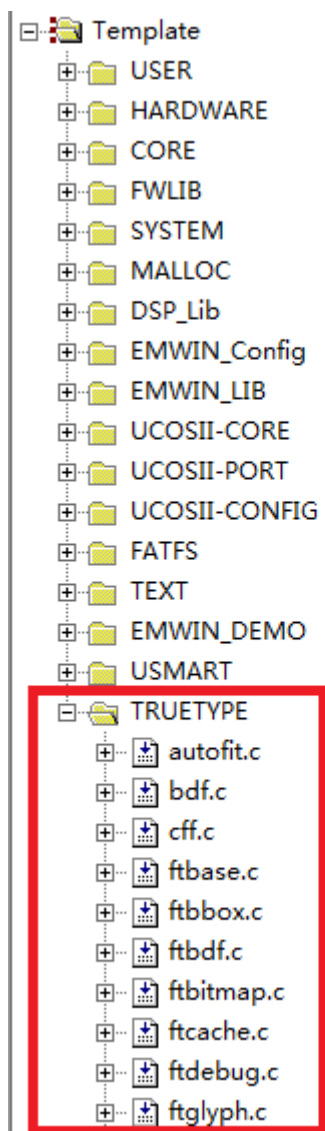


图 21.3.2 添加 Freetype 库

注意添加 Freetype 文件中相应的头文件路径，如图 21.3.3 所示。

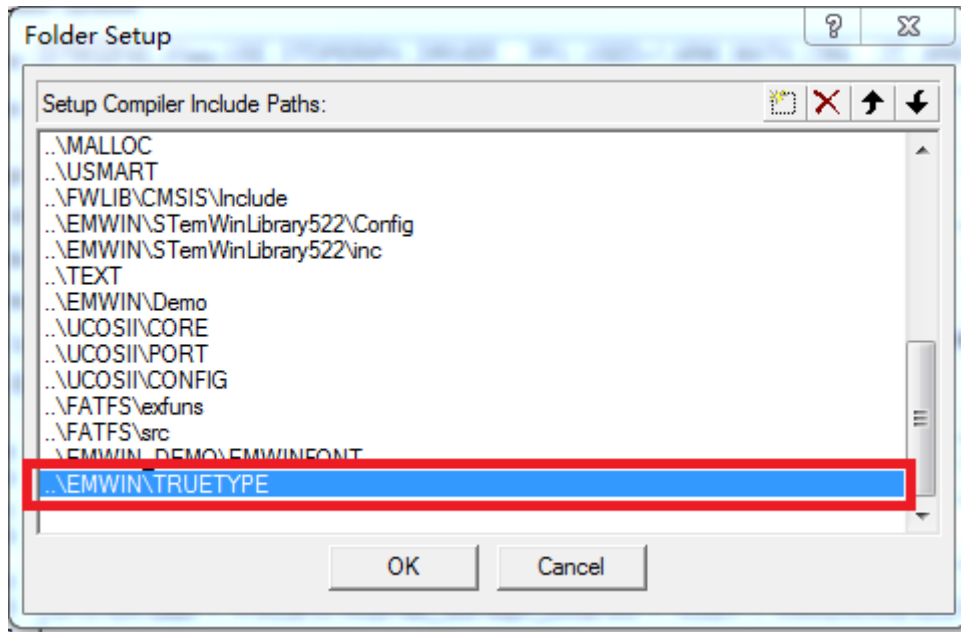


图 21.3.3 添加头文件路径

### 21.3.3 Freetype 内存管理

Freetype 需要使用到 malloc()和 free()函数来申请和释放内存，这里我们需要修改一下 ftstdlib.h 文件，让 Freetype 所使用的内存管理函数映射到 ALIENTEK 的内存管理函数上，打开 ftstdlib.h 文件，按照如图 21.3.4 所示修改。

```

139  /*****
140  /*
141  /*          memory allocation
142  /*
143  *****/
144
145
146  #define ttf_malloc(size)      mymalloc(SRAMIN, size)
147  #define ttf_free(block)      myfree(SRAMIN, block)
148  #define ttf_realloc(block, size) myrealloc(SRAMIN, block, size)
149
150  #define ft_scalloc      calloc
151  #define ft_sfree        ttf_free
152  #define ft_smallloc     ttf_malloc
153  #define ft_srealloc     ttf_realloc

```

图 21.3.4 修改内存管理函数

我们也可以采用另外一种方法，那就是不需要对 ftstdlib.h 做任何修改，让其使用 KEIL 自带的 malloc()函数和 free()函数，只需要将 startup\_stm32f40\_41xxx.s 中堆改大一点就可以了，如图 21.3.5 示，将 Heap\_Size 改为 0X0000F000。

```

49  ; <0> Heap Size (in Bytes) <0x0-0xFFFFFFFF:8>
50  ; </h>
51
52  Heap_Size      EQU      0x0000F000
53
54  AREA          HEAP, NOINIT, READWRITE, ALIGN=3
55  __heap_base

```

图 21.3.5 更改堆大小

### 21.3.4 准备 TTF 字库

前面下载的 Freetype 是 TTF 渲染文件，这里我们还需要下载 TTF 字库，我们已经整理了一些.ttf 格式的字库放在光盘中，路径：**3、EMWIN 学习资料->TTF 字库**。大家也可以自行下载一些 TTF 字库。由于开发板 RAM 限制，即使是使用外部 SRAM 最大也只有 1M，因此只能使用一些英文和数字的 TTF 字库，不能使用中文 TTF 字库！

将下载下来的 TTF 字库拷贝到 SD 卡中，在使用的时候需要将字库从 SD 卡读取到 RAM 中，SD 卡的保存路径为：**SYSTEM->EMWINFONT->TTF**，如图 23.3.4 所示。

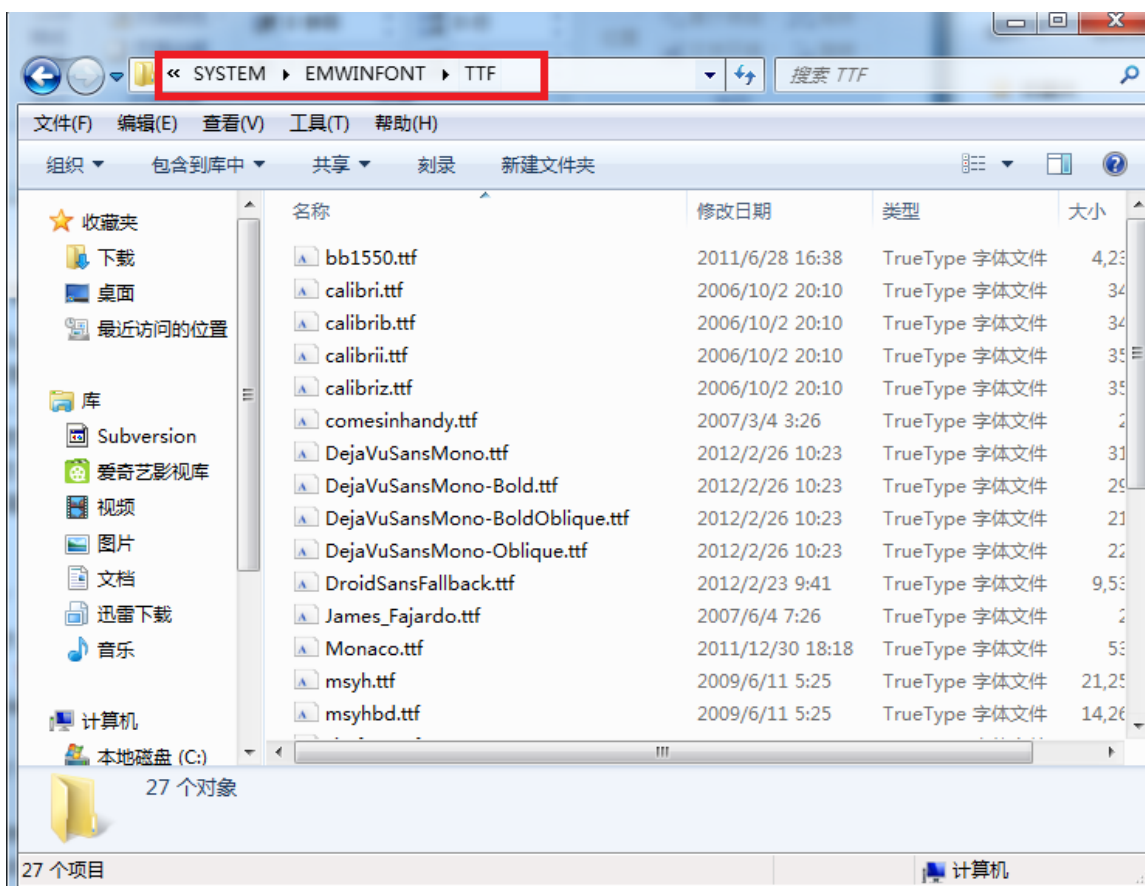


图 23.3.5 保存 TTF 字库到 SD 卡中

### 21.3.4 程序源码

经过上面 2 步，工程已经修改的差不多了，现在就开始编写代码了，本实验完整工程为：**EMWIN 实验 22 TTF 格式汉字显示**。打开文件 ttfcreate.c，程序如下：

```
#include "ttfcreate.h"
#include "ff.h"
#include "malloc.h"

#if SYSTEM_SUPPORT_UCOS
#include "includes.h"           //ucos 使用
#endif
```

```
static GUI_TTF_CS    Cs0,Cs1,Cs2,Cs3;

FIL TTFFontFile;

GUI_FONT TTF12_Font;
GUI_FONT TTF18_Font;
GUI_FONT TTF24_Font;
GUI_FONT TTF36_Font;

GUI_TTF_DATA TTFData;

//创建 TTF 字体，共 EMWIN 使用
//fxpath:存放 TTF 字库的路径
//返回值：0,成功；其他值，失败
int Create_TTFFont(u8 *fxpath)
{
    int result;
    u16 bread;
    char *TtfBuffer;

    #if SYSTEM_SUPPORT_UCOS
        OS_CPU_SR cpu_sr;
    #endif

    result = f_open(&TTFFontFile,(const TCHAR*)fxpath,FA_READ); //打开字库文件 (1)
    if(result != FR_OK) return 1;

    if(TTFFontFile.fsize>500*1024) return 2; //文件大于 500k，跳出! (2)
    else
    {
        TtfBuffer=mymalloc(SRAMEX,TTFFontFile.fsize);//申请内存 (3)
        if(TtfBuffer==NULL) return 3;//内存申请失败
    }

    //读取字体数据
    #if SYSTEM_SUPPORT_UCOS
        OS_ENTER_CRITICAL(); //临界区
    #endif

    result = f_read(&TTFFontFile,TtfBuffer,TTFFontFile.fsize,(UINT *)&bread); //读取数据(4)
    if(result != FR_OK) return 4; //文件打开失败，跳出
```



```

f_close(&TTFFontFile); //关闭 TTFFontFile 文件
#ifdef SYSTEM_SUPPORT_UCOS
    OS_EXIT_CRITICAL();    //退出临界区
#endif

TTFData.pData=TtfBuffer; //指向文件地址
TTFData.NumBytes=TTFFontFile.fsize; //文件大小

Cs0.pTTF      = &TTFData;
Cs0.PixelHeight = 12;
Cs0.FaceIndex= 0;

Cs1.pTTF      = &TTFData;
Cs1.PixelHeight = 18;
Cs1.FaceIndex= 0;

Cs2.pTTF      = &TTFData;
Cs2.PixelHeight = 24;
Cs2.FaceIndex= 0;

Cs3.pTTF      = &TTFData;
Cs3.PixelHeight = 36;
Cs3.FaceIndex= 0;

result = GUI_TTF_CreateFont(&TTF12_Font,&Cs0); //创建字体 (5)
if(result) return 5; //字体创建失败
result = GUI_TTF_CreateFont(&TTF18_Font,&Cs1); //创建字体
if(result) return 5; //字体创建失败
result = GUI_TTF_CreateFont(&TTF24_Font,&Cs2); //创建字体
if(result) return 5; //字体创建失败
result = GUI_TTF_CreateFont(&TTF36_Font,&Cs3); //创建字体
if(result) return 5; //字体创建失败
return 0;
}

```

(1)、从 SD 卡中打开 TTF 字库文件。

(2)、判断文件大小，最大可打开 500k 大小的 TTF 字库文件，然而普通的汉字 TTF 字库都要好几 M 的大小，所以只能打开一些英文字符和数字组成的 TTF 字库，不能显示汉字。

(3)、从外部 SRAM 中申请内存。

(4)、从 SD 卡中读取 TTF 字库

(5)、调用函数 GUI\_TTF\_CreateFont()创建 TTF 字体。

打开 main.c 文件，在函数 emwin\_maintask()中我们将调用函数 Create\_TTFFont()创建并使用 TTF 字库，emwin\_maintask()函数如下：

```
void emwin_maintask(void *pdata)
```

```
{  
    int result;  
    result=Create_TTFFont("0:/SYSTEM/EMWINFONT/TTF/calibrib.ttf");           (1)  
    if(result) printf("TTF 字体创建失败\r\n");  
  
    GUI_CURSOR_Show();  
    GUI_SetBkColor(GUI_CYAN);  
    GUI_SetColor(GUI_RED);  
    GUI_Clear();  
    GUI_SetFont(&TTF12_Font);                                               (2)  
    GUI_DispStringAt("ALIENTEK STemWin TTF Font Test 0123456789!",5,10);    (3)  
    GUI_SetFont(&TTF18_Font);  
    GUI_DispStringAt("ALIENTEK STemWin TTF Font Test 0123456789!",5,25);  
    GUI_SetFont(&TTF24_Font);  
    GUI_DispStringAt("ALIENTEK STemWin TTF Font Test 0123456789!",5,45);  
    GUI_SetFont(&TTF36_Font);  
    GUI_DispStringAt("ALIENTEK STemWin TTF Font Test 0123456789!",5,70);  
    while(1)  
    {  
        GUI_Delay(100);  
    }  
}
```

- (1)、调用函数 Create\_TTFFont()创建 TTF 字体。
- (2)、设置字体为创建的 TTF 字体。
- (3)、使用设置的 TTF 字体显示字符串。

## 21.4 下载验证

编译工程并下载到开发板中，注意 TTF 字库一定要放在 SD 卡中指定位置！下载完成以后 LCD 显示如图 21.4.1 所示，TTF 字体显示正常。

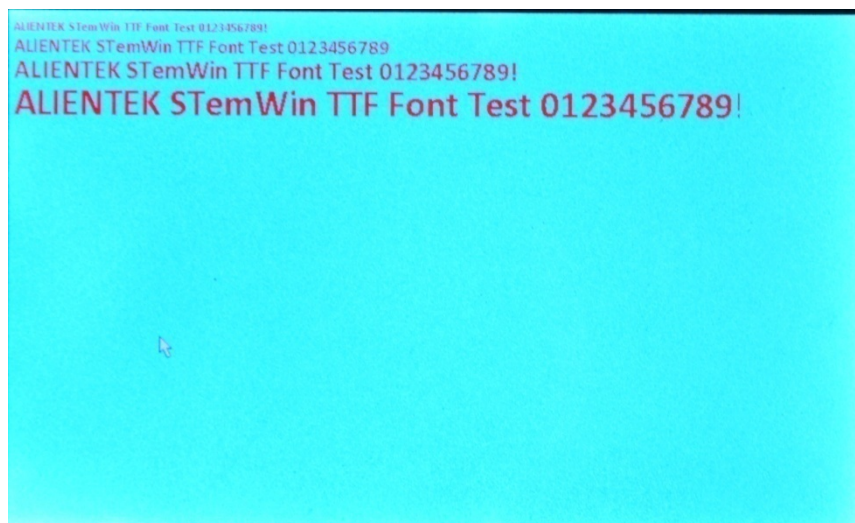


图 21.4.1 LCD 显示效果图

注意！只能加载小于 500k 的 TTF 字库，汉字 TTF 字库一般都比较大(大于 2M)，所以只能显示一些字母、数字和字符。

## 第二十二章 汉字显示(外部存储器)

前面讲了 XBF 字体和 TTF 字体的使用，XBF 在显示汉字的时候需要不断的从存储器中读取字库所以显示比较慢；TTF 受限于开发板 RAM 大小，不能加载汉字 TTF 字库，因此只能显示一些字母或者数字和字符，而 Segger 官方推荐的另外一种字体文件——SIF，更是因为生成的字库信息太大而不能使用。这里我们讲解一个显示速度快而且也不会占用大量 ram 的汉字显示方法，那就是将字库放在外部存储器中，比如外部 SPI Flash 等，这个也是我们常用的显示汉字的方法，本章分为如下几部分：

- 22.1 汉字显示简介
- 22.2 实验工程准备
- 22.3 字库制作
- 22.4 程序设计
- 22.5 下载验证

## 22.1 汉字显示简介

在 ALIENTEK 的《STM32F4 开发指南》中有讲解如何在外部 SPI FLASH 中制作汉字字库，并且在 LCD 上显示出来，本章我们就讲解一下如何在 emwin 中使用同样的方法显示汉字，至于这种方法的原理性知识本章就不讲解了，请参考 ALIENTEK 的《STM32F4 开发指南》中汉字显示实验章节，本章默认大家已经掌握了在不使用 emwin 时在 LCD 上显示汉字的方法。

## 22.2 实验工程准备

本实验要将基础实验中的汉字显示部分的文件和代码移植过来，这个很简单，就不详细讲解了，移植完成以后工程如图 22.2.1 所示。

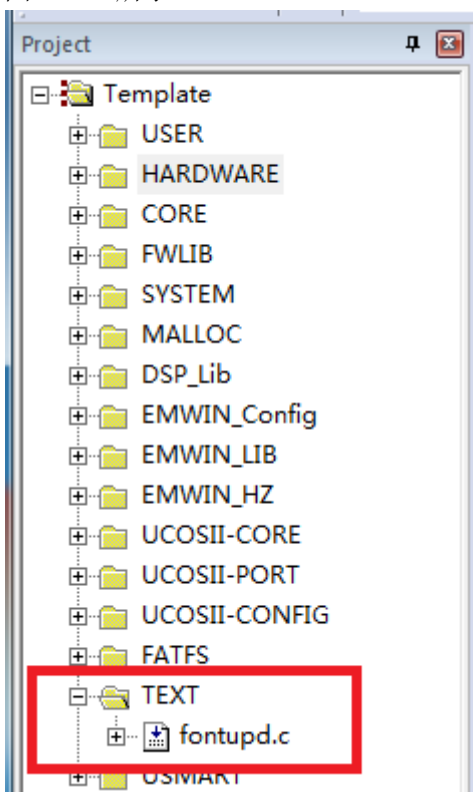


图 22.2.1 准备工程

## 22.3 字库制作

### 22.3.1 汉字字库制作

我们制作三个汉字字库，字体都为宋体，字库点阵大小为 12\*12、16\*16 和 24\*24，使用易木雨软件工作室设计的点阵字库生成器 V3.8 来制作字库，软件设置如图 22.3.1 所示，注意这里字库取模的方式选择的是“横向取模方式一”，在《STM32F4 开发指南》中是按照“纵向取模方式二”的，一定要注意这个差别，要不然会导致汉字显示出现异常！



图 22.3.1 汉字字库制作

这里字体大小和汉字点阵大小公式为：

$$fsize = dsize * 6 / 8$$

其中 fsize 是字体大小，dsize 是点阵字高和字宽(字高和字宽一样)，也就是字宽和字高。通过计算可以得到，12\*12 点阵对应着 9 号字体，16\*16 点阵对应着 12 号字体，24\*24 点阵对应 18 号字体，字库点阵大小和字体大小对应关系一定要设置好，否则制作出的字库将会显示不正常！字库制作完成以后放到 SD 卡中，路径为：SYSTEM->FONT，如图 22.3.2 所示。

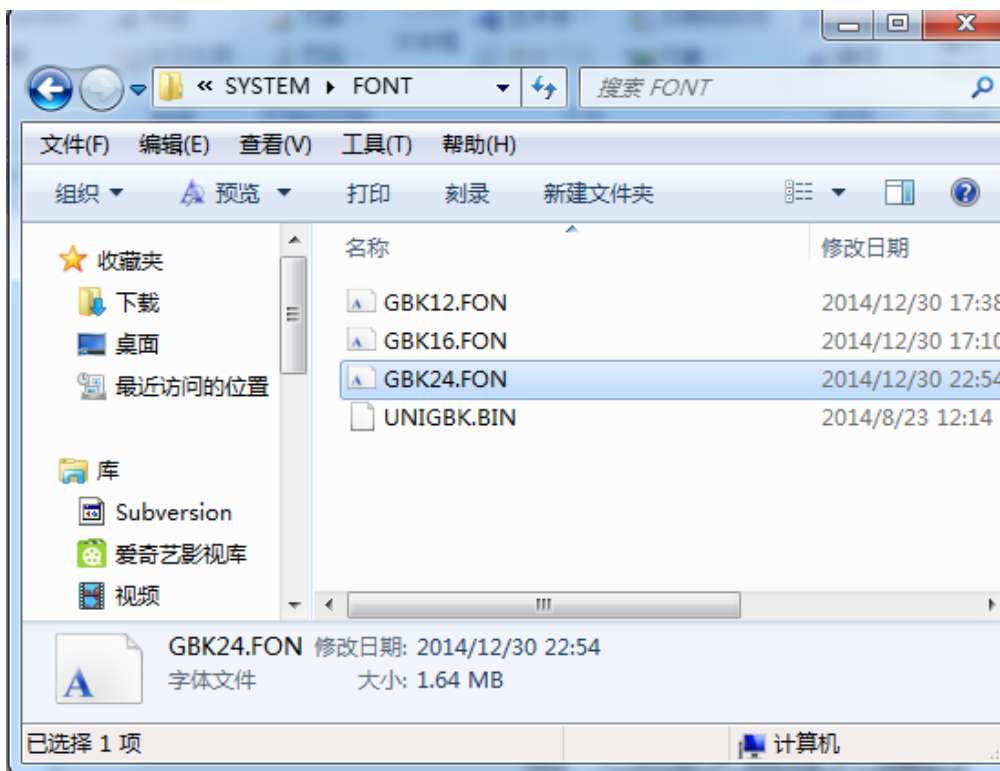


图 22.3.2 字库

由于基础例程中的汉字显示和 EMWIN 中汉字显示的字符取模方式不同，因此一旦在 SD 卡中放入为 EMWIN 制作的字库并且更新以后会导致基础例程和综合实验中的汉字显示出现问题，这个时候需要重新将字库更新为基础例程中字库。

### 22.3.2 英文字符字库制作

由于 EMWIN 的取模方式和《STM32F4 开发指南》中显示字符时的取模方式不同，所以英文、数字和符号的字模也需要重新制作一遍，使用 PCtoLCD2002 软件制作英文字模。对应的我们需要制作点阵大小为 12\*12、16\*16 和 24\*24，但是我们制作的是英文、数字和符号，实际上做出来的大小为 12\*6、16\*8 和 24\*12，取模方式设置如图 22.3.3 所示，取模方式选择为逐行式，格式为 C51。



图 22.3.3 PCtoLCD2002 取模方式设置

最终我们要定义 3 个数组来存放 12\*6、16\*8 和 24\*12 这 3 个不同点阵大小的字模，将制作好的字模数组放到 EmWin\_ASCII\_Font.h 头文件中，对应的数组分别为：emwin\_asc2\_1206、emwin\_asc2\_1608 和 emwin\_asc2\_2412。

## 22.4 程序设计

### 22.4.1 汉字字体制作

本实验完整工程为：**EMWIN 实验 23 汉字显示(FLASH 外置字库)**。fontupd.c 和 fontudp.h 文件是完成字库更新和初始化的，在《STM32F4 开发指南》里面有详细的讲解，这里就不做讲解了。打开实验工程的 EMWIN\_HZ 分组，如图 22.4.1 所示。

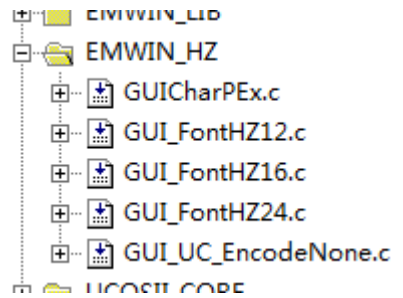


图 22.4.1 EMWIN\_HZ 分组

在 EMWIN\_HZ 分组中有 5 个.c 文件，分别为 GUICharPEX.c、GUI\_FontHZ12.c、GUI\_FontHZ16.c、GUI\_FontHZ24.c 和 GUI\_UC\_EncodeNone.c，打开 GUICharPEX.c 文件，代码如下：

```
#include <stddef.h>
#include "GUI_Private.h"
#include "ff.h"
#include "spi.h"
#include "fontupd.h"
#include "w25qxx.h"
#include "usart.h"
#include "sys.h"
#include "EmWin_ASCII_Font.h"
#include "ili93xx.h"
#include "mytext.h"
#include "string.h"
#include <stddef.h>
#include "EmWin_ASCII_Font.h"
#include "EmWinHZFont.h"

//字模数据暂存数组,单个字模的最大字节数
//单个字模最大为 1024
#define BYTES_PER_FONT 1024    //32*32=1024, 最大显示 32*32 点阵字库
U8 GUI_FontDataBuf[BYTES_PER_FONT];

//获取字模数据
static void GUI_GetDataFromMemory(const GUI_FONT_PROP GUI_UNI_PTR *pProp, U16P c)
{
    unsigned char qh,ql;
    unsigned char i;
    unsigned long foffset;
    unsigned char t;
    unsigned char *mat;
    u8 size,csize;//字体大小
    U16 BytesPerFont;
    GUI_FONT EMWINFONT;
```



```

EMWINFONT = *GUI_pContext->pAFont;
//每个字模的数据字节数
BytesPerFont = GUI_pContext->pAFont->YSize * pProp->paCharInfo->BytesPerLine;
if (BytesPerFont > BYTES_PER_FONT) BytesPerFont = BYTES_PER_FONT;

//判断字体的大小
//12 字体
if(memcmp(&EMWINFONT,&GUI_FontHZ12,sizeof(GUI_FONT)) == 0) size=12;      (1)
//16 字体
else if(memcmp(&EMWINFONT,&GUI_FontHZ16,sizeof(GUI_FONT)) == 0) size=16;
//24 字体
else if(memcmp(&EMWINFONT,&GUI_FontHZ24,sizeof(GUI_FONT)) == 0)size=24;

csize = (size/8+((size%8)?1:0))*(size); //得到字体一个字符对应点阵集所占的字节数 (2)
if (c < 0x80) //英文字符地址偏移算法 (3)
{
    switch(size)
    {
        case 12:
            for(t=0;t<12;t++) GUI_FontDataBuf[t]=emwin_asc2_1206[c-0x20][t];
            break;
        case 16:
            for(t=0;t<16;t++) GUI_FontDataBuf[t]=emwin_asc2_1608[c-0x20][t];
            break;
        case 24:
            for(t=0;t<48;t++) GUI_FontDataBuf[t]=emwin_asc2_2412[c-0x20][t];
            break;
    }
}
else
{
    ql=c/256;
    qh=c%256;
    if(qh<0x81||ql<0x40||ql==0xff||qh==0xff)//非常用汉字
    {
        for(i=0;i<(size*2);i++) *mat++=0x00;//填充满格
        return; //结束访问
    }
    if(ql<0x7f)ql-=0x40;
    else ql-=0x41;
    qh-=0x81;
    foffset=((unsigned long)190*qh+ql)*csize;//得到字库中的字节偏移量 (4)
    switch(size)

```

```

    {
        case 12:
            W25QXX_Read(GUI_FontDataBuf,foffset+ftinfo.f12addr,csiz);
            break;
        case 16:
            W25QXX_Read(GUI_FontDataBuf,foffset+ftinfo.f16addr,csiz);
            break;
        case 24:
            W25QXX_Read(GUI_FontDataBuf,foffset+ftinfo.f24addr,csiz);
            break;
    }
}

void GUIPROP_X_Dispatch(U16P c)
{
    int BytesPerLine;
    GUI_DRAWMODE DrawMode = GUI_pContext->TextMode;
    const GUI_FONT_PROP GUI_UNI_PTR *pProp = GUI_pContext->pAFont->pProp;
    //搜索定位字库数据信息
    for (; pProp; pProp = pProp->pNext)
    {
        if ((c >= pProp->First) && (c <= pProp->Last)) break;
    }
    if (pProp)
    {
        GUI_DRAWMODE OldDrawMode;
        const GUI_CHARINFO GUI_UNI_PTR * pCharInfo = pProp->pCharInfo;
        GUI_GetDataFromMemory(pProp, c); //取出字模数据
        BytesPerLine = pCharInfo->BytesPerLine;
        OldDrawMode = LCD_SetDrawMode(DrawMode);
        LCD_DrawBitmap(GUI_pContext->DispPosX, GUI_pContext->DispPosY,
            pCharInfo->XSize, GUI_pContext->pAFont->YSize,
            GUI_pContext->pAFont->XMag, GUI_pContext->pAFont->YMag,
            1, /* Bits per Pixel */
            BytesPerLine,
            &GUI_FontDataBuf[0],
            &LCD_BKCOLORINDEX
        );
        /* Fill empty pixel lines */
        if (GUI_pContext->pAFont->YDist > GUI_pContext->pAFont->YSize)
        {
            int YMag = GUI_pContext->pAFont->YMag;

```

```

    int YDist = GUI_pContext->pAFont->YDist * YMag;
    int YSize = GUI_pContext->pAFont->YSize * YMag;
    if (DrawMode != LCD_DRAWMODE_TRANS)
    {
        LCD_COLOR OldColor = GUI_GetColor();
        GUI_SetColor(GUI_GetBkColor());
        LCD_FillRect(GUI_pContext->DispPosX, GUI_pContext->DispPosY + YSize,
                    GUI_pContext->DispPosX + pCharInfo->XSize,
                    GUI_pContext->DispPosY + YDist);
        GUI_SetColor(OldColor);
    }
}

LCD_SetDrawMode(OldDrawMode); /* Restore draw mode */
GUI_pContext->DispPosX += pCharInfo->XDist * GUI_pContext->pAFont->XMag;
}
}

int GUIPROP_X_GetCharDistX(U16P c)
{
    const GUI_FONT_PROP GUI_UNI_PTR * pProp = GUI_pContext->pAFont->p.pProp;
    for (; pProp; pProp = pProp->pNext)
    {
        if ((c >= pProp->First) && (c <= pProp->Last))break;
    }
    return (pProp) ? (pProp->paCharInfo->XSize * GUI_pContext->pAFont->XMag : 0;
}

```

- (1)、得到字体大小，用来计算字符对应点阵所占的字节数
- (2)、利用 size 来计算出要显示的字符对应点阵集所占用的字节数
- (3)、英文字符地址偏移算法。
- (4)、得到在外部 SPI FLASH 字库中的字节偏移量。

上面计算汉字字符在 SPI FLASH 中的偏移量的公式和方法在《STM32F4 开发指南》中有详细的讲解，不懂的请查阅《STM32F4 开发指南》中汉字显示实验章节。

打开文件 GUI\_UC\_EncodeNone.c，代码如下：

```

#include "GUI_Private.h"
//当前字符的 UNICODE 码
static U16 _GetCharCode(const char GUI_UNI_PTR * s)
{
    if((*s) > 0XA0)
    {
        return *(const U16 GUI_UNI_PTR *)s;
    }
    return *(const U8 GUI_UNI_PTR *)s;
}

```

//返回当前字符字节数(也就是用一个字节表示还是两个，一般汉字用 2 个字节)

```
static int _GetCharSize(const char GUI_UNI_PTR * s)
```

```
{
    GUI_USE_PARA(s);
    if((*s) > 0XA0)
    {
        return 2;
    }
    return 1;
}
```

```
static int _CalcSizeOfChar(U16 Char)
```

```
{
    GUI_USE_PARA(Char);
    if(Char > 0XA0A0)
    {
        return 2;
    }
    return 1;
}
```

```
static int _Encode(char *s, U16 Char)
```

```
{
    if(Char > 0XA0A0)
    {
        *((U16*)s) = (U16)(Char);
        return 2;
    }
    *s = (U8)(Char);
    return 1;
}
```

```
const GUI_UC_ENC_APILIST GUI_UC_None = {
```

```
    _GetCharCode,
    _GetCharSize,
    _CalcSizeOfChar,
    _Encode
};
```

```
const GUI_UC_ENC_APILIST GUI__API_TableNone = {
```

```
    _GetCharCode,
    _GetCharSize,
    _CalcSizeOfChar,
```

```
_Encode
```

```
};
```

打开 GUI\_FontHZ12.c 文件，里面的代码如下：

```
#include "GUI.h"
```

```
extern void GUIPROP_X_DispChar(U16P c);
```

```
extern int GUIPROP_X_GetCharDistX(U16P c);
```

```
GUI_CONST_STORAGE GUI_CHARINFO GUI_FontHZ12_CharInfo[2] =
```

```
{
```

```
    { 6, 6, 1, (void*)"0"},
```

```
    { 12, 12, 2, (void*)"0"},
```

```
};
```

```
GUI_CONST_STORAGE GUI_FONT_PROP GUI_FontHZ12_PropHZ =
```

```
{
```

```
    0xA1A1,
```

```
    0xFFFF,
```

```
    &GUI_FontHZ12_CharInfo[1],
```

```
    (void *)0,
```

```
};
```

```
GUI_CONST_STORAGE GUI_FONT_PROP GUI_FontHZ12_PropASC =
```

```
{
```

```
    0x0000,
```

```
    0x007F,
```

```
    &GUI_FontHZ12_CharInfo[0],
```

```
    (void GUI_CONST_STORAGE *)&GUI_FontHZ12_PropHZ,
```

```
};
```

```
GUI_CONST_STORAGE GUI_FONT GUI_FontHZ12 =
```

```
{
```

```
    GUI_FONTTYPE_PROP_USER,
```

```
    12,
```

```
    12,
```

```
    1,
```

```
    1,
```

```
    (void GUI_CONST_STORAGE *)&GUI_FontHZ12_PropASC
```

```
};
```

```
GUI_CONST_STORAGE GUI_FONT GUI_FontHZ12x2 =
```

```
{
```

```
    GUI_FONTTYPE_PROP_USER,
```

```
    12,
```

(1)

```

    12,
    2,
    2,
    (void GUI_CONST_STORAGE *)&GUI_FontHZ12_PropASC
};

```

(1)、定义新的字体：GUI\_FontHZ12，GUI\_FontHZ16.c 和 GUI\_FontHZ24.c 这两个文件基本和 GUI\_FontHZ12.c 相同。

最后我们还需要在申明一下字体结构，在 GUI\_Type.h 中加入如下代码：

```

//声明一种新的字体结构
#define GUI_FONTTYPE_PROP_USER      \
    GUIPROP_X_Dispatch,              \
    GUIPROP_X_GetCharDistX,         \
    GUIMONO_GetFontInfo,             \
    GUIMONO_IsInFont,               \
    (GUI_GETCHARINFO *)0,            \
    (tGUI_ENC_APIList*)0

```

#### 22.4.2 字库更新及测试

打开 main.c 文件中的 main()函数，代码如下：

```

int main(void)
{
    delay_init(168);          //延时初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);    //中断分组配置
    uart_init(115200);        //串口波特率设置
    TFTLCD_Init();            //初始化 LCD
    KEY_Init();               //初始化按键
    LED_Init();               //LED 初始化
    FSMC_SRAM_Init();         //SRAM 初始化

    mem_init(SRAMIN);         //内部 RAM 初始化
    mem_init(SRAMEX);         //外部 RAM 初始化
    mem_init(SRAMCCM);        //CCM 初始化

    exfuns_init();            //为 fatfs 文件系统分配内存
    f_mount(fs[0], "0:", 1);   //挂载 SD 卡
    f_mount(fs[1], "1:", 1);   //挂载 FLASH
    TP_Init();                //初始化触摸屏

    while(font_init())        //初始化字库
    {
        LCD_ShowString(30, 70, 200, 16, 16, "Font Error!");
        while(SD_Init())      //检测 SD 卡
        {

```

```

        LCD_ShowString(30,90,200,16,16,"SD Card Failed!");
        delay_ms(200);
        LCD_Fill(30,90,200+30,70+16,WHITE);
        delay_ms(200);
    }
    update_font(30,90,16,"0:"); //如果字库不存在就更新字库
    delay_ms(2000);
    LCD_Clear(WHITE); //清屏
    break;
}

OSInit(); //初始化 UCOS
OSTaskCreate(start_task,                                //start_task 任务
              (void*)0,                                  //参数
              (OS_STK*)&START_TASK_STK[START_STK_SIZE-1], //任务堆栈栈顶
              START_TASK_PRIO);                          //任务优先级
OSSStart(); //开启 UCOS
}

```

上面 main()函数中的红色代码为初始化 SD 卡和字库，做过 ALIENTEK 的 STM32 开发板汉字显示实验的同学对这段代码应该会很熟悉的，在《STM32F4 开发指南》的汉字显示实验章节有非常详细的讲解。

我们可能会根据自己的需求制作其他字体的汉字字库，一旦制作完成就需要更新一下外部 SPI FLASH 字库，所以需要有一个字库更新任务。在 UCOS 系统的支持下，这个问题就很好解决，我们只需要新建一个任务来完成字库更新，在 main.c 中的任务函数 fontupdata\_task()就是来完成字库更新的，函数代码如下：

```

//字库更新任务
//KEY_UP 键长按 2s 更新字库
void fontupdata_task(void *pdata)
{
    while(1)
    {
        if(WK_UP == 1)                //KEY_UP 键按下
        {
            OSTimeDlyHMSM(0,0,2,0); //延时 2s
            if(WK_UP == 1)            //还是 KEY_UP 键
            {
                LCD_Clear(WHITE);
                OSSchedLock();        //调度器上锁
                LCD_ShowString(10,50,250,30,16,"Font Updataing,Please Wait...");
                update_font(10,70,16,"0:");//更新字库
                LCD_Clear(WHITE);
                POINT_COLOR = RED;
                LCD_ShowString(10,50,280,30,16,"Font Updata finshed,Please Restart!");
            }
        }
    }
}

```

```

        OSSchedUnlock(); //调度器解锁
    }
}
OSTimeDlyHMSM(0,0,0,10);    //延时 10ms
}
}

```

fontupdate\_task()任务函数很简单，通过判断 KEY\_UP 键的按下时间来判断是否更新字库，长按 KEY\_UP 键超过 2s 就调用函数 update\_font()更新字库。

emwin\_maintask()任务函数代码如下：

```

//EMWINDEMO 任务
void emwin_maintask(void *pdata)
{
    GUI_CURSOR_Show();
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_FontHZ12);
    GUI_DispStringAt("12 号字体：广州星翼电子 ALIENTEK!",10,10);
    GUI_SetFont(&GUI_FontHZ16);
    GUI_DispStringAt("16 号字体：广州星翼电子 ALIENTEK!",10,30);
    GUI_SetFont(&GUI_FontHZ24);
    GUI_DispStringAt("24 号字体：广州星翼电子 ALIENTEK!",10,50);

    while(1)
    {
        GUI_Delay(100);
    }
}

```

在 emwin\_maintask()中通过调用前面制作好的 3 种汉字字体在 LCD 上显示指定的字符串，这个很简单，就不多讲了。

## 22.5 下载验证

编译并下载到开发板中，下载完成以后首先按 KEY\_UP 键更新字库，字库更新完成以后 LCD 上会显示提示信息“**Font Updata finshed,Please Restart!**”。字库更新完成以后根据提示复位一次开发板，复位完成后 LCD 就可以显示出汉字了，如图 22.5.1 所示。





图 22.5.1 汉字显示

## 第二十三章 BMP 图片显示

emWin 支持 BMP 图片解码,可以直接从外部存储器(如 SD 卡)中读取 BMP 图片文件在 LCD 上显示,本章就讲解如何使用 emWin 的 BMP 解码函数在 LCD 上显示 BMP 图片,本章分为如下几部分:

23.1 支持格式

23.2 BMP 文件 API 函数

23.3 程序设计

23.4 下载验证

## 23.1 支持格式

BMP 文件格式由 Microsoft 定义，表 23.1.1 显示了许多不同的格式：

| 每像素位数 | 索引 | 压缩 | 支持 |
|-------|----|----|----|
| 1     | 是  | 否  | 是  |
| 4     | 是  | 否  | 是  |
| 4     | 是  | 是  | 是  |
| 8     | 是  | 否  | 是  |
| 8     | 是  | 是  | 是  |
| 16    | 否  | 否  | 是  |
| 24    | 否  | 否  | 是  |
| 32    | 否  | 否  | 是  |

表 23.1.1 BMP 格式

## 23.2 BMP 文件 API 函数

表 23.2.1 列出了关于 BMP 图片显示的 API 函数。

| 函数                     | 描述                        |
|------------------------|---------------------------|
| GUI_BMP_Draw()         | 绘制已加载到存储器的 BMP 文件。        |
| GUI_BMP_DrawEx()       | 绘制无需加载到存储器的 BMP 文件。       |
| GUI_BMP_DrawScaled()   | 绘制已加载到存储器的带比例的 BMP 文件。    |
| GUI_BMP_DrawScaledEx() | 绘制无需加载到存储器的带比例的 BMP 文件。   |
| GUI_BMP_GetXSize()     | 返回加载到存储器的 BMP 文件的 X 大小。   |
| GUI_BMP_GetXSizeEx()   | 返回无需加载到存储器的 BMP 文件的 X 大小。 |
| GUI_BMP_GetYSize()     | 返回加载到存储器的位图的 Y 大小。        |
| GUI_BMP_GetYSizeEx()   | 返回无需加载到存储器的 Y 大小。         |
| GUI_BMP_Serialize()    | 创建 BMP 文件。                |
| GUI_BMP_SerializeEx()  | 基于给定的矩形创建 BMP 文件。         |

表 23.2.1 BMP 文件 API 函数

## 23.3 程序设计

本实验完整工程为：**EMWIN 实验 24 BMP 图片显示**。打开文件 bmpdisplay.c，程序源码如下：

```
#include "bmpdisplay.h"
#include "EmWinHZFont.h"
#include "GUI.h"
#include "malloc.h"
#include "ff.h"
#include "ili93xx.h"

#if SYSTEM_SUPPORT_UCOS
#include "includes.h"           //ucos 使用
#endif
```

```

static FIL BMPFile;
static char bmpBuffer[BMPPERLINESIZE];
static int BmpGetData(void * p, const U8 ** ppData, unsigned NumBytesReq, U32 Off)
{
    static int readaddress=0;
    FIL * phFile;
    UINT NumBytesRead;
    #if SYSTEM_SUPPORT_UCOS
        OS_CPU_SR cpu_sr;
    #endif

    phFile = (FIL *)p;

    if (NumBytesReq > sizeof(bmpBuffer))
    {
        NumBytesReq = sizeof(bmpBuffer);
    }

    //移动指针到应该读取的位置
    if(Off == 1) readaddress = 0;
    else readaddress=Off;
    #if SYSTEM_SUPPORT_UCOS
        OS_ENTER_CRITICAL(); //临界区
    #endif
    f_lseek(phFile,readaddress);

    //读取数据到缓冲区中
    f_read(phFile,bmpBuffer,NumBytesReq,&NumBytesRead);
    #if SYSTEM_SUPPORT_UCOS
        OS_EXIT_CRITICAL(); //退出临界区
    #endif
    *ppData = (U8 *)bmpBuffer;
    return NumBytesRead;//返回读取到的字节数
}

//在指定位置显示加载到 RAM 中的 BMP 图片
//BMPFileName:图片在 SD 卡或者其他存储设备中的路径(需文件系统支持! )
//mode:显示模式
//      0 在指定位置显示, 有参数 x,y 确定显示位置
//      1 在 LCD 中间显示图片, 当选择此模式的时候参数 x,y 无效。
//x:图片左上角在 LCD 中的 x 轴位置(当参数 mode 为 1 时, 此参数无效)
//y:图片左上角在 LCD 中的 y 轴位置(当参数 mode 为 1 时, 此参数无效)

```

```
//member: 缩放比例的分子项
//denom:缩放比例的分母项
//返回值:0 显示正常,其他 失败
int dispbmp(u8 *BMPFileName,u8 mode,u32 x,u32 y,int member,int denom)
{
    u16 bread;
    char *bmpbuffer;
    char result;
    int XSize,YSize;
    float Xflag,Yflag;
    GUI_COLOR forecolor,bkcolor;

    #if SYSTEM_SUPPORT_UCOS
        OS_CPU_SR cpu_sr;
    #endif

    forecolor=GUI_GetColor();           //获取当前前景色
    bkcolor=GUI_GetBkColor();           //获取当前背景色
    result = f_open(&BMPFile,(const TCHAR*)BMPFileName,FA_READ); //打开文件
    //文件打开错误或者文件大于 BMPMEMORYSIZE
    if((result != FR_OK) || (BMPFile.fsize>BMPMEMORYSIZE)) return 1;

    bmpbuffer = mymalloc(SRAMEX,BMPFile.fsize);//申请内存
    if(bmpbuffer == NULL) return 2;

    #if SYSTEM_SUPPORT_UCOS
        OS_ENTER_CRITICAL(); //临界区
    #endif

    result = f_read(&BMPFile,bmpbuffer,BMPFile.fsize,(UINT *)&bread); //读取数据
    if(result != FR_OK) return 3;

    #if SYSTEM_SUPPORT_UCOS
        OS_EXIT_CRITICAL(); //退出临界区
    #endif

    switch(mode)
    {
        case 0: //在指定位置显示图片
            if((member == 1) && (denom == 1)) //无需缩放, 直接绘制
            {
                GUI_BMP_Draw(bmpbuffer,x,y); //在指定位置显示 BMP 图片
            }else //否则图片需要缩放
```

```

        {
            GUI_BMP_DrawScaled(bmpbuffer,x,y,member,denom);
        }
        break;
    case 1: //在 LCD 中间显示图片
        XSize = GUI_BMP_GetXSize(bmpbuffer); //获取图片的 X 轴大小
        YSize = GUI_BMP_GetYSize(bmpbuffer); //获取图片的 Y 轴大小
        if((member == 1) && (denom == 1)) //无需缩放, 直接绘制
        {
            //在 LCD 中间显示图片

            GUI_BMP_Draw(bmpbuffer,(lcddev.width-XSize)/2-1,(lcddev.height-YSize)/2-1);
        }else //否则图片需要缩放
        {
            Xflag = (float)XSize*((float)member/(float)denom);
            Yflag = (float)YSize*((float)member/(float)denom);
            XSize = (lcddev.width-(int)Xflag)/2-1;
            YSize = (lcddev.height-(int)Yflag)/2-1;
            GUI_BMP_DrawScaled(bmpbuffer,XSize,YSize,member,denom);
        }
        break;
    }
    f_close(&BMPFile); //关闭 BMPFile 文件
    myfree(SRAMEX,bmpbuffer); //释放内存
    GUI_SetColor(forecolor); //恢复前景色
    GUI_SetBkColor(bkcolor); //恢复背景色
    return 0;
}

//在指定位置显示无需加载到 RAM 中的 BMP 图片(需文件系统支持! 对于小 RAM, 推荐使用此方法!)
//BMPFileName:图片在 SD 卡或者其他存储设备中的路径
//mode:显示模式
//      0 在指定位置显示, 有参数 x,y 确定显示位置
//      1 在 LCD 中间显示图片, 当选择此模式的时候参数 x,y 无效。
//x:图片左上角在 LCD 中的 x 轴位置(当参数 mode 为 1 时, 此参数无效)
//y:图片左上角在 LCD 中的 y 轴位置(当参数 mode 为 1 时, 此参数无效)
//member: 缩放比例的分子项
//denom:缩放比例的分母项
//返回值:0 显示正常,其他 失败
int dispbmpex(u8 *BMPFileName,u8 mode,u32 x,u32 y,int member,int denom)
{
    char result;
    int XSize,YSize;

```

```

float Xflag,Yflag;
GUI_COLOR forecolor,bkcolor;
forecolor=GUI_GetColor();           //获取当前前景色
bkcolor=GUI_GetBkColor();           //获取当前背景色

result = f_open(&BMPFile,(const TCHAR*)BMPFileName,FA_READ); //打开文件
//文件打开错误
if(result != FR_OK)    return 1;

switch(mode)
{
    case 0: //在指定位置显示图片
        if((member == 1) && (denom == 1)) //无需缩放，直接绘制
        {
            GUI_BMP_DrawEx(BmpGetData,&BMPFile,x,y); //在指定位置显示 BMP 图片
        } else //否则图片需要缩放
        {
            GUI_BMP_DrawScaledEx(BmpGetData,&BMPFile,x,y,member,denom);
        }
        break;
    case 1: //在 LCD 中间显示图片
        XSize = GUI_BMP_GetXSizeEx(BmpGetData,&BMPFile); //获取图片的 X 轴大小
        YSize = GUI_BMP_GetYSizeEx(BmpGetData,&BMPFile); //获取图片的 Y 轴大小

        if((member == 1) && (denom == 1)) //无需缩放，直接绘制
        {
            //在 LCD 中间显示图片
            GUI_BMP_DrawEx(BmpGetData,&BMPFile,(lcddev.width-XSize)/2-1,\
                           (lcddev.height-YSize)/2-1);
        } else //否则图片需要缩放
        {
            Xflag = (float)XSize*((float)member/(float)denom);
            Yflag = (float)YSize*((float)member/(float)denom);
            XSize = (lcddev.width-(int)Xflag)/2-1;
            YSize = (lcddev.height-(int)Yflag)/2-1;
            GUI_BMP_DrawScaledEx(BmpGetData,&BMPFile,XSize,\
                                YSize,member,denom);
        }
        break;
}
f_close(&BMPFile); //关闭 BMPFile 文件
GUI_SetColor(forecolor); //恢复前景色

```

```
GUI_SetBkColor(bkcolor);    //恢复背景色
return 0;
}

//此函数被 GUI_BMP_Serialize()调用，用来向文件写入字节
static void _WriteByte2File(U8 Data, void * p)
{
    UINT nWritten;
    f_write((FIL*)p,&Data,1,&nWritten);
}

//生成一个 BMP 图片，保存到 SD 卡中,实现屏幕截图功能！
//*filepath:文件路径
//x0:创建 BMP 文件的 X 起始位置
//y0:创建 BMP 文件的 Y 起始位置
//Xsize:X 的大小
//Ysize:Y 的大小
void create_bmppicture(u8 *filename,int x0,int y0,int Xsize,int Ysize)
{
    static FIL hFile;
    GUI_COLOR forecolor,bkcolor;
    forecolor=GUI_GetColor();    //获取当前前景色
    bkcolor=GUI_GetBkColor();    //获取当前背景色
    //创建一个文件，路径为 filename,如果文件原本就存在的话会被新建的文件覆盖掉！
    f_open(&hFile,(const TCHAR*)filename,FA_READ|FA_WRITE|FA_CREATE_ALWAYS);
    GUI_BMP_SerializeEx(_WriteByte2File,x0,y0,Xsize,Ysize,&hFile);
    f_close(&hFile);    //关闭文件
    GUI_SetColor(forecolor);    //恢复前景色
    GUI_SetBkColor(bkcolor);    //恢复背景色
}

void bmpdisplay_demo(void)
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_FontHZ16);
    GUI_Clear();

    while(1)
    {
        GUI_DispStringHCenterAt("在指定位置显示一张加载到 RAM 中的 BMP 图片",400,5);
        dispbmp("0:/PICTURE/BMP/ALIENTEKLOGO.bmp",0,10, 78, 1,1);
        GUI_Delay(1000);
    }
}
```



```

GUI_Clear();

GUI_DispStringHCenterAt("在 LCD 中间显示一张缩小 1/2 的加载到\
                        RAM 中的 BMP 图片",400,5);
dispbmp("0:/PICTURE/BMP/ALIENTEKLOGO.bmp",1,0,0, 1,2);
GUI_Delay(1000);
GUI_Clear();

GUI_DispStringHCenterAt("在 LCD 中间显示一张扩大 4/3 倍的加载到\
                        RAM 中的 BMP 图片",400,5);
dispbmp("0:/PICTURE/BMP/ALIENTEKLOGO.bmp",1,0,0, 4,3);
GUI_Delay(1000);
GUI_Clear();

GUI_DispStringHCenterAt("在指定位置显示一张无需加载的 BMP 图片",400,5);
dispbmpex("0:/PICTURE/BMP/ALIENTEKLOGO.bmp",0,10, 78, 1,1);
GUI_Delay(1000);
GUI_Clear();

GUI_DispStringHCenterAt("在 LCD 中间显示一张缩小 1/2 的无需加载的\
                        BMP 图片",400,5);
dispbmpex("0:/PICTURE/BMP/ALIENTEKLOGO.bmp",1,0, 0, 1,2);
GUI_Delay(1000);
GUI_Clear();

GUI_DispStringHCenterAt("在 LCD 中间显示一张扩大 4/3 倍的无需加载的\
                        BMP 图片",400,5);
dispbmpex("0:/PICTURE/BMP/ALIENTEKLOGO.bmp",1,0, 0, 4,3);
GUI_Delay(1000);
GUI_Clear();
}
}

```

在 bmpdisplay.c 中有 6 个函数，如表 23.3.1 所示：

| 函数                  | 描述                                              |
|---------------------|-------------------------------------------------|
| BmpGetData()        | 从外部存储器获取 BMP 文件数据                               |
| dispbmp()           | 将 BMP 图片加载到 RAM 中，并显示到 LCD 上，显示速度快，但是需要大量的 RAM。 |
| dispbmpex()         | 不需要将 BMP 加载到 RAM 中，显示速度慢，但是不需要大量的 RAM。          |
| _WriteByte2File()   | 由函数 GUI_BMP_Serialize()调用，用于向文件写入数据。            |
| create_bmppicture() | 屏幕截图函数。                                         |
| bmpdisplay_demo()   | 本章实验的演示函数，在 LCD 上显示图片。                          |

表 23.3.1 bmpdisplay.c 文件函数

函数 BmpGetData()用于从外部存储器中获取 BMP 图片数据，在函数 dispbmpex ()显示 BMP

图片的时候做为参数传递给 GUI\_BMP\_DrawEx ()。

函数 dispbmp()用于显示 BMP 图片，该函数首先将 BMP 图片的所有数据加载到 RAM 中，然后调用 EMWIN 中的相应函数在 LCD 上显示加载到 RAM 中的 BMP 图片，由于需要将整副 BMP 图片加载到 RAM 中，所以需要大量的 RAM，但是显示速度快！此函数可以设置要显示的 BMP 图片的位置和放大倍数。

函数 dispbmpex()也是显示 BMP 图片的，与函数 dispbmp()不同的是，dispbmpex()不需要先将 BMP 图片加载到 RAM 中，可以直接从外部存储器中读取 BMP 图片并显示，这种方法需要的 RAM 小，但是速度相比 dispbmp()要慢，此函数也可以设置要显示的 BMP 图片的位置和放大倍数。

\_WriteByte2File()函数用于向外部存储器写入数据，在调用函数 create\_bmppicture()进行屏幕截图的时候会用到，

create\_bmppicture()是屏幕截图函数，截图为 BMP 格式，并保存到外部存储器中。

函数 bmpdisplay\_demo()是本章实验的演示 demo 函数，当开始截图的时候 LED1 点亮，截图完毕以后 LED1 熄灭！可以通过观察 LED1 的亮灭来观察截图是否完成。

在 main.c 文件中我们新建了一个任务 key\_task()，任务函数如下：

```
//按键处理任务
void key_task(void *pdata)
{
    u8 key;
    while(1)
    {
        key = KEY_Scan(0);
        switch(key)
        {
            case KEY0_PRES:
                OSSchedLock();           //调度器上锁
                LED1 = 0;
                create_bmppicture("0:/SCREENSHORT/屏幕截图 0.bmp",0,0,800,480);
                LED1 = 1;
                OSSchedUnlock(); //调度器解锁
                break;
        }
        OSTimeDlyHMSM(0,0,0,10);    //延时 10ms
    }
}
```

任务 key\_task()用来检测按键，如果检测到 KEY0 键被按下就进行屏幕截图。

## 23.4 下载验证

编译并下载到开发板中，LCD 轮流着按照指定的显示方法显示图片，如图 23.4.1 所示。

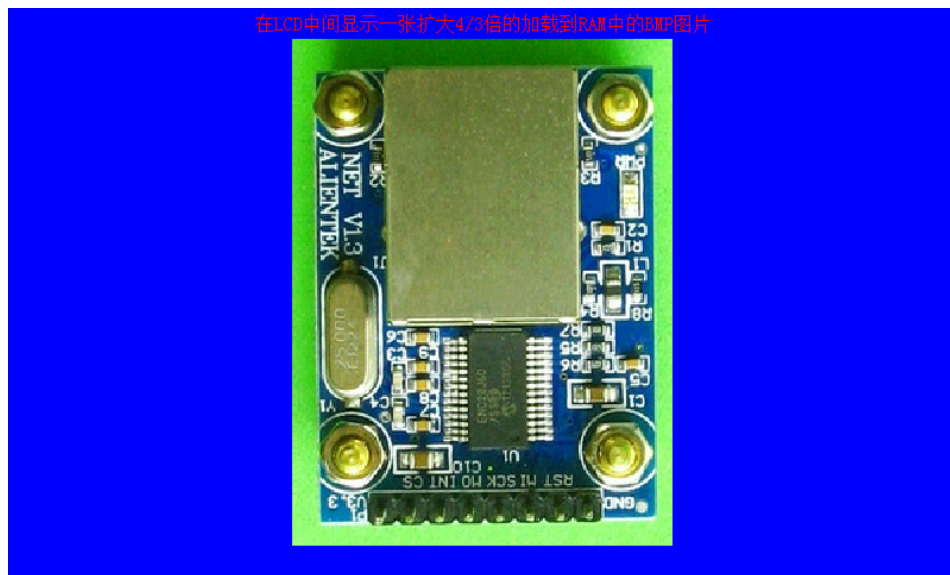


图 23.4.1 显示 BMP 图片

图 23.4.1 是在屏幕中间显示一张加载到 RAM 中并且扩大了 4/3 倍的 BMP 图片, 按下 KEY0 键可以进行屏幕截图, 图 23.4.2 显示的就是屏幕截图。

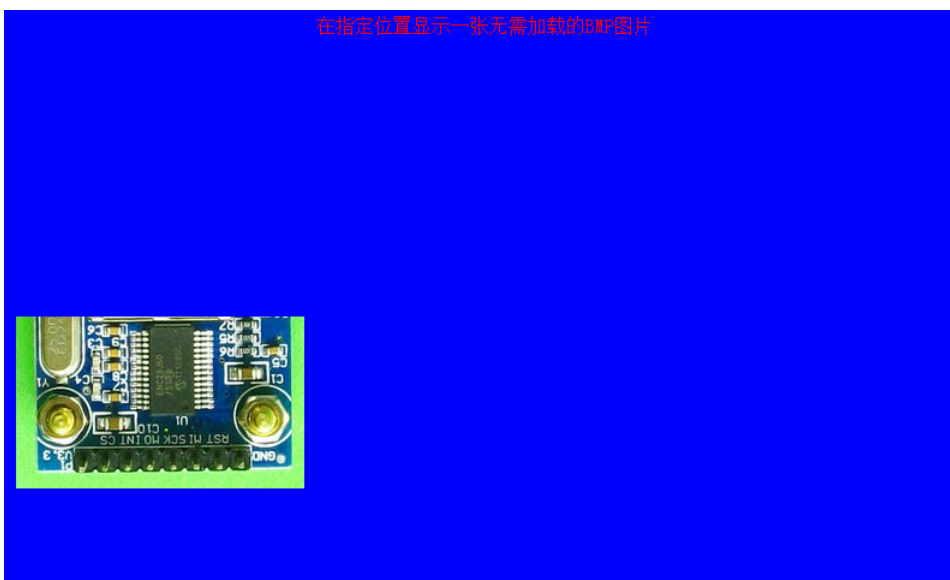


图 23.4.2 屏幕截图

图 23.4.2 是正在显示 的时候进行了屏幕截图, 可以看出此时图片显示并未完成就进行了屏幕截图。

注意! 进行屏幕截图的时候通过观察 LED1 来判断截图是否完成, LED1 点亮表示开始截图, LED1 熄灭表明截图完成。

## 第二十四章 JPEG 图片显示

JPEG 是全彩和灰度图像的标准压缩方法，JPEG 用于压缩“真实世界”的景象、线条画、卡通，其他非现实图像并不是其强项。JPEG 会有损耗，意指输出图像与输入图像并不完全相同，因此，如果需要通过完全相同的输出位，则不能使用 JPEG。不过，对于常见的照片图像，可以得到非常好的压缩级别，看不出变化。emWin 支持 JPEG 图片显示，可以直接从外部存储器(如 SD 卡)中读取 JPEG 图片文件在 LCD 上显示，本章就讲解如何使用 emWin 的 JPEG 显示函数在 LCD 上显示 JPEG 图片，本章分为如下几部分：

24.1 支持的 JPEG 压缩方法

24.2 支持的 JPEG 压缩方法

24.3 JPEG 文件 API 函数

24.4 程序设计

24.5 下载验证

## 24.1 支持的 JPEG 压缩方法

此软件执行 JPEG 基准、可扩展顺序以及渐进式的压缩流程。尽管尚未实施某些不常见的参数设置，但它支持这些流程的所有变体。出于法律原因，不得分发 JPEG 算术编码变体的代码。JPEG 规范的算术编码选项似乎属于归 IBM、AT&T 和 Mitsubishi 所有的专利。因此，从法律上讲，如未获得一个或多个许可，则不能使用算术编码。JPEG 文件支持不包含提供标准中定义的层次式或无损流程。

## 24.2 RAM 使用要求

JPEG 解压缩大约需要 33 Kb RAM 用于与图像大小无关的解压缩和依赖大小的字节量。RAM 要求可按以下方式计算：

大约 RAM 要求 = 图像的 X 大小 \* 80 字节 + 33 千字节

公示中 X 大小的取决于 JPEG 文件的压缩类型，表 24.2.1 列出了部分类型：

| 压缩   | 图像大小(像素) | RAM 使用<br>[千字节] | RAM 使用, 大小依赖<br>量[千字节] |
|------|----------|-----------------|------------------------|
| H1V1 | 160x120  | 45              | 12                     |
| H2V2 | 160x120  | 46              | 13                     |
| GRAY | 160x120  | 38              | 4                      |

表 24.2.1 RAM 使用量

JPEG 图片显示时解压缩所需的 RAM 由 emWin 存储器管理系统自动分配, 绘制 JPEG 图片后就会释放掉 RAM。

## 24.3 JPEG 文件 API 函数

表 24.3.1 列出了关于 BMP 图片显示的 API 函数。

| 函数                      | 描述                                      |
|-------------------------|-----------------------------------------|
| GUI_JPEG_Draw()         | 绘制已加载到存储器的 JPEG 文件。                     |
| GUI_JPEG_DrawEx()       | 绘制无需加载到存储器的 JPEG 文件。                    |
| GUI_JPEG_DrawScaled()   | 绘制已加载到存储器的带比例的 JPEG 文件。                 |
| GUI_JPEG_DrawScaledEx() | 绘制无需加载到存储器的带比例的 JPEG 文件。                |
| GUI_JPEG_GetInfo()      | 基于已加载到存储器的 JPEG 文件填充 GUI_JPEG_INFO 结构。  |
| GUI_JPEG_GetInfoEx()    | 基于无需加载到存储器的 JPEG 文件填充 GUI_JPEG_INFO 结构。 |

表 24.3.1 JPEG 文件 API 函数

## 24.4 程序设计

本实验完整工程为：EMWIN 实验 25 JPEG 图片显示，打开文件 jpegdisplay.c，程序源码如下：

```
#include "jpegdisplay.h"
#include "EmWinHZFont.h"
#include "GUI.h"
#include "malloc.h"
#include "ff.h"
#include "ili93xx.h"
```

```
#if SYSTEM_SUPPORT_UCOS
#include "includes.h"           //ucos 使用
#endif

static FIL JPEGFile;
static char jpegBuffer[JPEGPERSIZE];
static int JpegGetData(void * p, const U8 ** ppData, unsigned NumBytesReq, U32 Off)
{
    static int readaddress=0;
    FIL * phFile;
    UINT NumBytesRead;
    #if SYSTEM_SUPPORT_UCOS
        OS_CPU_SR cpu_sr;
    #endif

    phFile = (FIL *)p;

    if (NumBytesReq > sizeof(jpegBuffer))
    {
        NumBytesReq = sizeof(jpegBuffer);
    }

    //移动指针到应该读取的位置
    if(Off == 1) readaddress = 0;
    else readaddress=Off;

    #if SYSTEM_SUPPORT_UCOS
        OS_ENTER_CRITICAL(); //临界区
    #endif

    f_lseek(phFile,readaddress);

    //读取数据到缓冲区中
    f_read(phFile,jpegBuffer,NumBytesReq,&NumBytesRead);

    #if SYSTEM_SUPPORT_UCOS
        OS_EXIT_CRITICAL(); //退出临界区
    #endif

    *ppData = (U8 *)jpegBuffer;
    return NumBytesRead;//返回读取到的字节数
}
```

```

//在指定位置显示加载到 RAM 中的 JPEG 图片
//JPEGFileName:图片在 SD 卡或者其他存储设备中的路径(需文件系统支持!)
//mode:显示模式
//      0 在指定位置显示, 有参数 x,y 确定显示位置
//      1 在 LCD 中间显示图片, 当选择此模式的时候参数 x,y 无效。
//x:图片左上角在 LCD 中的 x 轴位置(当参数 mode 为 1 时, 此参数无效)
//y:图片左上角在 LCD 中的 y 轴位置(当参数 mode 为 1 时, 此参数无效)
//member: 缩放比例的分子项
//denom:缩放比例的分母项
//返回值:0 显示正常,其他 失败
int dispjpeg(u8 *JPEGFileName,u8 mode,u32 x,u32 y,int member,int denom)
{
    u16 bread;
    char *jpegbuffer;
    char result;
    int XSize,YSize;
    GUI_JPEG_INFO JpegInfo;
    float Xflag,Yflag;

    #if SYSTEM_SUPPORT_UCOS
        OS_CPU_SR cpu_sr;
    #endif

    result = f_open(&JPEGFile,(const TCHAR*)JPEGFileName,FA_READ); //打开文件
    //文件打开错误或者文件大于 JPEGMEMORYSIZE
    if((result != FR_OK) || (JPEGFile.fsize>JPEGMEMORYSIZE))    return 1;

    jpegbuffer=mymalloc(SRAMEX,JPEGFile.fsize);    //申请内存
    if(jpegbuffer == NULL) return 2;

    #if SYSTEM_SUPPORT_UCOS
        OS_ENTER_CRITICAL(); //临界区
    #endif

    result = f_read(&JPEGFile,jpegbuffer,JPEGFile.fsize,(UINT *)&bread); //读取数据
    if(result != FR_OK) return 3;

    #if SYSTEM_SUPPORT_UCOS
        OS_EXIT_CRITICAL(); //退出临界区
    #endif

    GUI_JPEG_GetInfo(jpegbuffer,JPEGFile.fsize,&JpegInfo); //获取 JEGP 图片信息

```

```

XSize = JpegInfo.XSize;//获取 JPEG 图片的 X 轴大小
YSize = JpegInfo.YSize;//获取 JPEG 图片的 Y 轴大小
switch(mode)
{
    case 0: //在指定位置显示图片
        if((member == 1) && (denom == 1)) //无需缩放，直接绘制
        {
            GUI_JPEG_Draw(jpegbuffer,JPEGFile.fsize,x,y);//在指定位置显示 JPEG 图片
        }else //否则图片需要缩放
        {
            GUI_JPEG_DrawScaled(jpegbuffer,JPEGFile.fsize,x,y,member,denom);
        }
        break;
    case 1: //在 LCD 中间显示图片
        if((member == 1) && (denom == 1)) //无需缩放，直接绘制
        {
            //在 LCD 中间显示图片
            GUI_JPEG_Draw(jpegbuffer,JPEGFile.fsize,(lcddev.width-XSize)/2-1,\
                (lcddev.height-YSize)/2-1);
        }else //否则图片需要缩放
        {
            Xflag = (float)XSize*((float)member/(float)denom);
            Yflag = (float)YSize*((float)member/(float)denom);
            XSize = (lcddev.width-(int)Xflag)/2-1;
            YSize = (lcddev.height-(int)Yflag)/2-1;
            GUI_JPEG_DrawScaled(jpegbuffer,JPEGFile.fsize,XSize,YSize,\
                member,denom);
        }
        break;
}
f_close(&JPEGFile); //关闭 JPEGFile 文件
myfree(SRAMEX,jpegbuffer); //释放内存
return 0;
}

```

//在指定位置显示无需加载到 RAM 中的 BMP 图片(需文件系统支持！对于小 RAM，推荐使用此方法！)

//JPEGFileName:图片在 SD 卡或者其他存储设备中的路径

//mode:显示模式

// 0 在指定位置显示，有参数 x,y 确定显示位置

// 1 在 LCD 中间显示图片，当选择此模式的时候参数 x,y 无效。

//x:图片左上角在 LCD 中的 x 轴位置(当参数 mode 为 1 时，此参数无效)

//y:图片左上角在 LCD 中的 y 轴位置(当参数 mode 为 1 时，此参数无效)



```

//member: 缩放比例的分子项
//denom:缩放比例的分母项
//返回值:0 显示正常,其他 失败
int displayjpegex(u8 *JPEGFileName,u8 mode,u32 x,u32 y,int member,int denom)
{
    char result;
    int XSize,YSize;
    float Xflag,Yflag;
    GUI_JPEG_INFO JpegInfo;

    result = f_open(&JPEGFile,(const TCHAR*)JPEGFileName,FA_READ); //打开文件
    //文件打开错误
    if(result != FR_OK)    return 1;

    GUI_JPEG_GetInfoEx(JpegGetData,&JPEGFile,&JpegInfo);
    XSize = JpegInfo.XSize;//JPEG 图片 X 大小
    YSize = JpegInfo.YSize;//JPEG 图片 Y 大小
    switch(mode)
    {
        case 0: //在指定位置显示图片
            if((member == 1) && (denom == 1)) //无需缩放, 直接绘制
            {
                GUI_JPEG_DrawEx(JpegGetData,&JPEGFile,x,y);//在指定位置显示 BMP 图片
            }else //否则图片需要缩放
            {
                GUI_JPEG_DrawScaledEx(JpegGetData,&JPEGFile,x,y,member,denom);
            }
            break;
        case 1: //在 LCD 中间显示图片
            if((member == 1) && (denom == 1)) //无需缩放, 直接绘制
            {
                //在 LCD 中间显示图片
                GUI_JPEG_DrawEx(JpegGetData,&JPEGFile,(lcddev.width-XSize)/2-1,\
                                (lcddev.height-YSize)/2-1);
            }else //否则图片需要缩放
            {
                Xflag = (float)XSize*((float)member/(float)denom);
                Yflag = (float)YSize*((float)member/(float)denom);
                XSize = (lcddev.width-(int)Xflag)/2-1;
                YSize = (lcddev.height-(int)Yflag)/2-1;
                GUI_JPEG_DrawScaledEx(JpegGetData,&JPEGFile,XSize,YSize,\
                                      member,denom);
            }
        }
    }
}

```

```

        }
        break;
    }
    f_close(&JPEGFile);    //关闭 BMPFile 文件
    return 0;
}

void jpegdisplay_demo(void)
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_FontHZ16);
    GUI_Clear();

    while(1)
    {
        GUI_DispStringHCenterAt("在指定位置显示一张加载到 RAM 中的 JPEG 图片",240,0);
        displyjpeg("0:/PICTURE/JPEG/实例图片.jpg",0,0,0, 1,1);
        GUI_Delay(1000);
        GUI_Clear();

        GUI_DispStringHCenterAt("在 LCD 中间显示一张扩大 2 倍的加载到\
                                RAM 中的 JPEG 图片",240,0);
        displyjpeg("0:/PICTURE/JPEG/jpeg0.jpg",1,0, 0,2,1);
        GUI_Delay(1000);
        GUI_Clear();

        GUI_DispStringHCenterAt("在 LCD 中间显示一张缩小 1/2 的加载到\
                                RAM 中的 JPEG 图片",240,0);
        displyjpeg("0:/PICTURE/JPEG/SIM900A.jpg",1,0,0,1,2);
        GUI_Delay(1000);
        GUI_Clear();

        GUI_DispStringHCenterAt("在指定位置显示一张无需加载的 JPEG 图片",240,0);
        displayjpegex("0:/PICTURE/JPEG/实例图片.jpg",0,0,0,1,1);
        GUI_Delay(1000);
        GUI_Clear();

        GUI_DispStringHCenterAt("在 LCD 中间显示一张扩大 2 倍的\
                                无需加载的 JPEG 图片",240,0);
        displayjpegex("0:/PICTURE/JPEG/jpeg0.jpg",1,0,0,2,1);
        GUI_Delay(1000);
        GUI_Clear();
    }
}

```

```

    GUI_DispStringHCenterAt("在 LCD 中间显示一张缩小 1/2 的\
                            无需加载的 JPEG 图片",240,0);
    displayjpegex("0:/PICTURE/JPEG/SIM900A.jpg",1,0,0,1,2);
    GUI_Delay(1000);
    GUI_Clear();
}
}

```

在 jpegdisplay.c 中有 3 个函数，如表 23.3.1 所示：

| 函数                  | 描述                                               |
|---------------------|--------------------------------------------------|
| JpegGetData()       | 从外部存储器获取 JPEG 文件数据                               |
| displyjpeg ()       | 将 JPEG 图片加载到 RAM 中，并显示到 LCD 上，显示速度快，但是需要大量的 RAM。 |
| displayjpegex()     | 不需要将 JPEG 加载到 RAM 中，显示速度慢，但是不需要大量的 RAM。          |
| jpegdisplay_demo () | 本章实验的演示函数，在 LCD 上显示图片。                           |

表 24.4.1 jpegdisplay.c 文件函数

函数 JpegGetData ()用于从外部存储器中获取 JPEG 图片数据，在函数 displayjpegex()显示 JPEG 图片的时候做为参数传递给 GUI\_JPEG\_GetInfoEx()。

函数 displyjpeg ()用于显示 JPEG 图片，该函数首先将 JPEG 图片的所有数据加载到 RAM 中，然后调用 EMWIN 中的相应函数在 LCD 上显示加载到 RAM 中的 JPEG 图片，由于需要将整副 JPEG 图片加载到 RAM 中，所以需要大量的 RAM，但是显示速度快！此函数可以设置要显示的 JPEG 图片的位置和放大倍数。

函数 displayjpegex ()也是显示 JPEG 图片的，与函数 displyjpeg ()不同的是，displayjpegex ()不需要将 JPEG 图片加载到 RAM 中，可以直接从外部存储器中读取 JPEG 图片并显示，这种方法需要的 RAM 小，但是速度相比 displyjpeg ()要慢，此函数也可以设置要显示的 JPEG 图片的位置和放大倍数。

函数 jpegdisplay\_demo ()是本章实验的演示 demo 函数。

## 24.5 下载验证

编译并下载到开发板中，LCD 轮流着按照指定的显示方法显示图片，图 24.5.1 是先将 JPEG 图片加载到 RAM 中，然后再显示出来，注意，图片扩大了 2 倍！



图 24.5.1 显示加载到 RAM 中的 JPEG 图片

图 24.5.2 是直接从 SD 卡中读取 JPEG 图片，并不需要事先将整幅 JPEG 读取到 RAM 中，图 24.5.2 中的图片缩小为原来的 1/2。

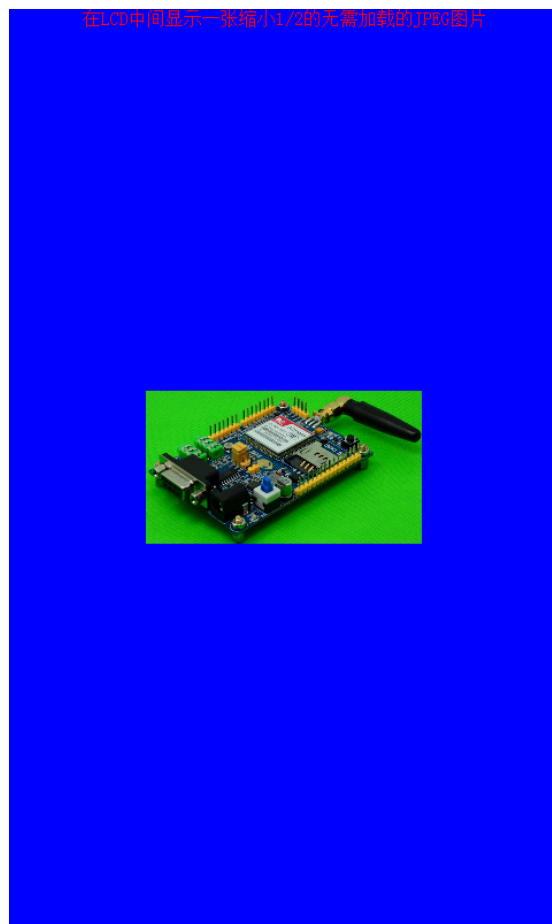


图 24.5.2 直接从 SD 卡读取显示 JPEG 图片

## 第二十五章 GIF 图片显示

GIF 图片用于跨数据网络传输，GIF 标准支持隔行扫描、透明、应用定义数据、动画以及原始文本渲染，emWin 将忽略原始文本或应用特定数据等不受支持的数据。GIF 文件使用 LZW (Lempel-Zif-Welch) 文件压缩方法来压缩图像数据，这种压缩方法运行起来不会丢失数据。输出图像与输入图像完全相同。本章就讲解如何使用 emWin 中的相关 API 函数在 LCD 上显示 GIF 图片，本章分为如下几部分：

25.1 GIF 文件支持

25.2 GIF 文件 API 函数

25.3 程序设计

25.4 下载验证

## 25.1 GIF 文件支持

图形库首先对图形信息进行解码，在显示 GIF 图像的时候，其中解码流程将花费相当长的时间。如果在窗口管理器经常调用的回调函数中使用 GIF 文件，同样，解码过程会花费相当长的时间。通过使用存储设备可缩短计算时间，最好的方法是先将图像绘制到存储设备中，在这种情况下，将只进行一次解压缩。

emWin 的 GIF 解压缩过程大约需要 16 千字节动态分配的 RAM 进行解压缩，绘制图像后，将释放用于解压缩的 RAM。

## 25.2 GIF 文件 API 函数

表 25.2.1 列出了关于 GIF 图片显示的 API 函数。

| 函数                        | 描述                            |
|---------------------------|-------------------------------|
| GUI_GIF_Draw()            | 绘制已加载到存储器的 GIF 文件的第一个图像。      |
| GUI_GIF_DrawEx()          | 绘制无需加载到存储器的 GIF 文件的第一个图像。     |
| GUI_GIF_DrawSub()         | 绘制已加载到存储器的 GIF 文件的给定子图像。      |
| GUI_GIF_DrawSubEx()       | 绘制无需加载到存储器的 GIF 文件的给定子图像。     |
| GUI_GIF_DrawSubScaled()   | 绘制已加载到存储器的带比例的 GIF 文件的给定子图像。  |
| GUI_GIF_DrawSubScaledEx() | 绘制无需加载到存储器的带比例的 GIF 文件的给定子图像。 |
| GUI_GIF_GetComment()      | 返回已加载到存储器的 GIF 文件给定的注释。       |
| GUI_GIF_GetCommentEx()    | 返回无需加载到存储器的 GIF 文件的给定注释       |
| GUI_GIF_GetImageInfo()    | 返回已加载到存储器的 GIF 文件的相关信息。       |
| GUI_GIF_GetImageInfoEx()  | 返回无需加载到存储器的 GIF 文件的相关信息。      |
| GUI_GIF_GetInfo()         | 返回已加载到存储器的 GIF 文件的相关信息。       |
| GUI_GIF_GetInfoEx()       | 返回无需加载到存储器的 GIF 文件的相关信息。      |
| GUI_GIF_GetXSize()        | 返回加载到存储器的位图的 X 大小。            |
| GUI_GIF_GetXSizeEx()      | 返回无需加载到存储器的位图的 X 大小。          |
| GUI_GIF_GetYSize()        | 返回加载到存储器的位图的 Y 大小。            |
| GUI_GIF_GetYSizeEx()      | 返回无需加载到存储器的位图的 Y 大小。          |

表 25.2.1 GIF 文件 API 函数

## 25.3 程序设计

本实验完整工程为：**EMWIN 实验 26 GIF 图片显示**，打开文件 gifdisplay.c，程序源码如下：

```
#include "gifdisplay.h"
#include "EmWinHZFont.h"
#include "GUI.h"
#include "malloc.h"
#include "ff.h"
#include "ili93xx.h"

#if SYSTEM_SUPPORT_UCOS
#include "includes.h"           //ucos 使用
```

```
#endif

static FIL GIFFile;
static char gifBuffer[GIFPERLINESIZE];

static int GifGetData(void * p, const U8 ** ppData, unsigned NumBytesReq, U32 Off)
{
    static int readaddress=0;
    FIL * phFile;
    UINT NumBytesRead;
    #if SYSTEM_SUPPORT_UCOS
        OS_CPU_SR cpu_sr;
    #endif

    phFile = (FIL *)p;

    if (NumBytesReq > sizeof(gifBuffer))
    {
        NumBytesReq = sizeof(gifBuffer);
    }

    //移动指针到应该读取的位置
    if(Off == 1) readaddress = 0;
    else readaddress=Off;

    #if SYSTEM_SUPPORT_UCOS
        OS_ENTER_CRITICAL(); //临界区
    #endif
    f_lseek(phFile,readaddress);

    //读取数据到缓冲区中
    f_read(phFile,gifBuffer,NumBytesReq,&NumBytesRead);

    #if SYSTEM_SUPPORT_UCOS
        OS_EXIT_CRITICAL(); //退出临界区
    #endif

    *ppData = (U8 *)gifBuffer;
    return NumBytesRead;//返回读取到的字节数
}

//在指定位置显示加载到 RAM 中的 GIF 图片
```



```

//GIFFileName:图片在 SD 卡或者其他存储设备中的路径(需文件系统支持! )
//mode:显示模式
//      0 在指定位置显示, 有参数 x,y 确定显示位置
//      1 在 LCD 中间显示图片, 当选择此模式的时候参数 x,y 无效。
//x:图片左上角在 LCD 中的 x 轴位置(当参数 mode 为 1 时, 此参数无效)
//y:图片左上角在 LCD 中的 y 轴位置(当参数 mode 为 1 时, 此参数无效)
//member: 缩放比例的分子项
//denom:缩放比例的分母项
//返回值:0 显示正常,其他 失败
int displaygif(char *GIFFileName,u8 mode,u32 x,u32 y,int member,int denom)
{
    int i;
    u16 bread;
    char *gifbuffer;
    char result;
    int XSize,YSize;
    GUI_GIF_INFO GifInfo;
    GUI_GIF_IMAGE_INFO ImageInfo;
    float Xflag,Yflag;

    #if SYSTEM_SUPPORT_UCOS
        OS_CPU_SR cpu_sr;
    #endif

    result = f_open(&GIFFile,(const TCHAR*)GIFFileName,FA_READ);    //打开文件
    //文件打开错误或者文件大于 JPEGMEMORYSIZE
    if((result != FR_OK) || (GIFFile.fsize>GIFMEMORYSIZE))    return 1;

    gifbuffer=mymalloc(SRAMEX,GIFFile.fsize);
    if(gifbuffer == NULL) return 2;

    #if SYSTEM_SUPPORT_UCOS
        OS_ENTER_CRITICAL(); //临界区
    #endif

    result = f_read(&GIFFile,gifbuffer,GIFFile.fsize,(UINT *)&bread); //读取数据
    if(result != FR_OK) return 3;

    #if SYSTEM_SUPPORT_UCOS
        OS_EXIT_CRITICAL(); //退出临界区
    #endif

    GUI_GIF_GetInfo(gifbuffer,GIFFile.fsize,&GifInfo); //获取 GIF 图片信息

```

```

XSize = GifInfo.xSize; //获取 GIF 图片的 X 轴大小
YSize = GifInfo.ySize; //获取 GIF 图片的 Y 轴大小
switch(mode)
{
    case 0: //在指定位置显示图片
        if((member == 1) && (denom == 1)) //无需缩放，直接绘制
        {
            //在指定位置显示 JPEG 图片
            for(i=0;i<GifInfo.NumImages;i++)
            {
                GUI_GIF_DrawSub(gifbuffer,GIFFile.fsize,x,y,i);
                GUI_GIF_GetImageInfo(gifbuffer,GIFFile.fsize,&ImageInfo,i);
                GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay*10:100);//延时
            }
        }
        else //否则图片需要缩放
        {
            for(i=0;i<GifInfo.NumImages;i++)
            {
                GUI_GIF_DrawSubScaled(gifbuffer,GIFFile.fsize,x,y,i,member,denom);
                GUI_GIF_GetImageInfo(gifbuffer,GIFFile.fsize,&ImageInfo,i);
                GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay*10:100);//延时
            }
        }
        break;
    case 1: //在 LCD 中间显示图片
        if((member == 1) && (denom == 1)) //无需缩放，直接绘制
        {
            //在 LCD 中间显示图片
            for(i=0;i<GifInfo.NumImages;i++)
            {
                GUI_GIF_DrawSub(gifbuffer,GIFFile.fsize,(lcddev.width-XSize)/2-1,\
                                (lcddev.height-YSize)/2-1,i);
                GUI_GIF_GetImageInfo(gifbuffer,GIFFile.fsize,&ImageInfo,i);
                GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay*10:100);//延时
            }
        }
        else //否则图片需要缩放
        {
            Xflag = (float)XSize*((float)member/(float)denom);
            Yflag = (float)YSize*((float)member/(float)denom);
            XSize = (lcddev.width-(int)Xflag)/2-1;
            YSize = (lcddev.height-(int)Yflag)/2-1;
        }
    }
}

```

```

        for(i=0;i<GifInfo.NumImages;i++)
        {
            GUI_GIF_DrawSubScaled(gifbuffer,GIFFile.fsize,XSize,\
                                   YSize,i,member,denom);
            GUI_GIF_GetImageInfo(gifbuffer,GIFFile.fsize,&ImageInfo,i);
            GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay*10:100); //延时
        }
    }
    break;
}
f_close(&GIFFile);    //关闭 JPEGFile 文件
myfree(SRAMEX,gifbuffer); //释放内存
return 0;
}

```

//在指定位置显示无需加载到 RAM 中的 GIF 图片(需文件系统支持! 对于小 RAM, 推荐使用此方法! )

//GIFFileName:图片在 SD 卡或者其他存储设备中的路径

//mode:显示模式

// 0 在指定位置显示, 有参数 x,y 确定显示位置

// 1 在 LCD 中间显示图片, 当选择此模式的时候参数 x,y 无效。

//x:图片左上角在 LCD 中的 x 轴位置(当参数 mode 为 1 时, 此参数无效)

//y:图片左上角在 LCD 中的 y 轴位置(当参数 mode 为 1 时, 此参数无效)

//member: 缩放比例的分子项

//denom:缩放比例的分母项

//返回值:0 显示正常,其他 失败

int displaygifex(char \*GIFFileName,u8 mode,u32 x,u32 y,int member,int denom)

```

{
    char result;
    int i;
    int XSize,YSize;
    float Xflag,Yflag;
    GUI_GIF_INFO GifInfo;
    GUI_GIF_IMAGE_INFO ImageInfo;

    result = f_open(&GIFFile,(const TCHAR*)GIFFileName,FA_READ);    //打开文件
    //文件打开错误
    if(result != FR_OK)    return 1;

    GUI_GIF_GetInfoEx(GifGetData,&GIFFile,&GifInfo);
    XSize = GifInfo.xSize; //GIF 图片 X 大小
    YSize = GifInfo.ySize; //GIF 图片 Y 大小
    switch(mode)

```

```

{
    case 0: //在指定位置显示图片
        if((member == 1) && (denom == 1)) //无需缩放，直接绘制
        {
            //在指定位置显示 BMP 图片
            for(i=0;i<GifInfo.NumImages;i++)
            {
                GUI_GIF_DrawSubEx(GifGetData,&GIFFile,x,y,i);
                GUI_GIF_GetImageInfoEx(GifGetData,&GIFFile,&ImageInfo,i);
                GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay*10:100);//延时
            }
        }
        else //否则图片需要缩放
        {
            for(i=0;i<GifInfo.NumImages;i++)
            {
                GUI_GIF_DrawSubScaledEx(GifGetData,&GIFFile,x,y,i,member,denom);
                GUI_GIF_GetImageInfoEx(GifGetData,&GIFFile,&ImageInfo,i);
                GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay*10:100);//延时
            }
        }
        break;
    case 1: //在 LCD 中间显示图片
        if((member == 1) && (denom == 1)) //无需缩放，直接绘制
        {
            //在 LCD 中间显示图片
            for(i=0;i<GifInfo.NumImages;i++)
            {
                GUI_GIF_DrawSubEx(GifGetData,&GIFFile,(lcddev.width-XSize)/2-1,\
                                   (lcddev.height-YSize)/2-1,i);
                GUI_GIF_GetImageInfoEx(GifGetData,&GIFFile,&ImageInfo,i);
                GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay*10:100);//延时
            }
        }
        else //否则图片需要缩放
        {
            Xflag = (float)XSize*((float)member/(float)denom);
            Yflag = (float)YSize*((float)member/(float)denom);
            XSize = (lcddev.width-(int)Xflag)/2-1;
            YSize = (lcddev.height-(int)Yflag)/2-1;

            for(i=0;i<GifInfo.NumImages;i++)
            {
                GUI_GIF_DrawSubScaledEx(GifGetData,&GIFFile,XSize,
                                         YSize,i,member,denom);
            }
        }
    }
}

```

```
        GUI_GIF_GetImageInfoEx(GifGetData,&GIFFile,&ImageInfo,i);
        GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay*10:100);//延时
    }
}
break;
}

f_close(&GIFFile);    //关闭 GIFFile 文件
return 0;
}

//GIF 图片显示例程
void gifdisplay_demo(void)
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetFont(&GUI_FontHZ16);
    GUI_SetColor(GUI_RED);
    GUI_Clear();
    while(1)
    {
        GUI_DispStringHCenterAt("在指定位置显示一张加载到 RAM 中的 GIF 图片",400,0);
        displaygif("0:/PICTURE/GIF/龙.gif",0,20,20,1,1);
        GUI_Delay(1000);
        GUI_Clear();

        GUI_DispStringHCenterAt("在 LCD 中间显示一张扩大 2 倍的加载到 RAM 中的\
                                GIF 图片",400,0);
        displaygif("0:/PICTURE/GIF/小黄人.gif",1,0,0,2,1);
        GUI_Delay(1000);
        GUI_Clear();

        GUI_DispStringHCenterAt("在 LCD 中间显示一张缩小 1/2 的加载到 RAM 中的\
                                GIF 图片",400,0);
        displaygif("0:/PICTURE/GIF/火柴人.gif",1,0,0,1,2);
        GUI_Delay(1000);
        GUI_Clear();

        GUI_DispStringHCenterAt("在指定位置显示一张无需加载的 GIF 图片",400,0);
        displaygifex("0:/PICTURE/GIF/龙.gif",0,20,20,1,1);
        GUI_Delay(1000);
        GUI_Clear();
    }
}
```

```

GUI_DispStringHCenterAt("在 LCD 中间显示一张扩大 2 倍的无需加载的\
                        GIF 图片",400,0);
displaygifex("0:/PICTURE/GIF/小黄人.gif",1,0,0,2,1);
GUI_Delay(1000);
GUI_Clear();

GUI_DispStringHCenterAt("在 LCD 中间显示一张缩小 1/2 的无需加载的\
                        GIF 图片",400,0);
displaygifex("0:/PICTURE/GIF/火柴人.gif",1,0,0,1,2);
GUI_Delay(1000);
GUI_Clear();
}
}

```

在 gifdisplay.c 中有 4 个函数，如表 23.3.1 所示：

| 函数                 | 描述                                              |
|--------------------|-------------------------------------------------|
| GifGetData ()      | 从外部存储器获取 GIF 文件数据                               |
| displaygif ()      | 将 GIF 图片加载到 RAM 中，并显示到 LCD 上，显示速度快，但是需要大量的 RAM。 |
| dispgifex()        | 不需要将 GIF 加载到 RAM 中，显示速度慢，但是不需要大量的 RAM。          |
| gifdisplay_demo () | 本章实验的演示函数，在 LCD 上显示图片。                          |

表 23.3.1 gifdisplay.c 文件函数

函数 GifGetData ()用于从外部存储器中获取 JPEG 图片数据，在函数 displaygifex ()显示 GIF 图片的时候做为参数传递给 GUI\_GIF\_GetImageInfoEx ()。

函数 displaygif ()用于显示 GIF 图片，该函数首先将 GIF 图片的所有数据加载到 RAM 中，然后调用 EMWIN 中的相应函数在 LCD 上显示加载到 RAM 中的 GIF 图片，由于需要将整副 GIF 图片加载到 RAM 中，所以需要大量的 RAM，但是显示速度快！此函数可以设置要显示的 GIF 图片的位置和放大倍数。

函数 displaygifex ()也是显示 GIF 图片的，与函数 displygif ()不同的是，displaygifex ()不需要将 GIF 图片加载到 RAM 中，可以直接从外部存储器中读取 GIF 图片并显示，这种方法需要的 RAM 小，但是速度相比 displygif ()要慢，此函数也可以设置要显示的 GIF 图片的位置和放大倍数。

函数 gifdisplay\_demo ()是本章实验的演示 demo 函数。

## 25.4 下载验证

编译并下载到开发板中，LCD 轮流着按照指定的显示方法显示图片，图 25.4.1 是先将 GIF 图片加载到 RAM 中，然后在指定的位置显示出来。

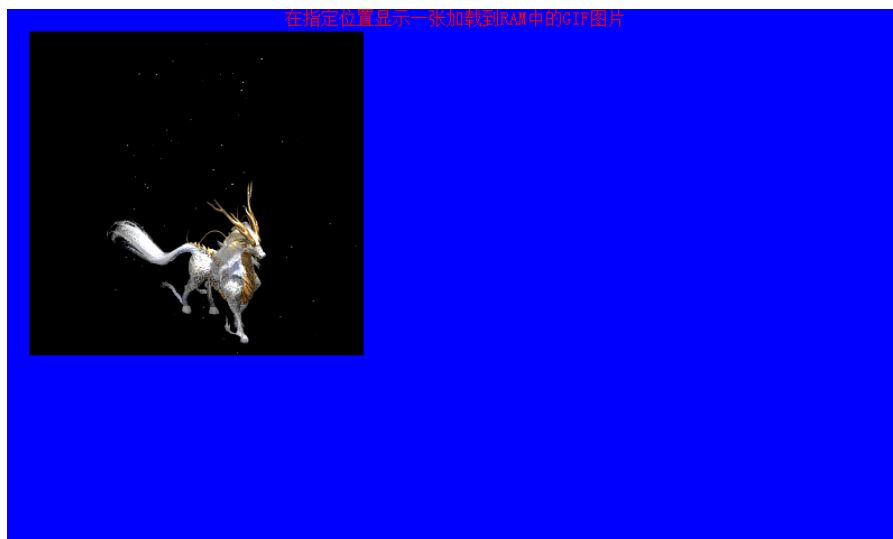


图 24.5.1 显示加载到 RAM 中的 GIF 图片

图 25.4.2 是直接从 SD 卡中读取 GIF 图片，并不需要事先将整幅 GIF 读取到 RAM 中，图 25.4.2 中的图片扩大了 2 倍。



图 24.5.2 直接从 SD 卡读取显示 GIF 图片

## 第二十六章 PNG 图片显示

PNG 格式是一种图像格式，其目的是试图替代 GIF 和 TIFF 文件格式，同时增加一些 GIF 文件格式所不具备的特性。PNG 用来存储灰度图像时，灰度图像的深度可多到 16 位，存储彩色图像时，彩色图像的深度可多到 48 位，并且还可存储多到 16 位的  $\alpha$  通道数据。emWin 是支持 PNG 解码的，本章我们就讲解一下如何在 LCD 上显示 PNG 图片，本章分为如下几部分：

26.1 基础工程准备

26.2 RAM 使用

26.3 PNG 文件 API 函数

26.4 程序设计

26.5 下载验证



## 26.1 基础工程准备

emWin源码中默认是不带PNG图片解码库的，我们需要在segger官方下载PNG解码库：libpng，下载地址为：[www.segger.com/link/emwin\\_png.zip](http://www.segger.com/link/emwin_png.zip) 获得，我们已经下载好放到光盘中，路径为：6、软件资料->3、EMWIN学习资料->emwin\_png，下载下来有多个版本的libpng，如图 26.1.1 所示，由于STemWin是 5.22 版本的emWin，所以libpng库我们选择emWin\_V5.18 版本的。

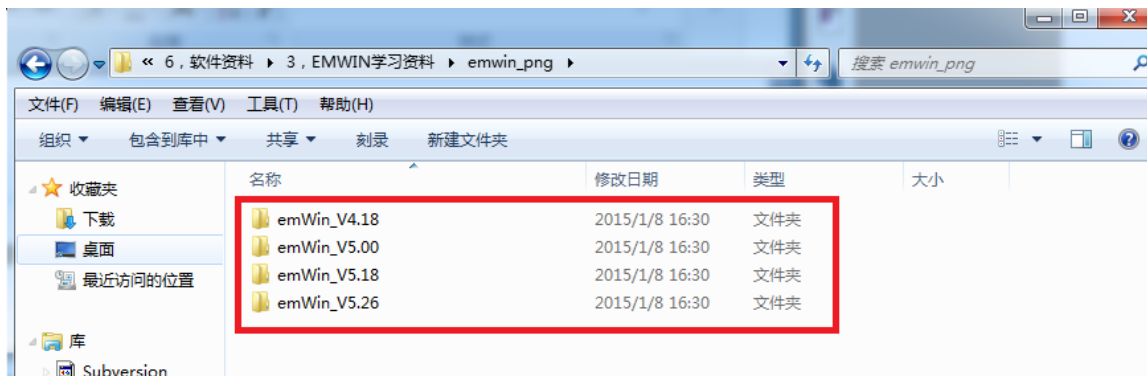


图 26.1.1 libpng 库

将 libpng 库添加到工程中，如图 26.1.2 所示。

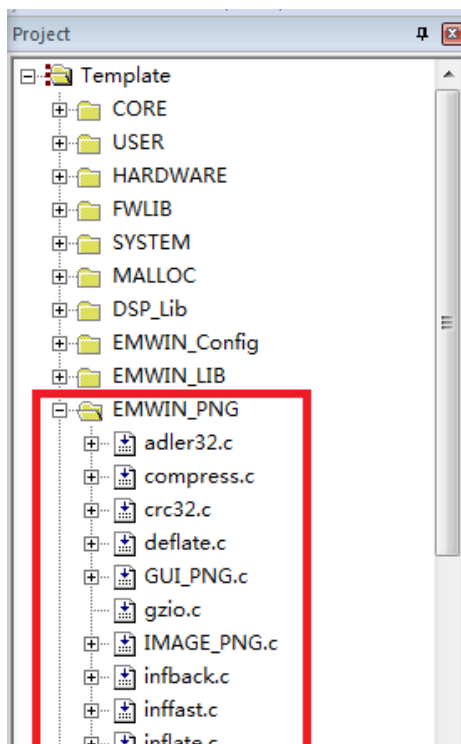


图 26.1.2 向工程中添加 libpng 库

## 26.2 RAM 使用

PNG 解压缩大约需要 21 Kb RAM 用于与图像大小无关的解压缩和依赖大小的字节量。RAM 要求可按以下方式计算：

大约 RAM 要求 = (X-Size + 1)\* Y 大小 \* 4 + 21Kbytes。

这里我们给 emWin 分配 500k 的外部 SRAM 内存，打开 GUIConf.c，图 26.2.1 红框就是为

emWin 分配的内存大小。

```

1  #include "GUI.h"
2  #include "sram.h"
3  #include "malloc.h"
4
5
6  #define USE_EXRAM 1//使用外部RAM
7  //设置EMWIN内存大小
8  #define GUI_NUMBYTES (500*1024)
9  #define GUI_BLOCKSIZE 0x80 //块大小
10
11

```

图 26.2.1 emWin 内存大小

### 26.3 PNG 文件 API 函数

表 26.3.1 列出了关于 PNG 图片显示的 API 函数。

| 函数                   | 描述                   |
|----------------------|----------------------|
| GUI_PNG_Draw()       | 绘制已加载到存储器的 PNG 文件。   |
| GUI_PNG_DrawEx()     | 绘制无需加载到存储器的 PNG 文件。  |
| GUI_PNG_GetXSize()   | 返回加载到存储器的位图的 X 大小    |
| GUI_PNG_GetXSizeEx() | 返回无需加载到存储器的位图的 X 大小。 |
| GUI_PNG_GetYSize()   | 返回加载到存储器的位图的 Y 大小。   |
| GUI_PNG_GetYSizeEx() | 返回无需加载到存储器的位图的 Y 大小。 |

表 26.3.1 PNG 文件 API 函数

### 26.4 程序设计

本实验完整工程为：EMWIN 实验 27 PNG 图片显示，打开文件 pngdisplay.c，程序源码如下：

```

#include "pngdisplay.h"
#include "EmWinHZFont.h"
#include "GUI.h"
#include "malloc.h"
#include "ff.h"
#include "ili93xx.h"
#if SYSTEM_SUPPORT_UCOS
#include "includes.h" //ucos 使用
#endif

static FIL PNGFile;
static int PngGetData(void * p, const U8 ** ppData, unsigned NumBytesReq, U32 Off)
{
    static int readaddress=0;
    FIL * phFile;

```

```

    U8 *pData;
    UINT NumBytesRead;
    #if SYSTEM_SUPPORT_UCOS
        OS_CPU_SR cpu_sr;
    #endif

    pData = (U8*)*ppData;
    phFile = (FIL *)p;

    //移动指针到应该读取的位置
    if(Off == 1) readaddress = 0;
    else readaddress=Off;

    #if SYSTEM_SUPPORT_UCOS
        OS_ENTER_CRITICAL(); //临界区
    #endif
    f_lseek(phFile,readaddress);

    //读取数据到缓冲区中
    f_read(phFile,pData,NumBytesReq,&NumBytesRead);

    #if SYSTEM_SUPPORT_UCOS
        OS_EXIT_CRITICAL(); //退出临界区
    #endif
    return NumBytesRead;//返回读取到的字节数
}

//在指定位置显示加载到 RAM 中的 PNG 图片(PNG 图片不能缩放!)
//PNGFileName:图片在 SD 卡或者其他存储设备中的路径(需文件系统支持!)
//mode:显示模式
//      0 在指定位置显示, 有参数 x,y 确定显示位置
//      1 在 LCD 中间显示图片, 当选择此模式的时候参数 x,y 无效。
//x:图片左上角在 LCD 中的 x 轴位置(当参数 mode 为 1 时, 此参数无效)
//y:图片左上角在 LCD 中的 y 轴位置(当参数 mode 为 1 时, 此参数无效)
//返回值:0 显示正常,其他 失败
int displaypng(char *PNGFileName,u8 mode,u32 x,u32 y)
{
    GUI_HMEM PNGMem;
    u16 bread;
    char *pngbuffer;
    char result;
    int XSize,YSize;

```

```

#if SYSTEM_SUPPORT_UCOS
    OS_CPU_SR cpu_sr;
#endif

result = f_open(&PNGFile,(const TCHAR*)PNGFileName,FA_READ); //打开文件
//文件打开错误或者文件大于 JPEGMEMORYSIZE
if((result != FR_OK) || (PNGFile.fsize>PNGMEMORYSIZE)) return 1;

pngbuffer=mymalloc(SRAMEX,PNGFile.fsize);
if(pngbuffer == NULL) return 2;

#if SYSTEM_SUPPORT_UCOS
    OS_ENTER_CRITICAL(); //临界区
#endif

result = f_read(&PNGFile,pngbuffer,PNGFile.fsize,(UINT *)&bread); //读取数据
if(result != FR_OK) return 3;

#if SYSTEM_SUPPORT_UCOS
    OS_EXIT_CRITICAL(); //退出临界区
#endif

XSize = GUI_PNG_GetXSize(pngbuffer,PNGFile.fsize); //获取 PNG 图片的 X 轴大小
YSize = GUI_PNG_GetYSize(pngbuffer,PNGFile.fsize); //获取 PNG 图片的 Y 轴大小
switch(mode)
{
    case 0: //在指定位置显示图片
        GUI_PNG_Draw(pngbuffer,PNGFile.fsize,x,y);//显示 PNG 图片
        break;
    case 1: //在 LCD 中间显示图片
        GUI_PNG_Draw(pngbuffer,PNGFile.fsize,(lcddev.width-XSize)/2-1,\
            (lcddev.height-YSize)/2-1);
        break;
}
f_close(&PNGFile); //关闭 PNGFile 文件
myfree(SRAMEX,pngbuffer);
return 0;
}

//在指定位置显示无需加载到 RAM 中的 PNG 图片(需文件系统支持! 对于小 RAM, 推荐使用
此方法! PNG 图片不能缩放!)
//PNGFileName:图片在 SD 卡或者其他存储设备中的路径

```

```

//mode:显示模式
//      0 在指定位置显示, 有参数 x,y 确定显示位置
//      1 在 LCD 中间显示图片, 当选择此模式的时候参数 x,y 无效。
//x:图片左上角在 LCD 中的 x 轴位置(当参数 mode 为 1 时, 此参数无效)
//y:图片左上角在 LCD 中的 y 轴位置(当参数 mode 为 1 时, 此参数无效)
//返回值:0 显示正常,其他 失败
int displaypngex(char *PNGFileName,u8 mode,u32 x,u32 y)
{
    char result;
    int XSize,YSize;

    result = f_open(&PNGFile,(const TCHAR*)PNGFileName,FA_READ); //打开文件
    //文件打开错误
    if(result != FR_OK)    return 1;

    XSize = GUI_PNG_GetXSizeEx(PngGetData,&PNGFile);//PNG 图片 X 大小
    YSize = GUI_PNG_GetYSizeEx(PngGetData,&PNGFile);//PNG 图片 Y 大小
    switch(mode)
    {
        case 0: //在指定位置显示图片
            GUI_PNG_DrawEx(PngGetData,&PNGFile,x,y);
            break;
        case 1: //在 LCD 中间显示图片
            GUI_PNG_DrawEx(PngGetData,&PNGFile,(lcddev.width-XSize)/2-1,\
                           (lcddev.height-YSize)/2-1);
            break;
    }
    f_close(&PNGFile); //关闭 PNGFile 文件
    return 0;
}

//PNG 图片显示例程
void pngdisplay_demo(void)
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetFont(&GUI_FontHZ16);
    GUI_SetColor(GUI_RED);
    GUI_Clear();
    while(1)
    {
        GUI_DispStringHCenterAt("在指定位置显示一张加载到 RAM 中的 PNG 图片",400,0);
        displaypng("0:/PICTURE/PNG/皇冠.png",0,200,100);
        GUI_Delay(1000);
    }
}

```

```

GUI_Clear();

displaypng("0:/PICTURE/PNG/CS.png",1,0,0);
GUI_Delay(1000);
GUI_Clear();

displaypng("0:/PICTURE/PNG/香蕉.png",1,0,0);
GUI_Delay(1000);
GUI_Clear();

GUI_DispStringHCenterAt("在指定位置显示一张无需加载的 PNG 图片",400,0);
displaypngex("0:/PICTURE/PNG/驴.png",0,200,100);
GUI_Delay(1000);
GUI_Clear();

displaypngex("0:/PICTURE/PNG/文件夹.png",1,0,0);
GUI_Delay(1000);
GUI_Clear();

displaypngex("0:/PICTURE/PNG/风扇.png",1,0,0);
GUI_Delay(1000);
GUI_Clear();
}
}

```

在 pngdisplay.c 中有 4 个函数，如表 26.4.1 所示：

| 函数                 | 描述                                              |
|--------------------|-------------------------------------------------|
| PngGetData ()      | 从外部存储器获取 PNG 文件数据                               |
| displaypng ()      | 将 PNG 图片加载到 RAM 中，并显示到 LCD 上，显示速度快，但是需要大量的 RAM。 |
| displaypngex ()    | 不需要将 PNG 加载到 RAM 中，显示速度慢，但是不需要大量的 RAM。          |
| pngdisplay_demo () | 本章实验的演示函数，在 LCD 上显示图片。                          |

表 26.4.1 pngdisplay.c 文件函数

函数 PngGetData ()用于从外部存储器中获取 PNG 图片数据，在函数 displaypngex ()显示 PNG 图片的时候做为参数传递给 GUI\_PNG\_DrawEx ()和 GUI\_PNG\_GetXSizeEx()这两个函数。

函数 displaypng ()用于显示 PNG 图片，该函数首先将 PNG 图片的所有数据加载到 RAM 中，然后调用 EMWIN 中的相应函数在 LCD 上显示加载到 RAM 中的 PNG 图片，由于需要将整副 PNG 图片加载到 RAM 中，所以需要大量的 RAM，但是显示速度快！此函数可以设置要显示的 PNG 图片的位置。

函数 displaypngex ()也是显示 PNG 图片的，与函数 displaypng ()不同的是，displaypngex ()不需要将 PNG 图片加载到 RAM 中，可以直接从外部存储器中读取 GIF 图片并显示，这种方法需要的 RAM 小，但是速度相比 displaypng ()要慢，此函数也可以设置要显示的 PNG 图片的位置。

函数 pngdisplay\_demo 是本章实验的演示 demo 函数。

## 26.4 下载验证

编译并下载到开发板中，LCD 轮流着按照指定的显示方法显示图片，图 26.4.1 是先将 PNG 图片加载到 RAM 中，然后在指定的位置显示出来。



图 26.4.1 显示加载到 RAM 中的 PNG 图片

图 26.4.2 为直接从 SD 卡中读取 PNG 图片然后直接显示在 LCD 上的。



图 26.4.2 直接显示 PNG 图片